

Reinforcement Learning Algorithms Set 3

Table of Contents

Autonomous reinforcement learning with experience replay	2
Prioritized Level Replay	13
Rainbow: Combining Improvements in Deep Reinforcement Learning	37
Soft Actor-Critic for Discrete Action Settings	50
Revisiting Discrete Soft Actor-Critic	57
Twin-Delayed DDPG: A Deep Reinforcement Learning Technique to Model a Continuous Movement of an Intelligent Robot Agent	85
A TD3-based multi-agent deep reinforcement learning method in mixed cooperation-competition environment	89
Scaling CrossQ with Weight Normalization	98
END	102



Contents lists available at SciVerse ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

2012 Special Issue

Autonomous reinforcement learning with experience replay

Paweł Wawrzyński^{a,*}, Ajay Kumar Tanwani^b^a Warsaw University of Technology, Institute of Control and Computation Engineering, Poland^b École Polytechnique Fédérale De Lausanne, Switzerland

ARTICLE INFO

Keywords:

Reinforcement learning
Autonomous learning
Step-size estimation
Actor–critic

ABSTRACT

This paper considers the issues of efficiency and autonomy that are required to make reinforcement learning suitable for real-life control tasks. A real-time reinforcement learning algorithm is presented that repeatedly adjusts the control policy with the use of previously collected samples, and autonomously estimates the appropriate step-sizes for the learning updates. The algorithm is based on the actor–critic with experience replay whose step-sizes are determined on-line by an enhanced fixed point algorithm for on-line neural network training. An experimental study with simulated octopus arm and half-cheetah demonstrates the feasibility of the proposed algorithm to solve difficult learning control problems in an autonomous way within reasonably short time.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

Reinforcement learning (RL) is a potentially useful methodology for control optimization. It may be the only feasible approach to many problems. However, its widespread use is limited by the total time many RL algorithms require to solve a given problem. This time encompasses duration of a single run in collecting appropriate number of samples multiplied by the number of runs necessary to tune problem dependent parameters. Selection of appropriate parameter setting requires learning to start over again, thereby, making the previously collected data useless. The cost of collecting data from control process and the elapsed time to handcraft the parameters makes learning infeasible for many real world tasks. Ideally, the learner should interact with the environment in a ‘plug-and-play’ manner and optimize a control policy in a single trial with minimum number of data samples. The main focus of this research is to make reinforcement learning suitable for real control applications and minimize the associated implementation costs.

A broad class of RL algorithms that encompass Actor–Critics (Bhatnagar, Sutton, Ghavamzadeh, & Lee, 2009; Konda & Tsitsiklis, 2003; Peters, Vijayakumar, & Schaal, 2005) and policy gradient methods (Sutton, McAllester, Singh, & Mansour, 2000; Williams, 1992) optimizes a control policy on the basis of stochastic gradient estimates. In these algorithms, the control policy is parameterized by a vector that is optimized incrementally on the basis of the data coming from the control process. The policy vector adjustment is

local in the sense that the algorithm moves a small step in the estimate of the improvement direction; its length is defined by the step-size of the learning process. The appropriate length of the step-size depends on the stage of the learning process and setting it wrong can make the process unstable or prohibitively slow. In this paper, we present a real-time reinforcement learning algorithm that is considerably efficient and autonomous due to: (1) *experience replay*—it stores the agent–environment interaction samples in a database and repeatedly draws previous samples—simultaneous to the interaction—to estimate the improvement direction of policy vector and speed up learning (Wawrzyński, 2009), and (2) *step-size estimation*—it autonomously determines the appropriate step-size to optimize the learning updates and eliminate the overhead of its manual tuning (Wawrzyński, 2010).

Optimization by means of improvement direction estimates is a tool commonly used in RL but it is not unique to this field. It is the main principle of a wide class of stochastic approximation algorithms (Kushner & Yin, 1997). In all these methods the step-size plays a key role in controlling their speed and stability. Despite significance of the problem of step-size estimation, its generally applicable solution has not been found.

1.1. Related work

A lot of effort is made in the reinforcement learning community to speed up the learning methods by using the previously collected samples. The concept of reusing samples evolved from recomputing previous experience by means of dynamic programming (Sutton, 1990). Application of the *importance sampling* technique (Rubinstein, 1981) made it possible to optimize a parametric control policy with the use of experience gained with other policies

* Corresponding author.

E-mail addresses: pwawrzyński@gmail.com, p.wawrzyński@elka.pw.edu.pl (P. Wawrzyński), ajay.tanwani@epfl.ch (A.K. Tanwani).

(Hachiya, Peters, & Sugiyama, 2011; Peshkin & Mukherjee, 2001). Kober and Peters (2011) proposed an algorithm called PoWER—Policy Learning by Weighting Exploration with the Returns—that combines the previous experience by per-decision importance weighting technique. The approach to sample reuse developed here is experience replay (Adam, Busoniu, & Babuska, 2012; Cichosz, 1999) applied to actor–critics with the use of the randomized truncated estimators (Wawrzyński, 2009).

Real world applications of reinforcement learning require a number of parameters to be defined and empirically tuned, e.g., reward function, policy structure, step-size, exploration noise, discount factor, and others specific to the learning algorithm in use. Despite several promising approaches to provide leverage for these overheads such as the use of apprenticeship learning to find reward function (Abbeel & Ng, 2005), dynamic evolution of policy parameters to get its appropriate size and representation (Kormushev, Ugurlu, Calinon, Tsagarakis, & Caldwell, 2011), the parameter that remains difficult to determine autonomously in reinforcement learning is the step-size of the learning process. The work (Sutton, 1992a, 1992b) established *delta-bar-delta* algorithm, introduced earlier by Jacobs (1988) as a generally applicable approach to step-size estimation in RL. The method requires the initial step-size to be specified and collapses if this value is too large, which delimits its autonomous use. Consequently, the solution to the problem of step-size determination in reinforcement learning is still a subject of active research (George & Powell, 2006; Noda, 2009; Schraudolph, Yu, & Aberdeen, 2006).

Many approaches to step-size estimation have been designed as tools for online neural network training. Algorithms of this type include *delta-delta* (Silva & Almeida, 1990), aforementioned *delta-bar-delta*, and stochastic meta-descent (Schraudolph & Giannakopoulos, 2000). Among others, step-sizes can be estimated to ensure Lyapunov-stability of the process (Behera, Kumar, & Patnaik, 2006; Kathirvalavakumar & Subavathi, 2009).

Most methods of autonomous or adaptive step-size estimation are designed as recursions that are updated from one sample to another (George & Powell, 2006). This design is often based on some problem-dependent parameters and may be sensitive to wrong initialization or nonstationarity. In this work we build upon the fixed-point method of step-size estimation (Wawrzyński, 2010) which has a different design to remedy the aforementioned problems. By computing gradient estimates always at two points it captures global characteristics of the process, thereby, obtaining robustness and independence from any parameters. In Wawrzyński and Papis (2011) this approach was specialized to neural-network on-line training, e.g., each weight of the network was assigned a separate step-size. Here the original algorithm is enhanced and adopted to provide autonomy to RL with experience replay.

1.2. Contributions

The purpose of this paper is to present a reinforcement learning framework that is fast and autonomous enough for on-line optimization of control of systems with complex dynamics and multidimensional, possibly continuous, state and input spaces. The contribution of this paper may be summarized in the following points:

- An enhanced fixed point method of step-size estimation (Wawrzyński, 2010) is presented.
- The above method of step-size estimation is combined with experience replay for actor–critic class of RL algorithms (Wawrzyński, 2009).
- The proposed approach is implemented on two simulated robot-control problems, both with complex dynamics and rich state and action spaces, namely *octopus arm* and *half-cheetah*.

The rest of the paper is organized as follows: Section 2 formulates the reinforcement learning problem followed by the description of actor–critic algorithm with experience replay in Section 3. In Section 4, we describe the fixed point method of step-size adaptation in a generic stochastic descent procedure. The resulting actor–critic reinforcement learning algorithm with experience replay and step-size estimation is proposed in Section 5. Experimental study is presented in Section 6 on challenging learning control problems. The last section concludes the paper with an outlook to future work.

2. Problem formulation

We consider the standard reinforcement learning setup under the Markov Decision Process (MDP) framework (Sutton & Barto, 1998). The problem concerns an agent that observes the state of its environment, s_t , in discrete time, $t = 1, 2, 3, \dots$, performs an action, a_t , which moves the environment to the next state, s_{t+1} , and gives the agent a reward, $r_t \in \mathcal{R}$. The environment is in general stochastic which means that the consecutive state, s_{t+1} , is a result of sampling from the transition distribution conditioned on the preceding state, s_t , and the action, a_t . Mathematically,

$$s_{t+1} \sim P_s(\cdot | s_t, a_t).$$

The reward may depend deterministically on the current action and the next state, $r_t = r(a_t, s_{t+1})$. A particular MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, P_s, r \rangle$ where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, respectively; $\{P_s(\cdot | s, a) : s \in \mathcal{S}, a \in \mathcal{A}\}$ is a set of state transition distributions and r is the reward function. The transition distributions, P_s , and the reward function, r , are initially unknown to the agent. The goal of learning is to determine a stochastic control policy π that assigns actions to states such that in each state the agent may expect the highest rewards in the future. Note that the control policy maps each state into a distribution of actions rather than a single action. Performing different actions in each state, the agent is able to differentiate good actions from inferior ones in each state. The control policy π is parameterized by the policy vector, θ , e.g., a set of weights in a neural network. It can be represented as

$$a \sim \pi(\cdot | s, \theta).$$

Let us denote by π_θ the policy of the form π with the fixed parameter θ . The quality of this policy is measured by the *value-function* V^{π_θ} defined as the expected value of the discounted sum of future rewards the agent may gain in the future, starting from the given state, s , at a certain time, t , and the policy π_θ :

$$V^{\pi_\theta}(s) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} | s_t = s, \text{ policy} = \pi_\theta \right). \quad (1)$$

The parameter $\gamma \in [0, 1)$ is the discount factor that defines the weight of distant rewards in relation to those obtained sooner. The quality of a policy may also be evaluated using the *action-value function* Q^{π_θ} , defined as

$$Q^{\pi_\theta}(s, a) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} | s_t = s, a_t = a, \text{ policy} = \pi_\theta \right). \quad (2)$$

It is the expected value of the same sum, but here it also depends on the first action to be performed, that is the action performed in the given state. This function is crucial for the policy optimization as it tells good actions from inferior ones in the same state.

We seek a learning algorithm that can optimize the control policy (i) without parameters determined experimentally, (ii) within short time of agent–environment interaction, but not necessarily after small amount of computation.

Algorithm 1. The classic actor–critic.

- 0: Set $y_\theta = 0, y_v = 0, t = 1$. Initialize θ and v
- 1: Choose the action: $a_t \sim \pi(\cdot; s_t, \theta)$
- 2: Execute a_t , evaluate s_{t+1} and r_t
- 3: Calculate the temporal difference $d_t(v)$
- 4: $d_t(v) = r_t + \gamma \bar{V}(s_{t+1}; v) - \bar{V}(s_t; v)$
- 5: Update the actor:
- 6: $y_\theta := (\gamma \lambda) y_\theta + \beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta)$
- 7: $\theta := \theta + y_\theta d_t$
- 8: Update the critic:
- 9: $y_v := (\gamma \lambda) y_v + \beta_t^v \nabla_v \bar{V}(s_t; v)$
- 10: $v := v + y_v d_t$
- 11: Assign $t := t + 1$ and repeat from Point 1

3. Actor–critic with experience replay

This section presents actor–critic algorithms (Barto, Sutton, & Anderson, 1983; Bhatnagar et al., 2009; Kimura & Kobayashi, 1998; Konda & Tsitsiklis, 2003) and the way of its speeding-up by experience replay (Adam et al., 2012; Cichosz, 1999; Wawrzyński, 2009). Particular attention is paid to the ‘classic’ actor–critic in the form presented by Kimura and Kobayashi (1998) because this algorithm is conceptually the simplest among its family to which the ideas presented in further sections are directly applicable.

3.1. Classic Actor–Critic

The classic actor–critic algorithm employs two data structures. The first one is an actor, $\pi(a; s, \theta)$, that represents a stochastic control policy to generate random actions on the basis of the given state and the policy vector $\theta \in \mathbb{R}^{n_\theta}$. The second one is a critic, $\bar{V}(s; v)$, that represents an approximator of the value function (1) parameterized by the critic vector $v \in \mathbb{R}^{n_v}$. The algorithm is based on a special estimator of the action-value function $Q^{\pi_\theta}(s_t, a_t)$ (Eq. (2)). This estimator has the form:

$$R_t^\lambda = r_t + \gamma \bar{V}(s_{t+1}; v) + \sum_{i \geq 1} (\gamma \lambda)^i (r_{t+i} + \gamma \bar{V}(s_{t+i+1}; v) - \bar{V}(s_{t+i}; v)). \quad (3)$$

The parameter $\lambda \in [0, 1]$ defines how much R_t^λ is influenced by the value-function approximator, $\bar{V}$, and the true rewards, r . For $\lambda = 1$ the estimator is based on true rewards only and thus unbiased, but its variance may be very large. For $\lambda = 0$ variance of the estimator is the smallest, but it is biased by the inaccuracy of the value-function approximator. Moderate values of λ balance the bias and the variance of the estimator.

The classic actor–critic is sketched in Algorithm 1. It can be verified that after a visit in state s_t , for $t = 1, 2, \dots$ the algorithm modifies the policy vector θ by the product $\beta_t^\theta \hat{\phi}_t$ where β_t^θ is a step-size (small positive number) and $\hat{\phi}_t$ is the policy vector improvement direction estimate given by

$$\hat{\phi}_t = (R_t^\lambda - \bar{V}(s_t; v)) \nabla_\theta \ln \pi(a_t; s_t, \theta), \quad (4)$$

where ∇_θ means gradient with respect to θ . (For $\lambda = 0$, the modification takes place in step t , for $\lambda > 0$ it is also partially distributed over further steps). Consequently, if the action a_t turns out to bring rewards, R_t^λ , larger than $\bar{V}(s_t; v)$, i.e., the rewards expected in state s_t , then the probability of action a_t in state s_t is being increased. If, conversely, the action turns out to bring rewards smaller than expected, then its probability is being decreased.

The purpose of the critic, $\bar{V}(s; v)$ is to approximate the expected value of R_t^λ . Therefore, a visit in state s_t induces a modification of the critic vector v by the product $\beta_t^v \hat{\psi}_t$ where β_t^v is a step-size and

Algorithm 2. Actor–Critic with Experience Replay. Estimators mentioned in Steps 6 and 7 are based on the samples in a database.

- 1: $t := 1$, Initialize θ and v
- 2: Choose and execute an action, $a_t \sim \pi(\cdot; s_t, \theta)$
- 3: Assign $\pi_t = \pi(a_t; s_t, \theta)$
- 4: Repeat $v(t)$ times, begin
- 5: Draw $i \in \{t - N + 1, t - N + 2, \dots, t\}$
- 6: Adjust θ along an estimator of $\phi(s_i, \theta, v)$:
- 7: $\theta := \theta + \beta_t^\theta \hat{\phi}_i^r(\theta, v)$
- 8: Adjust v along an estimator of $\psi(s_i, \theta, v)$:
- 9: $v := v + \beta_t^v \hat{\psi}_i^r(\theta, v)$
- 10: End
- 11: Register the tuple $\langle s_t, a_t, \pi_t, r_t, s_{t+1} \rangle$ in the database
- 12: Make sure only N most recent tuples remain in the database.
- 13: Assign $t := t + 1$ and repeat from Line 2

$\hat{\psi}_t$ is the critic vector improvement direction estimate:

$$\hat{\psi}_t = (R_t^\lambda - \bar{V}(s_t; v)) \nabla_v \bar{V}(s_t; v). \quad (5)$$

3.2. Classic actor–critic with experience replay

The idea of experience replay is to repeatedly recall previous pieces of experience and apply them to update the current policy as a sequential RL algorithm (like the classic actor–critic) would if they have just happened. This idea was applied to actor–critics in Wawrzyński (2009). Namely, let us denote by

$$\phi(s, \theta, v) = E(\hat{\phi}_t | s_t = s, \theta, v) \quad (6)$$

a direction in which θ is on average modified to increase the rewards expected in state s . Also, let us denote by

$$\psi(s, \theta, v) = E(\hat{\psi}_t | s_t = s, \theta, v) \quad (7)$$

a direction in which v is on average modified to increase the accuracy of the value function approximation in state s .

After each instant t , the classic actor–critic estimates $\phi(s_t, \theta, v)$, i.e., the direction of policy improvement, and adjusts the policy vector θ along the estimate. Before the next instant t , the modified algorithm repeatedly chooses one of the recently visited states at random, s_i , estimates $\phi(s_i, \theta, v)$, and modifies the policy vector along the estimate. Essentially both algorithms achieve the same goal, but the modified one improves the current actor and critic with the use of the whole gathered experience rather than only the present event, like the classic method. Due to more exhaustive exploitation of information experience replay leads to faster learning at the cost of additional computation.

In Wawrzyński (2009), the *randomized-truncated estimators* are introduced for estimating $\phi(s_i, \theta, v)$ and $\psi(s_i, \theta, v)$. To define them, let b be the upper limit of the randomized truncated estimators with $b > 1$, θ_{i+j} be the policy vector applied to generate a_{i+j} , $\alpha \in [0, 1]$ and K be drawn independently from $Geom(\rho)$ the geometric distribution¹ with parameter $\rho \in [0, 1]$. The randomized-truncated estimators, $\hat{\phi}_i^r(\theta, v)$ and $\hat{\psi}_i^r(\theta, v)$ have a generic form that in the case of classic actor–critic is reduced to

$$\times \sum_{k=0}^K \alpha^k d_{i+k}(v) \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi_{i+j}}, b \right\}, \quad (8)$$

¹ That is, random variable K of values in $\{0, 1, 2, \dots\}$ has distribution $Geom(\rho)$, iff $P(K = m) = (1 - \rho)\rho^m$ for nonnegative integer m .

$$\hat{\psi}_i^r(\theta, v) = \nabla_v \bar{V}(s_i; v) \times \sum_{k=0}^K \alpha^k d_{i+k}(v) \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi_{i+j}}, b \right\} \quad (9)$$

where

$$d_i(v) = r_i + \gamma \bar{V}(s_{i+1}; v) - \bar{V}(s_i; v),$$

is the temporal difference and π_{i+j} is the probability of the action taken at instant $i + j$ (or it is the density if $\mathcal{A}$ is continuous). Remembering the definition of R_t^k (Eq. (3)) makes it easy to notice the similarity between $\hat{\phi}_t$ (Eq. (4)) and $\hat{\phi}_i^r$ (Eq. (8)), and between $\hat{\psi}_t$ (Eq. (5)) and $\hat{\psi}_i^r$ (Eq. (9)). Though, there are two important differences. First, the infinite sum in Eq. (3) is replaced by the finite one with appropriately designed random limit in Eqs. (8) and (9). Second, truncated density ratios are introduced in order to compensate for the difference between the current policy and the ones that generated the actions $a_t, a_{t-1}, \dots$ present in the database.

The actor-critic with experience replay is presented in Algorithm 2. The number of computation steps after t -th control step is defined by the function $\nu(t)$. Since computation is carried out on-line, this function is bounded from above. It should also be limited for small t to prevent overtraining with too few samples in the database. (In Section 5 this function is discussed in more detail.)

3.3. Generalized actor-critic with experience replay

Experience replay may be applied to the classic actor-critic to increase its learning speed, but also to a wide class of sequential actor-critic algorithms. As shown in Wawrzyński (2009) the sufficient condition is that each step in the environment prompts the original algorithm to do modifications (possibly distributed in time) of the policy vector and the critic vector with estimates, respectively, $\hat{\phi}_t$ and $\hat{\psi}_t$ of the generic form

$$G_t(\theta, v) \sum_{k \geq 0} (\alpha \rho)^k z_{t,k}(\theta, v)$$

where G_t is a vector defined by s_t and a_t , $\alpha \in [0, 1)$, $\rho \in [0, 1)$, and $z_{t,k} \in \mathcal{R}$ is defined by $s_{t+k}, a_{t+k}, r_{t+k}, s_{t+k+1}$, and possibly a_{t+k+1} .

4. Fixed point method of step-size estimation

The actor-critic algorithm with experience replay repeatedly samples data and uses it to compute a certain vector with which it updates parameters θ and v . This pattern of operation places it in a wide class of optimization algorithms based on stochastic gradient descent (Kushner & Yin, 1997). Each algorithm of that type defines a sequence

$$\theta_{t+1} = \theta_t - \beta_t g(\theta_t; \xi_t), \quad t = 1, 2, \dots \quad (10)$$

in which $\theta \in \mathcal{R}^{n_\theta}$ is a parameter to be optimized, $\beta_t, t = 1, 2, \dots$ is a sequence of step-sizes, $\xi_t, t = 1, 2, \dots$ is a sequence of data samples and g is a function such that for random ξ the expected value of $-g(\theta, \xi)$ defines the direction towards the local minimum of a certain quality index, $J : \mathcal{R}^{n_\theta} \mapsto \mathcal{R}$. Typically this direction is the opposite of the gradient

$$Eg(\theta, \xi) = \nabla J(\theta).$$

For example, suppose a feedforward neural network is trained to approximate a function from given samples. Then θ is the vector of neural weights, J is the quality index, ξ represents an input-output pair, and g is the gradient computed by means of e.g., error backpropagation. In order to train the network, the input-output pairs may be sampled repeatedly, and its weights may be adjusted according to Eq. (10), which is exactly how the

well established back-propagation algorithm works (Rumelhart, Hinton, & Williams, 1988).

For the sequence (10) to have convergence guarantees, a set of requirements needs to be fulfilled (Kushner & Yin, 1997). Among others, the sequence of step-sizes has to be vanishing at an appropriate pace:

$$\begin{cases} \sum_{t \geq 1} \beta_t = +\infty, \\ \sum_{t \geq 1} \beta_t^2 < +\infty. \end{cases} \quad (11)$$

Unfortunately the practical value of the above conditions is limited. Even if they are satisfied, the step-sizes may still be too large and hence make the convergence process unstable, or too small and hence yield a very slow convergence.

The fixed point method of step-size estimation addresses this issue by autonomously setting the step-sizes (Wawrzyński, 2010). The method is based on the following principles:

1. The process (10) is divided into parts such that within each part the step-size remains constant. The length, n , of a part starting at moment t is kept large enough to enable estimation of the expected value of $g(\theta_t, \xi)$ with sufficient accuracy.
2. The sums $G_{t,n}$ and $G_{t,n}^*$ are computed for each part beginning at time t and ending at time $t + n$

$$G_{t,n} = \sum_{i=0}^{n-1} g(\theta_{t+i}, \xi_{t+i}), \quad G_{t,n}^* = \sum_{i=0}^{n-1} g(\theta_t, \xi_{t+i}). \quad (12)$$

where the vector $G_{t,n}$ is proportional to the displacement of the point θ between instant t and $t + n$, and the vector $G_{t,n}^*$ is proportional to the displacement of the point θ if all the gradient estimators g are calculated using θ_t .

3. At the end of each part the discrepancy between $G_{t,n}^*$ and $G_{t,n}$ is registered. On average, it increases with the ratio of the step-size used and the optimal one giving fastest convergence. Therefore, if the discrepancy is too small, the step-size is being increased. If it is too large, it is decreased. If it is radically too large, then instability of the process is detected, the step-size is radically decreased, and θ moves back to the point where the part began.

This last principle is illustrated in Fig. 1. For a very small β , the vector θ_{t+i} moves through a flat slope of the function J . This results in small differences between $g(\theta_t, \xi_{t+i})$ and $g(\theta_{t+i}, \xi_{t+i})$, thereby, making the discrepancy between $G_{t,n}$ and $G_{t,n}^*$ small as well. The speed of the local minimum search can be increased with larger values of β . However, if β is too large, θ_{t+i} jumps over the minimum of the function to be optimized. This creates large differences between $g(\theta_t, \xi_{t+i})$ and $g(\theta_{t+i}, \xi_{t+i})$ manifested by large discrepancy between $G_{t,n}$ and $G_{t,n}^*$.

The papers (Wawrzyński, 2010; Wawrzyński & Papis, 2011) present a statistical analysis to formalize the above intuitive principles in a simple, unidimensional model. Since each $g(\theta_t, \xi_{t+i})$ in (12) is an unbiased estimate of $\nabla J(\theta_t)$, the value $\frac{1}{n} G_{t,n}^*$ is also an unbiased estimator of $\nabla J(\theta_t)$ and its quality increases with n . The model considers n that ensures signal-to-noise ratio of 1 in $\frac{1}{n} G_{t,n}^*$, namely

$$n = \frac{E_{\theta_t} \|g(\theta_t, \xi)\|^2}{\|\nabla J(\theta_t)\|^2}. \quad (13)$$

For n of that size, and constant $\alpha_0 \cong 0.15$ the difference

$$\alpha_0 \|G_{t,n}^*\|^2 - \|G_{t,n}^* - G_{t,n}\|^2 \quad (14)$$

is on average equal to 0 if the step-size is optimal for fast convergence of (10). If the step-size is smaller than optimal, the

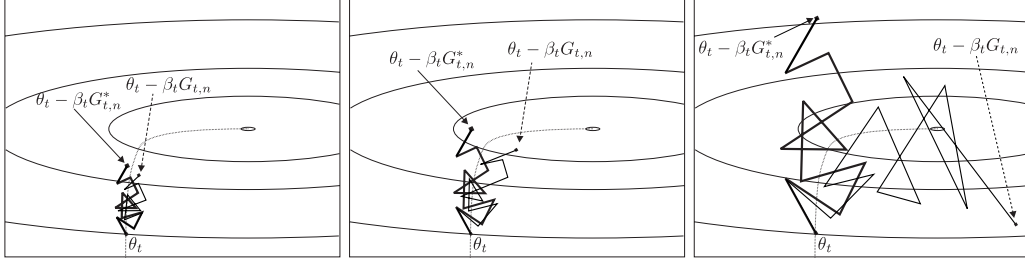


Fig. 1. The sketches depict contour lines of three different functions J with equal $\nabla J(\theta_t)$, β , and $g(\theta_t, \xi_{t+i})$ for $i \geq 0$. In n steps starting from t , the parameter in the J domain evolves from θ_t to $\theta_{t+n} = \theta_t - \beta_t G_{t,n}$. Left: Small curvature of J in relation to β , causes small difference of $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$. Middle: Moderate curvature of J causes moderate difference between $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$. Right: Very large curvature of J relatively to β causes instability of the optimization process which is manifested by divergence of $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$ that grows with n .

first term of (14) is larger than the latter, and for too large step-size, the relation is opposite. Therefore, the difference (14) on average indicates the direction in which the step-size should be modified, i.e., increased or decreased. In other words, the step-size is stabilized at the best level when on average the discrepancy $\|G_{t,n}^* - G_{t,n}\|^2$ is stabilized at the value of $\alpha_0 \|G_{t,n}^*\|^2$. Also, if the term (14) with larger α_0 is applied to adjust the step-size, then the step-size is being stabilized at a higher level than one that is best for the purpose of static optimization (10). This last property will become useful in the next section.

In order to apply the same principle to multidimensional θ , we will consider only the direction of gradient estimate, that is we will project $G_{t,n}$ on $G_{t,n}^*$ and consider the discrepancy between $G_{t,n}^*$ and the projection. The indicator of step-size adjustment (14) will thus take the form

$$\alpha_0 \|G_{t,n}^*\|^2 - \|G_{t,n}^* - \text{proj}(G_{t,n}, G_{t,n}^*)\|^2, \quad (15)$$

for

$$\text{proj}(G, G^*) = G^* G^T G / (\|G^*\|^2 + \epsilon) \quad (16)$$

and ϵ being a small number that prevents division by zero.

The discussion above leads to Algorithm 3 for autonomously setting the step-sizes. It divides the learning process into parts or *estimation periods* in which the step-size β remains fixed. In order to compute n according to (13) the fraction (L/M) estimates

$$E_{\theta_t} \|g(\theta_t, \xi)\|^2$$

such that L adds $\|g(\theta_t, \xi_{t+i})\|^2$ and M counts them. The fraction (L'/M') estimates $\|\nabla J(\theta_t)\|^2$ on the basis of the following property. Let ξ and ξ' be sampled independently from the same distribution. Then we have

$$\begin{aligned} E_{\theta_t} g(\theta_t, \xi)^T g(\theta_t, \xi') &= E_{\theta_t} g(\theta_t, \xi)^T E_{\theta_t} g(\theta_t, \xi') \\ &= \|\nabla J(\theta_t)\|^2. \end{aligned}$$

The estimator of $\|\nabla J(\theta_t)\|^2$ can thus take the form of the statistics

$$\frac{\sum_{i=1}^n \sum_{i'=0}^{i-1} g(\theta_t, \xi_{t+i})^T g(\theta_t, \xi_{t+i'})}{\sum_{i=1}^n i} = \frac{\sum_{i=1}^n g(\theta_t, \xi_{t+i})^T G_{t,i}^*}{\sum_{i=1}^n i}, \quad (17)$$

whose numerator and denominator are denoted by L' and M' , respectively.

We now explain the algorithm in a step-wise manner for better understanding. In Line 3, it receives a new training example. In Lines 6–8, it initializes variables of the fixed point algorithm. In Lines 9–12, it updates estimators L/M and L'/M' . In Lines 13–16 it updates the values $G_{t,n}$, θ_{t+n} , $G_{t,n}^*$, and

$$h = \max_{i=1, \dots, n} \|G_{t,i}^*\|^2,$$

respectively.

In Line 19, the algorithm checks whether the learning process is stable. Its instability is manifested by $\|G_{t,n} - G_{t,n}^*\|^2$ being larger than $h = \max_{1 \leq i \leq n} \|G_{t,i}^*\|^2$. This condition is additionally tightened in the k_0 initial periods. If instability is detected, the estimation period is reinitialized with halved step-size.

In Lines 21–22, the algorithm checks if the present estimation period has come to its end. It happens when

$$n \geq \frac{L/M}{L'/M'},$$

that is when the estimator of $Eg(\theta_t, \xi)$ has appropriate quality (13). The period is also finished when

$$n \geq \frac{3}{2} n',$$

where n' is the duration of the previous period. This prevents the algorithm from getting stuck in local minima ($Eg(\theta, \xi) = 0$ in a minimum and appropriate duration of an estimation period becomes infinite).

In the end of an estimation period, the step-size is modified (Line 25) and a new estimation period is started. Namely, the step-size is incremented proportionally to the statistics (15). The increment is scaled with the variable $\bar{h}$, that is determined in Line 26 as the maximal value of previous $\|G_{t,n}^*\|^2 / (1 - \alpha_1)$. Line 24 prevents very small values of the scaling factor $\bar{h}$, for instance in the first period.

Every time a new estimation period begins, the estimators (L/M) and (L'/M') become obsolete and the amount of information they carry is decreased. In Lines 27–28, the algorithm decreases proportionally the weight of previous samples in comparison to forthcoming ones.

The algorithm updates the step-size after each estimation period to optimize the process (10). One such period is defined by the number of steps sufficient to estimate the direction of movement of θ with unity information-noise ratio. While various processes may differ substantially, the algorithm divides them to similar sequences of periods. Coefficients of the algorithm are chosen to make these sequences converge fast for all underlying processes (10) at its all stages.

In comparison to its version presented in Wawrzyński (2010), the algorithm is improved in two directions. First, it estimates $\|Eg(\theta_t, \xi)\|^2$ better by means of the statistics (17). Second, after each of first k_0 estimation periods it moves the process back to its origin; at this time only data is collected to estimate $\|Eg(\theta_0, \xi)\|^2$ and $E\|g(\theta_0, \xi)\|^2$. The goal of these modifications is to make the algorithm exploit data better and be robust to too large initial step-size.

5. Actor-critic with experience replay and the fixed point method of step-size estimation

This section discusses combination of the actor-critic algorithm with experience replay (ac&er) and the fixed point method of

Algorithm 3. The fixed point method of step-size estimation. The coefficients applied: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $k_0 = 10$, initial $\beta = 1$.
FIXED_POINT

```

1: FP_INITIALIZE( $\theta_0$ )
2: Repeat
3:   Get  $\xi_{t+n}$ 
4:   FP_LOOP( $g(\theta_t, \xi_{t+n}), g(\theta_{t+n}, \xi_{t+n})$ )
5: Until some-stopping-condition

```

FP_INITIALIZE(θ_0)

```

6: Assign  $t = n := 0, k := 1, L = L' = M = M' := 0$ 
7: Assign  $G_{0,0} = G_{0,0}^* := 0, h = \bar{h} := 0, n' = +\infty$ 
8: Initialize  $\theta_0$ , and  $\beta$ 

```

FP_LOOP($g(\theta_t, \xi_{t+n}), g(\theta_{t+n}, \xi_{t+n})$)

```

9:  $L := L + \|g(\theta_t, \xi_{t+n})\|^2$ 
10:  $M := M + 1$ 
11:  $L' := \max\{L' + g(\theta_t, \xi_{t+n})^T G_{t,n}^*, 0\}$ 
12:  $M' := M' + n$ 
13:  $G_{t,n+1} := G_{t,n} + g(\theta_{t+n}, \xi_{t+n})$ 
14:  $\theta_{t+n+1} := \theta_{t+n} - \beta g(\theta_{t+n}, \xi_{t+n})$ 
15:  $G_{t,n+1}^* := G_{t,n}^* + g(\theta_t, \xi_{t+n})$ 
16:  $h := \max\{h, \|G_{t,n+1}^*\|^2\}$ 
17:  $n := n + 1$ 
18: newperiod := false
19: If  $\|G_{t,n} - G_{t,n}^*\|^2 > h \vee (k < k_0 \wedge \|G_{t,n} - G_{t,n}^*\|^2 > 0.3h)$ 
   then
20:    $\beta := \beta/2, \text{newperiod} := \text{true}$ 
21: If  $\neg \text{newperiod} \wedge n \geq 3 \wedge L' > 0$ 
22:    $\wedge n \geq \min\{(L/M)/(L'/M'), \frac{3}{2}n'\}$  then begin
23:   If  $k \geq k_0$  then begin
24:      $\bar{h} := \max\{\bar{h}, \alpha_0 \|G_{t,n}^*\|^2, \|\text{proj}(G_{t,n}, G_{t,n}^*) - G_{t,n}^*\|^2\}$ 
25:      $\beta := \beta \exp((\alpha_0 \|G_{t,n}^*\|^2 - \|\text{proj}(G_{t,n}, G_{t,n}^*) - G_{t,n}^*\|^2)/\bar{h})$ 
26:      $\bar{h} := \max\{\alpha_1 \bar{h}, (L/M)n/(1 - \alpha_1)\}$ 
27:      $L := \alpha_1 L, M := \alpha_1 M$ 
28:      $L' := \alpha_2 L', M' := \alpha_2 M'$ 
29:      $n' := n$ 
30:   end
31:    $t := t + n$ 
32:    $k := k + 1$ 
33:   newperiod := true
34: end
35: If newperiod then
36:    $n := 0, G_{t,0} = G_{t,0}^* := 0, h := h/2$ 

```

step-size adaptation. The original ac&er repeatedly selects data samples and computes the improvement direction estimators $\hat{\phi}_i^r$ and $\hat{\psi}_i^r$ to adjust the parameters of the actor and critic respectively. The combined algorithm optimizes the actor and critic parameters with separate modules that realize the fixed point method of step-size estimation (separate objects, as we would say in terms of object-oriented programming). Below, the generic terms of Section 4 are translated to the components of the ac&er algorithm. For the actor step-size optimization:

- ξ translates into the sequence of data that is required to compute $\hat{\phi}_i^r$.
- the vector that undergoes optimization is the actor parameter, θ ,
- its fixed value will be denoted by θ^* ,
- $g(\theta, \xi)$ represents $-\hat{\phi}_i^r(\theta, v)$; the minus results from the fact, that originally the fixed point method was meant for

minimization while here the direction of optimization is opposite,
– $g(\theta^*, \xi)$ represents $-\hat{\phi}_i^r(\theta^*, v)$.

For the critic step-size estimation we have:

- as previously, ξ translates into the sequence of data that is required to compute $\hat{\psi}_i^r$.
- the vector being optimized is now the critic parameter, v ,
- its fixed value will be denoted by v^* ,
- $g(\theta, \xi)$ represents $-\hat{\psi}_i^r(\theta, v)$; notice that now θ in the first term corresponds to v in the second while θ in the second is only one of the entities that define this random vector,
- $g(\theta^*, \xi)$ represents $-\hat{\psi}_i^r(\theta^*, v^*)$.

The above explanation suffices for straightforward combination of ac&er and the fixed point method of Section 4. This combination may be further enhanced by addressing the issues of overtraining, wrong fixed point initialization, and mutual adjustments of the actor and critic.

Overtraining. In general, a model is overtrained when it fits the data very well, but because the data is not sufficiently representative, the model does not describe the actual system that has generated the data. In the case of ac&er, the actor and critic may become overtrained when they are optimized on the basis of a short history of experiences. For instance, suppose the experience contains only several state-action pairs among which there are pairs with the same state, and the ac&er has enough resources for full optimization of the policy on the basis of these pairs. Then, it would readily make the better action in the same state infinitely more probable than the worse one. Consequently, the resulting policy would not try the worse actions at all which could lead to inability to explore certain area of the action space.

A way to overcome the above difficulty is to introduce regularization. Let

$$l : \mathcal{S} \times \mathcal{R}^{n_\theta} \mapsto \mathcal{R}_+$$

be a loss function that is continuous and assigns, for each state, zero to a bounded set of policy vectors that ensure exploration in a state, and an increasing positive score ($\in \mathcal{R}_+$) to the policy vectors with growing distance from this set. Then, the policy vector adjustment takes the form

$$\theta := \theta + \beta^\theta (\hat{\phi}_i^r(\theta, v) - \nabla l_\theta(s_i, \theta)).$$

As a result, even if the unrepresentative data pushes the policy vector outside the feasible exploration region, the loss function pulls it back.

Wrong fixed point initialization. In the initial period of the fixed point method, the algorithm only collects data and estimates $\|Eg(\theta, \xi)\|^2$ and $E\|g(\theta, \xi)\|^2$. It is assumed then that the g vectors are independently sampled from the appropriate distribution. If there are very few samples in the database, then this assumption is obviously violated and the estimator of $E\|g(\theta, \xi)\|^2$ becomes positively biased. As a result, the length of the estimation periods gets smaller. In order to stabilize the discrepancy between G and G^* for too short estimation periods, the step-sizes are being increased. This may lead to oscillations in the early stages of learning process.

A simple way to overcome this problem is to delay learning until a certain number of samples are collected in the database. To this end, we use the v function to regulate the intensity of replaying experience, of the form

$$v(t) = \begin{cases} 0 & \iff t \leq \text{min_capacity} \\ \text{comp_steps} & \iff t > \text{min_capacity} \end{cases} \quad (18)$$

for $\text{min_capacity} > \text{comp_steps} > 0$, i.e., $v(t)$ is a piecewise constant function that triggers the intensity of replaying experience given by comp_steps only after a certain time has elapsed denoted

Algorithm 4. Actor–Critic with experience replay and step-size estimation based on the fixed point method.

```

1:  $t := 1$ , Initialize  $\theta$  and  $v$ 
2: Actor.FP_INITIALIZE( $\theta_0$ )
3: Critic.FP_INITIALIZE( $v_0$ )
4: Draw and execute an action,  $a_t \sim \pi(\cdot; s_t, \theta)$ 
5: Assign  $\pi_t = \pi(a_t; s_t, \theta)$ 
6: Repeat  $v(t)$  times, begin
7:   Draw  $i \in \{t - N, t - N + 2, \dots, t - 1\}$ 
8:   Adjust  $\theta$  along an estimator of  $\phi(s_i, \theta, v)$ :
9:   Actor.FP_LOOP( $-\hat{\phi}_i^r(\theta^*, v) + \nabla_{\theta^*} l(s_i, \theta^*)$ ,
     $-\hat{\phi}_i^r(\theta, v) + \nabla_{\theta} l(s_i, \theta)$ )
10:  Adjust  $v$  along an estimator of  $\psi(s_i, \theta, v)$ :
11:  Critic.FP_LOOP( $-\hat{\psi}_i^r(\theta, v^*)$ ,  $-\hat{\psi}_i^r(\theta, v)$ )
12: End
13: Register the tuple  $\langle s_t, a_t, \pi_t, r_t, s_{t+1} \rangle$  in the database
14: Make sure only  $N$  most recent tuples remain in the
    database
15: Assign  $t := t + 1$  and repeat from Line 4

```

by *min_capacity*. While *comp_steps* is defined by available computation power, *min_capacity* should be set to exclude the use of too large computational power to optimize the actor and critic vectors on the basis of too few data samples.

Mutual adjustments of the actor and critic. The fixed point method of step-size estimation is originally designed as a tool for stochastic optimization. Actor–critic with experience replay may be understood as an algorithm of that type in which the actor parameter, θ , is optimized to adapt it to the current critic, and the critic parameter, v , is optimized to adapt it to the current actor. In both the cases, the ultimate goal of adaptation is constantly changing or running away. Consequently, the step-sizes should be estimated taking into account larger distance to the goal than currently perceived. The discrepancy between G and G^* in the fixed point method should be thus stabilized at a relatively higher level in ac&er as compared to stationary stochastic optimization. It is also inherent in Actor–Critics that the critic should adapt to the actor faster than *vice versa* (see e.g., Bhatnagar et al., 2009; Konda & Tsitsiklis, 2003).

The above remarks translate into the following guideline for the use of the fixed point method in ac&er: the discrepancies between G and G^* , regulated by the α_0 parameter, should be stabilized at relatively higher level for the critic than for the actor, and in both cases should be larger than optimal for stationary stochastic optimization.

The combined algorithm is presented in Algorithm 4. It defines the agent–environment interaction and the improvement directions for the actor and the critic. Their step-sizes and parameters are adjusted with the two fixed-point modules.

6. Experimental study

In the previous section, a reinforcement learning algorithm is introduced that combines experience replay with autonomous step-size estimation. In this section, it is confronted with two simulated control problems in multidimensional continuous spaces. The problems are selected to have general characteristics and the level of complexity corresponding to real-life robotic applications. The tasks analyzed are: (1) point reaching movement of octopus arm, and (2) cyclic running motion of half-cheetah.

6.1. Octopus arm

Octopus is well-known for exhibiting a high level of flexibility in controlling arm movements. The highly developed limbs

of octopus make it capable of bending, stretching, shortening and twisting its arm at any point and in any direction with virtually unlimited degrees of freedom. Artificial octopus arm has found its niche in continuum robotics to replace conventional serial manipulators with smooth, continuous and flexible links (McMahan et al., 2006). However, their use is severely limited by existing motion planning and control algorithms. Classical approaches of controlling redundant manipulators by applying functional constraints to the additional degrees of freedom with null-space projection do not scale to continuum robots due to added computational complexity (Chiaverini, Oriolo, & Walker, 2008). While the theoretical foundation of continuum kinematics is in its nascent stages, learning based approaches provide an interesting alternative to control such robots. To this end, Engel, Szabó, and Volkinshtein (2005) used the Gaussian process temporal difference based reinforcement learning algorithm to successfully demonstrate various learning tasks of a simulated octopus arm model. Because the original state and action space of an octopus arm has huge number of dimensions, it often serves as a test bed for solutions to the problem of dimensionality in RL (Koutnik, Gomez, & Schmidhuber, 2010; Woolley & Stanley, 2010). Here the proposed reinforcement learning algorithm is tested for solving the task of reaching a given goal by an octopus arm; the state and action spaces are reduced but still rich enough to represent current position and dynamics of the arm.

6.1.1. Control problem

We are interested in learning a reactive control policy for the octopus arm to reach an arbitrary goal point starting from some random position. The octopus arm is represented by a planar simulator model composed of 10 quadrilateral compartments with each segment containing a longitudinal or a transverse muscle, as described by Yekutieli et al. (2005). The mass of each compartment is distributed in its four corners as point masses which are connected by linear damped springs representing muscles. Each compartment follows the muscular hydrostat principle of maintaining a constant volume (area in this case) assuming the muscles to be incompressible. The activation pattern in muscles acts as input to the model and the resulting position of point masses is obtained on the basis of the net force acting on the arm. The forces modeled in the simulation are of four types: (1) internal force generated by muscles, (2) vertical force due to gravity and buoyancy of arm in seawater, (3) external force produced by water drag, and (4) constraint force induced by change in pressure to keep the area of each compartment constant. Moreover, the base is non-rotating in the model, i.e., the first spring is fixed to the ground. The planar dynamic model of the octopus arm provides a trade-off between its fidelity and real-time processing overheads. The model is simulated with the use of software made available for 2006 RL competition (Octopus-sources, 2006) originally written in Java and rewritten for the experiments presented here to C#. The simulator worked in its standard setting with 10 compartments, fixed base, each muscle 1 unit long, and action duration 0.1 s.

We now define the state and action spaces along with the associated reward function to formulate the control problem of octopus arm as MDP for reinforcement learning.

State space. In this study, we define three moving frames in polar coordinates that are used to localize the octopus arm movement towards its goal. The first frame (r_1, ϕ_1) is located at the center of the point masses of all the quadrilateral compartments, the second one (r_2, ϕ_2) is located at the center of the point masses of last 5 quadrilateral compartments, and the third one (r_3, ϕ_3) is located at the center of mass of the last compartment. The first two frames are movable with respect to the arm as they may or may not be located inside the model depending upon its configuration (see Fig. 2).

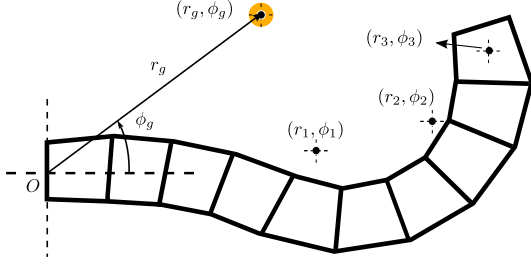


Fig. 2. Simulated octopus arm model with geometric description.

Table 1
State variables of octopus arm.

$s_0 = (r_g - 10)/3$	$s_4 = \dot{r}_1/0.01$	$s_8 = (r_3 - r_g)/3$
$s_1 = \phi_g$	$s_5 = \dot{\phi}_1/0.01$	$s_9 = \phi_3 - \phi_g$
$s_2 = (r_1 - r_g)/3$	$s_6 = (r_2 - r_g)/3$	$s_{10} = \text{sign}(r_3 - r_g)$
$s_3 = \phi_1 - \phi_g$	$s_7 = \phi_2 - \phi_g$	$s_{11} = \text{sign}(\phi_3 - \phi_g)$

Based on the three frames and the goal frame (r_g, ϕ_g) , the state space of our model consists of 12 state variables given in Table 1. Each state variable is normalized to roughly cover the interval $[-1, 1]$. Majority of the variables define the position of three frames with reference to the goal frame. Additionally, variables s_4 and s_5 express velocity of the center of mass, and discrete variables s_{10} and s_{11} describe whether the last compartment is 'in front of' or 'behind' the target. While these variables are redundant in defining arm configuration, they are quite useful inputs to the control policy.

Action space. The action space consists of a set of 6 actions each of which pre-defines the activation level in the arm's muscles, as used in Engel et al. (2005). The action space is shown in Fig. 3.

Episodes and reward function. The learning task is carried out in episodes. An episode terminates with success when the last compartment touches the goal (which is transparent for other compartments). If this goal is not reached within 500 steps, the episode terminates. A new target position is sampled in the beginning of every episode within the area described by a circular region:

$$r_g \in (7.5, 9.5)$$

$$\phi_g \in (-\pi/4, \pi/4).$$

The position of the arm is initialized such that it is set in its default position, an action is chosen randomly from the set {Action 1, Action 2}, and it is applied to the arm for a number of steps chosen randomly from the set $\{1, \dots, 100\}$.

The reward function is defined such that the controller is penalized with a negative score of -1 for all the learning steps in which the goal has not been reached. Moreover, the arm is rewarded for moving towards the goal in proportion to its speed. The reward function is given by:

$$r_t = \begin{cases} d_t - d_{t+1} - 1 & \text{if the target is not reached at } t + 1 \\ d_t & \text{if the target is reached at } t + 1. \end{cases} \quad (19)$$

The above term d_t denotes Euclidean distance between the tip of the arm and the goal position measured at instant t . The goal of the octopus arm is to maximize the reward function by reaching the goal position as quickly as possible. The total reward to be obtained within an episode is therefore equal to the difference between the initial distance of the last compartment to the goal and the number of steps in which the goal is reached.

6.1.2. Actor and critic structure

The actor and critic are based on feedforward neural networks, namely 2-layer perceptrons with sigmoidal neurons in their hidden layers and linear neurons in their output layers. The input of both the networks is the scaled state of the octopus arm. All neurons in the networks have a constant input (bias). The initial weights in the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and weights of the output layers are initially set to zero.

The actor is the combination of a neural network and a generator of discrete numbers. The network has N_A neurons in its hidden layer and 6 neurons in the output layer corresponding to the size of the discrete action set. Given state s and the observed neural network outputs

$$\mu_i(s, \theta), \quad i = 1 \dots 6,$$

the likelihood function for choosing a discrete action, Action i , is given by:

$$P(\text{Action } i | s) = \frac{\exp(\mu_i(s, \theta)/10)}{\sum_{j=1}^6 \exp(\mu_j(s, \theta)/10)}. \quad (20)$$

The exploration takes place as each action has a non-zero probability. This is ensured by the penalty function of the form

$$l(s, \theta) = \sum_{i=1}^6 \max\{0, |\mu_i(s, \theta)| - 20\}^2.$$

As a result, the policy is not penalized for the network's outputs in the range $[-20, 20]$, which means that the best action is not more than $e^4 \approx 54$ times more probable than the worst one.

The critic is a 2-layer perceptron with N_C neurons in its hidden layer and one neuron in the output layer.

6.1.3. Experiments and results

The experiments of the learning control problem of an octopus arm with the proposed algorithm are now presented. The parameter setting of the actor-critic reinforcement learning algorithm is as follows: $N_A = 50$, $N_C = 100$, $\gamma = 0.98$, $b = 2$, $\alpha = \gamma = 0.98$, and $\rho = \lambda = 0.8$. The experience database contained $M = 10^4$ events. The v function took the form (18) with $\min_capacity = 1000$ and $\text{comp_steps} = 100$. The fixed point algorithm parameters were: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $\alpha_0 = 0.2$ for the actor and $\alpha_0 = 0.3$ for the critic.

Fig. 4 shows different stages of the octopus arm in reaching the desired goal. Figs. 5–9 present the result of experiments performed with the above parameter setting. The actor-critic with experience replay and various constant step-sizes is compared against those estimated by the fixed point method. Fig. 5 presents learning curves (average rewards vs. episode number), Fig. 6 presents efficiency (ratio of runs in which the goal was reached), Figs. 7 and 8 present average duration of episodes, and Fig. 9 presents the step-sizes estimated by the fixed point method.

What is not demonstrated in the figures is that the learning process is unstable for constant step-sizes greater or equal to 0.1 for both the actor and the critic. For $\beta^\theta = \beta^v = 10^{-2}$, the algorithm does not converge and produces poor performance. Smaller step-sizes lead to better performance but for $\beta^\theta \leq 10^{-4}$ or $\beta^v \leq 10^{-4}$ the learning process becomes very slow. The fixed point method of adjusting step-sizes enables at least as good performance of the learning algorithm, at any stage of its operation, as given by any constant step-size. The training is completed (no further improvement is observed) after 240 episodes which is equivalent to 80 min of Octopus time.

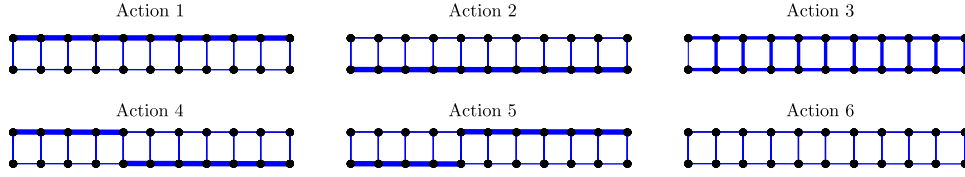


Fig. 3. Action set. The activated muscles are indicated by thick blue lines. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

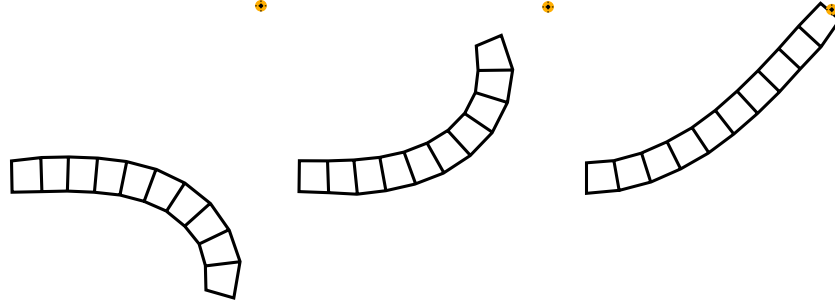


Fig. 4. Simulated goal reaching movement of octopus arm. *Left:* Arm position in the start of episode. *Middle:* Arm position trying to reach the goal. *Right:* Arm position when reached the goal. After learning, the arm reaches the goal in about 100 steps with maximum efficiency.

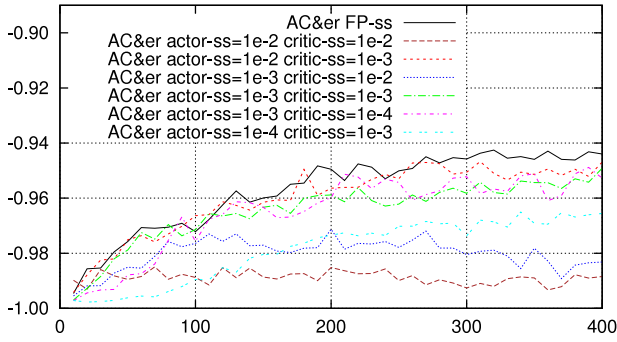


Fig. 5. Octopus arm: Average rewards vs. episode no. for actor-critic with experience replay and constant step-sizes against those estimated by the fixed point method. Each curve averages 10 runs.

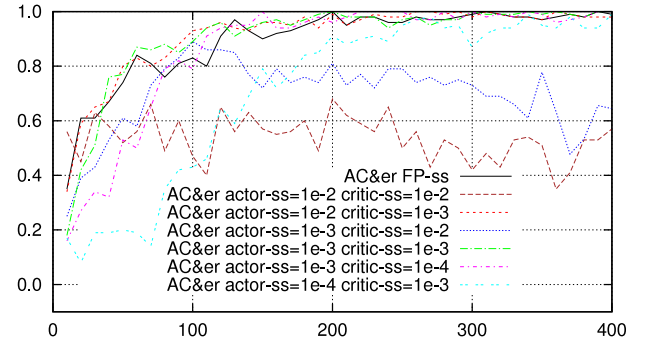


Fig. 6. Octopus arm: Efficiency vs. episode no. for actor-critic with experience replay, step-sizes estimated by the fixed point method, and constant step-sizes. Each curve averages 10 runs.

Fig. 9 shows the evolution of step-sizes driven by the fixed point method. They are averaged over runs and episodes according to the formula

$$\bar{\beta} = \exp \left(\frac{1}{N} \sum_i \ln \beta_i \right), \quad (21)$$

where N is the number of registered step-sizes. This way of averaging is better here than the normal one because it does not underweight very small step-sizes. It is seen that the actor step-size starts from a large initial value and then gradually decreases. The critic step-size initially decreases to a rather small value and then, after about 100 episodes, almost remains the same. This suggests that slowing down of the actor learning does not necessarily corresponds to slowing down of the critic learning.

6.2. Half-cheetah

Running in biological systems has largely inspired the development of legged robotics. Starting from the seminal spring-loaded hopping robots to the state-of-the-art quadrupeds and bipeds with compliant legs, a number of prototypes have been proposed to mimic versatility, speed and robustness in natural running. The dramatic progress is hindered by the lack of ability in robots to acquire these skills by online learning. In this section, we apply our proposed algorithm on a planar model of a large cat, called half-cheetah, to learn the complex skill of running.

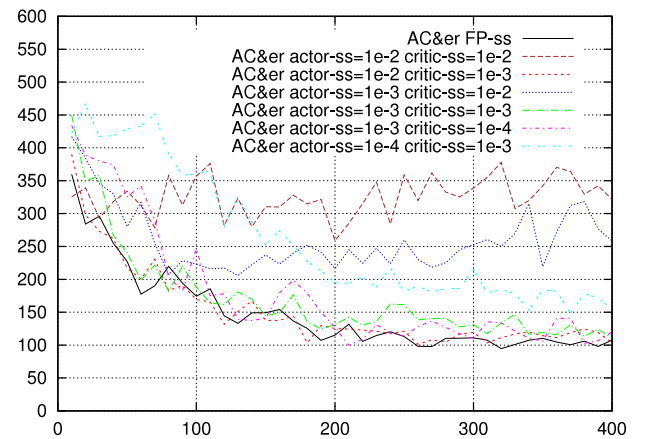


Fig. 7. Octopus arm: Duration in steps vs. episode no. for actor-critic with experience replay, step-sizes estimated by the fixed point method, and constant step-sizes. Each curve averages 10 runs.

6.2.1. Control problem

Half-Cheetah is a 6 degrees of freedom planar robot introduced in Wawrzyński (2009). It is composed of nine links, eight joints and two paws (see Fig. 10). The angles of the fourth and the fifth joint are fixed while others are controllable. The torque applied at each

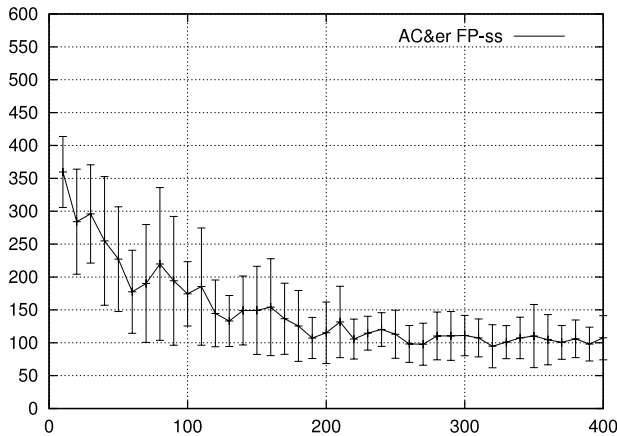


Fig. 8. Octopus arm: Duration in steps vs. episode no. for actor-critic with experience replay and step-sizes estimated by the fixed point method. The one-sigma limits are calculated to show run-to-run variability of trial averages.

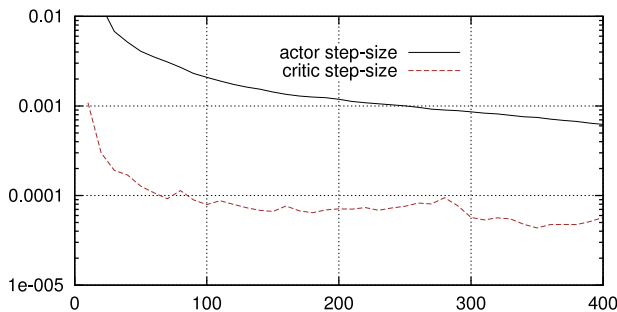


Fig. 9. Octopus arm: The fixed point method of step-size estimation vs. episode no. for actor-critic with experience replay. Each curve averages 10 runs.

joint acts as input to the model and the next position of the robot is obtained as output by integrating its dynamic equations of motion.

The control problem is to learn a reactive policy under the MDP framework to make half-cheetah run as fast as possible. State of half-cheetah is defined by 31 variables. The action space is continuous, contrary to that of octopus arm, with 6 dimensions each corresponding to one actuated joint independently. Learning is divided into episodes with an average duration of 250 steps, for 0.02 s duration of each step. The robot is mainly rewarded for its speed of moving forward. Other components are minor penalties for violating torque limits, joint limits, not moving the trunk in idle position and touching the ground with other body parts than paws. In this study, we use the simulator of half-cheetah, as reported in Wawrzyński (2009), with same experimental settings including its dynamics, state and action definition, and reward function.

6.2.2. Actor and critic structure

The actor, π , is composed of two parts: a neural network and a normal distribution. The input of the network is the state of half-cheetah. The network has a hidden layer with N_A sigmoidal neurons and six linear output neurons corresponding to the dimensionality of the action space. The output, $\mu(s; \theta)$, becomes a mean value of the normal distribution with covariance matrix equal to $I\sigma^2$. The distribution generates actions. Because each component of the action is projected on the interval $[-30, 30]$, the penalty function keeps each network output in this interval for a given state. It is of the form

$$l(s, \theta) = \sum_{i=1}^6 10^{-3} \max\{0, |\mu_i(s, \theta)| - 30\}^2.$$

The critic is a neural network of the same structure as the actor network with N_C neurons in its hidden layer. The initial

weights in the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and weights of the output layers are initially set to zero.

6.2.3. Experiments and results

The experiments to make half-cheetah run are configured with following parameters: $N_A = 160$, $N_C = 160$, $\sigma = 5$, $\gamma = 0.99$, $b = 2$, $\alpha = \gamma = 0.99$, $\rho = \lambda = 0.9$. The experience database contained $M = 10^5$ events. The v function took the form (18) with $\min_capacity = 100$ and $\text{comp_steps} = 30$. The fixed point algorithm parameters were kept same as for octopus arm with: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $\alpha_0 = 0.2$ for the actor and $\alpha_0 = 0.3$ for the critic.

Fig. 10 shows various stages of the learned running gait of half-cheetah. The cat robot starts from a standing position and first learns to move forward with the added noise in the control system. The awkward walk transforms into a trot gait which at the end of training becomes a smooth nimble run. The use of experience replay speeds up the learning process of running in proportion to the intensity of replaying computations.

In this study, we are interested in analyzing the effect of the proposed fixed-point method of step-size estimation on the learned policy. To this end, Figs. 11 and 12 compare the learning curve of the adaptive step-size estimation algorithm with the ones obtained by setting various constant step-sizes. Among the learning curves with constant step-sizes, the learning algorithm produces optimum results for $\beta^{\theta} = \beta^v = 10^{-5}$. For larger values, the policy evolves with a reasonable speed in the initial period but the ultimate performance is worse. Smaller step-sizes slow down the learning speed significantly. The fixed point method of estimating the step-sizes makes the learning algorithm perform better at every stage of the learning process than with any manually selected step-sizes. The algorithm required 3500 episodes (about 5 h of half-cheetah time) to learn to receive average reward of 6, which corresponds to a really nimble run.

Fig. 13 presents evolution of step-sizes driven by the fixed point method. They are averaged over runs and episodes according to formula (21). It is seen that the actor and critic step-sizes start descending from the same value and then gradually converge to about $5 \cdot 10^{-6}$ and 10^{-6} respectively.

6.3. Discussion

The purpose of this experimental study is to evaluate the feasibility of autonomously determining the step-sizes for actor-critic class of reinforcement learning algorithms with experience replay. The results are promising: the augmented algorithm has been used to find control policies for two challenging problems similar to those encountered in robotics. The algorithm gave at least as good performance at any stage of the learning process as Actor-Critic with experience replay and constant step-sizes.

In all the experiments, the step-sizes of the fixed point method were initialized blindly, with a value of 1.0 (Algorithm 3, Line 8). As experiments with constant step-sizes showed, the value of 1.0 was very large and surely caused instability of the learning, which was identified by the algorithm (Line 19), and the step-sizes were halved (Line 20) until they ensured normal learning. Therefore, despite their large initial value, improper for the problems at hand, the step-sizes are always smaller than 1.0 in Figs. 9 and 13. Further, the step-sizes were autonomously adjusted with time (Algorithm 3, Line 25), which usually (but not always) meant their slow decrease.

The fixed point method of estimating step-sizes gave at least as good performance as the manually defined constant step-sizes used in the experiments. We argue that constant step-sizes generally do not ensure good performance as even their

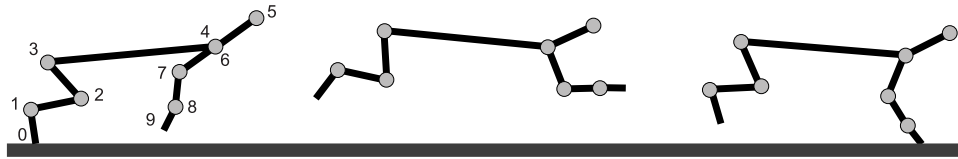


Fig. 10. Simulated run of Half-Cheetah. *Left:* Initial stance. *Middle:* Middle stance with feet in the air. *Right:* Landing stance. After learning, half-cheetah runs with a speed of about 6.5 m/s.

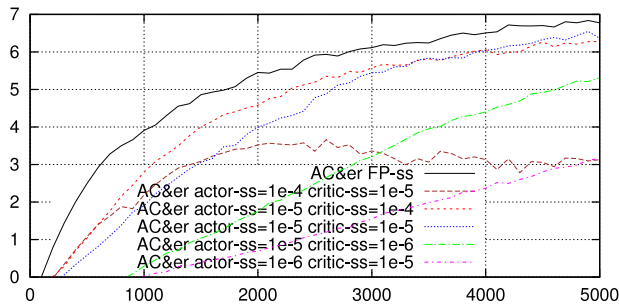


Fig. 11. Half-Cheetah: Average rewards vs. episode no. for actor-critic with experience replay and constant step-sizes against those estimated by the fixed point method. Each curve averages 10 runs.

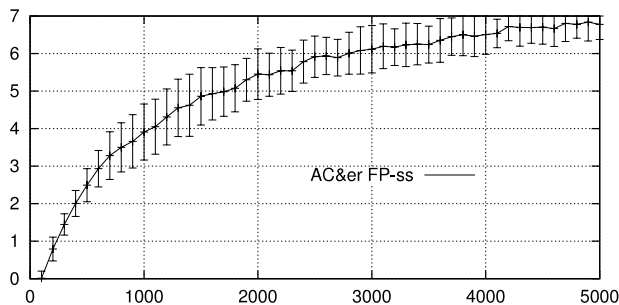


Fig. 12. Half-Cheetah: Average rewards vs. episode no. for actor-critic with experience replay and step-sizes estimated by the fixed point method. The one-sigma limits are calculated to show run-to-run variability of trial averages.

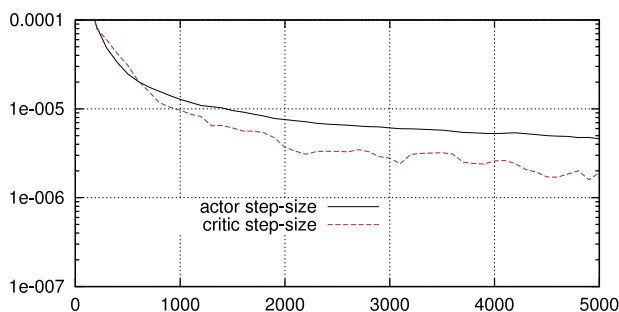


Fig. 13. Half-Cheetah: The fixed point method of step-size estimation vs. episode no. for actor-critic with experience replay. Each curve averages 10 runs.

handcrafted values yield slow learning in the early stages and fluctuations in the final stages. The fixed point method estimates large step-sizes initially and gradually decreases them during the course of learning. However, it has been observed in our supplementary experiments that the critic step-sizes that were twice larger than estimated by the fixed-point algorithm led to even better performance. Step-size estimation for the critic is an especially difficult problem as the critic learns mainly on the basis of values that it produces alone, thereby, going beyond the model of stochastic optimization for which the fixed-point algorithm along with most other methods of step-size estimation is designed. Still, the results presented here suggest the correctness of the principles that govern the fixed-point method.

Step-sizes are not the only input parameters of the learning algorithm. The rest of the parameters, however, either have the same values for both tasks or can be assigned by the experimenters based on their experience. The first category encompasses the parameters of the fixed-point method and parameter b of the randomized-truncated estimators (Eqs. (8) and (9)). The second category includes the parameters whose values are in principle problem-dependent, but the learning is not very sensitive to them and not much experience of an experimenter is required to set their proper values. Those are: (1) the sizes of the neural networks, N_A and N_C —the more difficult the control problem gets, the larger is the size of required networks, (2) the discount factor, γ , and the λ parameter—the more farsighted the control policy is required to be, the larger are the values of γ and λ , (3) the size, M , of the database with experience—it should be sufficiently large for this experience to be representative, (4) the intensity of experience replay, $comp_steps$, results from the available computer power.

In comparison to the parameters discussed above, the step-sizes have different nature: (1) the learning is quite sensitive to them, (2) their proper values differ orders of magnitude from one task to another, (3) for the same task they may differ substantially between stages of the learning. Their on-line estimation is therefore crucial for autonomy of learning.

7. Conclusions and future work

In this paper, an actor-critic reinforcement learning algorithm was introduced that combines experience replay (Wawrzyński, 2009) with the fixed point method of step-size estimation, improved upon its version presented in Wawrzyński (2010). Potentially problematic issues associated with this combination were addressed, namely overtraining, wrong fixed point initialization, and mutual adjustment of the actor and critic. The experimental study on octopus arm and half-cheetah revealed that the algorithm learned control policies in multidimensional continuous space within a short time. It is also autonomous in the sense that it achieved its goal essentially within a single run, without auxiliary experiments aiming at manual tuning of the problematic step-size parameters.

As the proposed approach has been designed for robotic applications, we plan its evaluation on a real humanoid walking robot. While the fixed point method of step-size estimation is justified by a simplified model and empirical performance, there is room for its further development and analysis of its convergence. This method has been applied here for optimization of the actor and critic step-sizes separately, while the learning of actor and of critic are coupled processes. The question whether the principles underlying the fixed point method can be formally extended to such coupling is another interesting subject of further research.

References

- Abbeel, P., & Ng, A. Y. (2005). Exploration and apprenticeship learning in reinforcement learning. In *Proc. of the 22nd ICML* (pp. 1–8). ACM.
- Adam, S., Busoniu, L., & Babuska, R. (2012). Experience replay for real-time reinforcement learning control. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 42(2), 201–212.
- Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can learn difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13, 834–846.

Prioritized Level Replay

Minqi Jiang^{1 2} Edward Grefenstette^{1 2} Tim Rocktäschel^{1 2}

Abstract

Environments with procedurally generated content serve as important benchmarks for testing systematic generalization in deep reinforcement learning. In this setting, each *level* is an algorithmically created environment instance with a unique configuration of its factors of variation. Training on a prespecified subset of levels allows for testing generalization to unseen levels. What can be learned from a level depends on the current policy, yet prior work defaults to uniform sampling of training levels independently of the policy. We introduce *Prioritized Level Replay* (PLR), a general framework for selectively sampling the next training level by prioritizing those with higher estimated learning potential when revisited in the future. We show TD-errors effectively estimate a level’s future learning potential and, when used to guide the sampling procedure, induce an emergent curriculum of increasingly difficult levels. By adapting the sampling of training levels, PLR significantly improves sample efficiency and generalization on Procgen Benchmark—matching the previous state-of-the-art in test return—and readily combines with other methods. Combined with the previous leading method, PLR raises the state-of-the-art to over 76% improvement in test return relative to standard RL baselines.

1. Introduction

Deep reinforcement learning (RL) easily overfits to training experiences, making generalization a key open challenge to widespread deployment of these methods. Procedural content generation (PCG) environments have emerged as a promising class of problems with which to probe and address this core weakness (Risi & Togelius, 2020; Chevalier-Boisvert et al., 2018; Cobbe et al., 2020a; Juliani et al., 2019;

Zhong et al., 2020; Küttler et al., 2020). Unlike singleton environments, like the Arcade Learning Environment games (Bellemare et al., 2013), PCG environments take on algorithmically created configurations at the start of each training episode, potentially varying the layout, asset appearances, or even game dynamics. Each environment instance generated this way is called a *level*. In mapping an identifier, such as a random seed, to each level, PCG environments allow us to measure a policy’s generalization to held-out test levels. In this paper, we assume only a blackbox generation process that returns a level given an identifier. We avoid the strong assumption of control over the generation procedure itself, explored by several prior works (see Section 6). Further, we assume levels follow common latent dynamics, so that in aggregate, experiences collected in individual levels reveal general rules governing the entire set of levels.

Despite its humble origin in games, the PCG abstraction proves far-reaching: Many control problems, such as teaching a robotic arm to stack blocks in a specific formation, easily conform to PCG. Here, each level may consist of a unique combination of initial block positions, arm state, and target formation. In a vastly different domain, Hanabi (Bard et al., 2020) also conforms to PCG, where levels map to initial deck orderings. These examples illustrate the generality of the PCG abstraction: Most challenging RL problems entail generalizing across instances (or levels) differing along some underlying factors of variation and thereby can be aptly framed as PCG. This highlights the importance of effective deep RL methods for PCG environments.

Many techniques have been proposed to improve generalization in the PCG setting (see Section 6), requiring changes to the model architecture, learning algorithm, observation space, or environment structure. Notably, these approaches default to uniform sampling of training levels. We instead hypothesize that the variation across levels implies that at each point of training, each level likely holds different potential for an agent to learn about the structure shared across levels to improve generalization. Inspired by this insight and selective sampling methods from active learning, we investigate whether sampling the next training level weighed by its learning potential can improve generalization.

We introduce Prioritized Level Replay (PLR), illustrated in Figure 1, a method for sampling training levels that exploits

¹Facebook AI Research, London, United Kingdom ²University College London, London, United Kingdom. Correspondence to: Minqi Jiang <msj@fb.com>.

Prioritized Level Replay

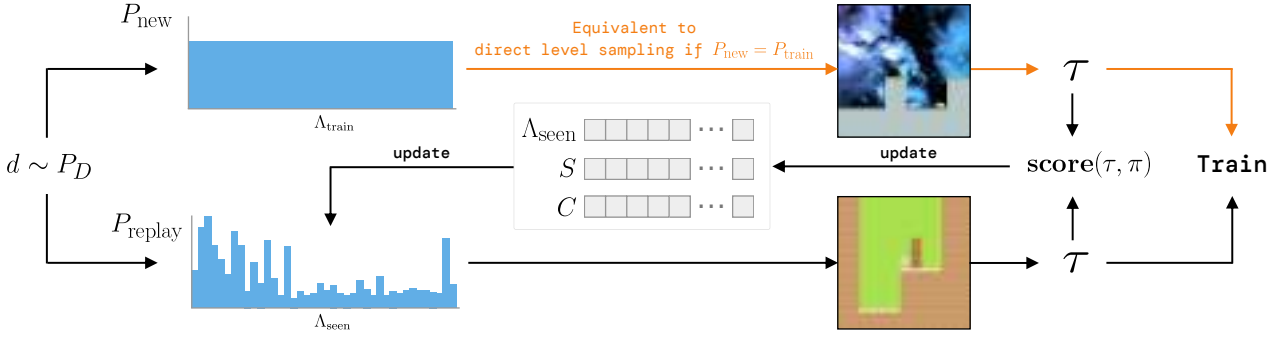


Figure 1. Overview of Prioritized Level Replay. The next level is either sampled from a distribution with support over unseen levels (top), which could be the environment’s (perhaps implicit) full training-level distribution, or alternatively, sampled from the replay distribution, which prioritizes levels based on future learning potential (bottom). In either case, a trajectory τ is sampled from the next level and used to update the replay distribution. This update depends on the lists of previously seen levels Λ_{seen} , their latest estimated learning potentials S , and last sampled timestamps C .

the differences in learning potential among levels to improve both sample efficiency and generalization. PLR selectively samples the next training level based on an estimated learning potential of replaying each level anew. During training, our method updates scores estimating each level’s learning potential as a function of the agent’s policy and temporal-difference (TD) errors collected along the last trajectory sampled on that level. Our method then samples the next training level from a distribution derived from a normalization procedure over these level scores. PLR makes no assumptions about how the policy is updated, so it can be used in tandem with any RL algorithm and combined with many other methods such as data augmentation. Our method also does not assume any external, predefined ordering of levels by difficulty or other criteria, but instead derives level scores dynamically during training based on how the policy interacts with the environment. The only requirements are as follows—satisfied by almost any problem that can be framed as PCG, including RL environments implemented as seeded simulators: (i) Some notion of “level” exists, such that levels follow common latent dynamics; (ii) such levels can be sampled from the environment in an identifiable way; and (iii) given a level identifier, the environment can be set to that level to collect new experiences from it.

While previous works in off-policy RL devised effective methods to directly reuse *past* experiences for training (Schaul et al., 2016; Andrychowicz et al., 2017), PLR uses past experiences to inform the collection of *future* experiences by estimating how much replaying each level anew will benefit learning. Our method can thus be seen as a forward-view variation of prioritized experience replay, and an online counterpart to this off-policy method.

This paper makes the following contributions¹: (i) We in-

¹Our code is available at <https://github.com/facebookresearch/level-replay>.

roduce a computationally-efficient algorithm for sampling levels during training based on an estimate of the future learning potential of collecting new experiences from each level; (ii) we show our method significantly improves generalization on 10 of 16 environments in Procgen Benchmark and two challenging MiniGrid environments; (iii) we demonstrate our method combines with the previous leading method to set a new state-of-the-art on Procgen Benchmark; and (iv) we show our method induces an implicit curriculum over training levels in sparse reward settings.

2. Background

In this paper, we refer to a *PCG environment* as any computational process that, given a level identifier (e.g. an index or a random seed), generates a *level*, defined as an environment instance exhibiting a unique configuration of its underlying factors of variation, such as layout, asset appearances, or specific environment dynamics (Risi & Togelius, 2020). For example, MiniGrid’s MultiRoom environment instantiates mazes with varying numbers of rooms based on the seed (Chevalier-Boisvert et al., 2018). We refer to sampling a new trajectory generated from the agent’s latest policy acting on a given level l as *replaying* that level l .

The level diversity of PCG environments makes them useful testbeds for studying the robustness and generalization ability of RL agents, measured by agent performance on unseen test levels. The standard test evaluation protocol for PCG environments consists of training the agent on a finite number of training levels Λ_{train} , and evaluating performance on unseen test levels Λ_{test} , drawn from the set of all levels. Training levels are sampled from an arbitrary distribution $P_{\text{train}}(l|\Lambda_{\text{train}})$. We call this training process *direct level sampling*. A common variation of this protocol sets Λ_{train} to the set of all levels, though in practice, the agent will still only effectively see a finite set of levels after training for a finite

number of steps. In the case of a finite training set, typically $P_{\text{train}}(l|\Lambda_{\text{train}}) = \text{Uniform}(l; \Lambda_{\text{train}})$. See Appendix C for the pseudocode outlining this procedure.

PCG environments naturally lend themselves to curriculum learning. Prior works have shown that directly altering levels to match their difficulty to the agent’s abilities can improve generalization (Justesen et al., 2018; Dennis et al., 2020; Chevalier-Boisvert et al., 2018; Zhong et al., 2020). These findings further suggest the levels most useful for improving an agent’s policy vary throughout the course of training. In this work, we consider how to automatically discover a curriculum that improves generalization for a general black-box PCG environment—crucially, without assuming any knowledge or control of how levels are generated (beyond providing the random seed or other indicial level identifier).

3. Prioritized Level Replay

In this section, we present *Prioritized Level Replay* (PLR), an algorithm for selectively sampling the next training level given the current policy, by prioritizing levels with higher estimated learning potential when replayed (that is, revisited). PLR is a drop-in replacement for the experience-collection process used in a wide range of RL algorithms. Algorithm 1 shows how it is straightforward to incorporate PLR into a generic policy-gradient training loop. For clarity, we focus on training on batches of complete trajectories (see Appendix C for pseudocode of PLR with T -step rollouts).

Our method, illustrated in Figure 1 and fully specified in Algorithm 2, induces a dynamic, nonparametric sampling distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ over previously visited training levels Λ_{seen} that prioritizes visited levels with higher learning potential based on properties of the agent’s past trajectories. We refer to $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ as the *replay distribution*. Throughout training, our method updates this replay distribution according to a heuristic score, assigning greater weight to visited levels with higher *future* learning potential. Using dynamic arrays S and C of equal length to Λ_{seen} , PLR tracks level scores $S_i \in S$ for each visited training level l_i based on the latest episode trajectory on l_i , as well as the episode count $C_i \in C$ at which each level $l_i \in \Lambda_{\text{seen}}$ was last sampled. Our method updates P_{replay} after each terminated episode by computing a mixture of two distributions, P_S , based on the level scores, and P_C , based on how long ago each level was last sampled:

$$P_{\text{replay}} = (1 \cup \rho) \cdot P_S + \rho \cdot P_C, \quad (1)$$

where the staleness coefficient $\rho \in [0, 1]$ is a hyperparameter. We discuss how we compute level scores S_i , parameterizing the scoring distribution P_S , and the staleness distribution P_C , in Sections 3.1 and 3.2, respectively.

PLR chooses the next level at the start of every training episode by first sampling a replay-decision from a

Algorithm 1 Policy-gradient training loop with PLR

Input: Training levels Λ_{train} , policy π_θ , policy update function $\mathcal{U}(\mathcal{B}, \theta) \rightarrow \theta'$, and batch size N_b .
 Initialize level scores S and level timestamps C
 Initialize global episode counter $c \leftarrow 0$
 Initialize the ordered set of visited levels $\Lambda_{\text{seen}} = \emptyset$
 Initialize experience buffer $\mathcal{B} = \emptyset$
while training **do**
 $\mathcal{B} \leftarrow \emptyset$
 while collecting experiences **do**
 $\mathcal{B} \leftarrow \mathcal{B} \cup \text{collect_experiences}(\Lambda_{\text{train}}, \Lambda_{\text{seen}}, \pi_\theta, S, C, c)$
 Using Algorithm 2
 end while
 $\theta \leftarrow \mathcal{U}(\mathcal{B}, \theta)$ *Update policy using collected experiences*
end while

Algorithm 2 Experience collection with PLR

Input: Training levels Λ_{train} , visited levels Λ_{seen} , policy π , global level scores S , global level timestamps C , and global episode counter c .
Output: A sampled trajectory τ
 $c \leftarrow c + 1$
 Sample replay decision $d \sim P_D(d)$
if $d = 0$ **and** $|\Lambda_{\text{train}} \setminus \Lambda_{\text{seen}}| > 0$ **then**
 Define new index $i \leftarrow |S| + 1$
 Sample $l_i \sim P_{\text{new}}(l|\Lambda_{\text{train}}, \Lambda_{\text{seen}})$ *Sample an unseen level*
 Add l_i to Λ_{seen}
 Add initial value $S_i = 0$ to S and $C_i = 0$ to C
else
 Sample $l_i \sim (1 - \rho) \cdot P_S(l|\Lambda_{\text{seen}}, S) + \rho \cdot P_C(l|\Lambda_{\text{seen}}, C, c)$
 Sample a level for replay
end if
 Sample $\tau \sim P_\pi(\tau|l_i)$
 Update score $S_i \leftarrow \text{score}(\tau, \pi)$ and timestamp $C_i \leftarrow c$

Bernoulli (or similar) distribution $P_D(d)$ to determine whether to replay a level sampled from the replay distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$ or to experience a new, unseen level from Λ_{train} , according to some distribution $P_{\text{new}}(l|\Lambda_{\text{train}}, \Lambda_{\text{seen}})$. In practice, for the case of a finite number of training levels, we implement P_{new} as a uniform distribution over the remaining unseen levels. For the case of a countably infinite number of training levels, we simulate P_{new} by sampling levels from P_{train} until encountering an unseen level. In our experiments based on a finite number of training levels, we opt to naturally anneal $P_D(d = 1)$ as $|\Lambda_{\text{seen}}|/|\Lambda_{\text{train}}|$, so replay occurs more often as more training levels are visited.

The following sections describes how Prioritized Level Replay updates the replay distribution $P_{\text{replay}}(l|\Lambda_{\text{seen}})$, namely through level scoring and staleness-aware prioritization.

3.1. Scoring Levels for Learning Potential

After collecting each complete episode trajectory τ on level l_i using policy π , our method assigns l_i a score $S_i = \text{score}(\tau, \pi)$ measuring the learning potential of replaying l_i in the future. We employ a function of the TD-error at timestep t , $\delta_t = r_t + \gamma V(s_{t+1}) \cup V(s_t)$, as a proxy

for this learning potential. The expectation of the TD-error over next states is equivalent to the advantage estimate, and therefore higher-magnitude TD-errors imply greater discrepancy between expected and actual returns, making δ_t a useful measure of the learning potential in revisiting a particular state transition. To prioritize the learning potential of future experiences resulting from replaying a level, we use a scoring function based on the *average magnitude* of the Generalized Advantage Estimate (GAE; Schulman et al., 2016) over each of T time steps in the latest trajectory τ from that level:

$$S_i = \text{score}(\tau, \pi) = \frac{1}{T} \sum_{t=0}^T \left| \sum_{k=t}^T (\gamma\lambda)^{k-t} \delta_k \right|. \quad (2)$$

While the GAE at time t is most commonly expressed as the discounted sum of all 1-step TD-errors starting at t as in Equation 2, it is equivalent to an exponentially-discounted sum of all k -step TD-errors from t , with discount factor λ . By considering all k -step TD-errors, the GAE mitigates the bias introduced by the bootstrap term in 1-step TD-errors. The discount factor λ then controls the trade-off between bias and variance. Our scoring function considers the absolute value of the GAE, as we assume the learning potential grows with the magnitude of the TD-error irrespective of its sign. This also avoids opposite signed errors canceling out.

Another useful interpretation of Equation 2 comes from observing that the GAE magnitude at t is equivalent to the L1 value loss $|\hat{V}_t - V_t|$ under a policy-gradient algorithm that uses GAE for its own advantage estimates (and therefore value targets $\hat{V}_t$), as done in state-of-the-art implementations of PPO (Schulman et al., 2017) used in our experiments. Unless otherwise indicated, PLR refers to the instantiation of our algorithm with L1 value loss as the scoring function.

We further formulate the *Value Correction Hypothesis* to motivate our approach: In sparse reward settings, prioritizing the sampling of training levels with greatest average absolute value loss leads to a curriculum that improves both sample efficiency and generalization. We reason that on threshold levels (i.e. those at the limit of the agent’s current abilities) the agent will see non-stationary returns (or value targets)—and therefore incur relatively high value errors—until it learns to solve them consistently. In contrast, levels beyond the agent’s current abilities tend to result in stationary value targets signaling failure and therefore low value errors, until the agent learns useful, transferable behaviors from threshold levels. Prioritizing levels by value loss then naturally guides the agent along the expanding threshold of its ability—without the need for any externally provided measure of difficulty. We believe that learning behaviors systematically aligned with the inherent complexities of the environment in this way may lead to better generalization, and will seek to verify this empirically in Section 5.2.

While we provide principled motivations for our specific choice of scoring function, we emphasize that in general, the scoring function can be any approximation of learning potential based on trajectory values. Note that candidate scoring functions should asymptotically decrease with frequency of level visitation to avoid mode collapse of P_{replay} to a limited set of levels and possible overfitting. In Section 4, we compare our choice of the GAE magnitude, or equivalently, the L1 value loss, to alternative TD-error-based and uncertainty-based scoring approaches, listed in Table 1.

Given level scores, we use normalized outputs of a prioritization function h evaluated over these scores and tuned using a temperature parameter β to define the score-prioritized distribution $P_S(\Lambda_{\text{train}})$ over the training levels, under which

$$P_S(l_i | \Lambda_{\text{seen}}, S) = \frac{h(S_i)^{1/\beta}}{\sum_j h(S_j)^{1/\beta}}. \quad (3)$$

The function h defines how differences in level scores translate into differences in prioritization. The temperature parameter β allows us to tune how much $h(S)$ ultimately determines the resulting distribution. We make the design choice of using rank prioritization, for which $h(S_i) = 1/\text{rank}(S_i)$, where $\text{rank}(S_i)$ is the rank of level score S_i among all scores sorted in descending order. We also experimented with proportional prioritization ($h(S_i) = S_i$) as well as greedy prioritization (the level with the highest score receives probability 1), both of which tend to perform worse.

3.2. Staleness-Aware Prioritization

As the scores used to parameterize P_S are a function of the state of the policy at the time the associated level was last played, they come to reflect a gradually more off-policy measure the longer they remain without an update through replay. We mitigate this drift towards “off-policy-ness” by explicitly mixing the sampling distribution with a staleness-prioritized distribution P_C :

$$P_C(l_i | \Lambda_{\text{seen}}, C, c) = \frac{c \cup C_i}{\sum_{C_j \in C} c \cup C_j} \quad (4)$$

which assigns probability mass to each level l_i in proportion to the level’s *staleness* $c \cup C_i$. Here, c is the count of total episodes sampled so far in training and C_i (referred to as the level’s timestamp) is the episode count at which l_i was last sampled. By pushing support to levels with staler scores, P_C ensures no score drifts too far off-policy.

Plugging Equations 3 and 4 into Equation 1 gives us a replay distribution that is calculated as

$$P_{\text{replay}}(l_i) = (1 \cup \rho) \cdot P_S(l_i | \Lambda_{\text{seen}}, S) + \rho \cdot P_C(l_i | \Lambda_{\text{seen}}, C, c).$$

Thus, a level has a greater chance of being sampled when its score is high or it has not been sampled for a long time.

4. Experimental Setting

We evaluate PLR on several PCG environments with various combinations of scoring functions and prioritization schemes, and compare to the most common direct level sampling baseline of $P_{\text{train}}(l|\Lambda_{\text{train}}) = \text{Uniform}(l; \Lambda_{\text{train}})$. We train and test on all 16 environments in the Procgen Benchmark on easy and hard difficulties, but focus discussion on the easy results, which allow direct comparison to several prior studies. We compare to UCB-DrAC (Raileanu et al., 2021), the state-of-the-art image augmentation method on this benchmark, and mixreg (Wang et al., 2020a), a recently introduced data augmentation method. We also compare to TSCL Window (Matiisen et al., 2020), which resembles PLR with an alternative scoring function using the slope of recent returns and no staleness sampling. For fair comparison, we also evaluate a custom TSCL Window variant that mixes in the staleness distribution P_C weighted by $\rho > 0$. Further, to demonstrate the ease of combining PLR with other methods, we evaluate UCB-DrAC using PLR for sampling training levels. Finally, we test the Value Correction Hypothesis on two challenging MiniGrid environments.

We measure episodic *test returns* per game throughout training, as well as the performance of the final policy over 100 unseen test levels of each game relative to PPO with uniform sampling. We also evaluate the mean normalized episodic test return and mean generalization gap, averaged over all games (10 runs each). We normalize returns according to Cobbe et al. (2019) and compute the generalization gap as train returns minus test returns. Thus, a larger gap indicates more overfitting, making it an apt measure of generalization. We assess statistical significance at $p = 0.05$, using the Welch t-test.

In line with the standard baseline for these environments, all experiments use PPO with GAE for training. For Procgen, we use the same ResBlock architecture as Cobbe et al. (2020a) and train for 25M total steps on 200 levels on the easy setting as in the original baselines. For MiniGrid, we use a 3-layer CNN architecture based on Igl et al. (2019), and provide approximately 1000 levels of each difficulty per environment during training. Detailed descriptions of the environments, architectures, and hyperparameters used in our experiments (and how they were set or obtained) can be found in Appendix A. See Table 1 for the full set of scoring functions investigated in our experiments.

Additionally, we extend PLR to support training on an unbounded number of levels by tracking a rolling, finite buffer of the top levels so far encountered by learning potential. Appendix B.3 reports the results of this extended PLR algorithm when training on the full level distribution of the MiniGrid environments featured in our main experiments.

Table 1. Scoring functions investigated in this work.

Scoring function	$\text{score}(\tau, \pi)$
Policy entropy	$\frac{1}{T} \sum_{t=0}^T \sum_a \pi(a, s_t) \log \pi(a, s_t)$
Policy min-margin	$\frac{1}{T} \sum_{t=0}^T (\max_a \pi(a, s_t) \cup \max_{a \neq \max_a} \pi(a, s_t))$
Policy least-confidence	$\frac{1}{T} \sum_{t=0}^T (1 \cup \max_a \pi(a, s_t))$
1-step TD error	$\frac{1}{T} \sum_{t=0}^T \delta_t $
GAE	$\frac{1}{T} \sum_{t=0}^T \sum_{k=t}^T (\gamma \lambda)^{k-t} \delta_k$
GAE magnitude (L1 value loss)	$\frac{1}{T} \sum_{t=0}^T \left \sum_{k=t}^T (\gamma \lambda)^{k-t} \delta_k \right $

5. Results and Discussion

Our main findings are that (i) PLR significantly improves both sample efficiency and generalization, attaining the highest normalized mean test and train returns and mean reduction in generalization gap on Procgen out of all individual methods evaluated, while matching UCB-DrAC in test improvement relative to PPO; (ii) alternative scoring functions lead to inconsistent improvements across environments; (iii) PLR combined with UCB-DrAC sets a new state-of-the-art on Procgen; and (iv) PLR induces an implicit curriculum over training levels, which substantially aids training in two challenging MiniGrid environments.

5.1. Procgen Benchmark

Our results, summarized in Figure 2, show PLR with rank prioritization ($\beta = 0.1$, $\rho = 0.1$) leads to the largest statistically significant gains in mean normalized test and train returns and reduction in generalization gap compared to uniform sampling, outperforming all other methods besides UCB-DrAC + PLR. PLR combined with UCB-DrAC sees the most drastic improvements in these metrics. As reported in Table 2, UCB-DrAC + PLR yields a 76% improvement in mean test return relative to PPO with uniform sampling, and a 28% improvement relative to the previous state-of-the-art set by UCB-DrAC. While PLR with rank prioritization leads to statistically significant gains in test return on 10 of 16 environments and proportional prioritization, on 11 of 16 games, we prefer rank prioritization: While we find the two comparable in mean normalized returns, Figure 3 shows rank prioritization results in higher mean *unnormalized* test returns and a significantly lower mean generalization gap, averaged over all environments.

Further, Figure 3 shows that gains only occur when P_{replay} considers *both* level scores and staleness ($0 < \rho < 1$), highlighting the importance of staleness-based sampling in keeping scores from drifting off-policy. Lastly, we also

Prioritized Level Replay

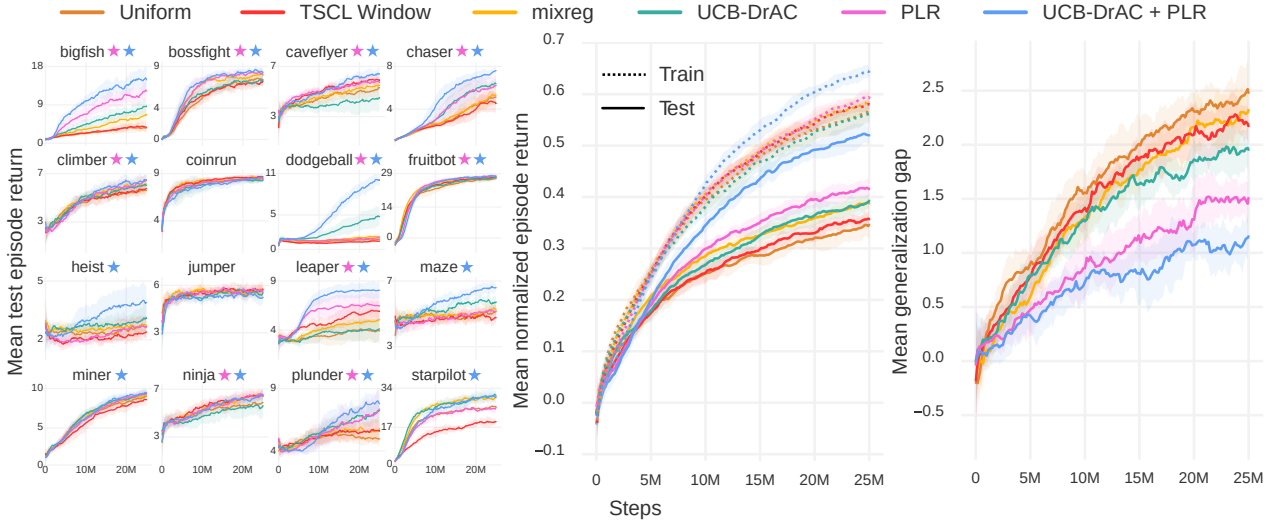


Figure 2. Left: Mean episodic test returns (10 runs) of each method. Each colored ★ indicates statistically significant ($p < 0.05$) gains in final test performance or sample complexity along the curve, relative to uniform sampling, for the PLR-based method of the same color. Center: Mean normalized train and test returns averaged across all games. Right: Mean generalization gaps averaged across all games.

benchmarked PLR on the hard setting against the same set of methods, where it again leads with 35% greater test returns relative to uniform sampling and 83% greater test returns when combined with UCB-DrAC. Figures 10–18 and Table 4 in Appendix B report additional details on these results.

The alternative scoring metrics based on TD-error and classifier uncertainty perform inconsistently across games. While certain games, such as BigFish, see improved sample-efficiency and generalization under various scoring functions, others, such as Ninja, see no improvement or worse, degraded performance. See Figure 3 for an example of this inconsistent effect across games. We find the best-performing variant of TSCL Window does not incorporate staleness information ($\rho = 0$) and similarly leads to inconsistent outcomes across games at test time, notably significantly worsening performance on StarPilot, as seen in Figure 2, and increasing the generalization gap on some environments as revealed in Figure 15 of Appendix B.

5.2. MiniGrid

We provide empirical support for the Value Correction Hypothesis (defined in Section 3) on two challenging MiniGrid environments, whose levels fall into discrete difficulties (e.g. by number of rooms to be traversed). In both, PLR with rank prioritization significantly improves sample efficiency and generalization over uniform sampling, demonstrating our method also works well in discrete state spaces. We find a staleness coefficient of $\rho = 0.3$ leads to the best test performance on MiniGrid. The top row of Figure 4 summarizes these results.

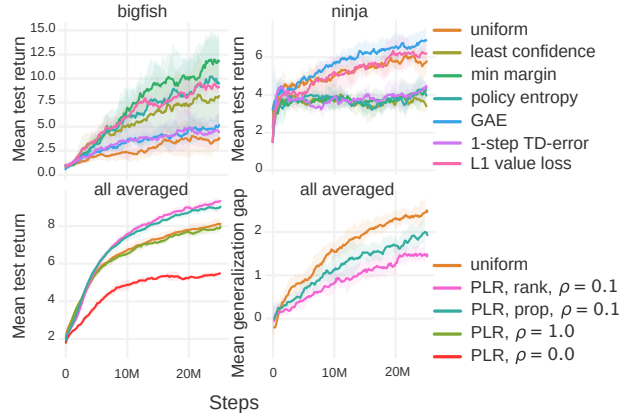


Figure 3. Top: Two example Procgen environments, between which all scoring functions except L1 value loss show inconsistent improvements to test performance (rank prioritization, $\beta = 0.1$, $\rho = 0.3$). This inconsistency holds across settings in our grid search. Bottom: Mean unnormalized episodic test returns (left) and mean generalization gap (right) for various PLR settings.

To test our hypothesis, we bin each level into its corresponding difficulty, expressed as ascending, discrete values (note that PLR does not have access to this privileged information). In the bottom row of Figure 4, we see how the expected difficulty of levels sampled using PLR changes during training for each environment. We observe that as P_{replay} is updated, levels become sampled according to an implicit curriculum over the training levels that prioritizes progressively harder levels. Of particular note, PLR seems to struggle to discover a useful curriculum for around the first 4,000 updates on ObstructedMazeGamut-Medium, at which point it discovers

Prioritized Level Replay

Table 2. Test returns of policies trained using each method with its best hyperparameters. Following Raileanu et al. (2021), the reported mean and standard deviations per environment are computed by evaluating the final policy’s average return on 100 test episodes, aggregated across multiple training runs (10 runs for Procgen Benchmark and 3 for MiniGrid, each initialized with a different training seed). Normalized test returns per run are computed by dividing the average test return per run for each environment by the corresponding average test return of the uniform-sampling baseline over all runs. We then report the means and standard deviations of normalized test returns aggregated across runs. We report the normalized return statistics for Procgen and MiniGrid environments separately. Bolded methods are not significantly different from the method with highest mean, unless all are, in which case none are bolded.

Environment	Uniform	TSCL	mixreg	UCB-DrAC	PLR	UCB-DrAC + PLR
BigFish	3.7 ± 1.2	4.3 ± 1.3	6.9 ± 1.6	8.7 ± 1.1	10.9 ± 2.8	14.3 ± 2.1
BossFight	7.7 ± 0.4	7.4 ± 0.8	8.1 ± 0.7	7.7 ± 0.7	8.9 ± 0.4	8.8 ± 0.8
CaveFlyer	5.4 ± 0.8	6.3 ± 0.6	6.0 ± 0.6	4.6 ± 0.9	6.3 ± 0.5	6.8 ± 0.7
Chaser	5.2 ± 0.7	4.9 ± 1.0	5.7 ± 1.1	6.8 ± 0.9	6.9 ± 1.2	8.0 ± 0.6
Climber	5.9 ± 0.6	6.0 ± 0.8	6.6 ± 0.7	6.4 ± 0.9	6.3 ± 0.8	6.8 ± 0.7
CoinRun	8.6 ± 0.4	9.2 ± 0.2	8.6 ± 0.3	8.6 ± 0.4	8.8 ± 0.5	9.0 ± 0.4
Dodgeball	1.7 ± 0.2	1.2 ± 0.4	1.8 ± 0.4	5.1 ± 1.6	1.8 ± 0.5	10.3 ± 1.4
FruitBot	27.3 ± 0.9	27.1 ± 1.6	27.7 ± 0.8	27.0 ± 1.3	28.0 ± 1.3	27.6 ± 1.5
Heist	2.8 ± 0.9	2.5 ± 0.6	2.7 ± 0.4	3.2 ± 0.7	2.9 ± 0.5	4.9 ± 1.3
Jumper	5.7 ± 0.4	6.1 ± 0.6	6.1 ± 0.3	5.6 ± 0.5	5.8 ± 0.5	5.9 ± 0.3
Leaper	4.2 ± 1.3	6.4 ± 1.2	5.2 ± 1.1	4.4 ± 1.4	6.8 ± 1.2	8.7 ± 1.0
Maze	5.5 ± 0.4	5.0 ± 0.3	5.4 ± 0.5	6.2 ± 0.5	5.5 ± 0.8	7.2 ± 0.8
Miner	8.7 ± 0.7	8.9 ± 0.6	9.5 ± 0.4	10.1 ± 0.6	9.6 ± 0.6	10.0 ± 0.5
Ninja	6.0 ± 0.4	6.8 ± 0.5	6.9 ± 0.5	5.8 ± 0.8	7.2 ± 0.4	7.0 ± 0.5
Plunder	5.1 ± 0.6	5.9 ± 1.1	5.7 ± 0.5	7.8 ± 0.9	8.7 ± 2.2	7.7 ± 0.9
StarPilot	26.8 ± 1.5	19.8 ± 3.4	32.7 ± 1.5	31.7 ± 2.4	27.9 ± 4.4	29.6 ± 2.2
Normalized test returns (%)	100.0 ± 4.5	103.0 ± 3.6	113.8 ± 2.8	129.8 ± 8.2	128.3 ± 5.8	176.4 ± 6.1
MultiRoom-N4-Random	0.80 ± 0.04	–	–	–	0.81 ± 0.01	–
ObstructedMazeGamut-Easy	0.53 ± 0.04	–	–	–	0.85 ± 0.04	–
ObstructedMazeGamut-Med	0.65 ± 0.01	–	–	–	0.73 ± 0.07	–
Normalized test returns (%)	100.0 ± 2.5	–	–	–	124.3 ± 4.7	–

a curriculum that gradually assigns more weight to harder levels. This curriculum enables PLR with access to only 6,000 training levels to attain even higher mean test returns than the uniform-sampling baseline with access to the full set of training levels, of which there are roughly 4 billion (so our training levels constitute 0.00015% of the total number).

We further tested an extended version of PLR that trains on the full level distribution on these two environments by tracking a buffer of levels with the highest estimated learning potential. We find it outperforms uniform sampling with access to the full level distribution. These additional results are presented in Appendix B.3.

6. Related Work

Several methods for improving generalization in deep RL adapt techniques from supervised learning, including stochastic regularization (Igl et al., 2019; Cobbe et al., 2020a), data augmentation (Kostrikov et al., 2020; Raileanu et al., 2021; Wang et al., 2020a), and feature distillation (Igl

et al., 2020; Cobbe et al., 2020b). In contrast, PLR modifies only how the next training level is sampled, thereby easily combining with any model or RL algorithm.

The selective-sampling performed by PLR makes it a form of active learning (Cohn et al., 1994; Settles, 2009). Our work also echoes ideas from Graves et al. (2017), who train a multi-armed bandit to choose the next task in multi-task supervised learning, so to maximize gradient-based progress signals. Sharma et al. (2018) extend these ideas to multi-task RL, but add the additional requirement of knowing a maximum target return for each task a priori. Zhang et al. (2020b) use an ensemble of value functions for selective goal sampling in the off-policy continuous control setting, requiring prior knowledge of the environment structure to generate candidate goals. Unlike PLR, these methods assume the ability to sample tasks or levels based on their structural properties, an assumption that does not typically hold for PCG simulators. Instead, our method automatically uncovers similarly difficult levels, giving rise to a curriculum without prior knowledge of the environment.

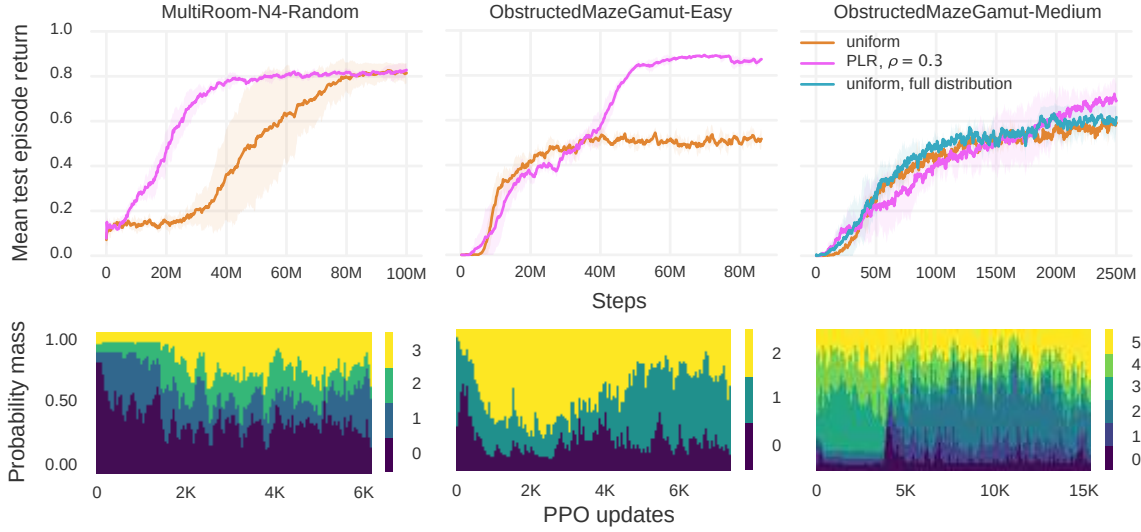


Figure 4. Top: Mean episodic test returns of PLR and the uniform-sampling baseline on MultiRoom-N4-Random (4 runs), ObstructedMazeGamut-Easy (3 runs), and ObstructedMazeGamut-Medium (3 runs). Bottom: The probability mass assigned to levels of varying difficulty over the course of training in a single, randomly selected run for the respective environment.

A recent theme in the PCG setting explores adaptively generating levels to facilitate learning (Sukhbaatar et al., 2017; Wang et al., 2019; 2020b; Khalifa et al., 2020; Akkaya et al., 2019; Campero et al., 2020; Dennis et al., 2020). Unlike these approaches, our method does not assume control over level generation, requiring only the ability to replay previously visited levels. These methods also require parameterizing level generation with additional learning modules. In contrast, our approach does not require such extensions of the environment, for example including teacher-specific action spaces in the case of Campero et al. (2020). Most similar to our method, Matiisen et al. (2020) proposes a teacher-student curriculum learning (TSCL) algorithm that samples training levels by considering the change in episodic returns per level, though they neither design nor test the method for generalization. As shown in Section 5.1, it provides inconsistent benefits at test time. Further, unlike TSCL, PLR does not assume access to all levels at the start of training, and as we show in Appendix B.3, PLR can be extended to improve sample efficiency and generalization by training on an unbounded number of training levels.

Like our method, Schaul et al. (2016) and Kapturowski et al. (2019) use TD-errors to estimate learning potential. While these methods make use of TD-errors to prioritize learning from *past* experiences, our method uses such estimates to prioritize revisiting levels for generating entirely new *future* experiences for learning.

Generalization requires sufficient exploration of environment states and dynamics. Thus, recent exploration strategies (e.g. Raileanu & Rocktäschel, 2020; Campero et al., 2020; Zhang et al., 2020a; Zha et al., 2021) shown to bene-

fit simple PCG settings are complementary to the aims of this work. However, as these studies focus on PCG environments with low-dimensional state spaces, whether such methods can be successfully applied to more complex PCG environments like Progen Benchmark remains to be seen. If so, they may potentially combine with PLR to yield additive improvements. We believe the interplay between such exploration methods and PLR to be a promising direction for future research.

7. Conclusion and Future Work

We introduced Prioritized Level Replay (PLR), an algorithm for selectively sampling the next training level in PCG environments based on the estimated learning potential of revisiting each level for the current policy. We showed that our method remarkably improves both the sample efficiency and generalization of deep RL agents in PCG environments, including the majority of environments in Progen Benchmark and two challenging MiniGrid environments. We further combined PLR with the prior leading method to set a new state-of-the-art on Progen Benchmark. Further, on MiniGrid environments, we showed PLR induces an emergent curriculum of increasingly more difficult levels.

The flexibility of the PCG abstraction makes PLR applicable to many problems of practical importance, for example, robotic object manipulation tasks, where domain randomized environment instances map to the notion of levels. We believe PLR may even be applicable to singleton environments, given a procedure for generating variations of the underlying MDP as a function of a level identifier, for ex-

ample, by varying the starting positions of entities. Another natural extension of PLR is to adapt the method to operate in the goal-conditioned setting, by incorporating goals into the level parameterization.

Despite the wide applicability of PCG and consequently PLR, not all problem domains can be effectively represented in seed-based simulation. Many real world problems require transfer into domains too complex to be adequately captured by simulation, such as car driving, where realizing a completely faithful simulation would entail solving the very same control problem of interest, creating a chicken-and-egg dilemma. Further, environment resets are not universally available, such as in the continual learning setting, where the agent interacts with the environment without explicit episode boundaries—arguably, a more realistic interaction model for a learning agent deployed in the wild.

Still, pre-training in simulation with resets can nevertheless benefit such settings, where the target domain is rife with open-ended complexity and where resets are unavailable, especially as training through real-world interactions can be slow, expensive, and precarious. In fact, in practice, we almost exclusively train deep RL policies in simulation for these reasons. As PLR provides a simple method to more fully exploit the simulator for improved test-time performance, we believe PLR can also be adapted to improve learning in these settings.

We further note that while we empirically demonstrated that the L1 value loss acts as a highly effective scoring function, there likely exist even more potent choices. Directly learning such functions may reveal even better alternatives. Lastly, combining PLR with various exploration strategies may further improve test performance in hard exploration environments. We look forward to investigating each of these promising directions in future work, prioritized accordingly, by learning potential.

Acknowledgements

We thank Roberta Raileanu, Heinrich Küttler, and Jakob Foerster for useful discussions and feedback on this work, and our anonymous reviewers, for their recommendations on improving this paper.

References

- Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., Ribas, R., Schneider, J., Tezak, N., Tworek, J., Welinder, P., Weng, L., Yuan, Q., Zaremba, W., and Zhang, L. Solving rubik’s cube with a robot hand. *CoRR*, abs/1910.07113, 2019. URL <http://arxiv.org/abs/1910.07113>.
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, O. P., and Zaremba, W. Hindsight experience replay. In *Advances in neural information processing systems*, pp. 5048–5058, 2017.
- Bard, N., Foerster, J. N., Chandar, S., Burch, N., Lanctot, M., Song, H. F., Parisotto, E., Dumoulin, V., Moitra, S., Hughes, E., Dunning, I., Mourad, S., Larochelle, H., Bellemare, M. G., and Bowling, M. The hanabi challenge: A new frontier for AI research. *Artif. Intell.*, 280:103216, 2020. doi: 10.1016/j.artint.2019.103216. URL <https://doi.org/10.1016/j.artint.2019.103216>.
- Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, Jun 2013. ISSN 1076-9757. doi: 10.1613/jair.3912. URL <http://dx.doi.org/10.1613/jair.3912>.
- Campero, A., Raileanu, R., Küttler, H., Tenenbaum, J. B., Rocktäschel, T., and Grefenstette, E. Learning with AMIGO: Adversarially Motivated Intrinsic Goals. *CoRR*, abs/2006.12122, 2020. URL <https://arxiv.org/abs/2006.12122>.
- Chevalier-Boisvert, M., Bahdanau, D., Lahlou, S., Willems, L., Saharia, C., Nguyen, T. H., and Bengio, Y. BabyAI: First steps towards grounded language learning with a human in the loop. *CoRR*, abs/1810.08272, 2018. URL <http://arxiv.org/abs/1810.08272>.
- Chevalier-Boisvert, M., Willems, L., and Pal, S. Minimalistic gridworld environment for OpenAI Gym. <https://github.com/maximecb/gym-minigrid>, 2018.
- Cobbe, K., Klimov, O., Hesse, C., Kim, T., and Schulman, J. Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning*, pp. 1282–1289. PMLR, 2019.
- Cobbe, K., Hesse, C., Hilton, J., and Schulman, J. Leveraging Procedural Generation to Benchmark Reinforcement Learning. In *International Conference on Machine Learning*, pp. 2048–2056. PMLR, November 2020a. URL <http://proceedings.mlr.press/v119/cobbe20a.html>. ISSN: 2640-3498.
- Cobbe, K., Hilton, J., Klimov, O., and Schulman, J. Phasic Policy Gradient. *CoRR*, abs/2009.04416, 2020b. URL <https://arxiv.org/abs/2009.04416>.
- Cohn, D., Atlas, L., and Ladner, R. Improving generalization with active learning. *Machine learning*, 15(2): 201–221, 1994.

A. Experiment Details and Hyperparameters

A.1. Procgen Benchmark

Procgen Benchmark consists of 16 PCG environments of varying styles, exhibiting a diversity of gameplay similar to that of the ALE benchmark. Game levels are determined by a random seed and can vary in navigational layout, visual appearance, and starting positions of entities. All Procgen environments share the same discrete 15-dimensional action space and produce $64 \times 64 \times 3$ RGB observations. (Cobbe et al., 2020a) provides a comprehensive description of each of the 16 environments. State-of-the-art RL algorithms, like PPO, lead to significant generalization gaps between test and train performance in all games, making Procgen a useful benchmark for assessing generalization performance.

We follow the standard protocol for testing generalization performance on Procgen: We train an agent for each game on a finite number of levels, N_{train} , and sample test levels from the full distribution of levels. Normalized test returns are computed as $(R \cup R_{\min}) / (R_{\max} \cup R_{\min})$, where R is the unnormalized return and each game’s minimum return, $R_{\min}$, and maximum return, $R_{\max}$, are provided in Cobbe et al. (2020a), which uses this same normalization.

To make the most efficient use of our computational resources, we perform hyperparameter sweeps on the easy setting. This also makes our results directly comparable to most prior works benchmarked on Procgen, which have likewise focused on the easy setting. In Procgen easy, our experiments use the recommended settings of $N_{\text{train}} = 200$ and 25M steps of training, as well as the same ResNet policy architecture and PPO hyperparameters shared across all games as in Cobbe et al. (2020a) and Raileanu et al. (2021). We find 25M steps to be sufficient for uncovering differences in generalization performance among our methods and standard baselines. Moreover, under this setup, we find Procgen training runs require much less wall-clock time than training runs on the two MiniGrid environments of interest over an equivalent number of steps needed to uncover differences in generalization performance. Therefore we survey the empirical differences across various settings of PLR on Procgen easy rather than MiniGrid.

To find the best hyperparameters for PLR, we evaluate each combination of the scoring function choices in Table 1 with both rank and proportional prioritization, performing a coarse grid search for each pair over different settings of the temperature parameter β in $\{0.1, 0.5, 1.0, 1.4, 2.0\}$ and the staleness coefficient ρ in $\{0.1, 0.3, 1.0\}$. For each setting, we run 4 trials across all 16 of games of the Procgen Benchmark, evaluating based on mean unnormalized test return across games. In our TD-error-based scoring functions, we set γ and λ equal to the same respective values used by the GAE in PPO during training. We found PLR offered the

most pronounced gains at $\beta = 0.1$ and $\rho = 0.1$ on Procgen, but these benefits also held for higher values ($\beta = 0.5$ and $\rho = 0.3$), though to a lesser degree.

For UCB-DrAC, we make use of the best-reported hyperparameters on the easy setting of Procgen in Raileanu et al. (2021), listed in Table 3.

We found the default setting of mixreg’s $\alpha = 0.2$, as used by Wang et al. (2020a) in the hard setting, performs poorly on the easy setting. Instead, we conducted a grid search over α in $\{0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.8, 0.2, 0.5, 0.8, 1\}$.

Since the TSCL Window algorithm was not previously evaluated on Procgen Benchmark, we perform a grid search over different settings for both Boltzmann and ϵ -greedy variants of the algorithm to determine the best hyperparameter settings for Procgen easy. We searched over window size K in $\{10, 100, 1000, 10000\}$, bandit learning rate α in $\{0.01, 0.1, 0.5, 1.0\}$, random exploration probability ϵ in $\{0.0, 0.01, 0.1, 0.5\}$ for the ϵ -greedy variant, and temperature τ in $\{0.1, 0.5, 1.0\}$ for the Boltzmann variant. Additionally, for a fairer comparison to PLR we further evaluated a variant of TSCL Window that, like PLR, incorporates the staleness distribution, by additionally searching over values of the staleness coefficient ρ in $\{0.0, 0.1, 0.3, 0.5\}$, though we ultimately found that TSCL Window performed best without staleness sampling ($\rho = 0$).

See Table 3 for a comprehensive overview of the hyperparameters used for PPO, UCB-DrAC, mixreg, and TSCL Window, shared across all Procgen environments to generate our reported results on Procgen easy.

The evaluation protocol on the hard setting entails training on 500 levels over 200M steps (Cobbe et al., 2020a), making it more compute-intensive than the easy setting. To save on computational resources, we make use of the same hyperparameters found in the easy setting for each method on Procgen hard, with one exception: As our PPO implementation does not use multi-GPU training, we were unable to quadruple our GPU actors as done in Cobbe et al. (2020a) and Wang et al. (2020a). Instead, we resorted to doubling the number of environments in our single actor to 128, resulting in mini-batch sizes half as large as used in these two prior works. As such, our baseline results on hard are not directly comparable to theirs. We found setting mixreg’s $\alpha = 0.2$ as done in Wang et al. (2020a) led to poor performance under this reduced batch size. We conducted an additional grid search, finding $\alpha = 0.01$ to perform best, as on Procgen easy.

A.2. MiniGrid

The MiniGrid suite (Chevalier-Boisvert et al., 2018) features a series of highly structured environments of increasing difficulty. Each environment features a task in a grid world

Table 3. Hyperparameters used for training on Procgen Benchmark and MiniGrid environments.

Parameter	Procgen	MiniGrid
<i>PPO</i>		
γ	0.999	0.999
λ_{GAE}	0.95	0.95
PPO rollout length	256	256
PPO epochs	3	4
PPO minibatches per epoch	8	8
PPO clip range	0.2	0.2
PPO number of workers	64	64
Adam learning rate	5e-4	7e-4
Adam ϵ	1e-5	1e-5
return normalization	yes	yes
entropy bonus coefficient	0.01	0.01
value loss coefficient	0.5	0.5
<i>PLR</i>		
Prioritization	rank	rank
Temperature, β , 0.1	0.1	0.1
Staleness coefficient, ρ	0.1	0.3
<i>UCB-DrAC</i>		
Window size, K	10	-
Regularization coefficient, α_r	0.1	-
UCB exploration coefficient, c	0.1	-
<i>mixreg</i>		
Beta shape, α	0.01	-
<i>TSCL Window</i>		
Bandit exploration strategy	ϵ -greedy	-
Window size, K	10	-
Bandit learning rate, α	1.0	-
Exploration probability, ϵ	0.5	-

setting, and as in Procgen, environment levels are determined by a seed. Harder levels require the agent to perform longer action sequences over a combinatorially-rich set of game entities, on increasingly larger grids. The clear ordering of difficulty over subsets of MiniGrid environments allows us to track the relative difficulty of levels sampled by PLR over the course of training.

MiniGrid environments share a discrete 7-dimensional action space and produce a 3-channel integer state encoding of the 7×7 grid immediately including and in front of the agent. However, following the training setup in Igl et al. (2019), we modify the environment to produce an $N \times M \times 3$ encoding of the full grid, where N and M vary according to the maximum grid dimensions of each environment. Full observability makes generalization harder, requiring the agent to generalize across different level layouts in their entirety.

We evaluate PLR with rank prioritization on two MiniGrid environments whose levels are uniformly distributed across several difficulty settings. Training on levels of varying difficulties helps agents make use of the easier levels as stepping stones to learn useful behaviors that help the agent make progress on harder levels. However, under the uniform-sampling baseline, learning may be inefficient, as the training process does not selectively train the agent on levels of increasing difficulty, leading to wasted training steps when a difficult level is sampled early in training. On the contrary, if PLR scores levels according to the time-averaged L1 value loss of recently experienced level trajectories, the average difficulty of the sampled levels should adapt to the agent’s current abilities, following the reasoning outlined in the Value Correction Hypothesis, stated in Section 3.

As in Igl et al. (2019), we parameterize the agent policy as a 3-layer CNN with 16, 32, and 32 channels, with a final hidden layer of size 64. All kernels are 2×2 and use a stride of 1. For the ObstructedMazeGamut environments, we increase the number of channels of the final CNN layer to 64. We follow the same high-level generalization evaluation protocol used for Procgen, training the agent on a fixed set of 4000 levels for MultiRoom-N4-Random, 3000 levels for ObstructedMazeGamut-Easy, and 6000 levels for ObstructedMazeGamut-Medium, and testing on the full level distribution. We chose these values for $|\Lambda_{\text{train}}|$ to ensure roughly 1000 training levels of each difficulty setting of each environment. We model our PPO parameters on those used by Igl et al. (2019) in their MiniGrid experiments. We performed a grid search to find that PLR with rank prioritization, $\beta = 0.1$, and $\rho = 0.3$ learned most quickly on the MultiRoom environment, and used this setting for all our MiniGrid experiments. Table 3 summarizes these hyperparameter choices.

The remainder of this section provides more details about the various MiniGrid environments used in this work.

MultiRoom-N4-Random This environment requires the agent to navigate through 1, 2, 3, or 4 rooms respectively to reach a goal object, resulting in a natural ordering of levels over four levels of difficulty. The agent always starts at a random position in the furthest room from the goal object, facing a random direction. The goal object is also initialized to a random position within its room. See Figure 5 for screenshots of example levels.

ObstructedMazeGamut-Easy This environment consists of levels uniformly distributed across the first three difficulty settings of the ObstructedMaze environment, in which the agent must locate and pick up the key in order to unlock the door to pick up a goal object in a second room. The agent and goal object are always initialized in random positions in different rooms separated by the locked door.

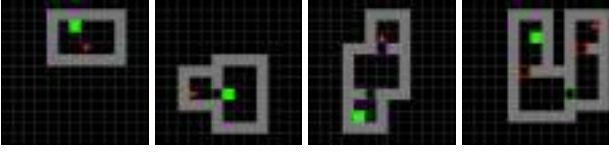


Figure 5. Example levels of each of the four difficulty levels of MultiRoom-N4-Random, in order of increasing difficulty from left to right. The agent (red triangle) must reach the goal (green square).

The second difficulty setting further requires the agent to first uncover the key from under a box before picking up the key. The third difficulty level further requires the agent to first move a ball blocking the door before entering the door. See Figure 6 for screenshots of example levels.



Figure 6. Example levels of each of the three difficulty levels of ObstructedMazeGamut-Easy, in order of increasing difficulty from left to right. The agent must find the key, which may be hidden under a box, to unlock a door, which may be blocked by an obstacle, to reach the goal object (blue circle).

ObstructedMazeGamut-Hard This environment consists of levels uniformly distributed across the first six difficulty levels of the ObstructedMaze environment. Harder levels corresponding to the fourth, fifth, and sixth difficulty settings include two additional rooms with no goal object to distract the agent. Each instance of these harder levels also contain two pairs of keys of different colors, each opening a door of the same color. The agent always starts one room away from the randomly positioned goal object. Each of the two keys is visible in the fourth difficulty setting and doors are unobstructed. The fifth difficulty setting hides the keys under boxes, and the sixth again places obstacles that must be removed before entering two of the doors, one of which is always the door to the goal-containing room. See Figure 7 for example screenshots.

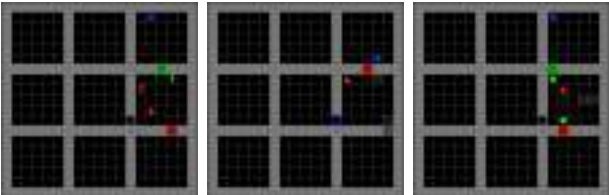


Figure 7. Example levels in increasing difficulty from left to right of each additional difficulty setting introduced by ObstructedMazeGamut-Hard in addition to those in ObstructedMazeGamut-Easy.

B. Additional Experimental Results

B.1. Extended Results on Procgen Benchmark

We present an overview of the improvements in test performance of each method across all 16 Procgen Benchmark games over 10 runs in Figure 14. For each game, Figure 15 further shows how the generalization gap changes over the course of training under each method tested. We show in Figures 16 and 17, the mean test episodic returns on the Procgen Benchmark (easy) for PLR with rank and proportional prioritization, respectively. In both of these plots, we can see that using only staleness ($\rho = 1$) or only L1 value loss scores ($\rho = 0$) is considerably worse than direct level sampling. Thus, we only observe gains compared to the baseline when both level scores and staleness are used for the sampling distribution. Comparing Figures 16 with 17 we find that PLR with proportional instead of rank prioritization provides statistically significant gains over uniform level sampling on an additional game (CoinRun), but rank prioritization leads to slightly larger mean improvements on several games.

Figure 18 shows that when PLR improves generalization performance, it also either matches or improves training sample efficiency, suggesting that when beneficial to test performance, the representations learned via the auto-curriculum induced by PLR prove similarly useful on training levels. However we see that our method reduces training sample efficiency on two games on which our method does not improve generalization performance. Since our method does not discover useful auto-curricula for these games, it is likely that uniformly sampling levels at training time allows the agent to better memorize useful behaviors on each of the training levels compared to the selective sampling performed by our method.

Finally, we also benchmarked PLR and UCB-DrAC + PLR against uniform sampling, TSCL Window, mixreg, and UCB-DrAC on Procgen hard across 5 runs per environment. Due to the high computational cost of the evaluation protocol for Procgen hard, which entails 200M training steps, we directly use the best hyperparameters found in the easy setting for each method. The results in Figure 10 show the two PLR-based methods significantly outperform all other methods in terms of normalized mean train and test episodic return, as well as reduction in mean generalization gap, attaining even greater margins of improvement than in the easy setting. As summarized by Table 4, the gains of PLR and UCB + PLR in mean normalized test return relative to uniform sampling in the hard setting are comparable to those in the easy setting. We provide plots of episodic test return over training for each individual environment in Figure 12.

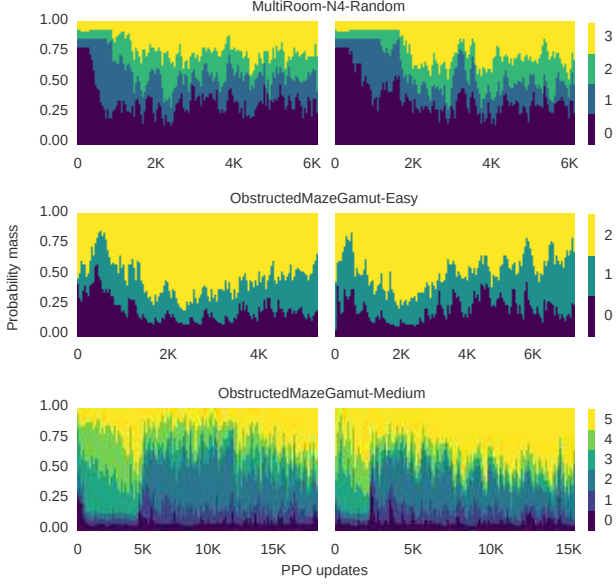


Figure 8. PLR consistently induces emergent curricula from easier to harder levels during training. Left and right correspond to two additional training runs independent of that in Figure 4.

B.2. Extended Results on Minigrid

To demonstrate that PLR consistently induces an emergent curriculum, we present plots showing the change in probability mass over different difficulty bins for additional training runs in Figure 8. Like in Figure 4, we see the probability mass assigned by P_{replay} gradually shifts from easier to harder levels over the course of training.

B.3. Training on the Full Level Distribution

While assessing generalization performance calls for using a fixed set of training levels, ideally our method can also make use of the full level distribution if given access to it. We take advantage of an unbounded number of training levels by modifying the list structures for storing scores and timestamps (see Algorithm 1 and 2) to track the top M levels by learning potential in our finite level buffer. When the lists are full, we set the next level for replacement to be

$$l_{\min} = \arg \min_l P_{\text{replay}}(l).$$

When the outcome of the Bernoulli P_D entails sampling a new level l , the score and timestamps of l replace those of $l_{\min}$ only if the score of $l_{\min}$ is lower than that of l . In this way, PLR keeps a running buffer throughout training of the top M levels appraised to have the highest learning potential for replaying anew.

Figure 9 shows that with access to the full level distribution at training, PLR improves sample efficiency and generalization performance in both environments compared to

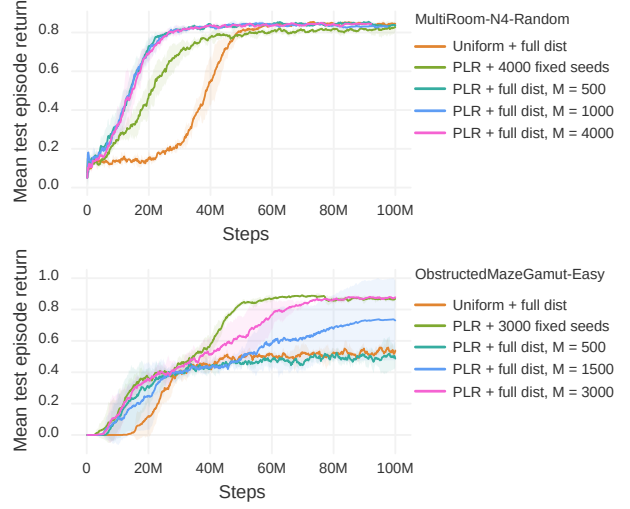


Figure 9. Mean test episodic returns on MultiRoom-N4-Random (top) and ObstructedMazeGamut-Easy (bottom) with access to the full level distribution at training. Plots are averaged over 3 runs. We set P_D to a Bernoulli parameterized as $p = 0.5$ for MultiRoom-N4-Random and $p = 0.95$ for ObstructedMazeGamut-Easy (found via grid search). As with all MiniGrid experiments using PLR, we use rank prioritization, $\beta = 0.1$, and $\rho = 0.3$.

uniform sampling on the full distribution. In MultiRoom-N4-Random, the value M makes little difference to test performance, and training with PLR on the full level distribution leads to a policy outperforming one trained with PLR on a fixed set of training levels. However, on ObstructedMazeGamut-Easy, a smaller M leads to worse test performance. Nevertheless, for all but $M = 500$, including the case of a fixed set of 3000 training levels, PLR leads to better mean test performance than uniform sampling on the full level distribution.

C. Algorithms

In this section, we provide detailed pseudocode for how PLR can be used for experience collection when using T -step rollouts. Algorithm 3 presents the extension of the generic policy-gradient training loop presented in Algorithm 1 to the case of T -step rollouts, and Algorithm 4 presents an implementation of experience collection in this setting (extending Algorithm 2). Note that when using T -step rollouts in the training loop, rollouts may start and end between episode boundaries. To compute level scores on full trajectories segmented across rollouts, we compute scores of partial episodes according to Equation 2, and record these partial scores alongside the partial episode step count in a separate buffer $\tilde{S}$. The function `score` then technically, optionally takes the additional input $\tilde{S}$ (through an abuse of notation) as an additional argument to stitch together this partial information into scores of full episodic trajectories.

Table 4. Comparison of test scores of PPO with PLR against PPO with uniform-sampling on the hard setting of Procgen Benchmark. Following (Raileanu et al., 2021), reported figures represent the mean and standard deviation of average test scores over 100 episodes aggregated across 5 runs, each initialized with a unique training seed. For each run, a normalized average return is computed by dividing the average test return for each game by the corresponding average test return of the uniform-sampling baseline over all 500 test episodes of that game, followed by averaging these normalized returns over all 16 games. The final row reports the mean and standard deviation of the normalized returns aggregated across runs. Bolded methods are not significantly different from the method with highest mean, unless all are, in which case none are bolded.

Environment	Uniform	TSCL	mixreg	UCB-DrAC	PLR	UCB-DrAC + PLR
BigFish	9.7 $\pm$ 1.8	11.9 $\pm$ 2.5	12.0 $\pm$ 2.5	10.9 $\pm$ 1.6	15.3 $\pm$ 3.6	15.5 $\pm$ 2.8
BossFight	9.6 $\pm$ 0.2	8.4 $\pm$ 0.7	9.3 $\pm$ 0.9	9.0 $\pm$ 0.2	9.7 $\pm$ 0.4	9.5 $\pm$ 1.1
CaveFlyer	3.5 $\pm$ 0.8	6.3 $\pm$ 0.6	4.0 $\pm$ 1.0	2.6 $\pm$ 0.8	6.4 $\pm$ 0.6	8.0 $\pm$ 0.9
Chaser	5.9 $\pm$ 0.5	6.2 $\pm$ 1.0	6.5 $\pm$ 0.8	7.0 $\pm$ 0.6	6.8 $\pm$ 2.2	7.6 $\pm$ 0.2
Climber	5.3 $\pm$ 1.1	5.2 $\pm$ 0.7	5.7 $\pm$ 0.7	6.1 $\pm$ 1.0	7.4 $\pm$ 0.6	7.6 $\pm$ 1.8
CoinRun	4.5 $\pm$ 0.4	5.8 $\pm$ 0.8	6.2 $\pm$ 1.0	5.2 $\pm$ 1.0	6.8 $\pm$ 0.6	7.1 $\pm$ 0.5
Dodgeball	3.9 $\pm$ 0.6	1.9 $\pm$ 0.9	4.7 $\pm$ 1.0	9.9 $\pm$ 1.2	7.4 $\pm$ 1.3	12.4 $\pm$ 0.7
FruitBot	11.9 $\pm$ 4.2	13.1 $\pm$ 2.3	14.7 $\pm$ 2.2	15.6 $\pm$ 3.7	16.7 $\pm$ 1.0	12.9 $\pm$ 5.1
Heist	1.5 $\pm$ 0.4	0.9 $\pm$ 0.3	1.2 $\pm$ 0.4	1.1 $\pm$ 0.3	1.3 $\pm$ 0.4	2.6 $\pm$ 2.2
Jumper	3.2 $\pm$ 0.3	3.2 $\pm$ 0.3	3.3 $\pm$ 0.4	2.9 $\pm$ 0.9	3.5 $\pm$ 0.5	3.3 $\pm$ 0.8
Leaper	7.1 $\pm$ 0.3	7.5 $\pm$ 0.5	7.5 $\pm$ 0.5	3.8 $\pm$ 1.6	7.4 $\pm$ 0.2	8.2 $\pm$ 0.7
Maze	3.6 $\pm$ 0.7	3.8 $\pm$ 0.6	3.9 $\pm$ 0.5	4.4 $\pm$ 0.2	4.0 $\pm$ 0.4	6.2 $\pm$ 0.4
Miner	12.8 $\pm$ 1.4	11.7 $\pm$ 0.9	13.3 $\pm$ 1.6	16.1 $\pm$ 0.6	11.3 $\pm$ 0.7	15.3 $\pm$ 0.8
Ninja	5.2 $\pm$ 0.1	5.9 $\pm$ 0.8	5.0 $\pm$ 1.0	5.2 $\pm$ 1.0	6.1 $\pm$ 0.6	6.9 $\pm$ 0.3
Plunder	3.2 $\pm$ 0.1	5.4 $\pm$ 1.1	3.7 $\pm$ 0.4	7.8 $\pm$ 1.1	8.6 $\pm$ 2.7	17.5 $\pm$ 1.3
StarPilot	5.5 $\pm$ 0.6	2.1 $\pm$ 0.4	6.9 $\pm$ 0.6	11.2 $\pm$ 1.7	5.4 $\pm$ 0.8	12.3 $\pm$ 1.5
Normalized test returns (%)	100.0 $\pm$ 2.0	103.9 $\pm$ 3.5	110.6 $\pm$ 3.9	126.6 $\pm$ 3.0	135.0 $\pm$ 6.1	182.9 $\pm$ 8.2

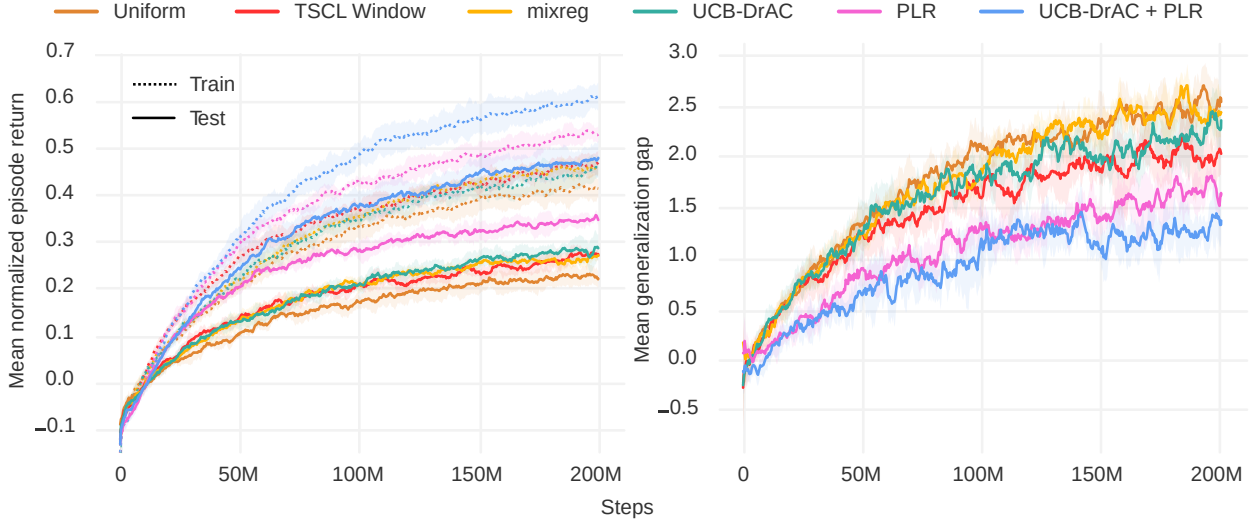


Figure 10. Left: Mean normalized train and test episode returns on Procgen Benchmark (hard). Right: Corresponding generalization gaps during training. All curves are averaged across all environments over 5 runs. The shaded area indicates one standard deviation around the mean. PLR-based methods statistically significantly outperform all others in both train and test returns. Only the PLR-based methods statistically significantly reduce the generalization gap ($p < 0.05$).

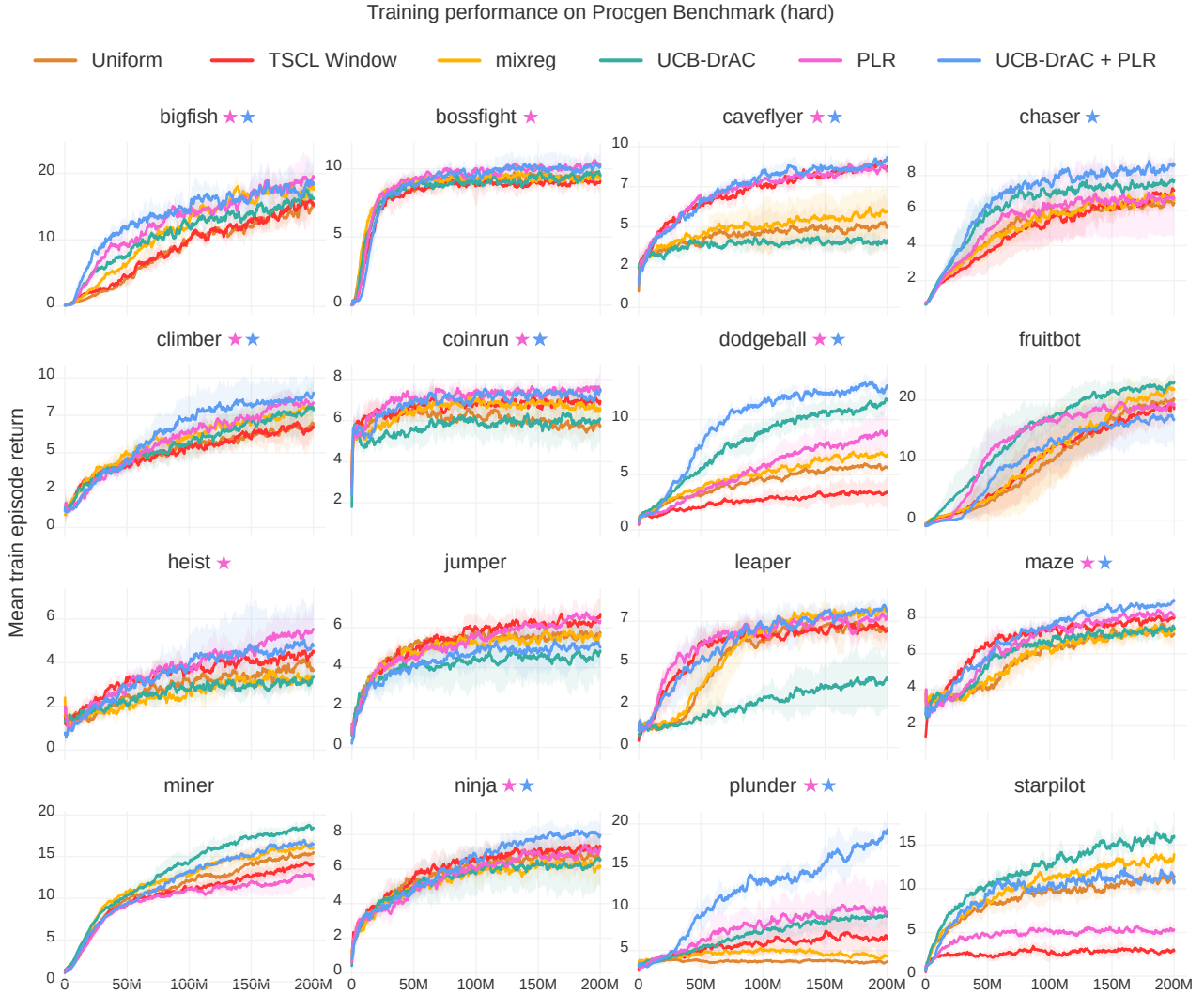


Figure 11. Mean train episode returns (5 runs) on Procgen Benchmark (hard), using the best hyperparameters found on the easy setting. The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$). Note that while PLR reduces training performance on StarPilot, it performs comparably to the uniform-sampling baseline at test time, indicating less overfitting to training levels.

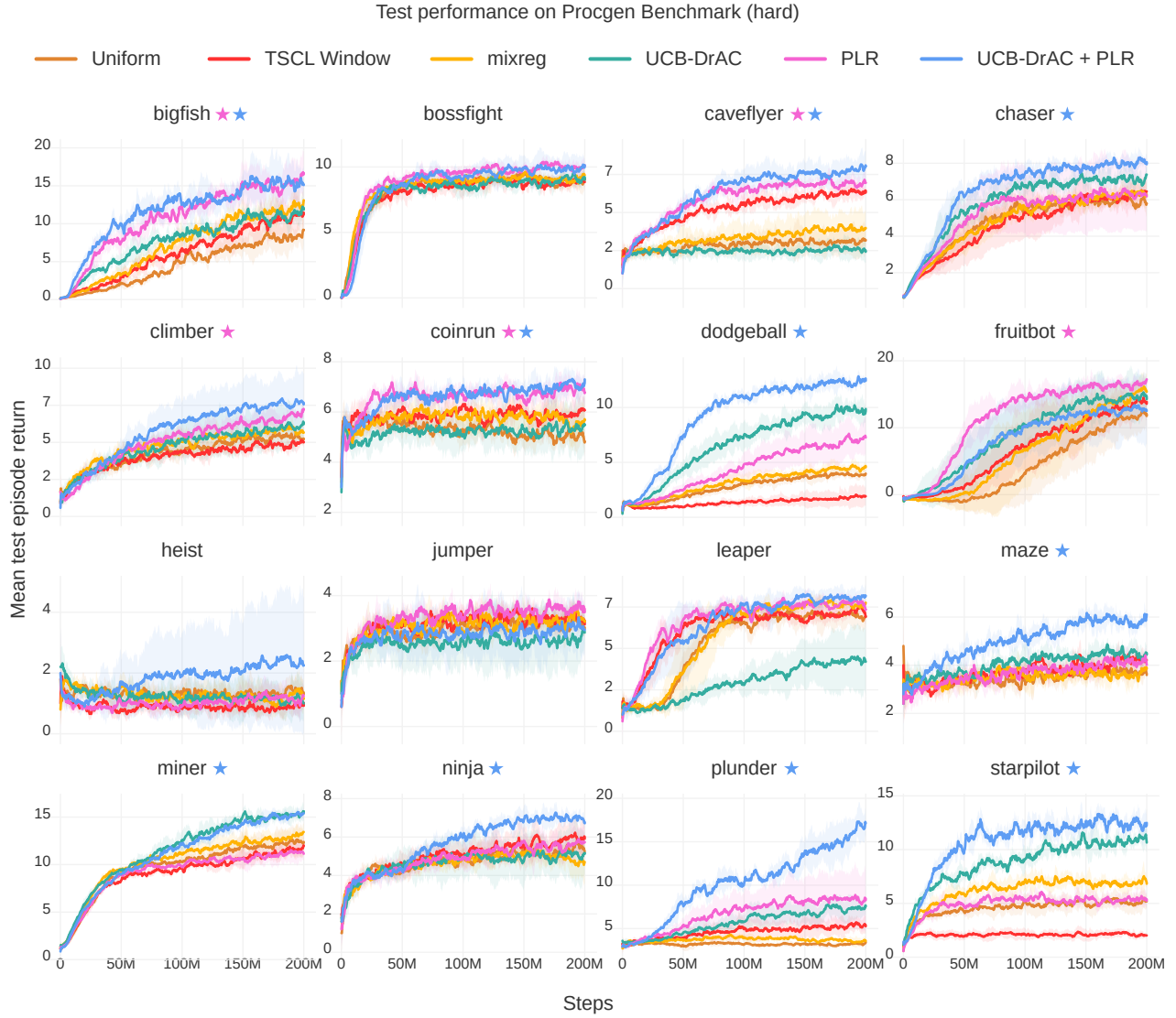


Figure 12. Mean test episode returns (5 runs) on Procgen Benchmark (hard), using best hyperparameters found on the easy setting. The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$).

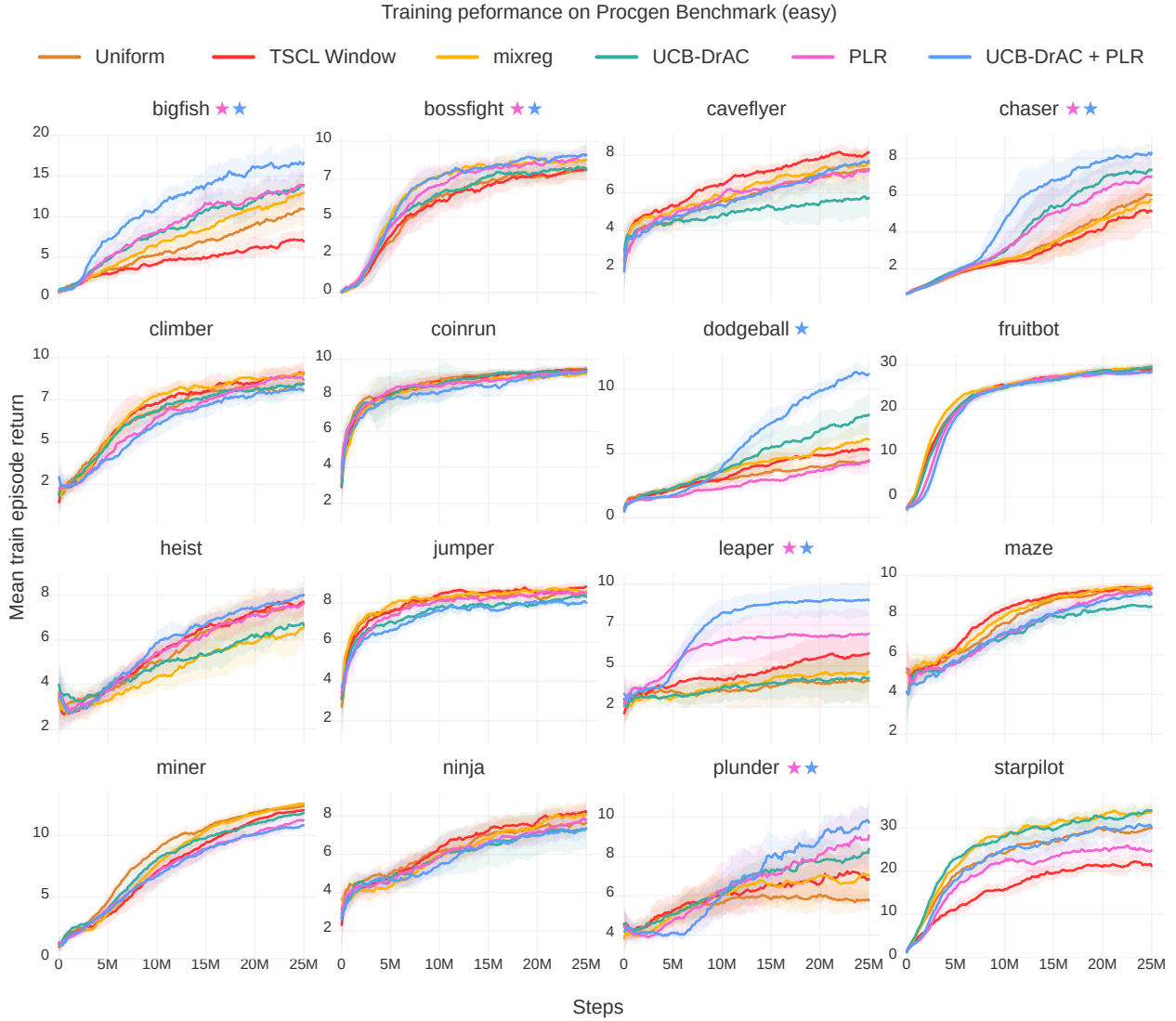


Figure 13. Mean train episode returns (5 runs) on Procgen Benchmark (easy). The shaded area indicates one standard deviation around the mean. A ★ indicates statistically significant improvement over the uniform-sampling baseline by the PLR-based method of the matching color ($p < 0.05$). PLR tends to improve or match training sample efficiency. Note that while PLR reduces training performance on StarPilot, it performs comparably to the uniform-sampling baseline at test time, indicating less overfitting to training levels.

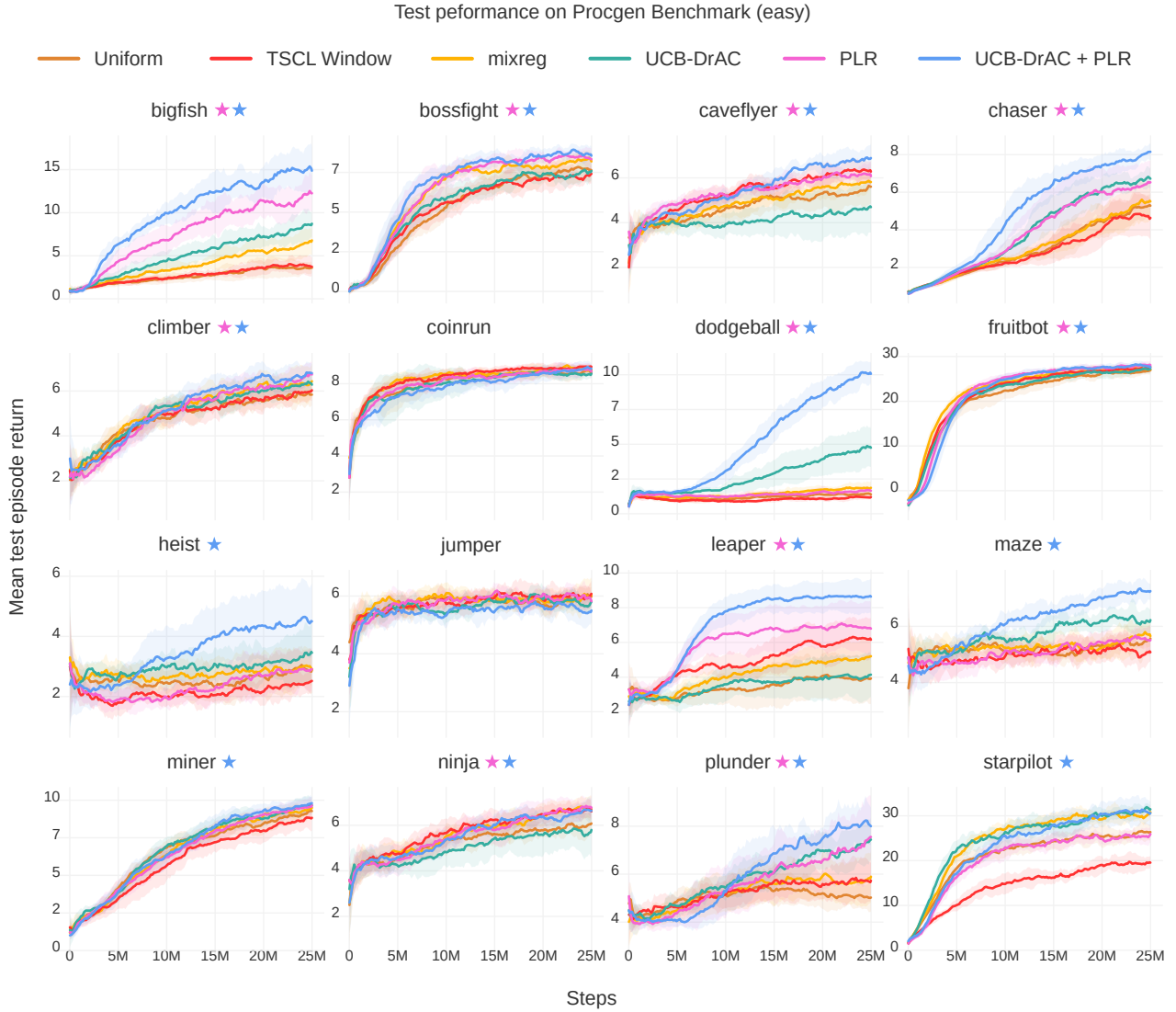


Figure 14. Mean test episode return (10 runs) on each Procgen Benchmark game (easy). The shaded area indicates one standard deviation around the mean. PLR-based methods consistently match or outperform uniform sampling with statistical significance ($p < 0.05$), indicated by a ★ of the corresponding color. We see that TSCL results in inconsistent outcomes across games, notably drastically lower test returns on StarPilot.

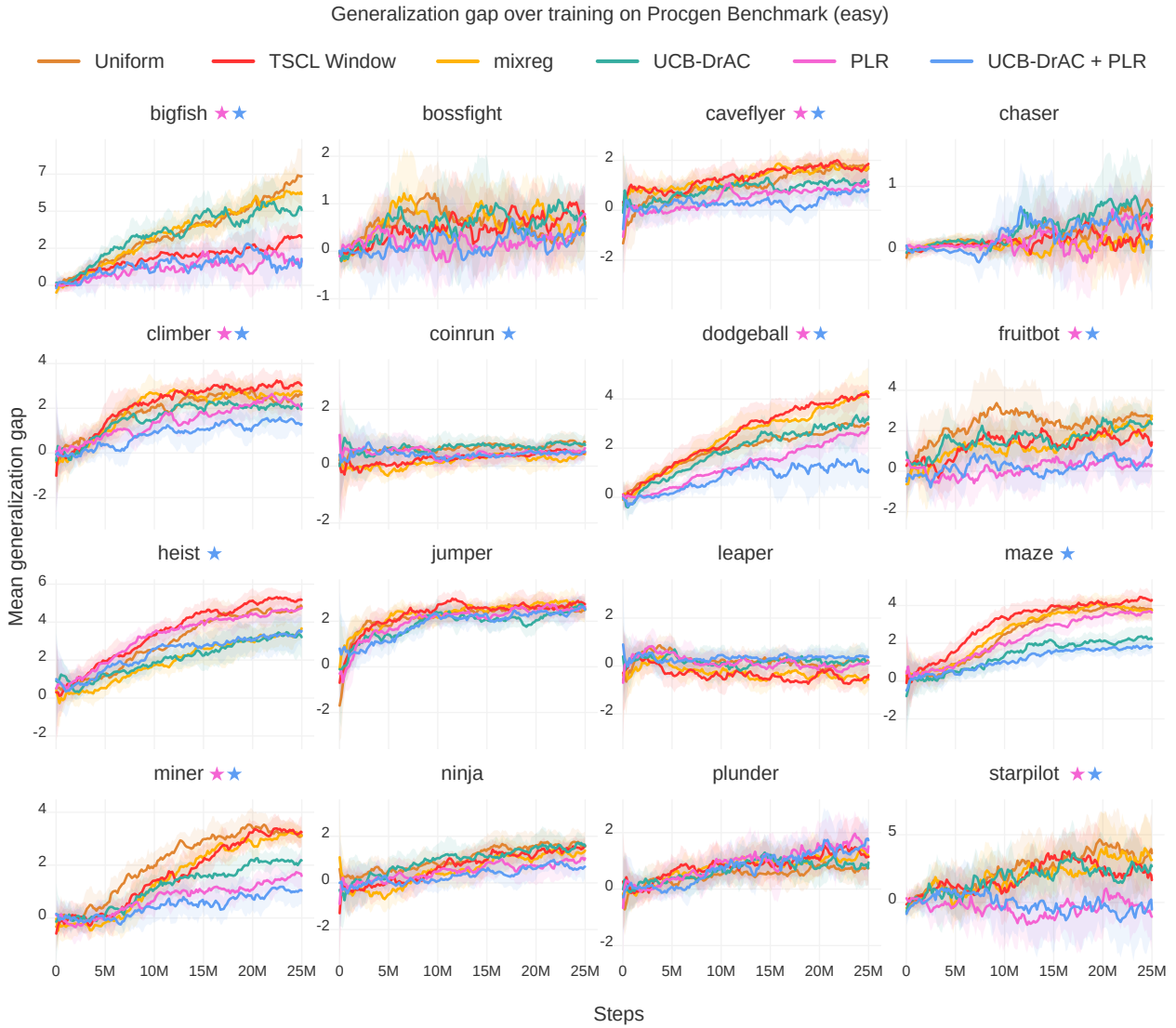


Figure 15. Mean generalization gaps throughout training (10 runs) on each Procgen Benchmark game (easy). The shaded area indicates one standard deviation around the mean. A ★ indicates the method of matching color results in a statistically significant ($p < 0.05$) reduction in generalization gap compared to the uniform-sampling baseline. By itself, PLR significantly reduces the generalization gap on 7 games, and UCB-DrAC, on 5 games. This number jumps to 10 of 16 games when these two methods are combined. TSCL only significantly reduces generalization gap on 2 of 16 games relative to uniform sampling, while increasing it on others, most notably on Dodgeball.

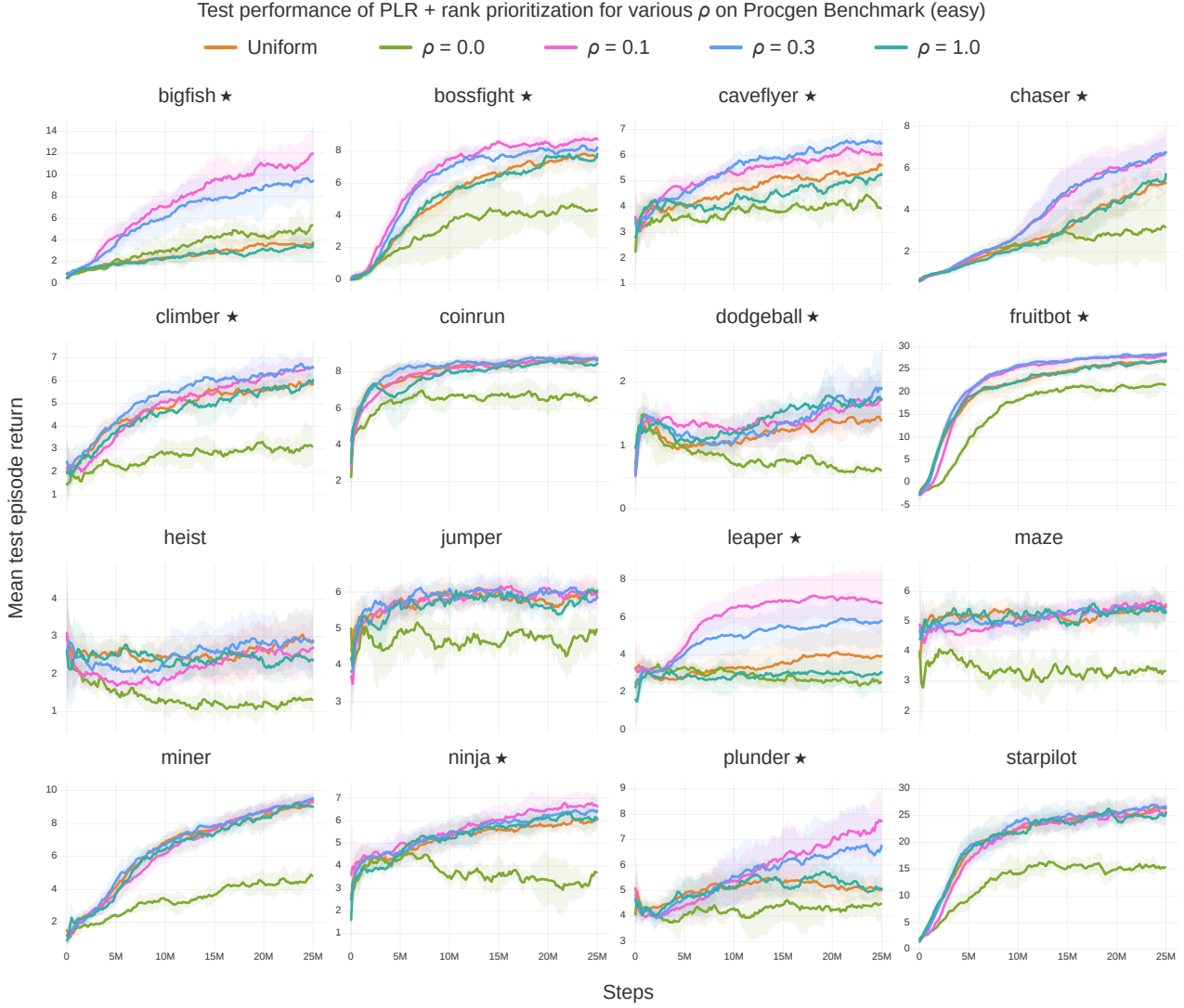


Figure 16. Mean test episode returns (10 runs) on the Procgen Benchmark (easy) for PLR with rank prioritization and $\beta = 0.1$ across a range of staleness coefficient values, ρ . The replay distribution must consider both the L1 value-loss and staleness values to realize improvements to generalization and sample efficiency. The shaded area indicates one standard deviation around the mean. A ★ next to the game name indicates that $\rho = 0.1$ exhibits statistically significantly better final test returns or sample efficiency along the test curve ($p < 0.05$), which we observe in 10 of 16 games.

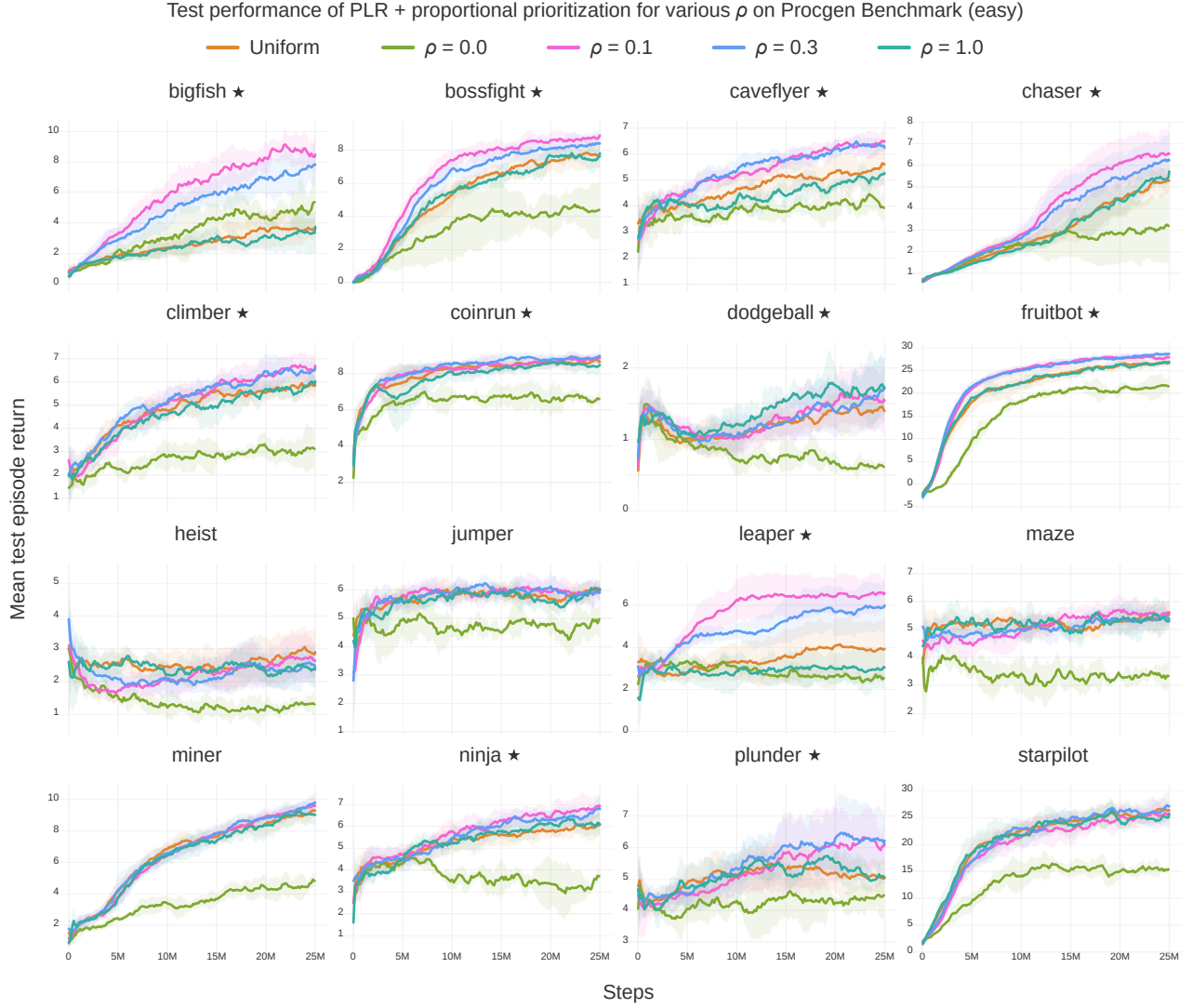


Figure 17. Mean test episode returns (10 runs) on the Procgen Benchmark (easy) for PLR with proportional prioritization and $\beta = 0.1$ across a range of values of ρ . As in the case of rank prioritization, the replay distribution must consider both the L1 value loss score and staleness values in order to realize performance improvements. The shaded area indicates one standard deviation around the mean. A ★ next to the game name indicates the condition $\rho = 0.1$ exhibits statistically significantly better final test returns or sample efficiency along the test curve ($p < 0.05$), which we observe in 11 of 16 games.

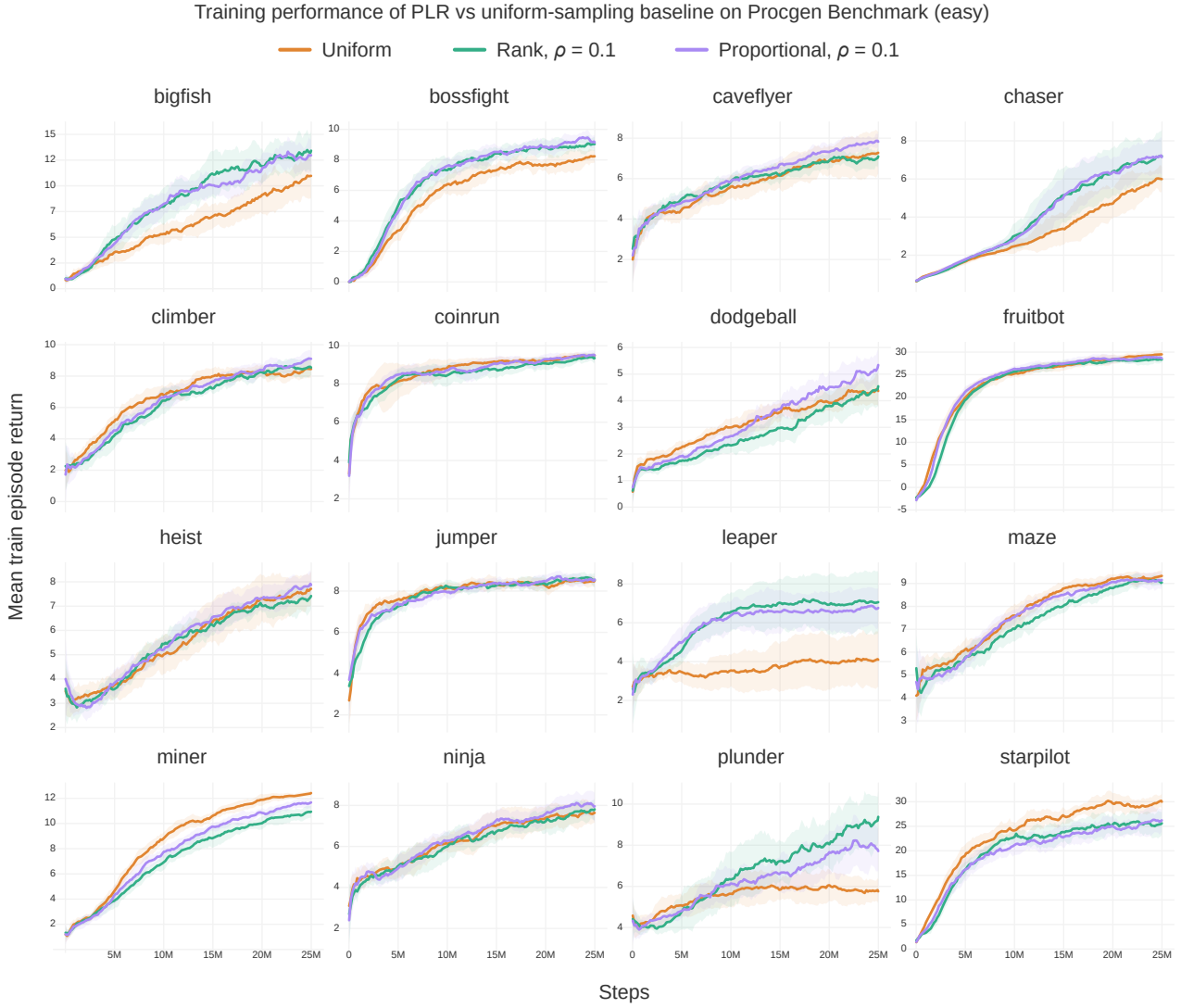


Figure 18. Mean training episode returns (10 runs) on the Procgen Benchmark for (easy) PLR with $\beta = 0.1$, $\rho = 0.1$, and each of rank and proportional prioritization. On some games, PLR improves both training sample efficiency and generalization performance (e.g. BigFish and Chaser), while on others, only generalization performance (e.g. CaveFlyer with rank prioritization). The shaded area indicates one standard deviation around the mean.

Algorithm 3 Generic T -step policy-gradient training loop with prioritized level replay

Input: Training levels Λ_{train} of an environment, policy π_{θ} , rollout length T , number of updates N_u , batch size N_b , policy update function $\mathcal{U}(\mathcal{B}, \theta) \rightarrow \theta'$.

Initialize level scores S , partial level scores $\tilde{S}$, and level timestamps C

Initialize global episode count $c \leftarrow 0$

Initialize set of visited levels $\Lambda_{\text{seen}} = \emptyset$

Initialize experience buffer $\mathcal{B} = \emptyset$

Initialize N_b parallel environment instances E , each set to a random level in $\in \Lambda_{\text{train}}$

for $u = 1$ **to** N_u **do**

$\mathcal{B} \leftarrow \emptyset$

for $k = 1$ **to** N_b **do**

$\mathcal{B} \leftarrow \mathcal{B} \quad \text{collect_experiences}(k, E, \Lambda_{\text{train}}, \Lambda_{\text{seen}}, \pi_{\theta}, T, S, \tilde{S}, C, c)$

end for

$\theta \leftarrow \mathcal{U}(\mathcal{B}, \theta)$

end for

Using Algorithm 4

Algorithm 4 Collect T -step rollouts with prioritized level replay

Input: Actor index k , batch environments E , training levels Λ_{train} , visited levels Λ_{seen} , current level l , policy π_θ , rollout length T , scoring function **score**, level scores S , partial scores $\tilde{S}$, staleness values C , and global episode count c .

Output: Experience buffer $\mathcal{B}$

Initialize $\mathcal{B} = \emptyset$, and set current level $l_i = E_k$

Observe current state s_0 , termination flag d_0

if d_0 **then**

 Define new index $i \leftarrow |S| + 1$

 Choose current level $l_i \leftarrow \text{sampleNextLevel}(\Lambda_{\text{train}}, S, C, c)$ and $E_k \leftarrow l_i$

 Update level timestamp $C_i \leftarrow c$

 Observe initial state s_0

end if

Choose $a_0 \sim \pi_\theta(\cdot | s_0)$

$t = 1$

Initialize episodic trajectory buffer $\tau = \emptyset$

while $t < T$ **do**

 Observe (s_t, r_t, d_t)

$\mathcal{B} \leftarrow \mathcal{B} \cup (s_{t-1}, a_{t-1}, s_t, r_t, d_t, \log \pi_\theta(a))$

$\tau \leftarrow \tau \cup (s_{t-1}, a_{t-1}, s_t, r_t, d_t, \log \pi_\theta(a))$

if d_t **then**

 Update level score $S_i \leftarrow \text{score}(\tau, \pi_\theta, \tilde{S}_i)$ and partial score $\tilde{S}_i \leftarrow 0$

$\tau \leftarrow \emptyset$

 Define new index $i \leftarrow |S| + 1$

 Update current level $l_i \leftarrow \text{sampleNextLevel}(\Lambda_{\text{train}}, S, C, c)$ and $E_k \leftarrow l_i$

 Update level timestamp $C_i \leftarrow c$

end if

 Choose $a_{t+1} \sim \pi_\theta(\cdot | s_t)$

$t \leftarrow t + 1$

end while

if not d_t **then**

$\tilde{S}_i \leftarrow (\text{score}(\tau, \pi_\theta), |\tau|)$

Track partial time-averaged score and $|\tau|$

end if

function $\text{sampleNextLevel}(\Lambda_{\text{train}}, S, C, c)$

$c \leftarrow c + 1$

 Sample replay decision $d \sim P_D(d)$

if $d = 0$ **and** $|\Lambda_{\text{train}} - \Lambda_{\text{seen}}| > 0$ **then**

 Define new index $i \leftarrow |S| + 1$

 Sample $l_i \sim P_{\text{new}}(l | \Lambda_{\text{train}}, \Lambda_{\text{seen}})$

Sample an unseen level, if any

 Add l_i to Λ_{seen} , add initial value $S_i = 0$ to S and $C_i = 0$ to C

else

 Sample $l_i \sim (1 \cup \rho) \cdot P_S(l | \Lambda_{\text{seen}}, S) + \rho \cdot P_C(l | \Lambda_{\text{seen}}, C, c)$

Sample a level for replay

end if

return l_i

end function

Rainbow: Combining Improvements in Deep Reinforcement Learning

Matteo Hessel
DeepMind

Joseph Modayil
DeepMind

Hado van Hasselt
DeepMind

Tom Schaul
DeepMind

Georg Ostrovski
DeepMind

Will Dabney
DeepMind

Dan Horgan
DeepMind

Bilal Piot
DeepMind

Mohammad Azar
DeepMind

David Silver
DeepMind

Abstract

The deep reinforcement learning community has made several independent improvements to the DQN algorithm. However, it is unclear which of these extensions are complementary and can be fruitfully combined. This paper examines six extensions to the DQN algorithm and empirically studies their combination. Our experiments show that the combination provides state-of-the-art performance on the Atari 2600 benchmark, both in terms of data efficiency and final performance. We also provide results from a detailed ablation study that shows the contribution of each component to overall performance.

Introduction

The many recent successes in scaling reinforcement learning (RL) to complex sequential decision-making problems were kick-started by the Deep Q-Networks algorithm (DQN; Mnih et al. 2013, 2015). Its combination of Q-learning with convolutional neural networks and experience replay enabled it to learn, from raw pixels, how to play many Atari games at human-level performance. Since then, many extensions have been proposed that enhance its speed or stability.

Double DQN (DDQN; van Hasselt, Guez, and Silver 2016) addresses an overestimation bias of Q-learning (van Hasselt 2010), by decoupling selection and evaluation of the bootstrap action. Prioritized experience replay (Schaul et al. 2015) improves data efficiency, by replaying more often transitions from which there is more to learn. The dueling network architecture (Wang et al. 2016) helps to generalize across actions by separately representing state values and action advantages. Learning from multi-step bootstrap targets (Sutton 1988; Sutton and Barto 1998), as used in A3C (Mnih et al. 2016), shifts the bias-variance trade-off and helps to propagate newly observed rewards faster to earlier visited states. Distributional Q-learning (Bellemare, Dabney, and Munos 2017) learns a categorical distribution of discounted returns, instead of estimating the mean. Noisy DQN (Fortunato et al. 2017) uses stochastic network layers for exploration. This list is, of course, far from exhaustive.

Each of these algorithms enables substantial performance improvements in isolation. Since they do so by addressing

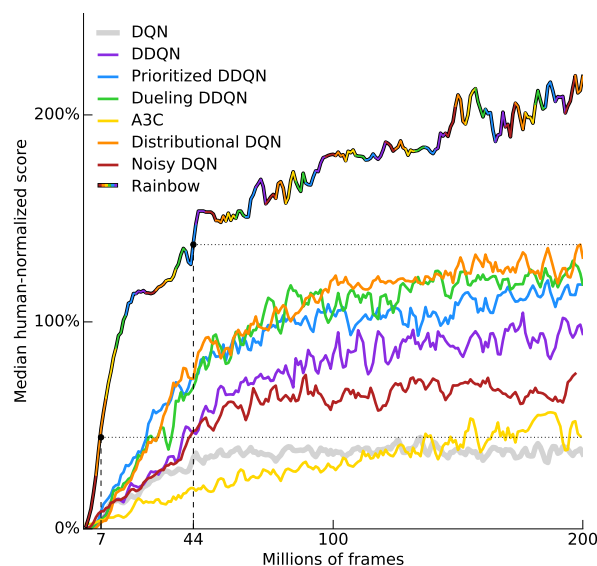


Figure 1: **Median human-normalized performance** across 57 Atari games. We compare our integrated agent (rainbow-colored) to DQN (grey) and six published baselines. Note that we match DQN’s best performance after 7M frames, surpass any baseline within 44M frames, and reach substantially improved final performance. Curves are smoothed with a moving average over 5 points.

radically different issues, and since they build on a shared framework, they could plausibly be combined. In some cases this has been done: Prioritized DDQN and Dueling DDQN both use double Q-learning, and Dueling DDQN was also combined with prioritized experience replay. In this paper we propose to study an agent that combines all the aforementioned ingredients. We show how these different ideas can be integrated, and that they are indeed largely complementary. In fact, their combination results in new state-of-the-art results on the benchmark suite of 57 Atari 2600 games from the Arcade Learning Environment (Bellemare et al. 2013), both in terms of data efficiency and of final performance. Finally we show results from ablation studies to help understand the contributions of the different components.

Background

Reinforcement learning addresses the problem of an *agent* learning to act in an *environment* in order to maximize a scalar *reward* signal. No direct supervision is provided to the agent, for instance it is never directly told the best action.

Agents and environments. At each discrete time step $t = 0, 1, 2, \dots$, the environment provides the agent with an observation S_t , the agent responds by selecting an action A_t , and then the environment provides the next reward R_{t+1} , discount γ_{t+1} , and state S_{t+1} . This interaction is formalized as a *Markov Decision Process*, or MDP, which is a tuple $\langle S, \mathcal{A}, T, r, \gamma \rangle$, where S is a finite set of states, $\mathcal{A}$ is a finite set of actions, $T(s, a, s') = P[S_{t+1} = s' | S_t = s, A_t = a]$ is the (stochastic) transition function, $r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor. In our experiments MDPs will be *episodic* with a constant $\gamma_t = \gamma$, except on episode termination where $\gamma_t = 0$, but the algorithms are expressed in the general form.

On the agent side, action selection is given by a policy π that defines a probability distribution over actions for each state. From the state S_t encountered at time t , we define the discounted return $G_t = \sum_{k=0}^{\infty} \gamma_t^{(k)} R_{t+k+1}$ as the discounted sum of future rewards collected by the agent, where the discount for a reward k steps in the future is given by the product of discounts before that time, $\gamma_t^{(k)} = \prod_{i=1}^k \gamma_{t+i}$. An agent aims to maximize the expected discounted return by finding a good policy.

The policy may be learned directly, or it may be constructed as a function of some other learned quantities. In value-based reinforcement learning, the agent learns an estimate of the expected discounted return, or value, when following a policy π starting from a given state, $v^\pi(s) = E_\pi[G_t | S_t = s]$, or state-action pair, $q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$. A common way of deriving a new policy from a state-action value function is to act ϵ -greedily with respect to the action values. This corresponds to taking the action with the highest value (the *greedy* action) with probability $(1 - \epsilon)$, and to otherwise act uniformly at random with probability ϵ . Policies of this kind are used to introduce a form of *exploration*: by randomly selecting actions that are sub-optimal according to its current estimates, the agent can discover and correct its estimates when appropriate. The main limitation is that it is difficult to discover alternative courses of action that extend far into the future; this has motivated research on more directed forms of exploration.

Deep reinforcement learning and DQN. Large state and/or action spaces make it intractable to learn Q value estimates for each state and action pair independently. In deep reinforcement learning, we represent the various components of agents, such as policies $\pi(s, a)$ or values $q(s, a)$, with deep (i.e., multi-layer) neural networks. The parameters of these networks are trained by gradient descent to minimize some suitable loss function.

In DQN (Mnih et al. 2015) deep networks and reinforcement learning were successfully combined by using a convolutional neural net to approximate the action values for a

given state S_t (which is fed as input to the network in the form of a stack of raw pixel frames). At each step, based on the current state, the agent selects an action ϵ -greedily with respect to the action values, and adds a transition $(S_t, A_t, R_{t+1}, \gamma_{t+1}, S_{t+1})$ to a replay memory buffer (Lin 1992), that holds the last million transitions. The parameters of the neural network are optimized by using stochastic gradient descent to minimize the loss

$$(R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t))^2, \quad (1)$$

where t is a time step randomly picked from the replay memory. The gradient of the loss is back-propagated only into the parameters θ of the *online network* (which is also used to select actions); the term $\bar{\theta}$ represents the parameters of a *target network*; a periodic copy of the online network which is not directly optimized. The optimization is performed using RMSprop (Tieleman and Hinton 2012), a variant of stochastic gradient descent, on mini-batches sampled uniformly from the experience replay. This means that in the loss above, the time index t will be a random time index from the last million transitions, rather than the current time. The use of experience replay and target networks enables relatively stable learning of Q values, and led to super-human performance on several Atari games.

Extensions to DQN

DQN has been an important milestone, but several limitations of this algorithm are now known, and many extensions have been proposed. We propose a selection of six extensions that each have addressed a limitation and improved overall performance. To keep the size of the selection manageable, we picked a set of extensions that address distinct concerns (e.g., just one of the many addressing exploration).

Double Q-learning. Conventional Q-learning is affected by an overestimation bias, due to the maximization step in Equation 1, and this can harm learning. Double Q-learning (van Hasselt 2010), addresses this overestimation by decoupling, in the maximization performed for the bootstrap target, the selection of the action from its evaluation. It is possible to effectively combine this with DQN (van Hasselt, Guez, and Silver 2016), using the loss

$$(R_{t+1} + \gamma_{t+1} q_{\bar{\theta}}(S_{t+1}, \arg\max_{a'} q_{\theta}(S_{t+1}, a')) - q_{\theta}(S_t, A_t))^2.$$

This change was shown to reduce harmful overestimations that were present for DQN, thereby improving performance.

Prioritized replay. DQN samples uniformly from the replay buffer. Ideally, we want to sample more frequently those transitions from which there is much to learn. As a proxy for learning potential, prioritized experience replay (Schaul et al. 2015) samples transitions with probability p_t relative to the last encountered absolute *TD error*:

$$p_t \propto |R_{t+1} + \gamma_{t+1} \max_{a'} q_{\bar{\theta}}(S_{t+1}, a') - q_{\theta}(S_t, A_t)|^\omega,$$

where ω is a hyper-parameter that determines the shape of the distribution. New transitions are inserted into the replay

buffer with maximum priority, providing a bias towards recent transitions. Note that stochastic transitions might also be favoured, even when there is little left to learn about them.

Dueling networks. The dueling network is a neural network architecture designed for value based RL. It features two streams of computation, the value and advantage streams, sharing a convolutional encoder, and merged by a special aggregator (Wang et al. 2016). This corresponds to the following factorization of action values:

$$q_\theta(s, a) = v_\eta(f_\xi(s)) + a_\psi(f_\xi(s), a) - \frac{\sum_{a'} a_\psi(f_\xi(s), a')}{N_{\text{actions}}},$$

where ξ , η , and ψ are, respectively, the parameters of the shared encoder f_ξ , of the value stream v_η , and of the advantage stream a_ψ ; and $\theta = \{\xi, \eta, \psi\}$ is their concatenation.

Multi-step learning. Q-learning accumulates a single reward and then uses the greedy action at the next step to bootstrap. Alternatively, forward-view *multi-step* targets can be used (Sutton 1988). We define the truncated n -step return from a given state S_t as

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma_t^{(k)} R_{t+k+1}. \quad (2)$$

A multi-step variant of DQN is then defined by minimizing the alternative loss,

$$(R_t^{(n)} + \gamma_t^{(n)} \max_{a'} q_\theta(S_{t+n}, a') - q_\theta(S_t, A_t))^2.$$

Multi-step targets with suitably tuned n often lead to faster learning (Sutton and Barto 1998).

Distributional RL. We can learn to approximate the distribution of returns instead of the expected return. Recently Bellemare, Dabney, and Munos (2017) proposed to model such distributions with probability masses placed on a discrete support $\mathbf{z}$, where $\mathbf{z}$ is a vector with $N_{\text{atoms}} \in \mathbb{N}^+$ atoms, defined by $z^i = v_{\min} + (i - 1) \frac{v_{\max} - v_{\min}}{N_{\text{atoms}} - 1}$ for $i \in \{1, \dots, N_{\text{atoms}}\}$. The approximating distribution d_t at time t is defined on this support, with the probability mass $p_\theta^i(S_t, A_t)$ on each atom i , such that $d_t = (\mathbf{z}, \mathbf{p}_\theta(S_t, A_t))$. The goal is to update θ such that this distribution closely matches the actual distribution of returns.

To learn the probability masses, the key insight is that return distributions satisfy a variant of Bellman’s equation. For a given state S_t and action A_t , the distribution of the returns under the optimal policy π^* should match a target distribution defined by taking the distribution for the next state S_{t+1} and action $a_{t+1}^* = \pi^*(S_{t+1})$, contracting it towards zero according to the discount, and shifting it by the reward (or distribution of rewards, in the stochastic case). A distributional variant of Q-learning is then derived by first constructing a new support for the target distribution, and then minimizing the Kullbeck-Leibler divergence between the distribution d_t and the target distribution $d_t' \equiv (R_{t+1} + \gamma_{t+1} \mathbf{z}, \mathbf{p}_\theta(S_{t+1}, a_{t+1}^*))$,

$$D_{\text{KL}}(\Phi_{\mathbf{z}} d_t' || d_t). \quad (3)$$

Here $\Phi_{\mathbf{z}}$ is a L2-projection of the target distribution onto the fixed support $\mathbf{z}$, and $\bar{a}_{t+1}^* = \arg\max_a q_\theta(S_{t+1}, a)$ is the greedy action with respect to the mean action values $q_\theta(S_{t+1}, a) = \mathbf{z}^\top \mathbf{p}_\theta(S_{t+1}, a)$ in state S_{t+1} .

As in the non-distributional case, we can use a frozen copy of the parameters $\bar{\theta}$ to construct the target distribution. The parametrized distribution can be represented by a neural network, as in DQN, but with $N_{\text{atoms}} \times N_{\text{actions}}$ outputs. A *softmax* is applied independently for each action dimension of the output to ensure that the distribution for each action is appropriately normalized.

Noisy Nets. The limitations of exploring using ϵ -greedy policies are clear in games such as Montezuma’s Revenge, where many actions must be executed to collect the first reward. Noisy Nets (Fortunato et al. 2017) propose a noisy linear layer that combines a deterministic and noisy stream,

$$\mathbf{y} = (\mathbf{b} + \mathbf{W}\mathbf{x}) + (\mathbf{b}_{\text{noisy}} \odot \epsilon^b + (\mathbf{W}_{\text{noisy}} \odot \epsilon^w) \mathbf{x}), \quad (4)$$

where ϵ^b and ϵ^w are random variables, and $\odot$ denotes the element-wise product. This transformation can then be used in place of the standard linear $\mathbf{y} = \mathbf{b} + \mathbf{W}\mathbf{x}$. Over time, the network can learn to ignore the noisy stream, but will do so at different rates in different parts of the state space, allowing state-conditional exploration with a form of self-annealing.

The Integrated Agent

In this paper we integrate all the aforementioned components into a single integrated agent, which we call *Rainbow*.

First, we replace the 1-step distributional loss (3) with a multi-step variant. We construct the target distribution by contracting the value distribution in S_{t+n} according to the cumulative discount, and shifting it by the truncated n -step discounted return. This corresponds to defining the target distribution as $d_t^{(n)} = (R_t^{(n)} + \gamma_t^{(n)} \mathbf{z}, \mathbf{p}_\theta(S_{t+n}, a_{t+n}^*))$. The resulting loss is

$$D_{\text{KL}}(\Phi_{\mathbf{z}} d_t^{(n)} || d_t),$$

where, again, $\Phi_{\mathbf{z}}$ is the projection onto $\mathbf{z}$.

We combine the multi-step distributional loss with double Q-learning by using the greedy action in S_{t+n} selected according to the *online network* as the bootstrap action a_{t+n}^* , and evaluating such action using the *target network*.

In standard proportional prioritized replay (Schaul et al. 2015) the absolute TD error is used to prioritize the transitions. This can be computed in the distributional setting, using the mean action values. However, in our experiments all distributional Rainbow variants prioritize transitions by the KL loss, since this is what the algorithm is minimizing:

$$p_t \propto \left(D_{\text{KL}}(\Phi_{\mathbf{z}} d_t^{(n)} || d_t) \right)^\omega.$$

The KL loss as priority might be more robust to noisy stochastic environments because the loss can continue to decrease even when the returns are not deterministic.

The network architecture is a dueling network architecture adapted for use with return distributions. The network

has a shared representation $f_\xi(s)$, which is then fed into a value stream v_η with N_{atoms} outputs, and into an advantage stream a_ξ with $N_{\text{atoms}} \times N_{\text{actions}}$ outputs, where $a_\xi^i(f_\xi(s), a)$ will denote the output corresponding to atom i and action a . For each atom z^i , the value and advantage streams are aggregated, as in dueling DQN, and then passed through a softmax layer to obtain the normalised parametric distributions used to estimate the returns' distributions:

$$p_\theta^i(s, a) = \frac{\exp(v_\eta^i(\phi) + a_\psi^i(\phi, a) - \bar{a}_\psi^i(s))}{\sum_j \exp(v_\eta^j(\phi) + a_\psi^j(\phi, a) - \bar{a}_\psi^j(s))},$$

where $\phi = f_\xi(s)$ and $\bar{a}_\psi^i(s) = \frac{1}{N_{\text{actions}}} \sum_{a'} a_\psi^i(\phi, a')$.

We then replace all linear layers with their noisy equivalent described in Equation (4). Within these noisy linear layers we use factorised Gaussian noise (Fortunato et al. 2017) to reduce the number of independent noise variables.

Experimental Methods

We now describe the methods and setup used for configuring and evaluating the learning agents.

Evaluation Methodology. We evaluated all agents on 57 Atari 2600 games from the arcade learning environment (Bellemare et al. 2013). We follow the training and evaluation procedures of Mnih et al. (2015) and van Hasselt et al. (2016). The average scores of the agent are evaluated during training, every 1M steps in the environment, by suspending learning and evaluating the latest agent for 500K frames. Episodes are truncated at 108K frames (or 30 minutes of simulated play), as in van Hasselt et al. (2016).

Agents' scores are normalized, per game, so that 0% corresponds to a random agent and 100% to the average score of a human expert. Normalized scores can be aggregated across all Atari levels to compare the performance of different agents. It is common to track the *median* human normalized performance across all games. We also consider the number of games where the agent's performance is above some fraction of human performance, to disentangle where improvements in the median come from. The *mean* human normalized performance is potentially less informative, as it is dominated by a few games (e.g., Atlantis) where agents achieve scores orders of magnitude higher than humans do.

Besides tracking the median performance as a function of environment steps, at the end of training we re-evaluate the best agent snapshot using two different testing regimes. In the *no-ops starts* regime, we insert a random number (up to 30) of no-op actions at the beginning of each episode (as we do also in training). In the *human starts* regime, episodes are initialized with points randomly sampled from the initial portion of human expert trajectories (Nair et al. 2015); the difference between the two regimes indicates the extent to which the agent has over-fit to its own trajectories.

Due to space constraints, we focus on aggregate results across games. However, in the appendix we provide full learning curves for all games and all agents, as well as detailed comparison tables of raw and normalized scores, in both the no-op and human starts testing regimes.

Hyper-parameter tuning. All Rainbow's components have a number of hyper-parameters. The combinatorial space of hyper-parameters is too large for an exhaustive search, therefore we have performed limited tuning. For each component, we started with the values used in the paper that introduced this component, and tuned the most sensitive among hyper-parameters by manual coordinate descent.

DQN and its variants do not perform learning updates during the first 200K frames, to ensure sufficiently uncorrelated updates. We have found that, with prioritized replay, it is possible to start learning sooner, after only 80K frames.

DQN starts with an exploration ϵ of 1, corresponding to acting uniformly at random; it anneals the amount of exploration over the first 4M frames, to a final value of 0.1 (lowered to 0.01 in later variants). Whenever using Noisy Nets, we acted fully greedily ($\epsilon = 0$), with a value of 0.5 for the σ_0 hyper-parameter used to initialize the weights in the noisy stream¹. For agents without Noisy Nets, we used ϵ -greedy but decreased the exploration rate faster than was previously used, annealing ϵ to 0.01 in the first 250K frames.

We used the Adam optimizer (Kingma and Ba 2014), which we found less sensitive to the choice of the learning rate than RMSProp. DQN uses a learning rate of $\alpha = 0.00025$. In all Rainbow's variants we used a learning rate of $\alpha/4$, selected among $\{\alpha/2, \alpha/4, \alpha/6\}$, and a value of 1.5×10^{-4} for Adam's ϵ hyper-parameter.

For replay prioritization we used the recommended proportional variant, with priority exponent ω of 0.5, and linearly increased the importance sampling exponent β from 0.4 to 1 over the course of training. The priority exponent ω was tuned comparing values of $\{0.4, 0.5, 0.7\}$. Using the KL loss of distributional DQN as priority, we have observed that performance is very robust to the choice of ω .

The value of n in multi-step learning is a sensitive hyper-parameter of Rainbow. We compared values of $n = 1, 3$, and 5. We observed that both $n = 3$ and 5 did well initially, but overall $n = 3$ performed the best by the end.

The hyper-parameters (see Table 1) are identical across all 57 games, i.e., the Rainbow agent really is a *single* agent setup that performs well across all the games.

¹The noise was generated on the GPU. Tensorflow noise generation can be unreliable on GPU. If generating the noise on the CPU, lowering σ_0 to 0.1 may be helpful.

Parameter	Value
Min history to start learning	80K frames
Adam learning rate	0.0000625
Exploration ϵ	0.0
Noisy Nets σ_0	0.5
Target Network Period	32K frames
Adam ϵ	1.5×10^{-4}
Prioritization type	proportional
Prioritization exponent ω	0.5
Prioritization importance sampling β	0.4 $\rightarrow$ 1.0
Multi-step returns n	3
Distributional atoms	51
Distributional min/max values	$[-10, 10]$

Table 1: Rainbow hyper-parameters

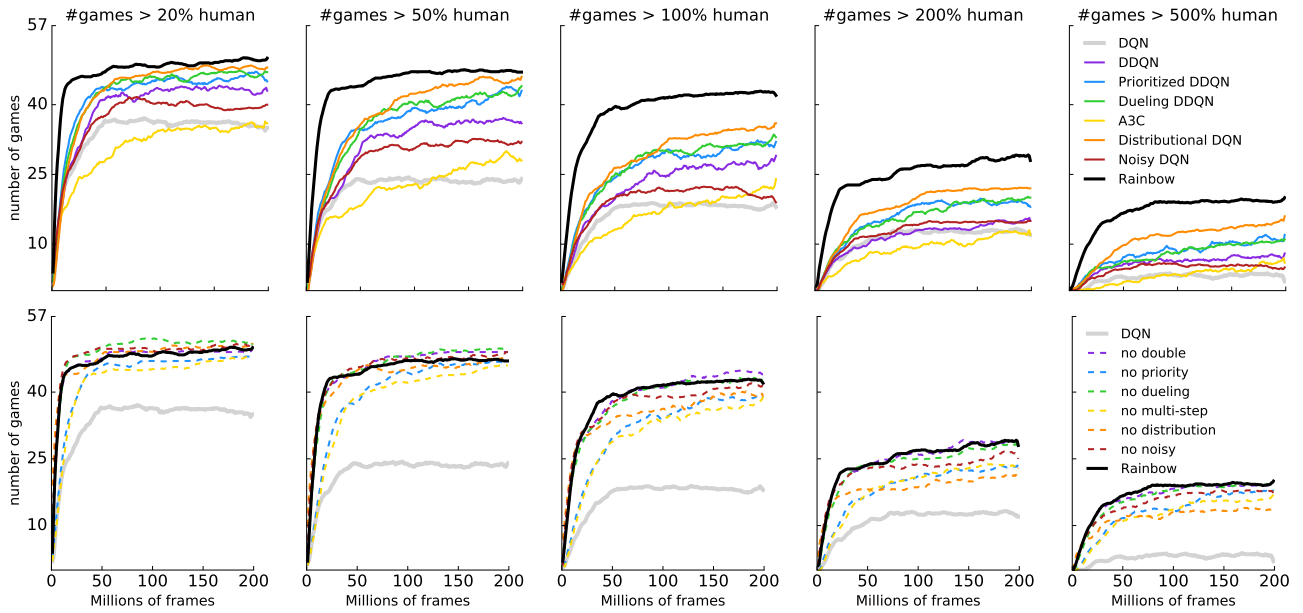


Figure 2: Each plot shows, for several agents, the number of games where they have achieved at least a given fraction of human performance, as a function of time. From left to right we consider the 20%, 50%, 100%, 200% and 500% thresholds. On the first row we compare Rainbow to the baselines. On the second row we compare Rainbow to its ablations.

Analysis

In this section we analyse the main experimental results. First, we show that Rainbow compares favorably to several published agents. Then we perform ablation studies, comparing several variants of the agent, each corresponding to removing a single component from Rainbow.

Comparison to published baselines. In Figure 1 we compare the Rainbow’s performance (measured in terms of the median human normalized score across games) to the corresponding curves for A3C, DQN, DDQN, Prioritized DDQN, Dueling DDQN, Distributional DQN, and Noisy DQN. We thank the authors of the Dueling and Prioritized agents for providing the learning curves of these, and report our own re-runs for DQN, A3C, DDQN, Distributional DQN and Noisy DQN. The performance of Rainbow is significantly better than any of the baselines, both in data efficiency, as well as in final performance. Note that we match final performance of DQN after 7M frames, surpass the best final performance of these baselines in 44M frames, and reach substantially improved final performance.

In the final evaluations of the agent, after the end of training, Rainbow achieves a median score of 223% in the no-ops regime; in the human starts regime we measured a median score of 153%. In Table 2 we compare these scores to the published median scores of the individual baselines.

In Figure 2 (top row) we plot the number of games where an agent has reached some specified level of human normalized performance. From left to right, the subplots show on how many games the different agents have achieved 20%, 50%, 100%, 200% and 500% human normalized perfor-

mance. This allows us to identify where the overall improvements in performance come from. Note that the gap in performance between Rainbow and other agents is apparent at all levels of performance: the Rainbow agent is improving scores on games where the baseline agents were already good, as well as improving in games where baseline agents are still far from human performance.

Learning speed. As in the original DQN setup, we ran each agent on a single GPU. The 7M frames required to match DQN’s final performance correspond to less than 10 hours of wall-clock time. A full run of 200M frames corresponds to approximately 10 days, and this varies by less than 20% between all of the discussed variants. The litera-

Agent	no-ops	human starts
DQN	79%	68%
DDQN (*)	117%	110%
Prioritized DDQN (*)	140%	128%
Dueling DDQN (*)	151%	117%
A3C (*)	-	116%
Noisy DQN	118%	102%
Distributional DQN	164%	125%
Rainbow	223%	153%

Table 2: Median normalized scores of the best agent snapshots for Rainbow and baselines. For methods marked with an asterisk, the scores come from the corresponding publication. DQN’s scores comes from the dueling networks paper, since DQN’s paper did not report scores for all 57 games. The others scores come from our own implementations.

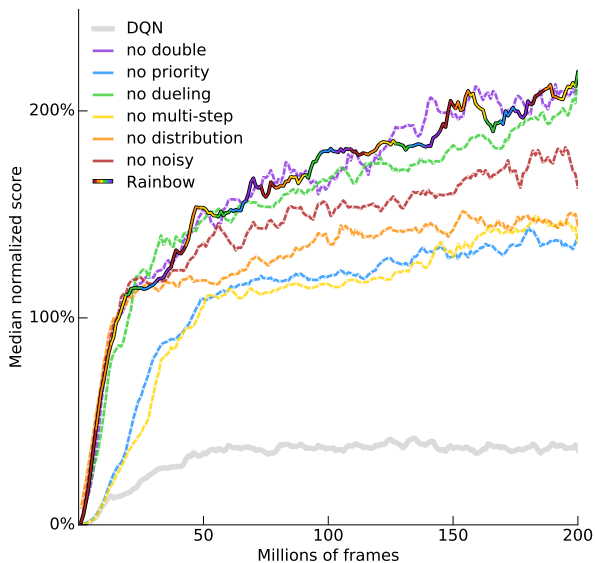


Figure 3: **Median human-normalized performance** across 57 Atari games, as a function of time. We compare our integrated agent (rainbow-colored) to DQN (gray) and to six different ablations (dashed lines). Curves are smoothed with a moving average over 5 points.

ture contains many alternative training setups that improve performance as a function of wall-clock time by exploiting parallelism, e.g., Nair et al. (2015), Salimans et al. (2017), and Mnih et al. (2016). Properly relating the performance across such very different hardware/compute resources is non-trivial, so we focused exclusively on algorithmic variations, allowing apples-to-apples comparisons. While we consider them to be important and complementary, we leave questions of scalability and parallelism to future work.

Ablation studies. Since Rainbow integrates several different ideas into a single agent, we conducted additional experiments to understand the contribution of the various components, in the context of this specific combination.

To gain a better understanding of the contribution of each component to the Rainbow agent, we performed ablation studies. In each ablation, we removed one component from the full Rainbow combination. Figure 3 shows a comparison for median normalized score of the full Rainbow to six ablated variants. Figure 2 (bottom row) shows a more detailed breakdown of how these ablations perform relative to different thresholds of human normalized performance, and Figure 4 shows the gain or loss from each ablation for every game, averaged over the full learning run.

Prioritized replay and multi-step learning were the two most crucial components of Rainbow, in that removing either component caused a large drop in median performance. Unsurprisingly, the removal of either of these hurt early performance. Perhaps more surprisingly, the removal of multi-step learning also hurt final performance. Zooming in on individual games (Figure 4), we see both components helped

almost uniformly across games (the full Rainbow performed better than either ablation in 53 games out of 57).

Distributional Q-learning ranked immediately below the previous techniques for relevance to the agent’s performance. Notably, in early learning no difference is apparent, as shown in Figure 3, where for the first 40 million frames the distributional-ablation performed as well as the full agent. However, without distributions, the performance of the agent then started lagging behind. When the results are separated relatively to human performance in Figure 2, we see that the distributional-ablation primarily seems to lag on games that are above human level or near it.

In terms of median performance, the agent performed better when Noisy Nets were included; when these are removed and exploration is delegated to the traditional ϵ -greedy mechanism, performance was worse in aggregate (red line in Figure 3). While the removal of Noisy Nets produced a large drop in performance for several games, it also provided small increases in other games (Figure 4).

In aggregate, we did not observe a significant difference when removing the dueling network from the full Rainbow. The median score, however, hides the fact that the impact of Dueling differed between games, as shown by Figure 4. Figure 2 shows that Dueling perhaps provided some improvement on games with above-human performance levels (# games > 200%), and some degradation on games with sub-human performance (# games > 20%).

Also in the case of double Q-learning, the observed difference in median performance (Figure 3) is limited, with the component sometimes harming or helping depending on the game (Figure 4). To further investigate the role of double Q-learning, we compared the predictions of our trained agents to the actual discounted returns computed from clipped rewards. Comparing Rainbow to the agent where double Q-learning was ablated, we observed that the actual returns are often higher than 10 and therefore fall outside the support of the distribution, spanning from -10 to $+10$. This leads to underestimated returns, rather than overestimations. We hypothesize that clipping the values to this constrained range counteracts the overestimation bias of Q-learning. Note, however, that the importance of double Q-learning may increase if the support of the distributions is expanded.

In the appendix, for each game we show final performance and learning curves for Rainbow, its ablations, and baselines.

Discussion

We have demonstrated that several improvements to DQN can be successfully integrated into a single learning algorithm that achieves state-of-the-art performance. Moreover, we have shown that within the integrated algorithm, all but one of the components provided clear performance benefits. There are many more algorithmic components that we were not able to include, which would be promising candidates for further experiments on integrated agents. Among the many possible candidates, we discuss several below.

We have focused here on value-based methods in the Q-learning family. We have not considered purely policy-based RL algorithms such as trust-region policy optimisa-

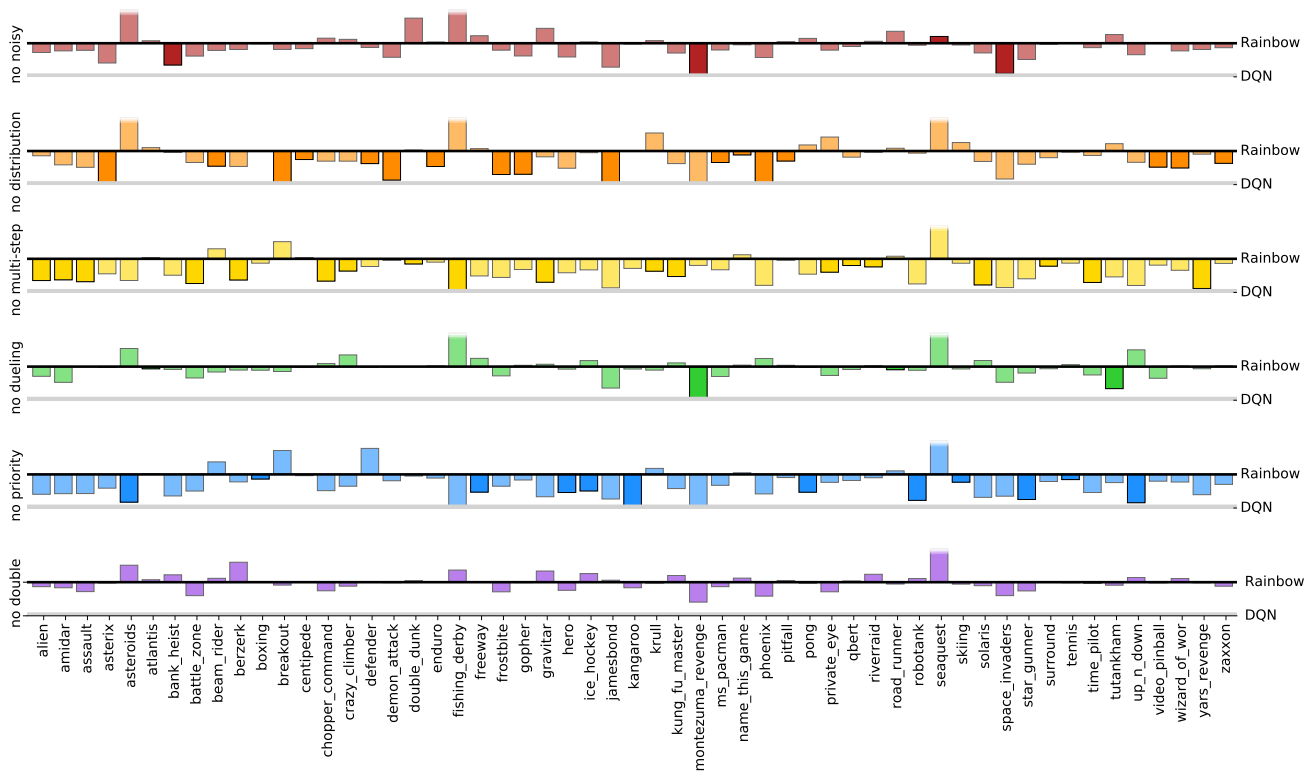


Figure 4: **Performance drops of ablation agents** on all 57 Atari games. Performance is the area under the learning curve, normalized relative to the Rainbow agent and DQN. Two games where DQN outperforms Rainbow are omitted. The ablation leading to the strongest drop is highlighted for each game. The removal of either prioritization or multi-step learning reduces performance across most games, but the contribution of each component varies substantially per game.

tion (Schulman et al. 2015), nor actor-critic methods (Mnih et al. 2016; O’Donoghue et al. 2016).

A number of algorithms exploit a sequence of data to achieve improved learning efficiency. Optimality tightening (He et al. 2016) uses multi-step returns to construct additional inequality bounds, instead of using them to replace the 1-step targets used in Q-learning. Eligibility traces allow a soft combination over n -step returns (Sutton 1988). However, sequential methods all leverage more computation per gradient than the multi-step targets used in Rainbow. Furthermore, introducing prioritized sequence replay raises questions of how to store, replay and prioritise sequences.

Episodic control (Blundell et al. 2016) also focuses on data efficiency, and was shown to be very effective in some domains. It improves early learning by using episodic memory as a complementary learning system, capable of immediately re-enacting successful action sequences.

Besides Noisy Nets, numerous other exploration methods could also be useful algorithmic ingredients: among these Bootstrapped DQN (Osband et al. 2016), intrinsic motivation (Stadie, Levine, and Abbeel 2015) and count-based exploration (Bellemare et al. 2016). Integration of these alternative components is fruitful subject for further research.

In this paper we have focused on the core learning up-

dates, without exploring alternative computational architectures. Asynchronous learning from parallel copies of the environment, as in A3C (Mnih et al. 2016), Gorila (Nair et al. 2015), or Evolution Strategies (Salimans et al. 2017), can be effective in speeding up learning, at least in terms of wall-clock time. Note, however, they can be less data efficient.

Hierarchical RL has also been applied with success to several complex Atari games. Among successful applications of HRL we highlight h-DQN (Kulkarni et al. 2016a) and Feudal Networks (Vezhnevets et al. 2017).

The state representation could also be made more efficient by exploiting auxiliary tasks such as pixel control or feature control (Jaderberg et al. 2016), supervised predictions (Dosovitskiy and Koltun 2016) or successor features (Kulkarni et al. 2016b).

To evaluate Rainbow fairly against the baselines, we have followed the common domain modifications of clipping rewards, fixed action-repetition, and frame-stacking, but these might be removed by other learning algorithm improvements. Pop-Art normalization (van Hasselt et al. 2016) allows reward clipping to be removed, while preserving a similar level of performance. Fine-grained action repetition (Sharma, Lakshminarayanan, and Ravindran 2017) enabled to learn how to repeat actions. A recurrent state network

(Hausknecht and Stone 2015) can learn a temporal state representation, replacing the fixed stack of observation frames. In general, we believe that exposing the real game to the agent is a promising direction for future research.

References

- Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res. (JAIR)* 47:253–279.
- Bellemare, M. G.; Srinivasan, S.; Ostrovski, G.; Schaul, T.; Saxton, D.; and Munos, R. 2016. Unifying count-based exploration and intrinsic motivation. In *NIPS*.
- Bellemare, M. G.; Dabney, W.; and Munos, R. 2017. A distributional perspective on reinforcement learning. In *ICML*.
- Blundell, C.; Uria, B.; Pritzel, A.; Li, Y.; Ruderman, A.; Leibo, J. Z.; Rae, J.; Wierstra, D.; and Hassabis, D. 2016. Model-Free Episodic Control. *ArXiv e-prints*.
- Dosovitskiy, A., and Koltun, V. 2016. Learning to act by predicting the future. *CoRR* abs/1611.01779.
- Fortunato, M.; Azar, M. G.; Piot, B.; Menick, J.; Osband, I.; Graves, A.; Mnih, V.; Munos, R.; Hassabis, D.; Pietquin, O.; Blundell, C.; and Legg, S. 2017. Noisy networks for exploration. *CoRR* abs/1706.10295.
- Hausknecht, M., and Stone, P. 2015. Deep recurrent Q-learning for partially observable MDPs. *arXiv preprint arXiv:1507.06527*.
- He, F. S.; Liu, Y.; Schwing, A. G.; and Peng, J. 2016. Learning to play in a day: Faster deep reinforcement learning by optimality tightening. *CoRR* abs/1611.01606.
- Jaderberg, M.; Mnih, V.; Czarnecki, W. M.; Schaul, T.; Leibo, J. Z.; Silver, D.; and Kavukcuoglu, K. 2016. Reinforcement learning with unsupervised auxiliary tasks. *CoRR* abs/1611.05397.
- Kingma, D. P., and Ba, J. 2014. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Kulkarni, T. D.; Narasimhan, K.; Saeedi, A.; and Tenenbaum, J. B. 2016a. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. *CoRR* abs/1604.06057.
- Kulkarni, T. D.; Saeedi, A.; Gautam, S.; and Gershman, S. J. 2016b. Deep successor reinforcement learning. *arXiv preprint arXiv:1606.02396*.
- Lin, L.-J. 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* 8(3):293–321.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. A. 2013. Playing atari with deep reinforcement learning. *CoRR* abs/1312.5602.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*.
- Nair, A.; Srinivasan, P.; Blackwell, S.; Alcicek, C.; Fearon, R.; De Maria, A.; Panneershelvam, V.; Suleyman, M.; Beattie, C.; Petersen, S.; Legg, S.; Mnih, V.; Kavukcuoglu, K.; and Silver, D. 2015. Massively parallel methods for deep reinforcement learning. *arXiv preprint arXiv:1507.04296*.
- O’Donoghue, B.; Munos, R.; Kavukcuoglu, K.; and Mnih, V. 2016. Pq: Combining policy gradient and q-learning. *CoRR* abs/1611.01626.
- Osband, I.; Blundell, C.; Pritzel, A.; and Roy, B. V. 2016. Deep exploration via bootstrapped dqn. In *NIPS*.
- Salimans, T.; Ho, J.; Chen, X.; and Sutskever, I. 2017. Evolution strategies as a scalable alternative to reinforcement learning. *CoRR* abs/1703.03864.
- Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2015. Prioritized experience replay. In *Proc. of ICLR*.
- Schulman, J.; Levine, S.; Moritz, P.; Jordan, M.; and Abbeel, P. 2015. Trust region policy optimization. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, ICML’15*, 1889–1897. JMLR.org.
- Sharma, S.; Lakshminarayanan, A. S.; and Ravindran, B. 2017. Learning to repeat: Fine grained action repetition for deep reinforcement learning. *arXiv preprint arXiv:1702.06054*.
- Stadie, B. C.; Levine, S.; and Abbeel, P. 2015. Incentivizing exploration in reinforcement learning with deep predictive models. *CoRR* abs/1507.00814.
- Sutton, R. S., and Barto, A. G. 1998. *Reinforcement Learning: An Introduction*. The MIT press, Cambridge MA.
- Sutton, R. S. 1988. Learning to predict by the methods of temporal differences. *Machine learning* 3(1):9–44.
- Tieleman, T., and Hinton, G. 2012. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning* 4(2):26–31.
- van Hasselt, H.; Guez, A.; Guez, A.; Hessel, M.; Mnih, V.; and Silver, D. 2016. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems* 29, 4287–4295.
- van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double Q-learning. In *Proc. of AAAI*, 2094–2100.
- van Hasselt, H. 2010. Double Q-learning. In *Advances in Neural Information Processing Systems* 23, 2613–2621.
- Vezhnevets, A. S.; Osindero, S.; Schaul, T.; Heess, N.; Jaderberg, M.; Silver, D.; and Kavukcuoglu, K. 2017. Feudal networks for hierarchical reinforcement learning. *CoRR* abs/1703.01161.

Appendix

Table 3 lists the preprocessing of environment frames, rewards and discounts introduced by DQN. Table 4 lists the additional hyper-parameters that Rainbow inherits from DQN and the other baselines considered in this paper. The hyper-parameters for which Rainbow uses non standard settings are instead listed in the main text. In the subsequent pages, we list the tables showing, for each game, the score achieved by Rainbow and several baselines in both the no-ops regime (Table 6) and the human-starts regime (Table 5). In Figures 5 and 6 we also plot, for each game, the learning curves of Rainbow, several baselines, and all ablation experiments. These learning curves are smoothed with a moving average over a window of 10.

Hyper-parameter	value
Grey-scaling	True
Observation down-sampling	(84, 84)
Frames stacked	4
Action repetitions	4
Reward clipping	[-1, 1]
Terminal on loss of life	True
Max frames per episode	108K

Table 3: **Preprocessing**: the values of these hyper-parameters are the same used by DQN and its variants. They are here listed for completeness. Observations are grey-scaled and rescaled to 84×84 pixels. 4 consecutive frames are concatenated as each state’s representation. Each action selected by the agent is repeated for 4 times. Rewards are clipped between $-1, +1$. In games where the player has multiple lives, transitions associated to the loss of a life are considered terminal. All episodes are capped after 108K frames.

Hyper-parameter	value
Q network: channels	32, 64, 64
Q network: filter size	$8 \times 8, 4 \times 4, 3 \times 3$
Q network: stride	4, 2, 1
Q network: hidden units	512
Q network: output units	Number of actions
Discount factor	0.99
Memory size	1M transitions
Replay period	every 4 agent steps
Minibatch size	32

Table 4: **Additional hyper-parameters**: the values of these hyper-parameters are the same used by DQN and its variants. The network has 3 convolutional layers: with 32, 64 and 64 channels. The layers use $8 \times 8, 4 \times 4, 3 \times 3$ filters with strides of 4, 2, 1, respectively. The value and advantage streams of the dueling architecture have both a hidden layer with 512 units. The output layer of the network has a number of units equal to the number of actions available in the game. We use a discount factor of 0.99, which is set to 0 on terminal transitions. We perform a learning update every 4 agent steps, using mini-batches of 32 transitions.

Game	DQN	A3C	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	634.0	518.4	1033.4	900.5	1,486.5	1,997.5	533.3	6,022.9
amidar	178.4	263.9	169.1	218.4	172.7	237.7	148.0	202.8
assault	3489.3	5474.9	6060.8	7,748.5	3,994.8	5,101.3	5,124.3	14,491.7
asterix	3170.5	22140.5	16837.0	31,907.5	15,840.0	395,599.5	8,277.3	280,114.0
asteroids	1458.7	4474.5	1193.2	1,654.0	2,035.4	2,071.7	4,078.1	2,249.4
atlantis	292491.0	911,091.0	319688.0	593,642.0	445,360.0	289,803.0	303,666.5	814,684.0
bank_heist	312.7	970.1	886.0	816.8	1,129.3	835.6	955.0	826.0
battle_zone	23750.0	12950.0	24740.0	29,100.0	31,320.0	32,250.0	26,985.0	52,040.0
beam_rider	9743.2	22707.9	17417.2	26,172.7	14,591.3	15,002.4	15,241.5	21,768.5
berzerk	493.4	817.9	1011.1	1,165.6	910.6	1,000.0	670.8	1,793.4
bowling	56.5	35.1	69.6	65.8	65.7	76.8	79.3	39.4
boxing	70.3	59.8	73.5	68.6	77.3	62.1	66.3	54.9
breakout	354.5	681.9	368.9	371.6	411.6	548.7	423.3	379.5
centipede	3973.9	3755.8	3853.5	3,421.9	4,881.0	7,476.9	4,214.4	7,160.9
chopper_command	5017.0	7021.0	3495.0	6,604.0	3,784.0	9,600.5	8,778.5	10,916.0
crazy_climber	98128.0	112646.0	113782.0	131,086.0	124,566.0	154,416.5	98,576.5	143,962.0
defender	15917.5	56533.0	27510.0	21,093.5	33,996.0	32,246.0	18,037.5	47,671.3
demon_attack	12550.7	113,308.4	69803.4	73,185.8	56,322.8	109,856.6	25,207.8	109,670.7
double_dunk	-6.0	-0.1	-0.3	2.7	-0.8	-3.7	-1.0	-0.6
enduro	626.7	-82.5	1216.6	1,884.4	2,077.4	2,133.4	1,021.5	2,061.1
fishing_derby	-1.6	18.8	3.2	9.2	-4.1	-4.9	-3.7	22.6
freeway	26.9	0.1	28.8	27.9	0.2	28.8	27.1	29.1
frostbite	496.1	190.5	1448.1	2,930.2	2,332.4	2,813.9	418.8	4,141.1
gopher	8190.4	10022.8	15253.0	57,783.8	20,051.4	27,778.3	13,131.0	72,595.7
gravitar	298.0	303.5	200.5	218.0	297.0	422.0	250.5	567.5
hero	14992.9	32464.1	14892.5	20,506.4	15,207.9	28,554.2	2,454.2	50,496.8
ice_hockey	-1.6	-2.8	-2.5	-1.0	-1.3	-0.1	-2.4	-0.7
kangaroo	4496.0	94.0	11204.0	10,241.0	10,334.0	9,555.5	7,465.0	10,841.0
krull	6206.0	5560.0	6796.1	7,406.5	8,051.6	6,757.8	6,833.5	6,715.5
kung_fu_master	20882.0	28819.0	30207.0	31,244.0	24,288.0	33,890.0	27,921.0	28,999.8
montezuma_revenge	47.0	67.0	42.0	13.0	22.0	130.0	55.0	154.0
ms_pacman	1092.3	653.7	1241.3	1,824.6	2,250.6	2,064.1	1,012.1	2,570.2
name_this_game	6738.8	10476.1	8960.3	11,836.1	11,185.1	11,382.3	7,186.4	11,686.5
phoenix	7484.8	52894.1	12366.5	27,430.1	20,410.5	31,358.3	15,505.0	103,061.6
pitfall	-113.2	-78.5	-186.7	-14.8	-46.9	-342.8	-154.4	-37.6
pong	18.0	5.6	19.1	18.9	18.8	18.9	18.0	19.0
private_eye	207.9	206.9	-575.5	179.0	292.6	5,717.5	5,955.4	1,704.4
qbert	9271.5	15148.8	11020.8	11,277.0	14,175.8	15,035.9	9,176.6	18,397.6
road_runner	35215.0	34216.0	43156.0	56,990.0	58,549.0	56,086.0	35,376.5	54,261.0
robotank	58.7	32.8	59.1	55.4	62.0	49.8	50.9	55.2
sequest	4216.7	2355.4	14498.0	39,096.7	37,361.6	3,275.4	2,353.1	19,176.0
skiing	-12142.1	-10911.1	-11490.4	-10,852.8	-11,928.0	-13,247.7	-13,905.9	-11,685.8
solaris	1295.4	1956.0	810.0	2,238.2	1,768.4	2,530.2	2,608.2	2,860.7
space_invaders	1293.8	15,730.5	2628.7	9,063.0	5,993.1	6,368.6	1,697.2	12,629.0
star_gunner	52970.0	138218.0	58365.0	51,959.0	90,804.0	67,054.5	31,864.5	123,853.0
surround	-6.0	-9.7	1.9	-0.9	4.0	4.5	-3.1	7.0
tennis	11.1	-6.3	-7.8	-2.0	4.4	22.6	-2.1	-2.2
time_pilot	4786.0	12,679.0	6608.0	7,448.0	6,601.0	7,684.5	5,311.0	11,190.5
tutankham	45.6	156.3	92.2	33.6	48.0	124.3	123.3	126.9
venture	136.0	23.0	21.0	244.0	200.0	462.0	10.5	45.0
video_pinball	154414.1	331628.1	367823.7	374,886.9	110,976.2	455,052.7	241,851.7	506,817.2
wizard_of_wor	1609.0	17,244.0	6201.0	7,451.0	7,054.0	11,824.5	4,796.5	14,631.5
yars_revenge	4577.5	7157.5	6270.6	5,965.1	25,976.5	8,267.7	5,487.3	93,007.9
zaxxon	4412.0	24,622.0	8593.0	9,501.0	10,164.0	15,130.0	7,650.5	19,658.0

Table 5: **Human Starts** evaluation regime: Raw scores across all games, averaged over 200 testing episodes, from the agent snapshot that obtained the highest score during training. We report the published scores for DQN, A3C, DDQN, Dueling DDQN, and Prioritized DDQN. For Distributional DQN and Rainbow we report our own evaluations of the agents.

Game	DQN	DDQN	Prior. DDQN	Duel. DDQN	Distrib. DQN	Noisy DQN	Rainbow
alien	1620.0	3747.7	6,648.6	4,461.4	4,055.8	2,394.9	9,491.7
amidar	978.0	1793.3	2,051.8	2,354.5	1,267.9	1,608.0	5,131.2
assault	4280.0	5393.2	7,965.7	4,621.0	5,909.0	5,198.6	14,198.5
asterix	4359.0	17356.5	41,268.0	28,188.0	400,529.5	12,403.8	428,200.3
asteroids	1364.5	734.7	1,699.3	2,837.7	2,354.7	4,814.1	2,712.8
atlantis	279987.0	106056.0	427,658.0	382,572.0	273,895.0	329,010.0	826,659.5
bank_heist	455.0	1030.6	1,126.8	1,611.9	1,056.7	1,323.0	1,358.0
battle_zone	29900.0	31700.0	38,130.0	37,150.0	41,145.0	32,050.0	62,010.0
beam_rider	8627.5	13772.8	22,430.7	12,164.0	13,213.4	12,534.0	16,850.2
berzerk	585.6	1225.4	1,614.2	1,472.6	1,421.8	837.3	2,545.6
bowling	50.4	68.1	62.6	65.5	74.1	77.3	30.0
boxing	88.0	91.6	98.8	99.4	98.1	83.3	99.6
breakout	385.5	418.5	381.5	345.3	612.5	459.1	417.5
centipede	4657.7	5409.4	5,175.4	7,561.4	9,015.5	4,355.8	8,167.3
chopper_command	6126.0	5809.0	5,135.0	11,215.0	13,136.0	9,519.0	16,654.0
crazy_climber	110763.0	117282.0	183,137.0	143,570.0	178,355.0	118,768.0	168,788.5
defender	23633.0	35338.5	24,162.5	42,214.0	37,896.8	23,083.0	55,105.0
demon_attack	12149.4	58044.2	70,171.8	60,813.3	110,626.5	24,950.1	111,185.2
double_dunk	-6.6	-5.5	4.8	0.1	-3.8	-1.8	-0.3
enduro	729.0	1211.8	2,155.0	2,258.2	2,259.3	1,129.2	2,125.9
fishing_derby	-4.9	15.5	30.2	46.4	9.1	7.7	31.3
freeway	30.8	33.3	32.9	0.0	33.6	32.0	34.0
frostbite	797.4	1683.3	3,421.6	4,672.8	3,938.2	583.6	9,590.5
gopher	8777.4	14840.8	49,097.4	15,718.4	28,841.0	15,107.9	70,354.6
gravitar	473.0	412.0	330.5	588.0	681.0	443.5	1,419.3
hero	20437.8	20130.2	27,153.9	20,818.2	33,860.9	5,053.1	55,887.4
ice_hockey	-1.9	-2.7	0.3	0.5	1.3	-2.1	1.1
kangaroo	7259.0	12992.0	14,492.0	14,854.0	12,909.0	12,117.0	14,637.5
krull	8422.3	7920.5	10,263.1	11,451.9	9,885.9	9,061.9	8,741.5
kung_fu_master	26059.0	29710.0	43,470.0	34,294.0	43,009.0	34,099.0	52,181.0
montezuma_revenge	0.0	0.0	0.0	0.0	367.0	0.0	384.0
ms_pacman	3085.6	2711.4	4,751.2	6,283.5	3,769.2	2,501.6	5,380.4
name_this_game	8207.8	10616.0	13,439.4	11,971.1	12,983.6	8,332.4	13,136.0
phoenix	8485.2	12252.5	32,808.3	23,092.2	34,775.0	16,974.3	108,528.6
pitfall	-286.1	-29.9	0.0	0.0	-2.1	-18.2	0.0
pong	19.5	20.9	20.7	21.0	20.8	21.0	20.9
private_eye	146.7	129.7	200.0	103.0	15,172.9	3,966.0	4,234.0
qbert	13117.3	15088.5	18,802.8	19,220.3	16,956.0	15,276.3	33,817.5
road_runner	39544.0	44127.0	62,785.0	69,524.0	63,366.0	41,681.0	62,041.0
robotank	63.9	65.1	58.6	65.3	54.2	53.5	61.4
seaquest	5860.6	16452.7	44,417.4	50,254.2	4,754.4	2,495.4	15,898.9
skiing	-13062.3	-9021.8	-9,900.5	-8,857.4	-14,959.8	-16,307.3	-12,957.8
solaris	3482.8	3067.8	1,710.8	2,250.8	5,643.1	3,204.5	3,560.3
space_invaders	1692.3	2525.5	7,696.9	6,427.3	6,869.1	2,145.5	18,789.0
star_gunner	54282.0	60142.0	56,641.0	89,238.0	69,306.5	34,504.5	127,029.0
surround	-5.6	-2.9	2.1	4.4	6.2	-3.3	9.7
tennis	12.2	-22.8	0.0	5.1	23.6	0.0	-0.0
time_pilot	4870.0	8339.0	11,448.0	11,666.0	7,875.0	6,157.0	12,926.0
tutankham	68.1	218.4	87.2	211.4	249.4	231.6	241.0
venture	163.0	98.0	863.0	497.0	1,107.0	0.0	5.5
video_pinball	196760.4	309941.9	406,420.4	98,209.5	478,646.7	270,444.6	533,936.5
wizard_of_wor	2704.0	7492.0	10,373.0	7,855.0	15,994.5	5,432.0	17,862.5
yars_revenge	18089.9	11712.6	16,451.7	49,622.1	16,608.6	9,570.1	102,557.0
zaxxon	5363.0	10163.0	13,490.0	12,944.0	18,347.5	9,390.0	22,209.5

Table 6: **No-op starts** evaluation regime: Raw scores across all games, averaged over 200 testing episodes, from the agent snapshot that obtained the highest score during training. We report the published scores for DQN, DDQN, Dueling DDQN, and Prioritized DDQN. For Distributional DQN and Rainbow we report our own evaluations of the agents. A3C is not listed since the paper did not report the scores for the no-ops regime.

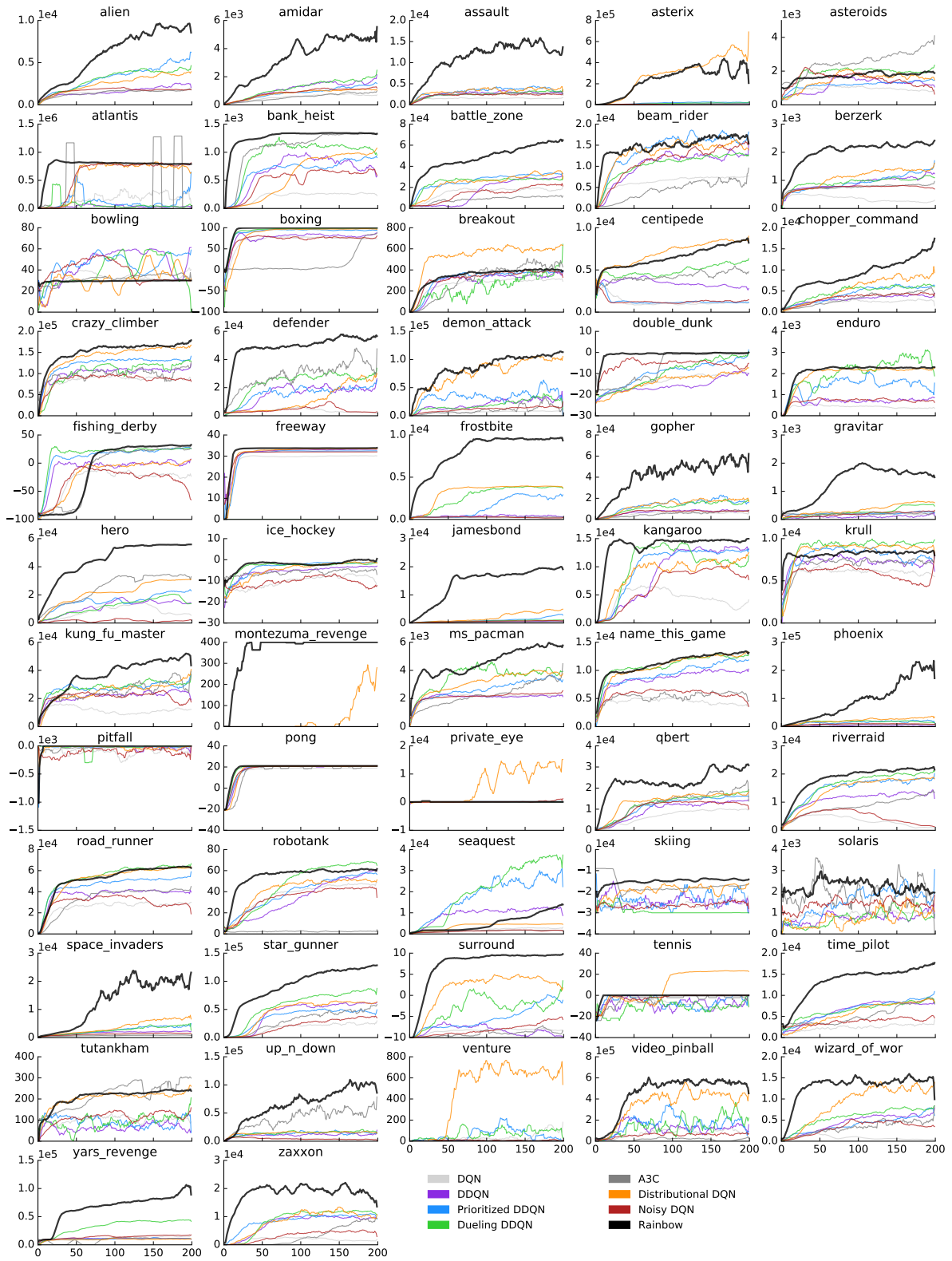


Figure 5: Learning curves for Rainbow and the baselines discussed in the paper, for each individual game. Every curve is smoothed with a moving average of 10 to improve readability.

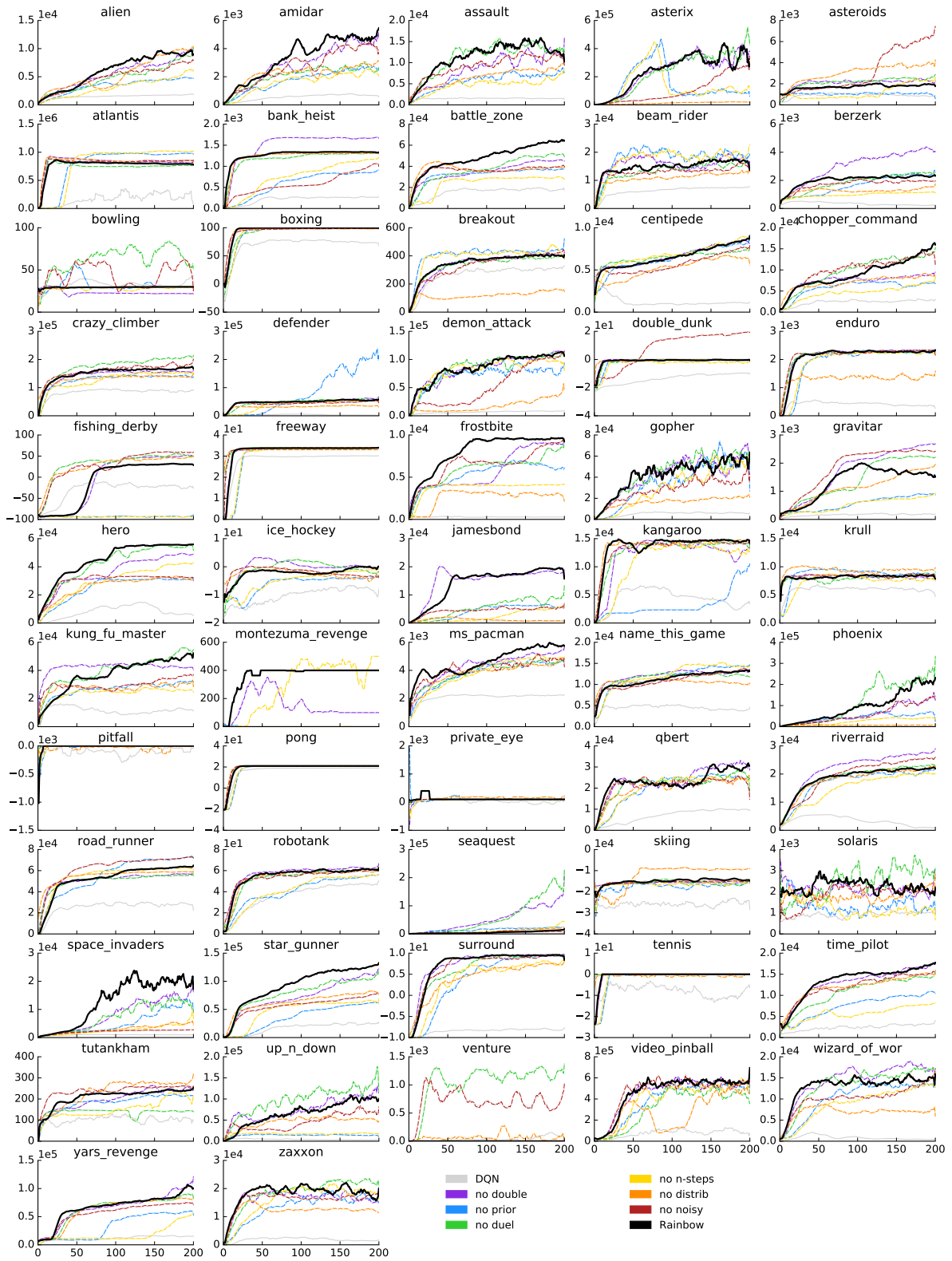


Figure 6: Learning curves for Rainbow and its ablations, for each individual game. Every curve is smoothed with a moving average of 10 to improve readability.

SOFT ACTOR-CRITIC FOR DISCRETE ACTION SETTINGS

A PREPRINT

Petros Christodoulou
Imperial College London
petros.christodoulou18@imperial.ac.uk

October 21, 2019

ABSTRACT

Soft Actor-Critic is a state-of-the-art reinforcement learning algorithm for continuous action settings that is not applicable to discrete action settings. Many important settings involve discrete actions, however, and so here we derive an alternative version of the Soft Actor-Critic algorithm that *is* applicable to discrete action settings. We then show that, even without any hyperparameter tuning, it is competitive with the tuned model-free state-of-the-art on a selection of games from the Atari suite.

1 Introduction

Reinforcement Learning (RL) has famously made great progress in recent years, successfully being applied to settings such as board games [Silver et al., 2017], video games [Mnih et al., 2015] and robot tasks [OpenAI et al., 2018]. However, widespread adoption of RL in real-world domains has remained slow primarily because of its poor sample efficiency which Wu et al. (2017) see as a "dominant concern in RL".

Haarnoja et al. (2018) provide the Soft Actor-Critic (SAC) algorithm which helps deal with this concern in continuous action settings. It has achieved model-free state-of-the-art sample efficiency in multiple challenging continuous control domains. Many domains however involve *discrete* rather than continuous actions and in these environments SAC is not currently applicable. This paper derives a version of SAC that is applicable to discrete action domains and then shows that it is competitive with the model-free state-of-the-art for discrete action domains in terms of sample efficiency on a selection of games from the Atari [Bellemare et al., 2013] suite.

We proceed as follows: first we explain the derivation of Soft Actor-Critic for continuous action settings found in Haarnoja et al. (2018) and Haarnoja et al. (2019), then we derive and explain the changes required to create a discrete action version of the algorithm, and finally we test the discrete action algorithm on the Atari suite.

2 Soft Actor-Critic

Soft Actor-Critic [Haarnoja et al., 2018] attempts to find a policy that maximises the maximum entropy objective:

$$\pi^* = \operatorname{argmax}_{\pi} \sum_{t=0}^T E_{(s_t, a_t) \sim \tau_{\pi}} [\gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))] \quad (1)$$

where π is a policy, π^* is the optimal policy, T is the number of timesteps, $r : S \times A \rightarrow \mathbb{R}$ is the reward function, $\gamma \in [0, 1]$ is the discount rate, $s_t \in S$ is the state at timestep t , $a_t \in A$ is the action at timestep t , τ_{π} is the distribution of trajectories induced by policy π , α determines the relative importance of the entropy term versus the reward and is called the temperature parameter, and $\mathcal{H}(\pi(\cdot|s_t))$ is the entropy of the policy π at state s_t and is calculated as $\mathcal{H}(\pi(\cdot|s_t)) = -\log \pi(\cdot|s_t)$.

To maximise the objective the authors use soft policy iteration which is a method of alternating between policy evaluation and policy improvement within the maximum entropy framework.

The policy evaluation step involves computing the value of policy π . To do this they first define the soft state value function as:

$$V(s_t) := E_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log(\pi(a_t|s_t))] \quad (2)$$

They then prove that in a tabular setting (i.e. when the state space is discrete) we can obtain the soft q-function by starting from a randomly initialised function $Q : S \times A \rightarrow \mathbb{R}$ and repeatedly applying the modified Bellman backup operator T^π given by:

$$T^\pi Q(s_t, a_t) := r(s_t, a_t) + \gamma E_{s_{t+1} \sim p(s_t, a_t)}[V(s_{t+1})] \quad (3)$$

where $p : S \times A \rightarrow S$ gives the distribution over the next state given the current state and action.

In the continuous state (instead of tabular) setting they explain that we instead firstly parameterise the soft q-function $Q_\theta(s_t, a_t)$ using a neural network with parameters θ . Then we train the soft Q-function to minimise the soft Bellman residual:

$$J_Q(\theta) = E_{(s_t, a_t) \sim D}[\frac{1}{2}(Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma E_{s_{t+1} \sim p(s_t, a_t)}[V_{\bar{\theta}}(s_{t+1})]))^2] \quad (4)$$

where D is a replay buffer of past experiences and $V_{\bar{\theta}}(s_{t+1})$ is estimated using a target network for Q and a monte-carlo estimate of (2) after sampling experiences from the replay buffer .

The policy improvement step then involves updating the policy in a direction that maximises the rewards it will achieve. To do this they use the soft Q-function calculated in the policy evaluation step to guide changes to the policy. Specifically, they update the policy towards the exponential of the new soft Q-function. Because they also want the policy to be tractable however, they restrict the possible policies to a parameterised family of distributions (e.g. Gaussian). To account for this, after updating the policy towards the exponential of the soft Q-function they then project it back into the space of acceptable policies using the information projection defined in terms of Kullback-Leibler divergence. So overall the policy improvement step is given by:

$$\pi_{\text{new}} = \underset{\pi \in \Pi}{\operatorname{argmin}} D_{\text{KL}}\left(\pi(\cdot|s_t) \parallel \frac{\exp(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)}\right) \quad (5)$$

They note that partition function $Z^{\pi_{\text{old}}}(s_t)$ is intractable but does not contribute to the gradient with respect to the new policy and so it can be ignored.

In the continuous state setting they parameterise the policy $\pi_\phi(a_t|s_t)$ using a neural network with parameters ϕ that outputs a mean and covariance that is then used to define a Gaussian policy. The policy parameters are then learned by minimizing the expected KL-divergence (5) after multiplying by the temperature parameter α and ignoring the partition function $Z^{\pi_{\text{old}}}(s_t)$ as it does not impact the gradient:

$$J_\pi(\phi) = E_{s_t \sim D}[E_{a_t \sim \pi_\phi}[\alpha \log(\pi_\phi(a_t|s_t)) - Q_\theta(s_t, a_t)]] \quad (6)$$

This involves taking an expectation over the policy's output distribution which means errors cannot be backpropagated in the normal way. To deal with this they use the reparameterisation trick [Kingma and Welling, 2013] - instead of using the output of the policy network to form a stochastic action distribution directly, they combine its output with an input noise vector sampled from a spherical Gaussian. For example, in the one-dimensional case our network outputs a mean m and standard deviation s . We could randomly sample our action directly $a \sim N(m, s)$ but then we could not backpropagate the errors through this operation. So instead we do $a = m + s\epsilon$ where $\epsilon \sim N(0, 1)$ which allows us to backpropagate as normal. To signify that they are reparameterising the policy in this way they write:

$$a_t = f_\phi(\epsilon_t; s_t) \quad (7)$$

where $\epsilon_t \sim N(0, I)$. The new policy objective then becomes:

$$J_\pi(\phi) = E_{s_t \sim D, \epsilon_t \sim N}[\alpha \log(\pi_\phi(f_\phi(\epsilon_t; s_t)|s_t)) - Q_\theta(s_t, f_\phi(\epsilon_t; s_t))] \quad (8)$$

where π_ϕ is now defined implicitly in terms of f_ϕ . They then go on to prove that in the tabular setting, alternating between policy evaluation and policy improvement as above will converge to the optimal policy.

Haarnoja et al. (2019) also provide an optional way of learning the temperature parameter so that we do not need to set it as a hyperparameter. They provide a long derivation for the temperature objective value, however because the details are not strictly relevant for our derivation of the discrete action version of SAC we do not repeat it here. The final objective they get to for the temperature parameter is however relevant and given by:

$$J(\alpha) = E_{a_t \sim \pi_t} [-\alpha(\log \pi_t(a_t|s_t) + \bar{H})] \quad (9)$$

where $\bar{H}$ is a constant vector equal to the hyperparameter representing the target entropy. They are unable to minimise this expression directly because of the expectation operator and so instead they minimise a monte-carlo estimate of it after sampling experiences from the replay buffer.

Lastly, in practice the authors maintain two separately trained soft Q-networks and then use the minimum of their two outputs to be the soft Q-network output in the above objectives. They do this because Fujimoto, Hoof, and Meger (2018) showed that it helps combat state-value overestimation.

3 Soft Actor-Critic for Discrete Action Settings (SAC-Discrete)

We now derive a discrete action version of the above SAC algorithm. The first thing to note is that all the critical steps involved in deriving the objectives above hold whether the actions are continuous or discrete. All that changes is that $\pi_\phi(a_t|s_t)$ now outputs a probability instead of a density. Therefore the three objective functions $J_Q(\theta)$ (4), $J_\pi(\phi)$ (6) and $J(\alpha)$ (9) still hold. We must however make five important changes to the process of optimising these objective functions:

- i) It is now more efficient to have the soft Q-function output the Q-value of each possible action rather than simply the action provided as an input, i.e. our Q function moves from $Q : S \times A \rightarrow \mathbb{R}$ to $Q : S \rightarrow \mathbb{R}^{|A|}$. This was not possible before when there were infinitely many possible actions we could take.
- ii) There is now no need for our policy to output the mean and covariance of our action distribution, instead it can directly output our action distribution. The policy therefore changes from $\pi : S \rightarrow \mathbb{R}^{2|A|}$ to $\pi : S \rightarrow [0, 1]^{|A|}$ where now we are applying a softmax function in the final layer of the policy to ensure it outputs a valid probability distribution.
- iii) Before, in order to minimise the soft Q-function cost $J_Q(\theta)$ (4) we had to plug in our sampled actions from the replay buffer to form a monte-carlo estimate of the soft state-value function (2). This was because estimating the soft state-value function in (2) involved taking an expectation over the action distribution. However, now, because our action set is discrete we can fully recover the action distribution and so there is no need to form a monte-carlo estimate and instead we can calculate the expectation directly. This change should reduce the variance involved in our estimate of the objective $J_Q(\theta)$ (4). This means that we change our soft state-value calculation equation from (2) to:

$$V(s_t) := \pi(s_t)^T [Q(s_t) - \alpha \log(\pi(s_t))] \quad (10)$$

- iv) Similarly, we can make the same change to our calculation of the temperature loss to also reduce the variance of that estimate. The temperature objective changes from (9) to:

$$J(\alpha) = \pi_t(s_t)^T [-\alpha(\log(\pi_t(s_t)) + \bar{H})] \quad (11)$$

- v) Before, to minimise $J_\pi(\phi)$ (6) we had to use the reparameterisation trick to allow gradients to pass through the expectations operator. However, now our policy outputs the exact action distribution we are able to calculate the expectation directly. Therefore there is no need for the reparameterisation trick and the new objective for the policy changes from (8) to:

$$J_\pi(\phi) = E_{s_t \sim D} [\pi_t(s_t)^T [\alpha \log(\pi_\phi(s_t)) - Q_\theta(s_t)]] \quad (12)$$

Combining all these changes, our algorithm for SAC with discrete actions (SAC-Discrete) is given by Algorithm 1.

Algorithm 1 Soft Actor-Critic with Discrete Actions (SAC-Discrete)

Initialise $Q_{\theta_1} : S \rightarrow \mathbb{R}^{|A|}$, $Q_{\theta_2} : S \rightarrow \mathbb{R}^{|A|}$, $\pi_\phi : S \rightarrow [0, 1]^{|A|}$ ▷ Initialise local networks
Initialise $\bar{Q}_{\theta_1} : S \rightarrow \mathbb{R}^{|A|}$, $\bar{Q}_{\theta_2} : S \rightarrow \mathbb{R}^{|A|}$ ▷ Initialise target networks
 $\bar{\theta}_1 \leftarrow \theta_1$, $\bar{\theta}_2 \leftarrow \theta_2$ ▷ Equalise target and local network weights
 $\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay buffer
for each iteration do
 for each environment step do
 $a_t \sim \pi_\phi(a_t|s_t)$ ▷ Sample action from the policy
 $s_{t+1} \sim p(s_{t+1}|s_t, a_t)$ ▷ Sample transition from the environment
 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$ ▷ Store the transition in the replay buffer
 for each gradient step do
 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
 $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
 $\alpha \leftarrow \alpha - \lambda_{\hat{\nabla}_\alpha} J(\alpha)$ ▷ Update temperature
 $\bar{Q}_i \leftarrow \tau Q_i + (1 - \tau) \bar{Q}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
Output θ_1, θ_2, ϕ ▷ Optimized parameters

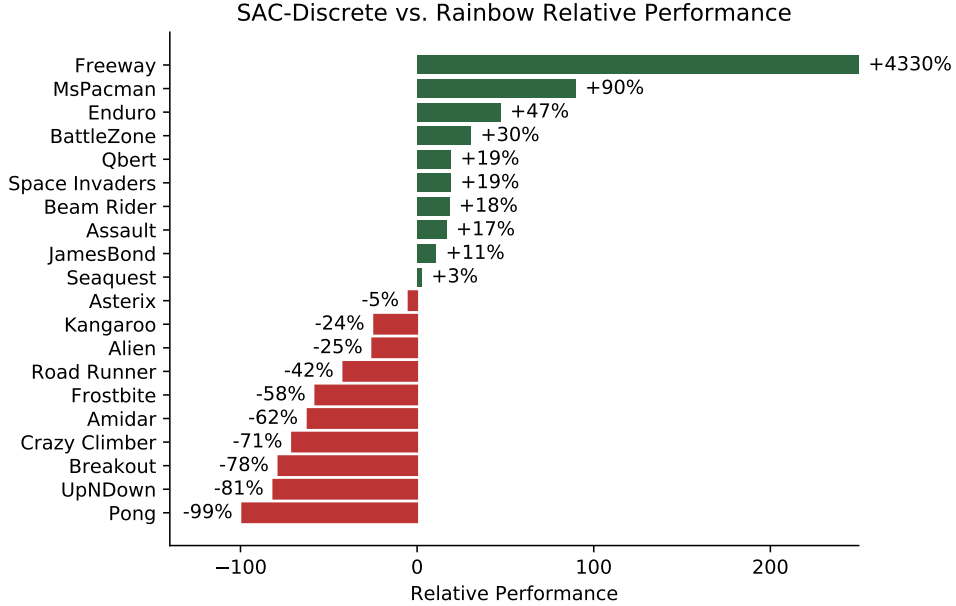


Figure 1: Comparing SAC-Discrete to Rainbow for 20 Atari games. The graph shows the average relative performance of SAC-Discrete over Rainbow over 5 random seeds where evaluation scores are calculated at the end of 100,000 steps of training. Note that no hyperparameter tuning was done to support the SAC scores compared to the Rainbow scores which benefited from substantial hyperparameter tuning

4 Results

To test the effectiveness of SAC-Discrete we run it for 100,000 steps on 20 Atari games for 5 random seeds each and compare its results with Rainbow which is a state-of-the-art model-free algorithm in terms of sample efficiency. The games vary significantly and were chosen a priori and so we believe the results on these 20 games are a good estimate for relative performance on the whole Atari suite of 49 games. We chose to run the algorithm for 100,000 steps because we are most interested in sample efficiency and Kaiser et al. (2019) demonstrated that Rainbow can make significant progress on Atari games within 100,000 steps.

For SAC-Discrete actions we did no hyperparameter tuning and instead used a mixture of the hyperparameters found in Haarnoja et al. (2019) and Kaiser et al. (2019). The hyperparameters can be found in Appendix B. The Rainbow

results we compare to come from Kaiser et al. (2019) and as they explain were the result of a significant amount of hyperparameter tuning.¹ Therefore we are comparing the tuned Rainbow algorithm to our untuned SAC algorithm and so it is highly likely the relative performance of SAC could be improved if we spent time tuning its hyperparameters.

We find that SAC-Discrete achieves a better score than Rainbow in 10 out of 20 games with a median performance of -1%, maximum performance of +4330% and minimum of -99% - Figure 1 summarises the results and Appendix A provides them in a table. Overall, we therefore consider the SAC-Discrete algorithm as roughly competitive with the model-free state-of-the-art on the Atari suite in terms of sample efficiency.

5 Conclusion

The original Soft Actor-Critic algorithm achieved state-of-the-art results on numerous continuous action settings but was not applicable to discrete action settings. To correct this we have derived a version of the algorithm called SAC-Discrete that is applicable to discrete action settings and have shown that it performs competitively with the model-free state-of-the-art on the Atari suite even without any hyperparameter tuning. We provide a Python implementation of the algorithm at the project’s GitHub repository.²

References

- Bellemare, M., Y. Naddaf, J. Veness, and M. Bowling (2013). “The Arcade Learning Environment: An Evaluation Platform for General Agents”. In: *Journal of Artificial Intelligence*.
- Castro, P., S. Moitra, C. Geland, S. Kumar, and M. Bellemare (2018). “Dopamine: A Research Framework for Deep Reinforcement Learning”. In: *arXiv preprint*.
- Fujimoto, S., H. van Hoof, and D. Meger (2018). “Addressing Function Approximation Error in Actor-Critic Methods”. In: *arXiv preprint*.
- Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine (2018). “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. In: *International Conference on Learning Representations*.
- Haarnoja, T., A. Zhou, K. Hartikainen, G. Tucker, J. Tan S. Ha, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine (2019). “Soft Actor-Critic Algorithms and Applications”. In: *arXiv preprint*.
- Kaiser, L., M. Babaeizadeh, P. Milos, B. Osinski, R. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, A. Mohiuddin, R. Sepassi, G. Tucker, and H. Michalewski (2019). “Model Based Reinforcement Learning for Atari”. In: *arXiv preprint*.
- Kingma, D. and M. Welling (2013). “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations*.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. Rusu, J. Veness, M. Bellemare, A. Graves, M. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015). “Human-Level Control Through Deep Reinforcement Learning”. In: *Nature*.
- OpenAI, Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub W. Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba (2018). “Learning Dexterous In-Hand Manipulation”. In: *ArXiv abs/1808.00177*.
- Silver, D., J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. Driessche, T. Graepel, and D. Hassabis (2017). “Mastering the Game of Go Without Human Knowledge”. In: *Nature*.
- Wu, Y., E. Mansimov, S. Liao, R. Grosse, and J. Ba (2017). “Scalable Trust-Region Method for Deep Reinforcement Learning Using Kronecker-Factored Approximation”. In: *Advances in Neural Information Processing Systems*.

¹For two games (Enduro and SpaceInvaders) Kaiser et al. (2019) provide no results for Rainbow and so for these two games only we ran the Rainbow algorithm ourselves. We used the Dopamine [Castro et al., 2018] codebase to do this (as Kaiser et al. (2019) also did) along with the same (tuned) hyperparameters used in Kaiser et al. (2019). We share the code used to do this in the colabory notebook: <https://colab.research.google.com/drive/11prfRfM5qrMsfUXV6cGY868HtwDphxaF>

²<https://github.com/p-christ/Deep-Reinforcement-Learning-Algorithms-with-PyTorch>

Appendix

A SAC and Rainbow Atari Results

Table 1: SAC and Rainbow results on 20 Atari games. The mean SAC result of 5 random seeds is shown with the standard deviation in brackets. As a benchmark we also provide a column indicating the score an agent would get if it acted purely randomly. The Rainbow results come from Kaiser et al., 2019.

Game	Random	Rainbow	SAC
Freeway	0.0	0.1	4.4 (9.9)
MsPacman	235.2	364.3	690.9 (141.8)
Enduro	0.0	0.53	0.8 (0.8)
BattleZone	2895.0	3363.5	4386.7 (1163.0)
Qbert	166.1	235.6	280.5 (124.9)
Space Invaders	148.0	135.1	160.8 (17.3)
Beam Rider	372.1	365.6	432.1 (44.0)
Assault	233.7	300.3	350.0 (40.0)
JamesBond	29.2	61.7	68.3 (26.2)
Seaquest	61.1	206.3	211.6 (59.1)
Asterix	248.8	285.7	272.0 (33.3)
Kangaroo	42.0	38.7	29.3 (55.1)
Alien	184.8	290.6	216.9 (43.0)
Road Runner	0.0	524.1	305.3 (557.4)
Frostbite	74.0	140.1	59.4 (16.3)
Amidar	11.8	20.8	7.9 (5.1)
Crazy Climber	7339.5	12558.3	3668.7 (600.8)
Breakout	0.9	3.3	0.7 (0.5)
UpNDown	488.4	1346.3	250.7 (176.5)
Pong	-20.4	-19.5	-20.98 (0.0)

B SAC-Discrete Hyperparameters

Table 2: Hyperparameters used for SAC-Discrete results

Hyperparameter	Value
Layers	3 convolutional layers and 2 fully connected layers
Convolutional channels per layer	[32, 64, 64]
Convolutional kernel sizes per layer	[8, 4, 3]
Convolutional strides per layer	[4, 2, 1]
Convolutional padding per layer	[0, 0, 0]
Fully connected layer hidden units	[512, number of moves in game]
Batch size	64
Replay buffer size	1,000,000
Discount rate	0.99
Steps per learning update	4
Learning iterations per round	1
Learning rate	0.0003
Optimizer	Adam
Weight initialiser	He
Fixed network update frequency	8000
Loss	Mean squared error
Clip rewards	Clip to [-1, +1]
Initial random steps	20,000
Entropy target	$0.98 * (-\log (1 / A))$

Revisiting Discrete Soft Actor-Critic

Haibin Zhou
Tencent Inc.

haibinzhou@tencent.com

Tong Wei
Tsinghua University

wt22@mails.tsinghua.edu.cn

Zichuan Lin
Tencent Inc.

zichuanlin@tencent.com

Junyou Li
Tencent Inc.

junyouli@tencent.com

Junliang Xing
Tsinghua University

jlxing@tsinghua.edu.cn

Yuanchun Shi
Tsinghua University

shiyu@tsinghua.edu.cn

Li Shen
Sun Yat-sen University

mathshenli@gmail.com

Chao Yu
Sun Yat-sen University

yuchao3@mail.sysu.edu.cn

Deheng Ye*
Tencent

dericye@tencent.com

Reviewed on OpenReview: <https://openreview.net/forum?id=EUF2R6VBeU>

Abstract

We study the adaptation of Soft Actor-Critic (SAC), which is considered as a state-of-the-art reinforcement learning algorithm, from continuous action space to discrete action space. We revisit vanilla discrete SAC, i.e., SAC for discrete action space, and provide an in-depth understanding of its Q value underestimation and performance instability issues when applied to discrete settings. We thereby propose Stable Discrete SAC (SD-SAC), an algorithm that leverages entropy-penalty and double average Q-learning with Q-clip to address these issues. Extensive experiments on typical benchmarks with discrete action space, including Atari games and a large-scale MOBA game, show the efficacy of SD-SAC. Our code is at: <https://github.com/coldsummerday/SD-SAC.git>.

1 Introduction

In the conventional model-free reinforcement learning (RL) paradigm, an agent can be trained by learning an approximator of action-value (Q) function (Mnih et al., 2015; Bellemare et al., 2017). The class of actor-critic algorithms (Mnih et al., 2016; Fujimoto et al., 2018) evaluates the policy function by approximating the value function. Motivated by maximum-entropy RL (Ziebart et al., 2008; Rawlik et al., 2012; Abdolmaleki et al., 2018), soft actor-critic (SAC) (Haarnoja et al., 2018a) introduces action entropy in the framework of

*Corresponding author.

actor-critic to balance exploitation and exploration. It has achieved remarkable performance in a range of environments with continuous action spaces (Haarnoja et al., 2018b), and is considered as the state-of-the-art algorithm for domains with continuous action space, e.g., Mujoco (Todorov et al., 2012).

However, while SAC solves problems with continuous action space, it cannot be directly applied to discrete domains since it relies on the reparameterization of Gaussian policies to sample actions, in which the action in discrete domains is categorical. Soft-DQN (Vieillard et al., 2020) provides a simple way to discretize SAC by adopting the maximum-entropy RL to DQN (Mnih et al., 2013). However, Soft-DQN utilizes only a Q-value parametrization to bypass the policy parameterization. Another discretization of the continuous action output and Q value in vanilla SAC is suggested by previous work (Christodoulou, 2019) to adapt SAC to discrete domains, resulting in discrete SAC (DSAC). However, it is counter-intuitive that the empirical experiments in subsequent efforts (Xu et al., 2021) indicate that discrete SAC performs poorly in discrete domains, e.g., Atari games. We believe that the idea of maximum entropy RL applies to both discrete and continuous domains. However, extending the maximum-entropy-based SAC algorithm to discrete domains still lacks a commonly accepted practice in the community. Therefore, in this paper, similar to the motivation of DDPG (deep deterministic policy gradient) (Lillicrap et al., 2016), which adapts DQN (deep Q networks) (Mnih et al., 2013) from discrete action space to continuous action space, we aim to optimize SAC algorithm for discrete domains.

Previous studies (Xu et al., 2021; Wang & Ni, 2020) have analyzed the reasons for the performance disparity of SAC between continuous and discrete domains. Reviewing from the perspective of automating entropy adjustment, an unreasonable setting of target-entropy for temperature α may break the SAC value-entropy trade-off (Wang & Ni, 2020; Xu et al., 2021). Furthermore, the function approximation errors of Q-value are known to lead to estimation bias and hurt performance in actor-critic methods (Fujimoto et al., 2018). To avoid overestimation bias, both discrete SAC and continuous SAC resort to clipped double Q-learning (Fujimoto et al., 2018) for actor-critic algorithms. On the contrary, using the lower bound approximation on the critic can lead to underestimation bias, which makes the policy fall into pessimistic underexplored, as pointed by (Ciosek et al., 2019; Pan et al., 2020), mainly when the reward is sparse. However, existing works only focus on continuous domains (Ciosek et al., 2019; Pan et al., 2020), while SAC for discrete cases remains less explored.

In addition to the abovementioned issues, we conjecture that discrete SAC fails also due to the absence of policy update constraints. Intuitively, the unstable training causes a shift in the Q function distribution and policy entropy, which generates a rapidly changing target for the critic network due to the soft Q-learning objective. Meanwhile, the critic network in SAC needs time to adapt to the oscillating target process, exacerbating policy instability.

To address the above challenges, we first design test cases to replicate the failure modes of vanilla discrete SAC, exposing its inherent weaknesses regarding training instability and Q-value underestimation. Then, accordingly, we propose Stable Discrete SAC (SD-SAC) to stabilize the training. We develop an entropy penalty on the policy optimization objective to constrain policy updates. We also develop double average Q-learning with Q-clip to confine the Q value within a reasonable range. We use Atari games (the default testbed for the RL algorithm for discrete action space) to verify the effectiveness of our optimizations. We also deploy our method to the Honor of Kings 1v1 game, a large-scale MOBA game used extensively in recent RL advances (Ye et al., 2020b;ca; Wei et al., 2022), to demonstrate the scale-up capacity of our Stable Discrete SAC.

To sum up, our contributions are:

- We pinpoint two failure modes of discrete SAC, regarding training instability and underestimated Q values. We find that the underlying causes are the environment’s deceptive rewards and SAC’s double Q learning respectively.
- To alleviate the training instability issue, we propose entropy-penalty to constrain the policy update in discrete SAC.
- To deal with the underestimation bias of Q value in discrete SAC, we propose double average Q-learning with Q-clip to estimate the state-action value.

With the above contributions, we have obtained the Stable Discrete SAC (SD-SAC) algorithm. Extensive experiments on Atari games and a large-scale MOBA game show SD-SAC’s superiority compared to baselines, with a 68% improvement of normalized scores in Atari and around 100% ELO increase in the Honor of Kings 1v1 game environment.

2 Related Work

Adaptation of Action Space. The most relevant works to this paper are vanilla discrete SAC (Christodoulou, 2019), TES-SAC (Xu et al., 2021) and Soft-DQN (Vieillard et al., 2020). Discrete SAC replaces the Gaussian policy with a categorical one and discretizes the Q-value output to adapt SAC from continuous to discrete action space. However, as we will point out, a direct discretization of SAC will have specific failure modes with poor performance. TES-SAC proposes a new scheduling method for the target entropy parameters in discrete SAC. Soft-DQN has discretized SAC by adopting the maximum-entropy RL to DQN, utilizing only a Q value parametrization and directly applies a softmax operation to the Q-values to take action.

Q Estimation. Previous works (Fujimoto et al., 2018; Ciosek et al., 2019; Pan et al., 2020; Duan et al., 2021) have already expressed concerns about the estimation bias of Q value for SAC. SD3 (Pan et al., 2020) proposes to reduce the Kurtosis distribution of Q approximately by using the softmax operator on the original Q value output to reduce the overestimation bias. OAC (Ciosek et al., 2019) constrains the Q value approximation objective by calculating the upper and lower boundaries of two Q-networks. Distributional SAC (Duan et al., 2021) replaces the Q learning target with the expected reward sum obtained from the current state to the end of the episode and uses a multi-frame estimates target to reduce overestimation. Maxmin Q-learning (Lan et al., 2020) controls estimation bias by minimizing the complete ensemble in the target. MME (Han & Sung, 2021) extends max-min operation to the entropy framework to adapt to SAC. REM (Agarwal et al., 2020) ensembles Q-value estimations with a random convex combination to enhance generalization in the offline setting. REDQ (Chen et al., 2021b) reduces the estimation bias by minimizing a random subset of Q-functions. AEQ (Gong et al., 2023) adjusts the estimation bias by using the mean of Q-functions minus their standard deviation. However, little research is on discrete settings. Our approach focuses on reducing the underestimation bias for the double Q-estimators to enhance exploration.

Performance Stability. Flow-SAC (Ward et al., 2019) applies a technique called normalizing flows policy on continuous SAC leading to the finer transformation that improves training stability when exploring complex states. However, applying normalizing flows to discrete domains will cause a degeneracy problem (Horvat & Pfister, 2021), making it difficult to transfer to discrete actions. SAC-AWMP (Hou et al., 2020) improves the stability of the final policy by using a weighted mixture to combine multiple policies. Based on this method, the cost of network parameters and inference speed is significantly increased. ISAC (Banerjee et al., 2022) increases SAC stability by mixing prioritized and on-policy samples, enabling the actor to repeat learns states with drastic changes. Repeatedly learning priority samples, however, runs the risk of settling into a local optimum. By comparison, our method improves policy stability in case of drastic state changes with an entropy constraint.

3 Preliminaries

This section briefly overviews the symbol definitions of SAC for discrete action space. Following the maximum entropy framework, SAC adds an entropy term $\mathcal{H}(\pi(\cdot | s))$ as regularization to the policy gradient objective:

$$\pi^* = \operatorname{argmax}_{\pi} \sum_{t=0}^T \left[\gamma^t \mathbb{E}_{\substack{s_t \sim p \\ a_t \sim \pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \right], \quad (1)$$

$$\mathcal{H}(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [-\log \pi(a | s)] \quad (2)$$

where π is a policy, π^* is the optimal policy, and α is the temperature parameter that determines the relative importance of the entropy term versus the reward r , thus controls the stochasticity of the optimal policy.

Soft Bellman Backup The soft Q-function, parametrized by θ , is updated by reducing the soft Bellman error as described in the next subsection:

$$J_Q(\theta) = \frac{1}{2} (r(s_t, a_t) + \gamma V(s_{t+1}) - Q_\theta(s_t, a_t))^2, \quad (3)$$

where $V(s_t)$ defines the soft state value function, which represents the expected reward estimate that policy obtains from the current state to the end of the trajectory.

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q_\theta(s_t, a_t) - \alpha \log(\pi(a_t | s_t))]. \quad (4)$$

Soft actor-critic minimizes soft Q-function with final soft Bellman error:

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim D} \left[\frac{1}{2} (Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t)} [V(s_{t+1})]))^2 \right], \quad (5)$$

where D is a replay buffer replenished by rollouts of the policy π interacting with the environment. In the implementation, SAC (Haarnoja et al., 2018a) uses the minimum of two delayed-update target-critic network outputs as the soft bellman learning objective to reduce overestimation. The formula is expressed as

$$V(s_{t+1}) = \min_{i=1,2} \mathbb{E}_{a_t \sim \pi} [Q_{\theta'_i}(s_{t+1}, a_{t+1}) - \alpha \log(\pi(a_{t+1} | s_{t+1}))], \quad (6)$$

where $Q_{\theta'_i}$ represents i -th target-critic network.

Policy Update Iteration The policy, parameterized by ϕ , distills the softmax policy induced by the soft Q-function. The discrete SAC policy directly maximizes the probability of discrete actions, in contrast to the continuous SAC policy, which optimizes the two parameters of the Gaussian distribution. Then, the discrete SAC policy is updated by minimizing KL divergence between the policy distribution and the soft Q-function.

$$\pi_{\phi_{new}} = \operatorname{argmin}_{\pi_{\phi_{old}} \in \Pi} D_{\text{KL}} \left(\pi_{\phi_{old}}(\cdot | s_t) \parallel \frac{\exp(\frac{1}{\alpha} Q^{\pi_{\phi_{old}}}(s_t, \cdot))}{Z^{\pi_{\phi_{old}}}(s_t)} \right). \quad (7)$$

Note that the partition function $Z^{\pi_{\phi_{old}}}(s_t)$ is a normalization term that can be ignored since it does not affect the gradient concerning the new policy. The resulting optimization objective of the policy is as follows:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} [\mathbb{E}_{a_t \sim \pi_\phi} [\alpha \log(\pi_\phi(a_t | s_t)) - Q_\theta(s_t, a_t)]]. \quad (8)$$

Automating Entropy Adjustment The entropy parameter temperature α regulates the value-entropy balance in soft Q learning. The SAC paper proposes using the temperature Lagrange term to tune the temperature α automatically. The following equation can be regarded as the optimization objective satisfying an entropy constraint.

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [-\log(\pi_t(a_t | s_t))] \geq \mathcal{H}, \forall t, \quad (9)$$

where $\mathcal{H}$ is the desired minimum expected entropy. Optimizing the Lagrangian term α involves minimizing:

$$J(\alpha) = \mathbb{E}_{(a|s) \sim \pi_t} [-\alpha \log \pi_t(a_t | s_t) - \alpha \mathcal{H}]. \quad (10)$$

By setting a loose upper limit on the target entropy $\mathcal{H}$, SAC achieves automatic adjustment of temperature α . Typically, the target entropy is set to $0.98 * -\log(\frac{1}{\dim(\text{Actions})})$ for discrete (Christodoulou, 2019) and $-\dim(\text{Actions})$ for continuous actions (Haarnoja et al., 2018b).

4 Failure Modes of Vanilla Discrete SAC

We start by outlining the failure modes of the vanilla discrete SAC and then analyze under what circumstances the standard choices of vanilla discrete SAC perform poorly.

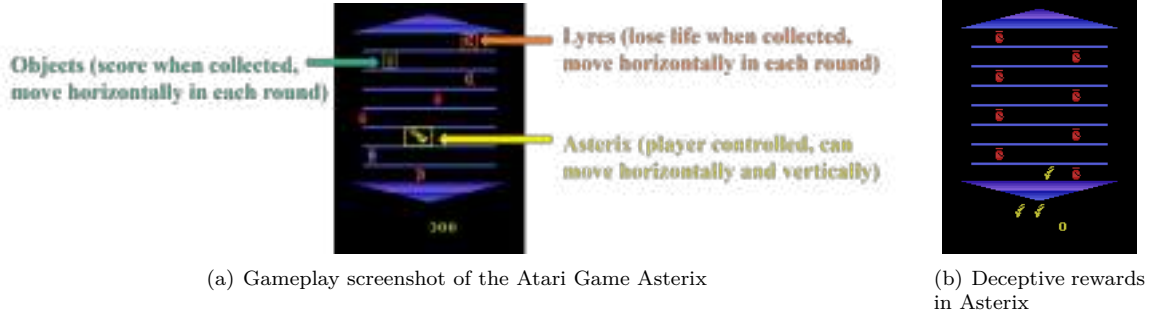


Figure 1: Gameplay screenshot of the Atari Game Asterix, including the player-controlled Asterix (yellow box), scoring objects (green box) and life-losing lyres (orange box) that appear in rounds. Deceptive rewards appear in the early stage of game when there are only scoring objects.

4.1 Unstable Coupling Training

The first failure mode comes from the instability caused by fluctuations in Q function distribution and policy entropy during training. The maximum entropy mechanism in SAC effectively balances exploration and exploitation. However, due to the existence of entropy term in the soft Bellman error, and the mechanism in discrete SAC that aligns the policy with the Q function, the policy update iteration (Eq. 8) is strongly coupled with Q-learning iteration (Eq. 5).

In environments with deceptive rewards (Hong et al., 2018), the agent can gain substantial returns in the early stages of training through short-term rewards, causing the Q value of specific actions to rise rapidly and the Q function distribution to become sharper. The coupling learning paradigm of discrete SAC leads to a sharper policy distribution and, thus, a decline in entropy. Consequently, the Q learning target becomes unstable, which can, in turn, deteriorate the policy learning. As a result, the agent falls into local optima and struggles to discover alternative strategies with larger long-term payoffs. To illustrate this issue more concretely, we take the training process of discrete SAC in the Atari game Asterix as an example.

As shown in Fig. 1(a), the player controls Asterix, which can move horizontally and vertically. In each round, horizontally moving objects appear. Asterix scores points by collecting objects and loses a life when collecting a lyre. In the early stage of the game, rounds often appear where there are only scoring objects and no life-losing lyres (Fig. 1(b)), allowing the agent to score quickly by collecting objects, resulting in deceptive rewards. These rewards make the Q function sharper, thereby reducing the entropy of the policy. In Fig. 2(a), we sample a fixed set of states, and measure the variance of Q function across different actions for these states. We find that the Q function variance increases rapidly, indicating that the Q function becomes sharp quickly. Policy entropy also decreased during this period.

As the learning process continues, the policy entropy drops rapidly, and the action probabilities become deterministic quickly (Fig. 2(c)). The agent can collect objects but struggles to avoid obstacles effectively. After the policy entropy reaches its lowest point at round 2 million steps, neither the episode length (Fig. 2(d)) nor the number of steps with rewards (Fig. 2(e)) increases significantly. At the same time, the drastic change of policy entropy misleads the learning process, and thus, both Q-value and policy fall into local optimum (Fig. 2(c) and Fig. 2(b)). Since both policy and Q-value converge to the local optimum, it becomes hard for the policy to explore efficiently in the later training stage. Even the policy entropy re-rises in the later stage (Fig. 2(c)), the performance of policy does not improve anymore (Fig. 2(f)). Similar situations also occur in other Atari environments, and we provide more examples in Appendix A.6.

To better understand why this undesirable behavior occurs, we inspect the gradient of the soft Bellman object calculated by the formula 5

$$\hat{\nabla}_{\theta} J_Q(\theta) = \nabla_{\theta} Q_{\theta}(a_t, s_t) (Q_{\theta}(s_t, a_t) - (r(s_t, a_t) + \gamma(Q_{\theta}(s_{t+1}, a_{t+1}) - \alpha \log(\pi_{\phi}(a_{t+1} | s_{t+1}))))). \quad (11)$$

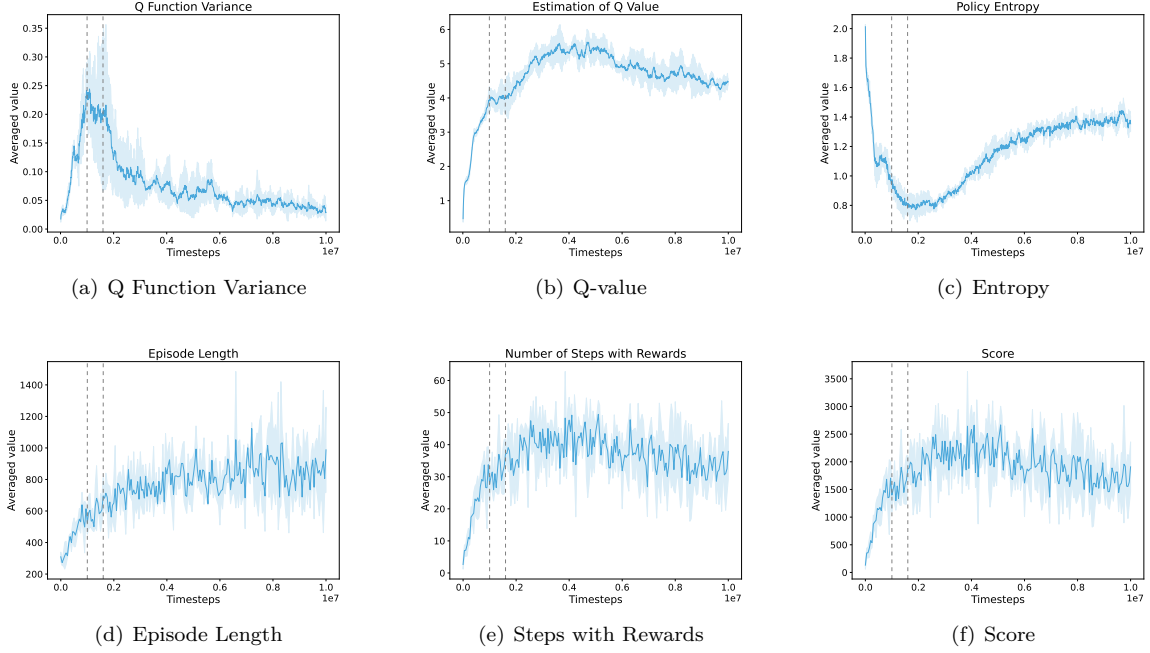


Figure 2: Measuring Q variance, estimation of Q-value, policy entropy, episode length, steps with rewards, and score on Atari Game Asterix with discrete SAC over 10 million timesteps.

As shown in Eq. 11, the improvement of $Q_\theta(s_t, a_t)$ relies on the Q-estimation of the following states and policy entropy. However, a sharper Q function causes the drastically shifting entropy, increasing the uncertainty of gradient updates and misleading the learning of the Q-network. Since the soft Q-network induces the policy, the policy can also become misleading and hurt performance. To mitigate this phenomenon, the key is to ensure the smoothness of policy entropy change to maintain stable training. In the next section, we will introduce how to constrain the policy’s randomness to ensure smooth policy changes.

4.2 Pessimistic Exploration

The second failure mode comes from pessimistic exploration due to the double Q-learning mechanism. To address the issue of overestimation in DQN, double Q-learning was proposed. This approach mitigates the problem by employing two independent Q-networks, and using the minimum value between them as the final Q-value. The concept was initially introduced by Double DQN (Van Hasselt et al., 2016) in the discrete domain. In the continuous domain, TD3 (Fujimoto et al., 2018) and SAC (Haarnoja et al., 2018a) also adopt clipped double Q-learning to mitigate overestimation, making it a favored technique across various reinforcement learning algorithms.

Empirical results demonstrate that the clipped double Q-learning trick can boost SAC performance in continuous domains, but its impact remains unclear in discrete domains. Therefore, we need to revisit clipped double Q-learning for discrete SAC.

In our experiments, in discrete domains, we find that discrete SAC tends to suffer from underestimation bias instead of overestimation bias. This underestimation bias can cause pessimistic exploration, especially in the case of sparse reward. Here, we illustrate how the popularly used clipped double Q-learning trick causes underestimation bias and how the policy used with this trick tends to converge to suboptimal actions for discrete action spaces. Our work complements previous work with a more in-depth analysis of clipped double Q-learning. We demonstrate the existence of underestimation bias and then illustrate its impact on Atari games.

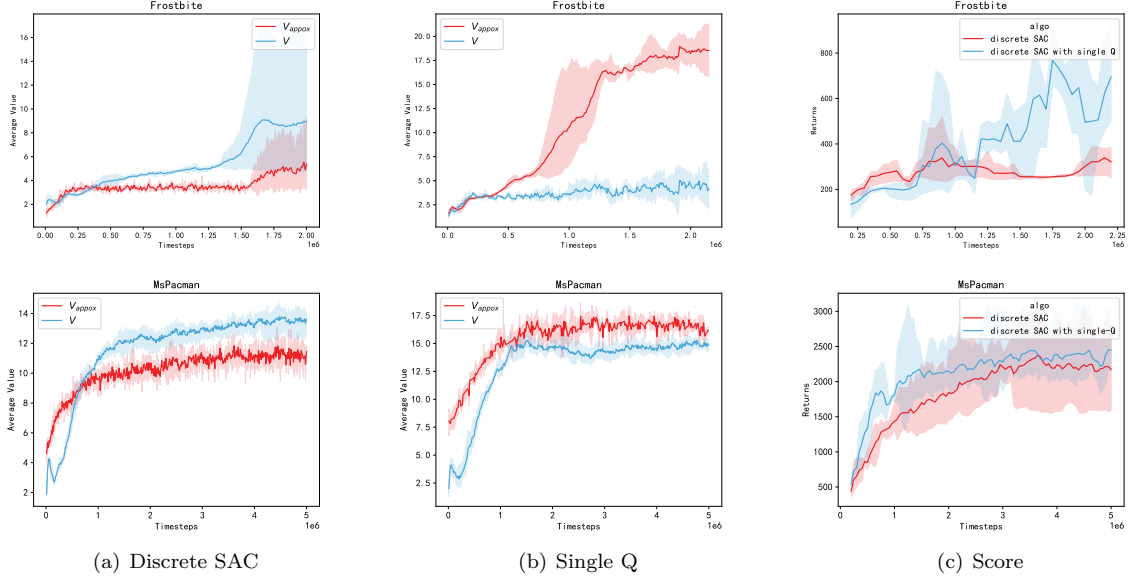


Figure 3: The results of Atari game Frostbite/MsPacman environment over 2/5 million time steps: a) Measuring Q-value estimates of discrete SAC; b) Measuring Q-value estimates of discrete SAC with single Q; c) Score comparison between discrete SAC and discrete SAC with single Q.

To analyze the estimated bias ϵ , we introduce the mathematical expression of the soft state-value function:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \log(\pi(a_t | s_t))], \quad (12)$$

where $Q(s_t, a_t)$ represents the true Q-value. In practice, SAC uses the clipped double Q-learning trick. The learning target of soft state-value function can be written as:

$$V_{approx}(s_t) = \mathbb{E}_{a_t \sim \pi} \min_{i=1,2} [Q_{\theta'_i}(s_t, a_t) - \alpha \log(\pi(a_t | s_t))], \quad (13)$$

where $Q_{\theta'_i}$ represents estimation of target-critic networks parameterized by θ'_i . The estimated bias for $Q_{\theta'_i}$ can be calculated as $\epsilon_i = Q_{\theta'_i}(s, a) - Q(s, a)$. On the one hand, when $\epsilon_1 > \epsilon_2 > 0$, the clipped double Q-learning trick can help mitigate overestimation error due to the $\min$ operation. On the other hand, when $\epsilon_1 < \epsilon_2 < 0$ or $\epsilon_1 < 0 < \epsilon_2$, the clipped double Q-learning trick will lead to underestimation (i.e., $V_{approx} < V$) and consequently result in pessimistic exploration (Pan et al., 2020; Ciosek et al., 2019).

Does this theoretical underestimate occur in practice for discrete SAC and hurt the performance? We answer this question by showing the influence of the clipped double Q-learning trick for discrete SAC in Atari games, as shown in Fig. 3. Here, we compare the true value to the estimated value. The results are averaged over three independent experiments with different random seeds. We find that, in Fig. 3(a), the approximate values are lower than the true value over time, demonstrating the underestimation bias issue. At the same time, we also run experiments for discrete SAC with a single Q (DSAC-S), which uses a single Q-value for bootstrapping instead of clipped double Q-values. As shown in Fig. 3(b), without the clipped double Q-learning trick, the estimated value of DSAC-S is higher than the true value and thus has an overestimation bias. However, in Fig. 3(c), we discover that even though DSAC-S suffers from overestimation bias, it performs much better than discrete SAC which adopts the clipped double Q-learning mechanism. This indicates that the clipped double Q-learning trick can lead to pessimistic exploration issues and hurt the agent's performance.

5 Improvements of SAC Failure Modes

We provide two simple alternatives, which are the surrogate objective with entropy-penalty and double average Q-learning with Q-clip, to avoid the two failure modes of discrete SAC discussed in Section 4. Combining these two modifications, we propose stable discrete SAC (SD-SAC). □

5.1 Entropy-Penalty

The drastic change of Q function distribution and entropy affects the optimization of the Q-value. Due to the mutual coupling of the Q function and policy training in discrete SAC, we optimize policy entropy to alleviate the unstable effect on training caused by a sharp Q function distribution and a rapid drop in entropy. Simply removing the entropy term will injure the exploration ability under the framework of maximum entropy RL. An intuitive solution is to introduce an entropy penalty in the objective of policy to avoid entropy chattering. We will introduce how to incorporate the entropy penalty in the learning process for the discrete SAC algorithm.

Recall the objective of policy in discrete SAC as in Eq. 8. For a mini-batch transition data pair (s_t, a_t, r_t, s_{t+1}) sampled from the replay buffer, we add an extra entropy term $\mathcal{H}_{\pi_{old}}$ to the transition tuple which reflects the randomness of policy (i.e., $(s_t, a_t, r_t, s_{t+1}, \mathcal{H}_{\pi_{old}})$), where π_{old} denotes the policy used for data sampling. We calculate the entropy penalty by measuring the distance between $\mathcal{H}_{\pi_{old}}$ and $\mathcal{H}_{\pi}$. Formally, the objective of the policy is as follows:

$$\begin{aligned} J_{\pi}(\phi) = & \mathbb{E}_{s_t \sim D} [\mathbb{E}_{a_t \sim \pi_{\phi}} [\alpha \log(\pi_{\phi}(a_t | s_t)) - Q_{\theta}(s_t, a_t)]] \\ & + \beta \cdot \frac{1}{2} \mathbb{E}_{s_t \sim D} ([\mathbb{E}_{a_t \sim \pi_{\phi_{old}}} [-\log(\pi_{\phi_{old}})] - \mathbb{E}_{a_t \sim \pi_{\phi}} [-\log(\pi_{\phi})])^2, \end{aligned} \quad (14)$$

where $\mathbb{E}_{a_t \sim \pi_{\phi_{old}}} [-\log(\pi_{\phi_{old}})]$ represents policy entropy of $\pi_{\phi_{old}}$, $\mathbb{E}_{a_t \sim \pi_{\phi}} [-\log(\pi_{\phi})]$ represents policy entropy of π_{ϕ} , and β denotes a coefficient for the penalty term and is set to 0.5 in this paper. By constraining the policy objective with this penalty term, we increase the stability of the policy learning process.

Fig. 4 shows the training curves to demonstrate how the entropy penalty mitigates the failure mode of policy drastic change. In Fig. 4(b), the entropy of discrete SAC (the blue curve) drops quickly, and the policy falls into a local optimum at the early training stage. Later, the policy stops improving and even suffers from performance deterioration, as shown in the blue curves in Fig. 4(c) and Fig. 4(d). □

On the contrary, our proposed method (i.e., discrete SAC with entropy-penalty) demonstrates better stability than discrete SAC. As shown in Fig. 4(a), entropy penalty effectively constrains the sharpness of Q function, as a result, the policy changes smoothly during training (Fig. 4(b)). Consequently, compared with discrete SAC, the policy in our approach can keep improving during the whole training stage and does not suffer from a performance drop at the later training stage (the red curves in Fig. 4(c) and Fig. 4(d)). □

It is worth noting that, since the instability in training mainly manifests as the existence of policy entropy term in optimization, imposing constraints in the entropy space is more effective than constraints in the policy space. Other common methods, such as the KL penalty, limit the magnitude of policy updates and impose additional restrictions on policy updates. This is proved in experiments: KL penalty (the yellow curve) cannot effectively constrain the rise in Q variance (Fig. 4(a)) and the decrease in entropy (Fig. 4(b)). Consequently, the final Q-value and score of the KL penalty are lower than those with the entropy penalty, with a difference of 12% and 23%, respectively. □

The entropy-penalty term $\frac{1}{2} \mathbb{E}_{s_t \sim D} ([\mathbb{E}_{a_t \sim \pi_{\phi_{old}}} [-\log(\pi_{\phi_{old}})] - \mathbb{E}_{a_t \sim \pi_{\phi}} [-\log(\pi_{\phi})])^2$, in conjunction with the temperature α , jointly regulates the exploration of policy. Different from other trust region methods such as KL constraint (Schulman et al., 2015) or clipping surrogate objective (Schulman et al., 2017), our method penalizes the change of action entropy between old and new policies to address policy instability during training. By adding regularization in entropy space instead of policy space, our method can mitigate the drastic changes of policy entropy while maintaining the inherent exploratory ability of discrete SAC (as shown in Fig. 4(b)), the policy entropy changes smoothly. It keeps at a relatively high value to encourage exploration). □

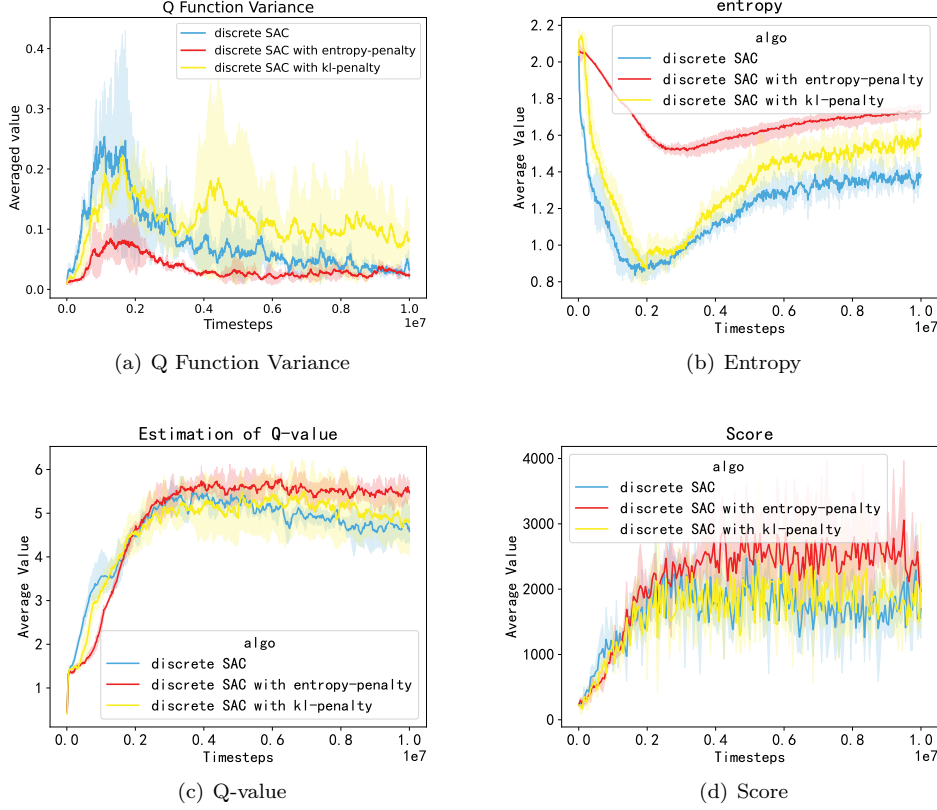


Figure 4: Measuring Q function variance, policy action entropy, estimation of Q-value, and score on Atari game Asterix compared between discrete SAC, discrete SAC with KL-penalty and discrete SAC with entropy-penalty over 10 million time steps.

5.2 Double Average Q-learning with Q-clip

While several approaches (Ciosek et al., 2019; Pan et al., 2020) have been proposed to reduce underestimation bias, they are not straightforward to be applied to discrete SAC due to the use of Gaussian distribution. In this section, we introduce a novel variant of double Q-learning to mitigate the underestimation bias for discrete SAC.

In practice, discrete SAC uses clipped double q-learning with a pair of target critics (Q_{θ_1} , Q_{θ_2}), and the learning target of these two critics is:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i}(s', \pi(s')). \quad (15)$$

When neural networks approximate the Q-function, there exists an unavoidable bias in the critics. Since policy is optimized concerning the low bound of double critics, for some states, we will have $Q_{\theta_2}(s, \pi_\phi(s)) > Q_{true} > Q_{\theta_1}(s, \pi_\phi(s))$. This is problematic because $Q_{\theta_1}(s, \pi_\phi(s))$ will generally underestimate the true value, and this underestimated bias will be further exaggerated during the whole training phase, which results in pessimistic exploration.

To address this problem, we propose to mitigate the underestimation bias by replacing the *min* operator with *avg* operator. This results in taking the average between the two estimates, which we refer to as *double average Q-learning*:

$$y = r + \gamma \cdot \text{avg}(Q_{\theta_1}(s', \pi(s')), Q_{\theta_2}(s', \pi(s'))). \quad (16)$$

By doing so, the other critic can mitigate the underestimated bias of the lower bound of double critics. To improve the stability of the Q-learning process, inspired by value clipping in PPO (Schulman et al., 2017), we further add a clip operator on the Bellman error to prevent drastic updates of the Q-network. The modified Bellman loss of Q-network is as follows:

$$\mathcal{L}(\theta_i) = \max \left((Q_{\theta_i} - y)^2, (Q_{\theta_i} + \text{clip}(Q_{\theta_i} - Q_{\theta_i'}, -c, c) - y)^2 \right), \quad (17)$$

where Q_{θ_i} represents the critic network’s estimate, $Q_{\theta_i'}$ represents estimation of target-critic networks, and c is the hyperparameter denoting the clip range. This clipping operator prevents the Q-network from performing an incentive update beyond the clip range. In this way, the Q-learning process is more robust to the abrupt change in data distribution. Combining the clipping mechanism (Eq. 17) with double average Q-learning (Eq. 16), we refer to our proposed approach as *double average Q-learning with Q-clip*.

Fig. 5 demonstrates the effectiveness of our approach. We compare the discrete SAC and various ensemble Q-estimation methods, including Randomized Ensembled Double Q-learning (REDQ) (Chen et al., 2021b) and Random Ensemble Mixture (REM) (Agarwal et al., 2020), with our proposed method, SD-SAC. In Fig. 5(a), the Q-value estimate of discrete SAC is underestimated than the true value. Therefore, the policy of discrete SAC suffers from pessimistic exploration and results in poor performance (purple curve in Fig. 5(e)). On the contrary, in Fig. 5(d), with double average Q-learning and Q-clip, the Q-value estimate eliminates underestimation bias and improves quickly at the early training stage. The improvement of Q-value carries over to the performance of policy. Consequently, our approach outperforms baseline discrete SAC by a large margin (Fig. 5(e)). The result also demonstrates that even though REDQ has less estimation bias in Fig. 5(b), it still suffers from underestimation bias, leading to suboptimal performance due to pessimistic exploration. Although REM addresses the underestimation issue in Fig. 5(c), the overestimation bias of REM significantly exceeds that of our proposed method, resulting in a rapid decline in performance at 8 million steps. In Fig. 5(d), we also notice that the Q-value overestimates the true value during the early training stage but finally converges to the true value after the training process. This encourages early exploration, which is consistent with the principle of optimism in the face of uncertainty (Kearns & Singh, 2002).

5.3 Pseudocode

Finally, we provide the pseudo code for SD-SAC (i.e., Stable Discrete SAC with entropy-penalty and double average Q-learning with Q-clip), as shown in Algorithm 1.

6 Experiments

6.1 Experimental Setup

To evaluate our algorithm, we compare our SD-SAC with most related baselines, i.e., discrete SAC (Christodoulou, 2019), TES-SAC (Xu et al., 2021), Soft-DQN (Vieillard et al., 2020) and Rainbow (Hessel et al., 2018) which is widely accepted algorithm in the discrete domain. We measure their performance in 20 Atari games chosen as the same as (Christodoulou, 2019) for a fair comparison. We evaluate for 10 episodes for every 50000 steps during training, and execute 3 random seeds for each algorithm for 10 million environment steps (or 40 million frames). For the baseline implementation of discrete-SAC, we use Tianshou. We find that Tianshou’s implementation performs better than the original paper by Christodoulou (Christodoulou, 2019), thus we use the default hyperparameters in Tianshou on all 20 games.

We start the game with up to 30 no-op actions, similar to (Mnih et al., 2013), to provide the agent with a random starting position. To obtain summary statistics across games, following Hasselt (Van Hasselt et al., 2016), we normalize the score for each game as follows: $\text{Score}_{\text{normalized}} = \frac{\text{Score}_{\text{agent}} - \text{Score}_{\text{random}}}{\text{Score}_{\text{human}} - \text{Score}_{\text{random}}}$.

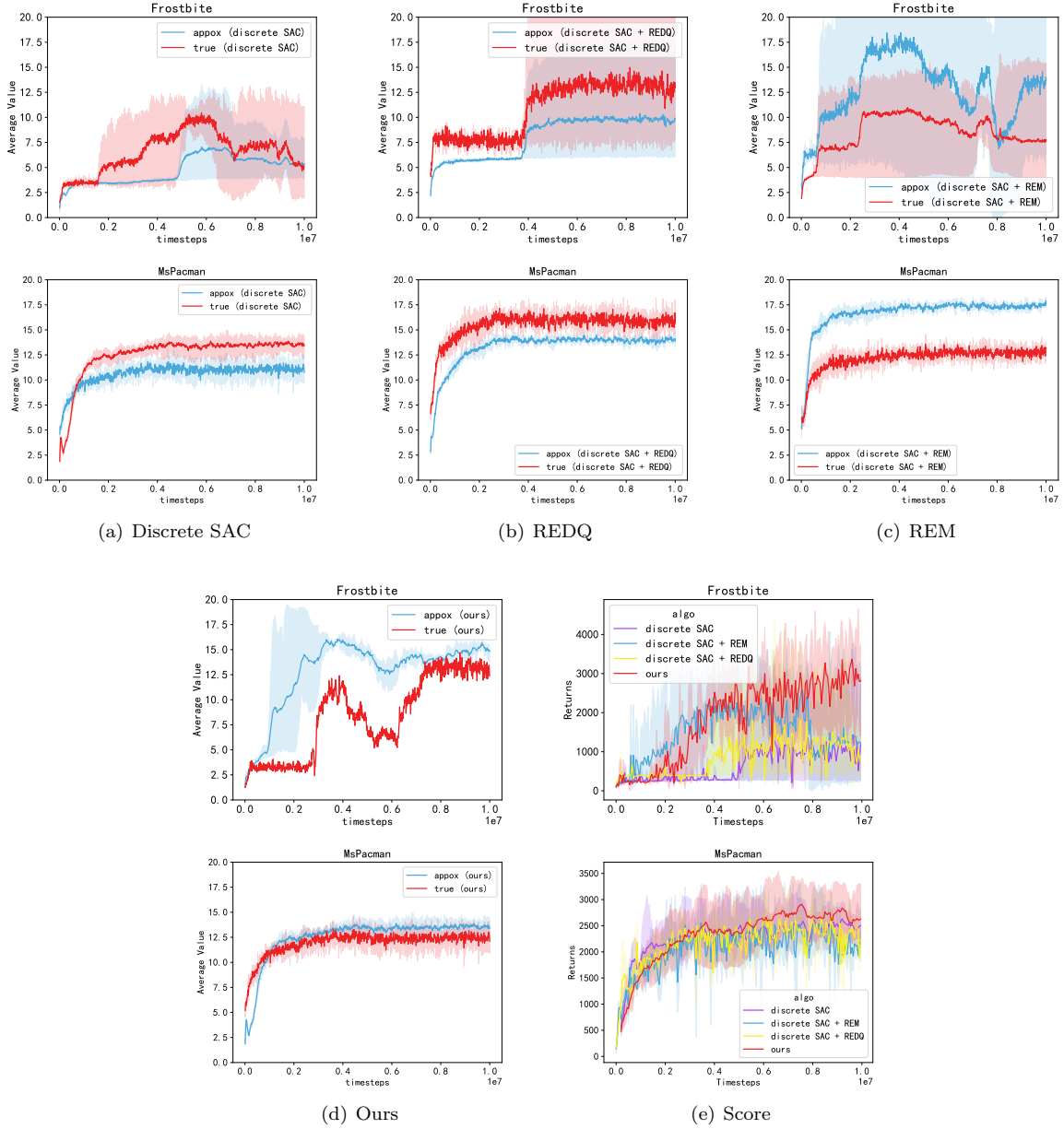


Figure 5: Measuring estimation of Q-value and score on Atari Game Frostbite/MsPacman environment compared between discrete SAC, discrete SAC with REDQ, discrete SAC with REM, and ours (SD-SAC) over 10 million steps.

Table 1: Mean and median normalized scores of discrete SAC, TES-SAC, Rainbow, Soft-DQN and our method across all 20 Atari games at 1M and 10M steps.

	Discrete SAC(1M)	TES-SAC(1M)	Soft-DQN(1M)	Ours(1M)	Rainbow(10M)	Discrete SAC(10M)	Soft-DQN(10M)	Ours(10M)
Mean	0.5%	3.0%	41.7%	38.5%	187.4 %	151.4%	199.2%	220.0%
Median	0.4%	2.1%	20.0%	11.1%	79.2 %	90.8%	107.7%	114.1%

Algorithm 1 SD-SAC: Stable Discrete SAC with entropy-penalty and double average Q-learning with Q-clip

Input: θ_1, θ_2, ϕ ▷ Initial parameters
 Output: θ_1, θ_2, ϕ ▷ Optimized parameters
 Hyperparameters: γ, β, c, τ
 Initialise $Q_{\theta_1} : S \rightarrow \mathbb{R}^{|A|}, Q_{\theta_2} : S \rightarrow \mathbb{R}^{|A|}, \pi_\phi : S \rightarrow [0, 1]^{|A|}$ ▷ Initialise local networks
 Initialise $Q_{\theta'_1} : S \rightarrow \mathbb{R}^{|A|}, Q_{\theta'_2} : S \rightarrow \mathbb{R}^{|A|}$ ▷ Initialise target networks
 $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2$ ▷ Equalise target and local network weights
 $\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay buffer
for each iteration **do**
 for each environment step **do**
 $a_t \sim \pi_\phi(a_t | s_t)$ ▷ Sample action from the policy
 $s_{t+1} \sim p(s_{t+1} | s_t, a_t)$ ▷ Sample transition from the environment
 $\mathcal{H}_{\pi_{old}} \sim \mathbb{E}_{a \sim \pi_\phi(\cdot | s_t)} [-\log \pi_\phi(a | s_t)]$ ▷ Calculate the entropy $\mathcal{H}_{\pi_{old}}$ of the current policy ϕ
 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1}, \mathcal{H}_{\pi_{old}})\}$ ▷ Store the transition in the replay buffer
 end for
 for each gradient step **do**
 $y \sim r(s_t, a_t) + \gamma \cdot \text{avg}(Q_{\theta'_1}(s_{t+1}, \pi(s_{t+1})), Q_{\theta'_2}(s_{t+1}, \pi(s_{t+1})))$ ▷ Double average Q-value estimation
 $\mathcal{L}(\theta_i) \sim \max \left((Q_{\theta_i} - y)^2, (Q_{\theta_i} + \text{clip}(Q_{\theta_i} - Q_{\theta'_i}, -c, c) - y)^2 \right)$ for $i \in \{1, 2\}$ ▷ Clip the Q-value estimation from target critic network
 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} \mathcal{L}(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
 $\mathcal{H}_\pi \sim \mathbb{E}_{a \sim \pi_\phi(\cdot | s_t)} [-\log \pi_\phi(a | s_t)]$ ▷ Calculate the entropy $\mathcal{H}_\pi$ of policy ϕ
 $J_\pi(\phi) \sim \mathbb{E}_{s_t \sim \mathcal{D}} [\mathbb{E}_{a_t \sim \pi_\phi} [\alpha \log(\pi_\phi(a_t | s_t)) - Q_\theta(s_t, a_t)]] + \beta \cdot \frac{1}{2} (\mathcal{H}_{\pi_{old}} - \mathcal{H}_\pi)^2$
 $\phi \sim \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
 $\alpha \sim \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$ ▷ Update temperature
 $Q_{\theta'_i} \leftarrow \tau Q_{\theta_i} + (1 - \tau) Q_{\theta'_i}$ for $i \in \{1, 2\}$ ▷ Update target network weights
 end for
end for

Table 2: Raw scores across all 20 Atari games. For methods discrete SAC (1M) and TES-SAC(1M), the scores come from the corresponding paper, and the NE means the score does not exist in the original paper.

Game	Discrete SAC (1M)	TES-SAC(1M)	Soft-DQN(1M)	Ours(1M)	Rainbow(10M)	Discrete SAC (10M)	Soft-DQN(10M)	Ours (10M)
Alien	216.90	685.93	726.33	981.67	1798.33	2717.67	2018.00	2158.33
Amidar	7.9	42.07	130.03	132.97	394.23	354.77	438.80	407.20
Assault	350.0	337.03	881.97	1664.77	1802.53	7189.97	7258.87	6785.60
Asterix	272.0	378.5	676.67	733.33	5853.33	2860.00	3761.67	5993.33
BattleZone	4386.7	5790	7933.33	6266.67	24266.67	16850.00	24733.33	9466.67
BeamRider	432.1	NE	3321.60	3468.60	3310.40	7169.60	7048.20	10506.60
Breakout	0.7	2.65	46.17	11.47	492.93	29.03	155.83	60.43
CrazyClimber	3668.7	4.0	25390.00	20753.33	30286.67	126320.00	95156.67	140726.67
Enduro	0.8	NE	54.23	0.93	1517.70	1326.77	1144.07	2246.40
Freeway	4.4	13.57	17.70	20.17	20.13	15.73	32.30	20.17
Frostbite	59.4	81.03	294.33	347.00	4163.67	646.33	2959.00	4806.00
Jamesbond	68.3	31.33	273.33	368.33	656.67	1386.67	965.00	2085.00
Kangaroo	29.3	307.33	160.00	120.00	3716.67	2426.67	2703.33	5556.67
MsPacman	690.9	1408	1528.00	1639.00	2738.67	3221.33	2386.33	3175.67
Pong	-20.98	-20.84	20.00	15.53	20.93	20.37	20.73	20.37
Qbert	280.5	74.93	1400.83	986.67	15299.17	12946.67	14293.33	15325.83
RoadRunner	305.3	NE	5510.00	12793.33	45173.33	34043.33	33370.00	43203.33
SpaceInvaders	160.8	NE	488.83	383.50	1330.50	458.83	816.00	586.50
Seaquest	211.6	116.73	681.33	744.00	2105.33	1853.33	3438.67	2764.00
UpNDown	250.7	207.6	8727.33	8114.67	9110.00	17803.33	79313.00	63441.33

6.2 Overall Performance

Table 1 provides an overview of results and detailed results are presented in the Table 2 and Appendix A.1. Since TES-SAC is not open-sourced and our re-implement algorithm following the paper underperforms the

¹ <https://github.com/thu-ml/tianshou>

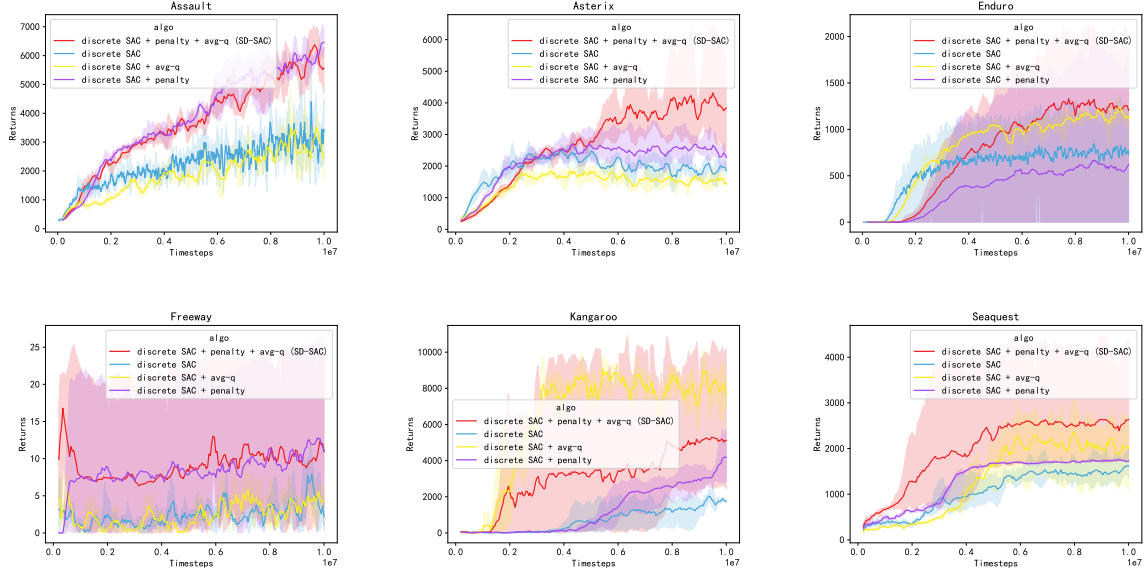


Figure 6: Scores of variant discrete SAC, which includes discrete SAC, discrete SAC with entropy-penalty, discrete SAC with double average Q learning with Q-clip, for Atari games Assault, Asterix, Enduro, Freeway, Kangaroo and Seaquest.

reported results, we adopt the normalized scores of discrete SAC and TES-SAC reported in the corresponding publication (Xu et al., 2021). When comparing our method to the discrete SAC and TES-SAC, mean normalized scores increase by 38% and 35.5%, respectively. And our method improves the median normalized scores by 10.7% and 9.0% while compared with discrete SAC and TES-SAC.

To verify the effect of a longer training process, table 1 also compares discrete SAC, Rainbow, Soft-DQN, and our method performance on 10 million steps. Compared with discrete SAC, our method has improved the normalized scores by 68.6% and 23.3% on mean and median, respectively. Additionally, our proposed method outperformed Rainbow by 32.6% on the mean and by 34.9% on the median. Better Q-estimation and steady policy updates are responsible for the performance increase in average scores. The experimental results demonstrate that benefiting from the deterministic greedy policy and entropy regularization in the evaluation step, Soft-DQN’s performance improves rapidly in the early stages and achieves the best results at 1 million steps. However, due to the early convergence of the deterministic greedy policy, Soft-DQN’s performance stagnates after 4 million steps, as seen in Fig. 8. Our method outperforms Soft-DQN in the final 10 million steps by 20.8% on average and 6.4% on median, due to the training stability brought by entropy penalty and the optimistic exploration altered by the double avg-Q with Q-clip.

6.3 Ablation Study

Fig. 6 shows the learning curves for 6 environments. Entropy-penalty (purple curve) increases performance compared to the discrete SAC in each of the six environments and even increases 2x scores in Assault. This shows that discrete SAC can perform excellently after removing unstable training. Except for Asterix, the alternative choice of clipped double Q-learning, which is double average Q learning with Q-clip (yellow curve), also shows some improvement compared to the discrete SAC in 5 environments. Additional improvements can be derived when the combination of both alternative design choices is used simultaneously.

To evaluate the influence of hyperparameter tuning, we also conducted a comprehensive hyperparameter analysis. By experimenting with different α and learning rate in the discrete SAC algorithm, we identify the performance upper bound of discrete SAC. The results show that, under various β values, SD-SAC consistently outperforms this upper bound, demonstrating that entropy penalty serves as a better and more

balanced constraint. This further confirms that SD-SAC can significantly achieve a more stable training process. We present the experiment details and results in Appendix B.2.1

6.4 Qualitative Analysis

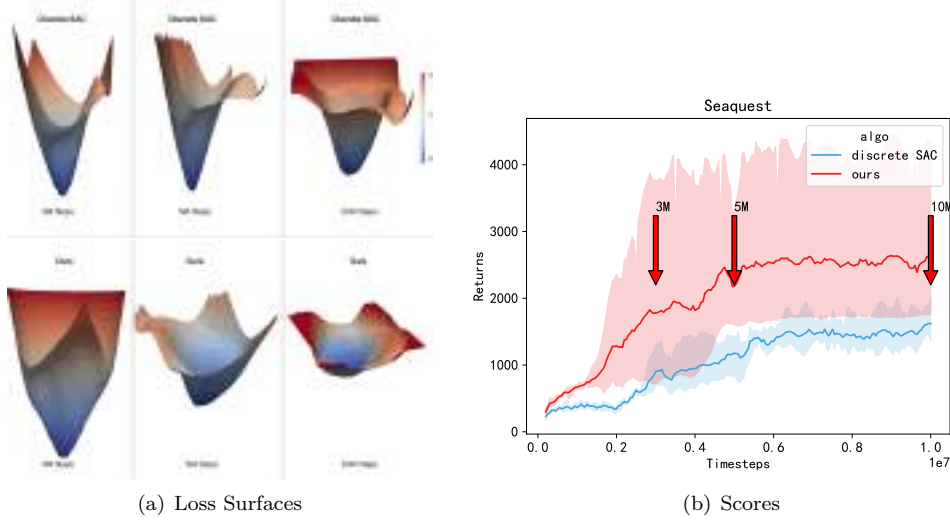


Figure 7: The loss surfaces of discrete SAC and our method on Atari game Seaquest with trained weights at 3 million, 5 million and 10 million steps.

Fig. 7 shows loss surfaces of the discrete SAC and our method by using the visualization method proposed in (Li et al., 2018; Ota et al., 2021) with the loss of TD error of Q functions. According to the sharpness/flatness in these two sub-figures, our method has a nearly convex surface, while discrete SAC has a more complex loss surface. The surface of our method has fewer saddle points than the discrete SAC, which further shows that it can be more smoothly optimized during the training process.

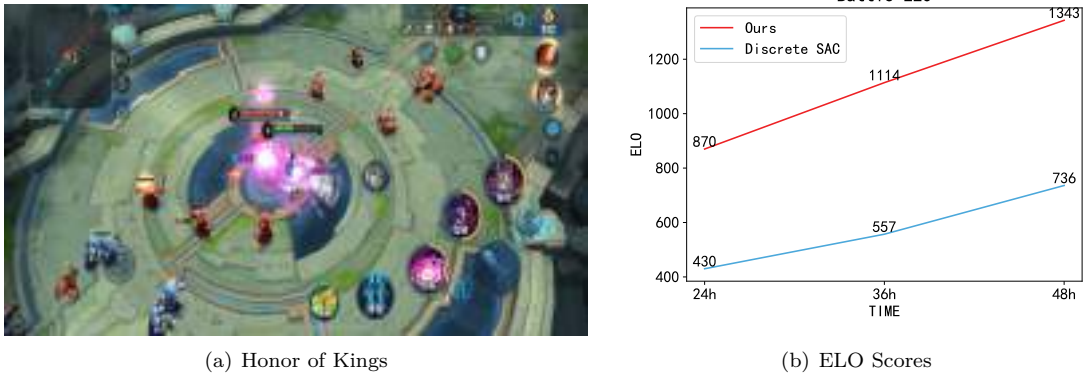


Figure 8: a) A screenshot of the Honor of Kings 1v1 game. b) The ELO scores, compared with discrete SAC and our method, were tested for three snapshots of 24, 36, and 48 hours during training.

6.5 Case Study using Honor of Kings

We further deploy our method into Honor of Kings 1v1, a commercial game in the industry, to investigate the scale-up ability of our proposed SD-SAC algorithm.

Honor of Kings² is a popular MOBA (Multiplayer Online Battle Arena) game and a good testbed for RL research (Ye et al., 2020b;ca; Chen et al., 2021a; Wei et al., 2022). The game descriptions are in (Ye et al., 2020ca). In our experiments, we use the one-versus-one mode (1v1 solo), with both sides being the same hero: Diao Chan. The state of the game is represented by feature vectors, as reported in (Ye et al., 2020c; Wei et al., 2022). The action space is discrete, i.e., we discretize the direction of movement and skill, same to (Ye et al., 2020ca). The goal of the game is to destroy the opponent’s turrets and base crystals while protecting its own. We use the ELO rating system (Elo & Sloan 1978), which calculate scores from the win rate, to measure the ability of two agents. A detailed introduction of the ELO system is presented in Appendix A.5.

We selected three snapshots of 24, 36, and 48 hours during the training process, resulting in 6 agents (SD-SAC-24h, SD-SAC-36h, SD-SAC-48h, DSAC-24h, DSAC-36h, DSAC-48h). We conducted 48 one-on-one matches for each agent, resulting in a total of 720 matches and thus serving as the basis of ELO calculation.

The results are shown in Fig. 8. Throughout the entire training period, our method outperforms discrete SAC by a significant margin, which indicates our method’s efficiency in large-scale cases. Specifically, SD-SAC-48h achieved 35 wins, 7 draws, and 6 losses, with a win rate of 72.92%. The agent also exhibits higher skill hit rate, higher Kill/Death ratio and better turret-dashing ability.

7 Conclusions and Future Work

Many algorithmic design choices in reinforcement learning are limited to the regime of the chosen benchmark tasks. We highlight that soft actor-critic (SAC), that widely accepted design choices in continuous action space do not necessarily generalize to new discrete environments. We conduct failure mode analysis and obtain two main insights: 1) due to the deceptive reward, the unstable coupling update of policy and Q function will further disturb training; 2) the underestimation bias caused by double Q-learning results in the agent’s pessimistic exploration and inefficient sample usage. We thereby propose two alternative design choices for SAC: entropy-penalty and double-average Q-learning with Q-clip, resulting in a new algorithm, called SD-SAC. Experiments show that our alternative design choices increase the training stability and Q-value estimation accuracy, which ultimately improves overall performance. In addition, we also apply our method to the large-scale MOBA game Honor of Kings 1v1 to show the scalability of our optimizations.

Finally, the success obscures certain flaws, one of which is that our improved discrete SAC still performs poorly in instances involving long-term decision-making. One possible reason is that SAC can not accurately estimate the future only by rewarding the current frame. In order to accomplish long-term choices with SAC, our next study will concentrate on improving the usage of the incentive signal across the whole episode.

References

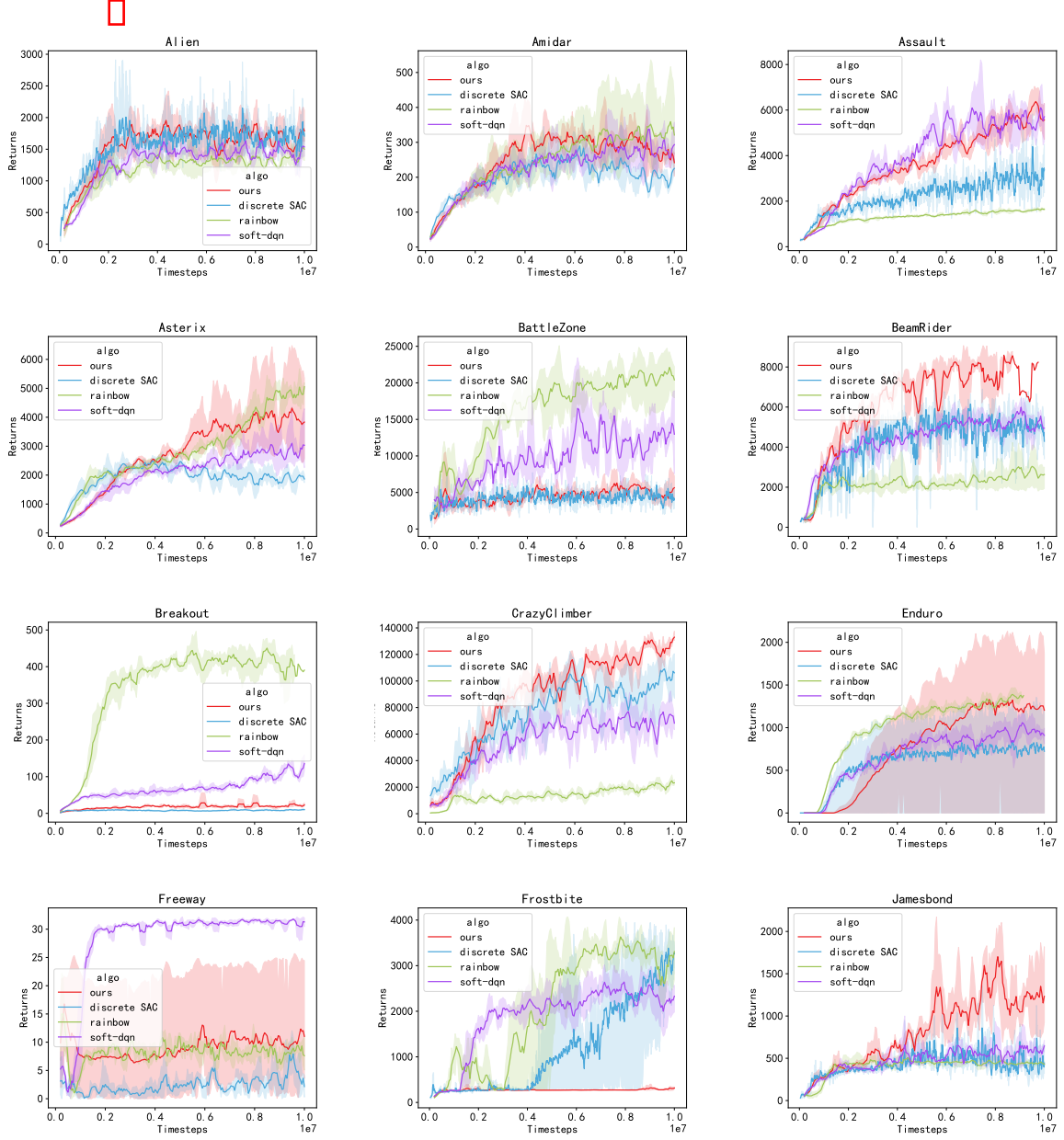
- Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.
- Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event, volume 119 of Proceedings of Machine Learning Research*, pp. 104–114. PMLR, 2020. URL <http://proceedings.mlr.press/v119/agarwal20c.html>.
- Chayan Banerjee, Zhiyong Chen, and Nasimul Noman. Improved soft actor-critic: Mixing prioritized off-policy samples with on-policy experiences. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pp. 449–458. PMLR, 2017.

²https://github.com/tencent-ailab/hok_env

A Additional Details and Experiment Results

A.1 Detailed Experiment Results on 20 Atari Game Environments

In Figure 8 we present the learning curves of all 20 experiments.



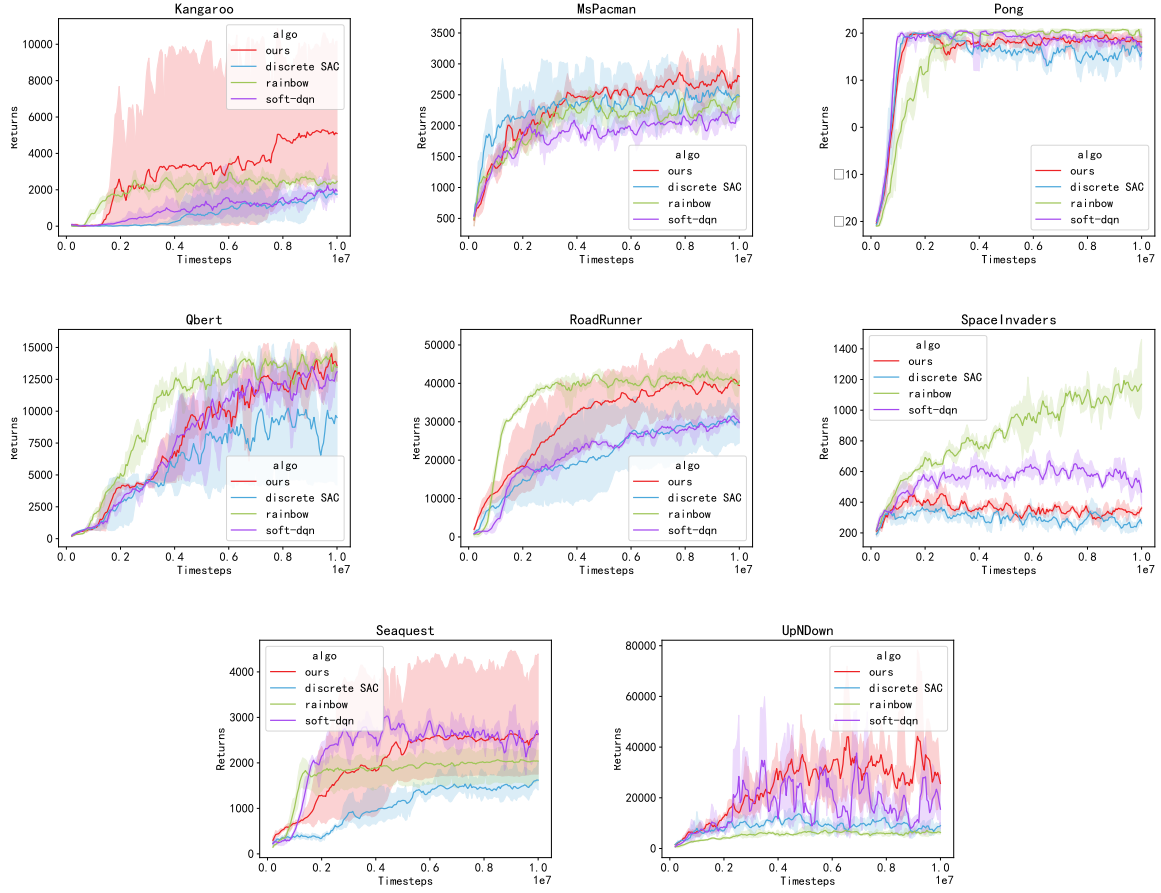


Figure 8: Learning curves for discrete SAC, Rainbow, Soft-DQN, and ours, for each game. Every curve is smoothed with a moving average of 10 to improve readability.

A.2 Further Comparison of SD-SAC and DSAC

Beyond the comparison of Figure 4, we further measure SD-SAC and DSAC in terms of episode length and number of steps with rewards in Figure 9. After $2e6$ steps, the algorithm with entropy penalty demonstrates significant longer episode lengths and more reward steps compared to discrete SAC. This indicates that the entropy penalty helps the agent learn both scoring and avoidance skills, leading to continued performance improvement.

A.3 Result Curves of Individual Runs

For clearer demonstration, we show the results of individual run curves for figures during our major analysis in Section 4 and 5.

A.3.1 Comparison Between True Q Values and Estimate Q Values

We plot Figure 3 by individual runs in Figure 10. The results show that for each individual seed, discrete SAC consistently suffers from an underestimation problem, while using a single Q leads to an overestimation issue.

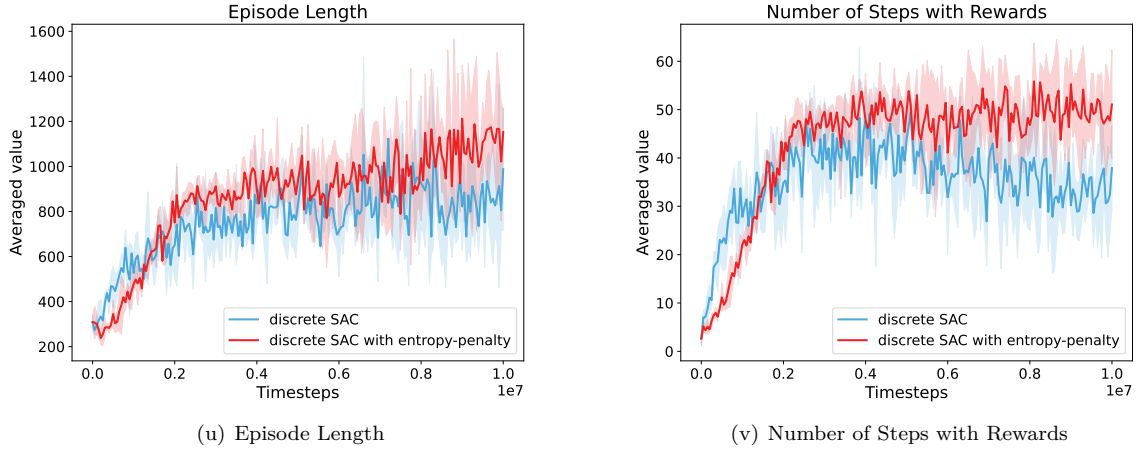


Figure 9: Comparison of SD-SAC and DSAC in terms of episode length and number of steps with rewards in Asterix.

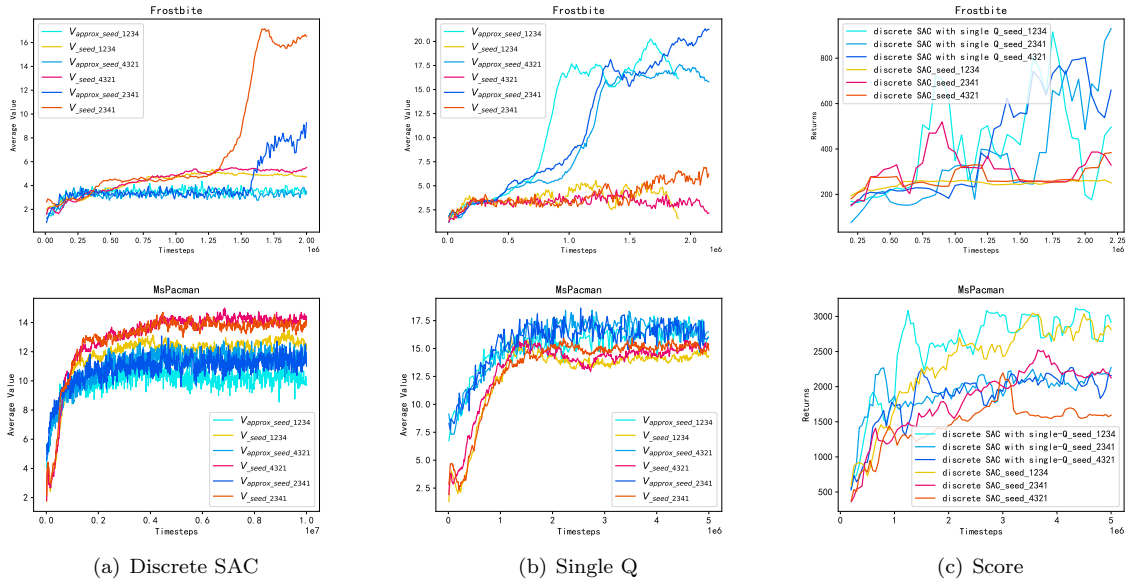


Figure 10: The results of Atari game Frostbite/MsPacman environment over 2/5 million time steps: a) Measuring Q-value estimates of discrete SAC; b) Measuring Q-value estimates of discrete SAC with single Q; c) Score comparison between discrete SAC and discrete SAC with single Q.

A.3.2 Comparison Between Different Policy Constraints

We provide Figure 4 by individual runs in Figure 11. The results show that entropy penalty enables a stable training and better performance across all seeds.

A.3.3 Comparison Between Different Q-value Estimation Methods

We provide Figure 5 by individual runs in Figure 12. Our approach demonstrates effectiveness in alleviating underestimation and reduce bias in all individual runs.

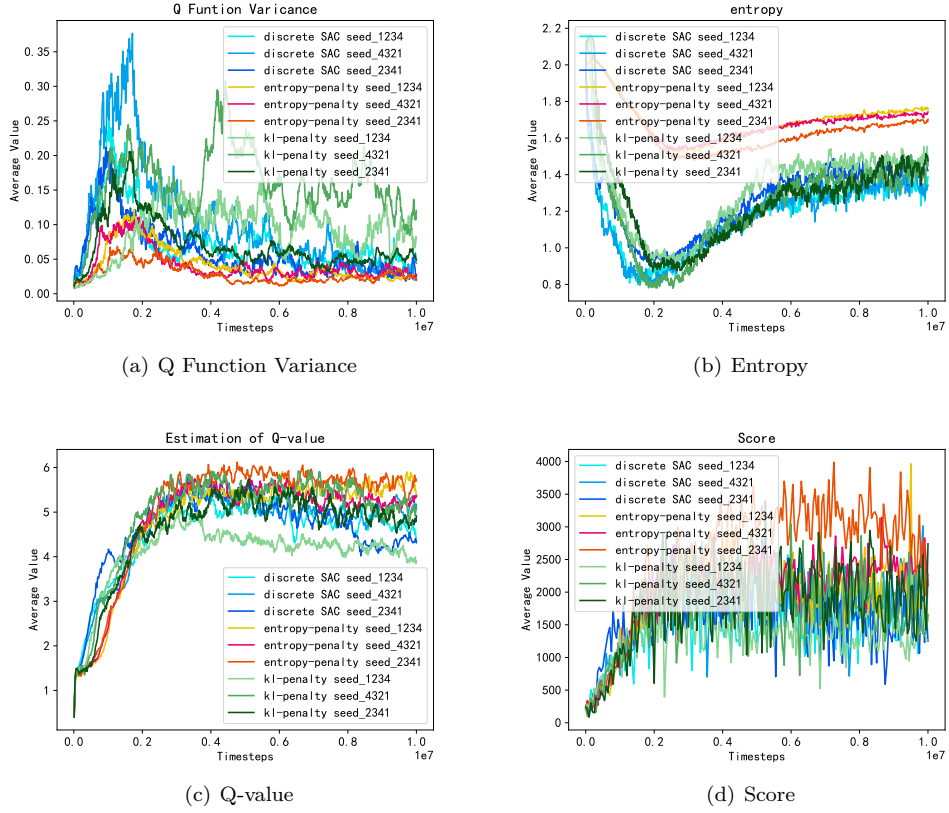


Figure 11: Measuring Q function variance, policy action entropy, estimation of Q-value, and score on Atari game Asterix compared between discrete SAC, discrete SAC with KL-penalty and discrete SAC with entropy-penalty over 10 million time steps.

Table 3: Hyperparameter for Discrete SAC and SD-SAC

Hyperparameter	Discrete SAC	SD-SAC
learning rate	10^{-5}	10^{-5}
optimizer	Adam	Adam
mini-batch size	64	64
discount (γ)	0.99	0.99
buffer size	10^5	10^5
hidden layers	2	2
hidden units per layer	512	512
target smoothing coefficient (τ)	0.005	0.005
Learning iterations per round	1	1
alpha	0.05	0.05
n-step	3	3
β	False	0.5
c	False	0.5

A.4 Hyperparameter Used in SD-SAC

Please refer to Table 3.

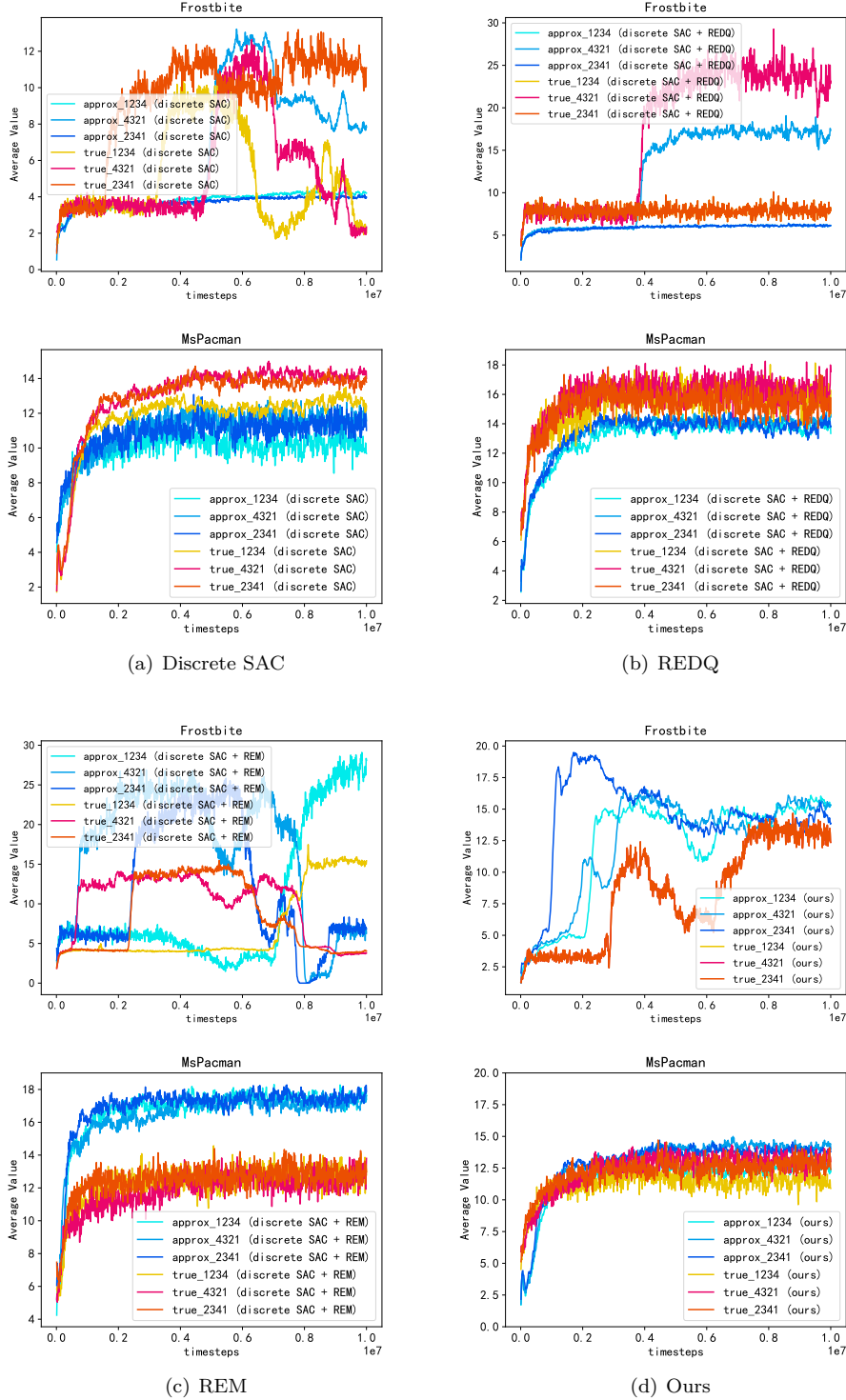


Figure 12: Measuring estimation of Q-value on Atari Game Frostbite/MsPacman environment compared between discrete SAC, discrete SAC with REDQ, discrete SAC with REM, and SD-SAC (discrete SAC with double average Q-learning with Q-clip) over 10 million steps.

A.5 Introduction of the ELO System

The ELO rating system (Elo & Sloan, 1978) is a widely-used mechanism for assessing the relative skill levels of players or agents, commonly applied in chess, competitive games, and other adversarial environments. In our study, the ELO ratings of agents are calculated through the following process:

1. Each agent is assigned an initial ELO rating R_{base} .
2. Before agent A competes against agent B, the expected score for each agent is calculated based on their current ELO ratings: $E_A = \frac{1}{1+10^{(R_B-R_A)/R_{base}}}$
3. The ELO ratings are updated based on the outcome of the match between A and B, where S_A is the actual score and K is a constant: $R'_A = R_A + K \cdot (S_A - E_A)$
4. Through multiple matches among various agents, the ELO ratings are adjusted according to the results of these matches. The final ELO ratings reflect each agent's relative strength compared to the others.

A.6 Other Atari Environments for Unstable Coupling Training of Discrete SAC

We conduct cross-validation in other Atari environments, as presented in Fig. 14-16. The result show that in other environments with deceptive rewards, the rapid decrease in policy entropy due to target Q variance similarly affects training.



A.6.1 Games with Deceptive Rewards

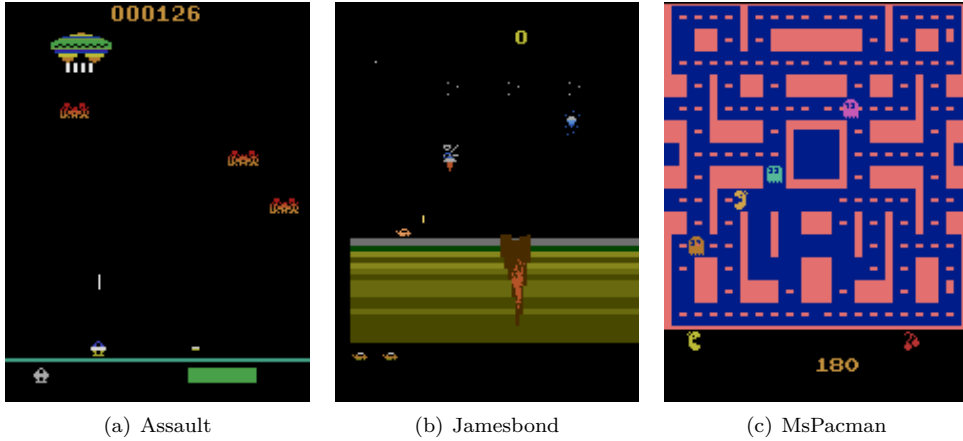


Figure 13: Three examples of Atari game environments with deceptive rewards.

We take the Atari games Assault, Jamesbond and MsPacman as examples to further illustrate the manifestation and impact of deceptive rewards in game environments. (Fig. 13)

In the game Assault, a mothership releases different kinds of aliens. They move along the screen, with the bottom-most alien firing various types of weapons. The player controls a cannon that can shoot bullets horizontally or vertically to attack the aliens and fireballs they shot. Hitting an alien scores points, while being hit or cannon overheating results in a loss of life.

In Jamesbond, the player controls a craft that needs to complete various mission to achieve final victory. In the first mission, the player must navigate through a desert with craters, avoid overhead satellite scans and helicopter bombings, and score points by hitting diamonds through fixed-angle shooting.

As for MsPacman, the player controls a Pacman, who scores points by eating dots in a maze while avoiding floating ghosts. When Pacman eats an energy pill, she can attack the ghosts to gain higher scores.

In all three environments, the agent can quickly gain deceptive rewards through short-term payoffs. For Assault and Jamesbond, all points come from shooting actions that hit specific targets, while avoiding obstacles can prevent the loss of life but does not bring clear rewards. Thus, agents often excel at shooting but struggle with dodging. In MsPacman, the numerous dots in the maze provide many rewards for the agent’s movement. As a result, the agent finds it difficult to learn advanced strategies such as avoiding ghosts and picking up energy pills to attack ghosts. The presence of deceptive rewards leads to the training process stuck in local optima, making it challenging to explore better, long-term strategies.

A.6.2 Plots of Training Process

We present the training process in the three aforementioned environments with deceptive rewards in Fig. 14-16. It can be observed that in each case, deceptive rewards cause a rapid increase in Q variance and a decrease in policy entropy, leading the training process to fall into local optima.

□□

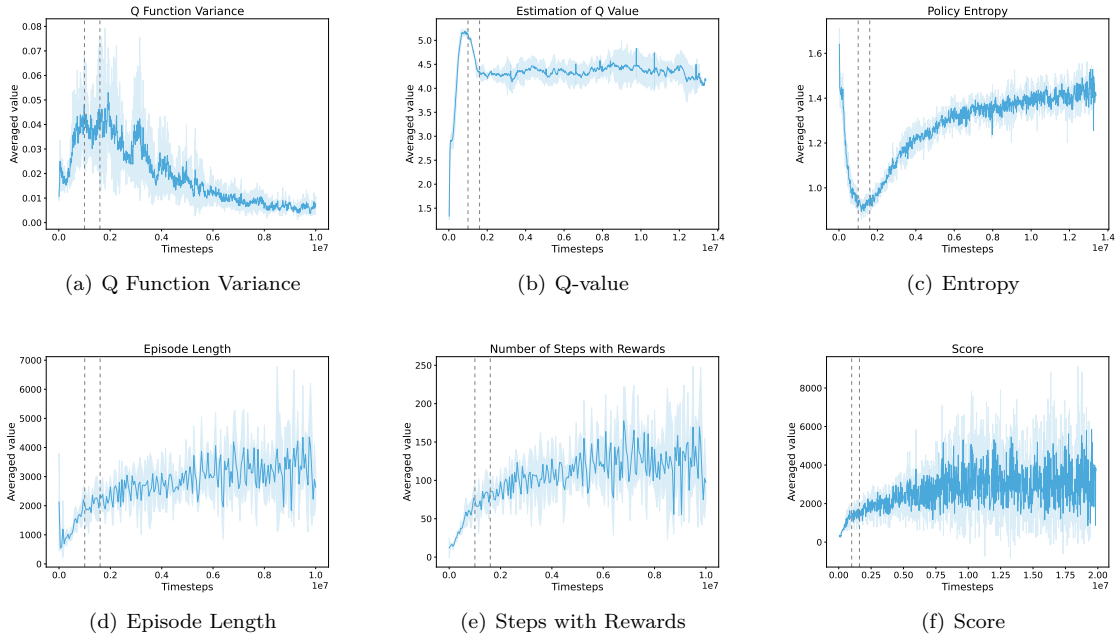


Figure 14: Plots of Q function variance, estimation of Q-value, policy action entropy, episode length, number of steps with rewards and score on Atari Game Assault environment with discrete SAC over 10 million time steps.

A.6.3 Comparison of Different Algorithms on Additional Atari Environments

Here in Figure 17 we provide comparative performance curves of DSAC, DSAC with entropy penalty, and DSAC with KL penalty in three additional Atari game environments: Assault, Jamesbond, and MsPacman. As shown in the results, the entropy penalty consistently offers the best early-stage regulation of entropy changes across all three environments. This regulation helps prevent the agent from falling into local optimum during the learning process, thereby improving the final score performance.

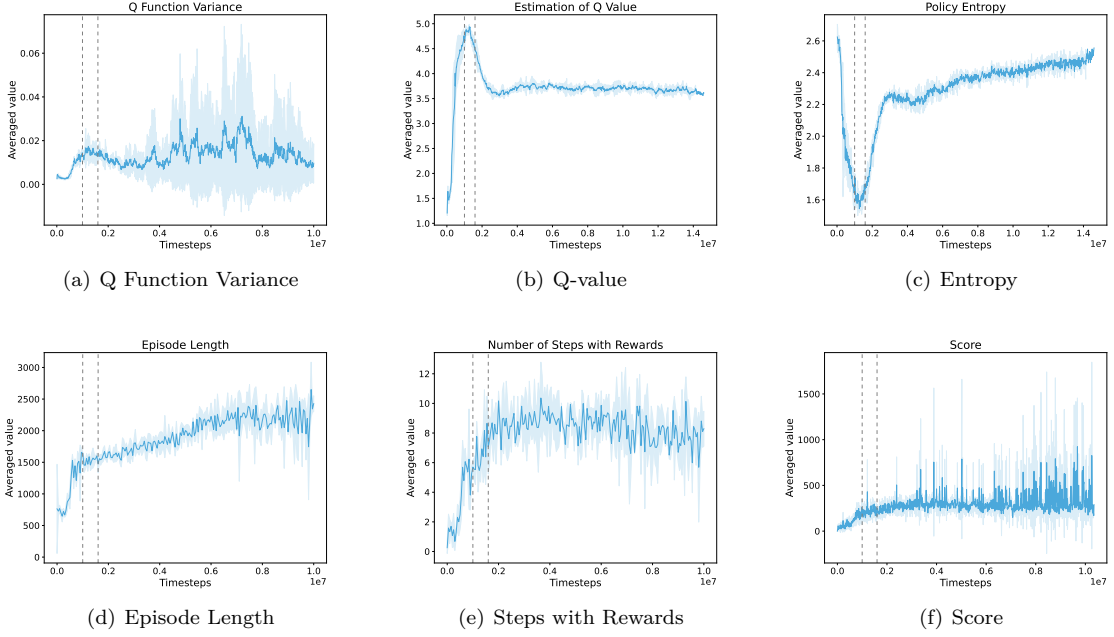


Figure 15: Plots of Q function variance, estimation of Q-value, policy action entropy, episode length, number of steps with rewards and score on Atari Game Jamesbond environment with discrete SAC over 10 million time steps.

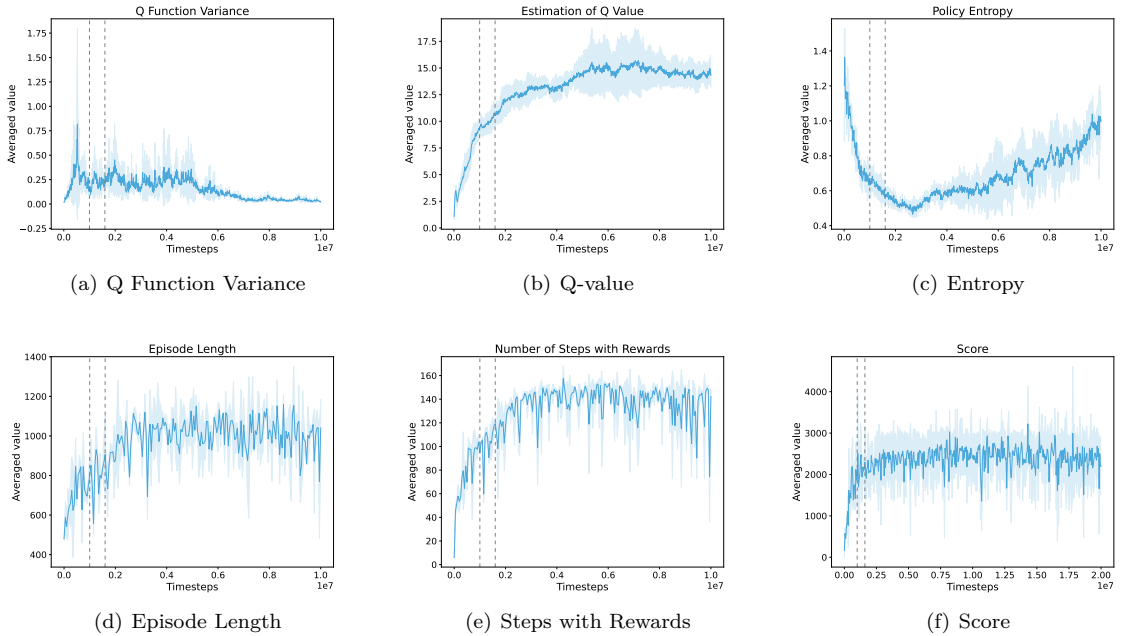


Figure 16: Plots of Q function variance, estimation of Q-value, policy action entropy, episode length, number of steps with rewards and score on Atari Game MsPacman environment with discrete SAC over 10 million time steps.

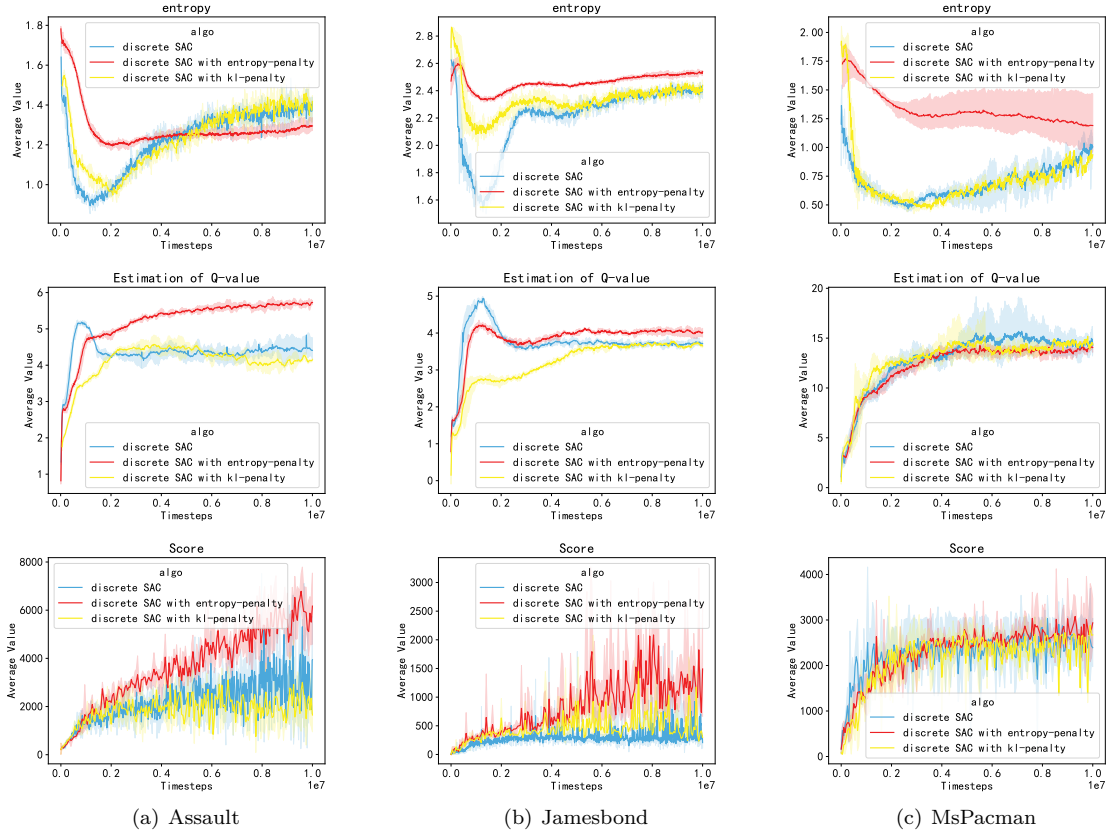


Figure 17: Measuring Q function variance, policy action entropy, estimation of Q-value, and score on Atari game Assault, Jamesbond and MsPacman, comparing between discrete SAC, discrete SAC with KL-penalty and discrete SAC with entropy-penalty over 10 million time steps.

B Further Analysis

B.1 SAC Training Pattern on MuJoCo

We only observe the failure modes in discrete SAC. The reason SAC does not exhibit these failure modes in continuous environments is twofold. First, SAC employs the reparameterization trick, fitting actions with a Gaussian distribution, allowing it to adapt to deceptive rewards without sacrificing policy diversity. Second, in continuous environments, actions that deviate slightly from the best response may have minimal impact on the outcome, whereas in discrete settings, different actions can have entirely distinct meanings. Therefore, our analysis primarily focuses on the challenges SAC faces in discrete environments.

To validate this point, we test SAC on three tasks of the MuJoCo environment. Results in Figure 18 indicate that in MuJoCo, the SAC algorithm does not encounter local optimum issues; policy entropy changes are minimal and gradual, while the scores steadily increase. This suggests that SAC does not face the problems described in the paper when applied to continuous tasks.

B.2 Hyperparameter Analysis

B.2.1 Different Hyperparameter Choices of DSAC

Our design method incorporates two hyperparameters, i.e., entropy-penalty coefficient β and Q-clip range c . Fig. 19 compares various entropy-penalty coefficient β and Q-clip range c values. The constraint proportion

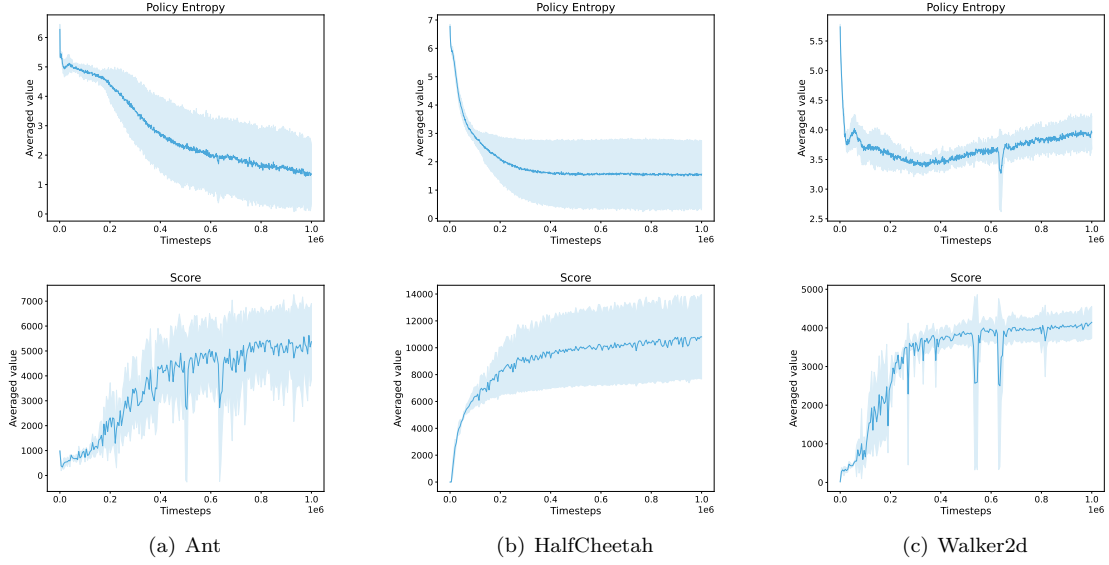
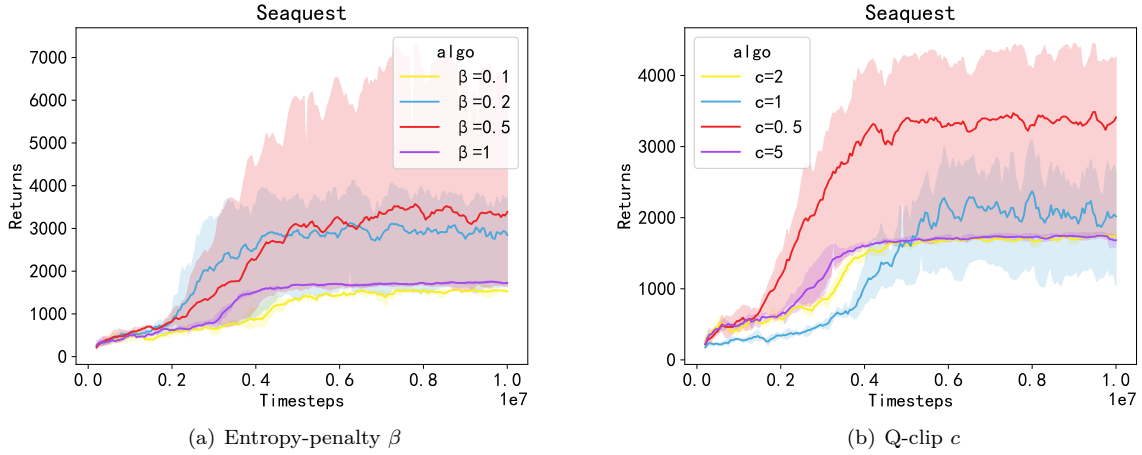


Figure 18: The results of SAC in the MuJoCo environment.

Figure 19: Scores on Seaquest: a) variants entropy-penalty coefficient β with 0.1, 0.2, 0.5 and 1. b) variants Q-clip c with 0.5, 1, 2 and 5.

of policy change is determined by the entropy-penalty coefficient β . Intuitively, an excessive penalty term will lead to policy under-optimization. We experiment with different β in $\{0.1, 0.2, 0.5, 1\}$. We find that $\beta = 0.5$ can effectively limit entropy randomness while improving performance. The Q-clip constrains different ranges of Q value range c , and experiments with different ranges c in $\{0.5, 1, 2, 5\}$ show that 0.5 is a reasonable constraint value.

B.2.2 Different Choices of Clip Ratio

In Figure [20](#), we compare the clip ratio and final scores of different c in our Q-clip.

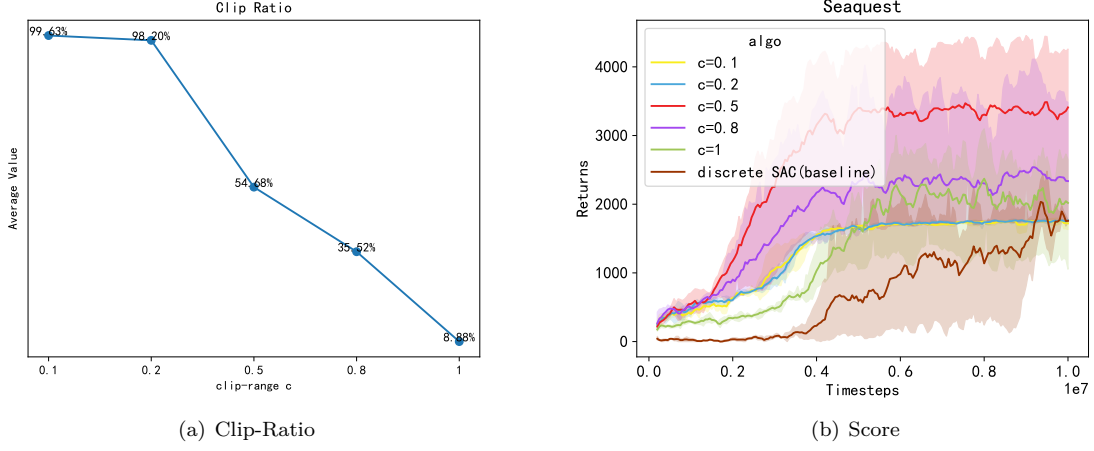


Figure 20: Measuring clip-ratio and score on Atari Game Seaquest environment with our method over 10 million time steps using variants Q-clip c with 0.1, 0.2, 0.5, 0.8 and 1.0 .

B.2.3 Various Learning Rates for Discrete SAC

We introduce various learning rates for experiments on Asterix using vanilla discrete SAC in Fig. 21. An excessively high learning rate leads to early convergence of entropy, while an excessively low learning rate results in insufficient optimization. The experiments show that the entropy instability issue of discrete SAC is not caused by inappropriate learning rate settings.

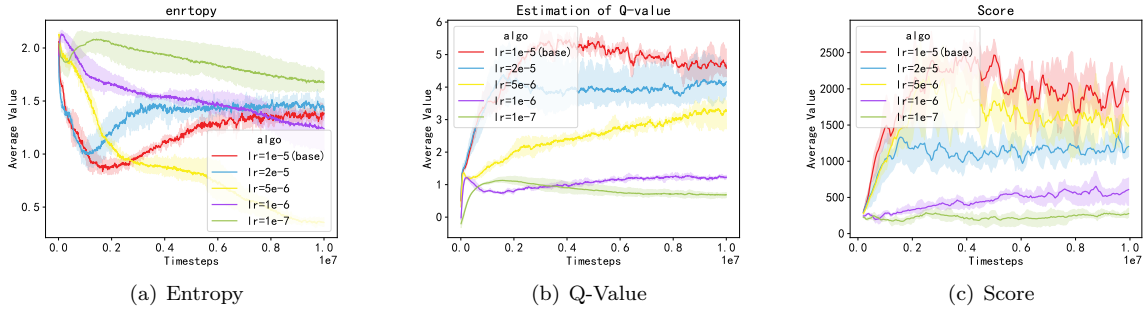


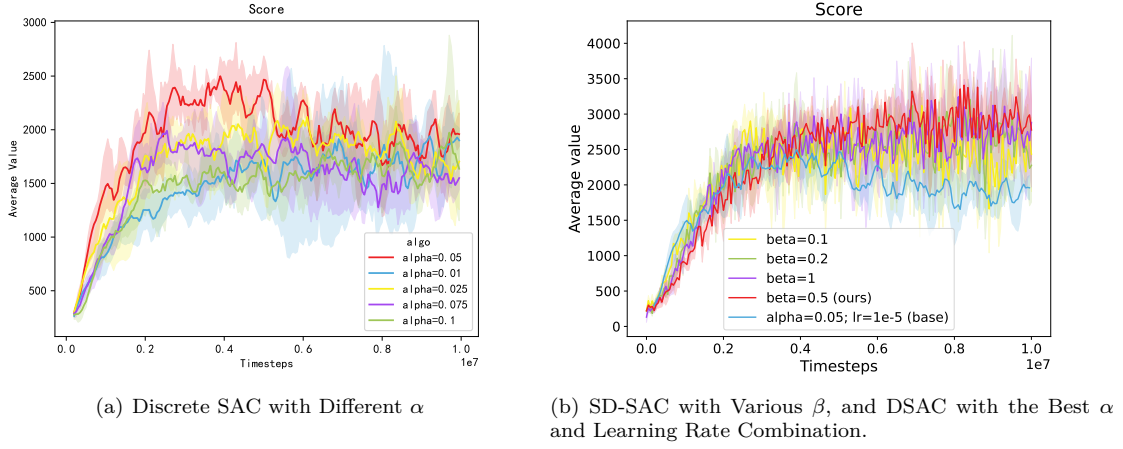
Figure 21: Measuring policy action entropy, estimation of Q-value and score on Atari Game Asterix environment with discrete SAC over 10 million time steps using different learning rates.

B.2.4 Different Choices of Temperature α in Discrete SAC

In Figure 22(a), we compare scores on Asterix by discrete SAC using variants α with 0.01, 0.025, 0.05, 0.075 and 0.1 over 10 million time steps.

B.2.5 Comparison Across SD-SAC and DSAC

We first determine discrete SAC's best combination of α (Fig. 22(a)) and learning rate (Fig. 21(a)). Then, we compare the scores between SD-SAC with different β , and DSAC with this combination ($\alpha = 0.5$; $lr = 1e-5$) in Figure 22(b). The result shows that across various β values, SD-SAC consistently outperforms DSAC. This indicates that the entropy penalty is a better and more balanced constraint than merely limiting the extent of policy updates.

Figure 22: Comparison of different hyperparameters, including α , learning rate and β .

B.3 Computation Overhead

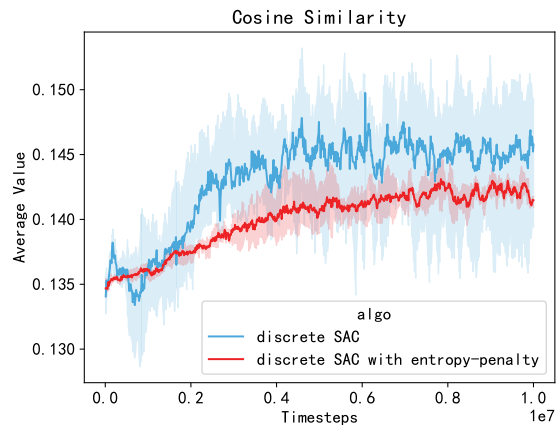
We test the computational speed on a machine equipped with an Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50GHz with 24 cores and a single Tesla T4 GPU. The unit "it/s" represents the number of steps interacting with the environment per second. Detailed data are shown in the Table 4 below. The results demonstrate that our method has a 10.86% reduction(265.41->236.58) in speed compared to the vanilla discrete SAC, while maintaining the same parameter size. $\square$

Table 4: Computational speed our method and discrete SAC.

algorithm	speed
discrete SAC	265.41it/s
discrete SAC + entropy-penalty	246.83it/s(-18.58)
discrete SAC + avg-q + q-clip	250.27it/s(-15.14)
SD-SAC	236.58it/s(-28.83)

B.4 Cosine Similarity Comparison

We visualize the changes in cosine similarity between adjacent states before and after incorporating the entropy penalty in the DSAC algorithm in Figure 23. The results indicate that, following the addition of the entropy penalty, state transitions exhibit smaller and more stable changes. This observation further substantiates that the entropy penalty contributes to more stable policy updates, thereby enhancing the overall performance of the algorithm. $\square$



(a) State Similarity

Figure 23: Measuring cosine similarity of states on Atari game Asterix compared between discrete SAC and discrete SAC with entropy-penalty over 10 million time steps.



Twin-Delayed DDPG: A Deep Reinforcement Learning Technique to Model a Continuous Movement of an Intelligent Robot Agent

Stephen Dankwa
Automation Engineering
University of Electronic Science and Technology of
China
nabistephen@yahoo.com

Wenfeng Zheng
Automation Engineering
University of Electronic Science and Technology of
China
winfirms@uestc.edu.cn

ABSTRACT

In this current research, Twin-Delayed DDPG (TD3) algorithm has been used to solve the most challenging virtual Artificial Intelligence application by training a 4-ant-legged robot as an Intelligent Agent to run across a field. Twin-Delayed DDPG (TD3) is an incredibly smart AI model of a Deep Reinforcement Learning which combines the state-of-the-art methods in Artificial Intelligence. These includes Policy gradient, Actor-Critics, and continuous Double Deep Q-Learning. These Deep Reinforcement Learning approaches trained an Intelligent agent to interact with an environment with automatic feature engineering, that is, necessitating minimal domain knowledge. For the implementation of the TD3, we used a two-layer feedforward neural network of 400 and 300 hidden nodes respectively, with Rectified Linear Units (ReLU) as an activation function between each layer for both the Actor and Critics. We, then added a final tanh unit after the output of the Actor. The Critic receives both the state and action as input to the first layer. Both the network parameters were updated using Adam optimizer. The idea behind the Twin-Delayed DDPG (TD3) is to reduce overestimation bias in Deep Q-Learning with discrete actions which are ineffective in an Actor-Critic domain setting. Based on the Maximum Average Reward over the evaluation time-step, our model achieved an approximate maximum of 2364. Therefore, we can truly say that, TD3 has obviously improved on both the learning speed and performance of the Deep Deterministic Policy Gradient (DDPG) in a challenging environment in a continuous control domain.

CCS Concepts

•Computing methodologies → Modeling and simulation → Model development and analysis → Model verification and validation

Keywords

Deep Reinforcement Learning; Artificial Intelligence; Twin-Delayed Deep Deterministic Policy Gradient; Actor-Critic

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICVISP 2019, August 26–28, 2019, Vancouver, BC, Canada

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7625-9/19/08...\$15.00

<https://doi.org/10.1145/3387168.3387199>

1. INTRODUCTION

Recently, Deep Reinforcement Learning has gain tremendous attention in the machine learning community as a result to the essential amount of progress that has been achieved for the past few years. Reinforcement Learning is the ability of an agent to learn what to do, how to map situations to actions, in order to maximize a numerical reward signal [12]. Figuring out how to deal with continuous action spaces was one of the subsequent challenges that the Reinforcement Learning community faced. Solving complex tasks from unprocessed, high-dimensional, input sensory is one of the primary aims in the field of Artificial Intelligence [5]. Deep Q-Network algorithms have been attempted and tried to solve problems with high-dimensional observation spaces but, however, it is able to handle discrete and low-dimensional action spaces [5]. Addition, several practical and physical control tasks are characterized with continuous and high-dimensional spaces. And as a result, Deep Q-Network cannot be directly applied to continuous domain settings since it depends on a result the action which maximizes the action-value function. The reason is that continuous valued-based request an iterative optimization process at every step [5]. In the course of solving the continuous and high-dimensional spaces problem, evolved the Deterministic Policy Gradient (DPG) algorithms [11], which is an off-policy actor-critic algorithm that learns a deterministic target policy. In their work, they demonstrated that deterministic policy Gradient algorithms significantly outperformed their stochastic counterparts in high-dimensional spaces. In addition, Deep Deterministic Policy Gradient (DDPG) algorithms [5] also emerged in the process of solving the continuous and high-dimensional spaces problem. The DDPG algorithm was built upon the DPG algorithm, which is also an off-policy actor-critic algorithm that uses deep function approximators that can learn policies in high-dimensional, continuous action spaces [5]. In their work, they stated that they combined the actor-critic technique with intuitions from the recent success of the Deep Q-Network (DQN). Deep Q-Network (DQN) [6] has the capability to learn value functions based on function approximators in a stable and robust manner as a result to two novelties [5]:

- The network is trained off-policy with samples from a replay buffer to minimize correlations between samples
- The network is trained with a target Q-Network to give reliable targets during temporal difference backups.

Furthermore, based on the success works from the Deep Q-Network (DQN) [6], the Deterministic Policy Gradient (DPG) [11] and Deep Deterministic Policy Gradient (DDPG) [5] algorithms paved way to the Twin-Delayed DDPG (TD3) algorithms [10]. Twin-Delayed Deep Deterministic Policy Gradient (TD3) is an

off-policy model and recently has achieved breakthrough in Artificial Intelligence state-of-the-art models with continuous high-dimensional spaces. Reinforcement Learning algorithms that are characterized as off-policy generally utilized a separate behavior policy which is independent of the policy which is being improved upon. The key advantage of the separation is that the behavior policy can operate by sampling all actions, while the estimation policy can be deterministic [12].

Twin-Delayed Deep Deterministic Policy Gradient algorithm (TD3) was built on the Deep Deterministic Policy Gradient algorithm (DDPG) to increase stability and performance with consideration of function approximation error [10]. The uniqueness about the TD3 algorithm is in the combination of three powerful Deep Reinforcement Learning techniques such as continuous Double Deep Q-Learning [13], Policy Gradient [11] and Actor-Critic [7]. Most at times, the Researcher had tried to experiment the algorithm with a MuJoCo [5,6,10,11] continuous control tasks but, we experimented with the pybullet continuous control environment.

Therefore, in this current research, we applied the TD3 algorithm to model and train a 3D 4-Legged Ant robot in a continuous high-dimensional action space. We experimented with the pybullet continuous control environment which was interfaced through the OpenAI Gym.

The three main ideas behind this current research are to (1) present the evolution process of the TD3 model, (2) develop a model using the TD3 algorithm, and (3) evaluate its performance. The importance of this current research is to present to the Artificial Intelligence community how Twin-Delayed Deep Deterministic Policy Gradient algorithm (TD3) algorithm has been applied to solve the most challenging virtual Artificial Intelligence application by training a 3D 4-Legged-Ant robot as Intelligent Agent to walk and run across a field.

2. METHODOLOGY

Twin-Delayed DDPG (TD3) consists of three Deep Reinforcement Learning methods such the continuous Double Deep Q-Learning, Policy Gradient and Actor-Critic. This section presents the experimental details, the TD3 algorithm and the steps used in implementing the model.

2.1 Experiment Details

We implemented the TD3 model by using two-layer feedforward neural network of 400 and 300 hidden nodes respectively. We used Rectified Linear Units (ReLU) as an activation function between each layer for both the Actor and Critics, and then a final tanh unit following the output of the Actor. The Critic receives both the state and action as input to the first layer. The parameters set and utilized in the model were the name of the environment (AntBulletEnv-v0), random seed number (0), start timesteps (10000), evaluation steps (5000), maximum timesteps (500000), save model (true), exploration noise (0.1), batch size (100), discount factor (0.99), target network update rate (0.005), policy noise (0.2), clip noise (0.5) and policy frequency (2). Both the network parameters were updated using Adam optimizer [4]. The environment was achieved using the pybullet [2] continuous control tasks. Pybullet is a python module for physics stimulation, robotics, and reinforcement learning based on the Physics Bullet SDK. We interfaced the pybullet continuous control task, which is the environment (AntBulletEnv-v0) through the OpenAI Gym [1]. The parameters and their values used in this research work are been summarized in Table 1. The TD3 algorithm can be seen from

Algorithm 1.

In this work, we used the Python version 3 environment as the programming language. We wrote and executed our code using the Jupyter notebook through the Google Colaboratory (<https://colab.research.google.com/>). Google Colaboratory is a free jupyter notebook that requires no setup and execute completely in the cloud. It offered us a free virtual machine server with GPU to execute the code.

The only package that was installed was the pybullet version 2.5.3. The rest of the libraries that were utilized in this work were, the numpy, gym, and torch. These libraries were imported directly from the google colab platform without installations.

Table 1. The Parameters of the TD3 model.

Parameter	Value	Description
Env_name	AntBulletEnv-v0	The name of the PyBullet environment
seed	0	Random seed number
Eval_freq	5e3	The number of times the evaluation step is performed
Max_timesteps	5e5	Total number of iterations
Save_model	true	Boolean checker, whether or not to save the pre-trained model
Expl_noise	0.1	Exploration noise
Batch_size	100	Size of the batch
discount	0.99	Discount factor gamma
Noise_clip	0.5	Maximum value of the Gaussian noise
Policy_freq	2	Number of iterations to wait before the policy network

Algorithm 1 Twin-Delayed Deep Deterministic Policy Gradient TD3
Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_{ϕ} with random parameters θ_1, θ_2, ϕ
Initialize target networks $\hat{Q}_{\theta_1} \leftarrow Q_{\theta_1}, \hat{Q}_{\theta_2} \leftarrow Q_{\theta_2}, \hat{\phi} \leftarrow \phi$
Initialize replay buffer B
For $t = 1$ to T do
Select action with exploration noise $a \leftarrow \pi_{\phi}(s) + \epsilon$
$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'
Store transition tuple (s, a, r, s') in B
Sample mini-batch of N transitions (s, a, r, s') from B
$\hat{r} \leftarrow r_{\theta_1}(s) + a, \hat{a} \leftarrow \text{clip}(\pi_{\hat{\phi}}(s), \theta), \tau \leftarrow r, s'$
$y \leftarrow r + \gamma \max_{\theta_1, \theta_2} Q_{\theta_i}(s', \hat{a})$
Update critics $\theta_1 \leftarrow \argmin_{\theta} N^{-1} \sum_j (y - Q_{\theta_1}(s, a))^2$
If $t \bmod \tau = 0$ then
Update $\hat{\phi}$ by the deterministic policy gradient
$V_{\theta_1}(s) \leftarrow \pi^{-1} \sum_a V_{\theta_1}(s, a) \pi_{\hat{\phi}}(a s)$
Update target networks
$\hat{\theta}_1 \leftarrow \theta_1 + (1 - \tau)\theta_1$
$\hat{\phi} \leftarrow \phi + (1 - \tau)\phi$
End if
End for

2.2 Steps Used in Implementing the TD3 Algorithm

In this current research work, the steps that were implemented based on the TD3 algorithm can be followed as:

STEP 1: Initialize the Experience Replay Memory with a size of

20000 which will then be populated with each new transition

STEP 2: Build one neural network for the Actor model and one neural network for the Actor target

STEP 3: Build two neural networks for the two critic models and two neural networks for the two critic targets

STEP 4: Sample a batch of transitions (s, s', a, r) from the memory, where s=state, s'=next state, a=action, r=reward.

Then for each element of the batch:

STEP 5: From the next state s', the Actor target plays the next action a'.

STEP 6: Add Gaussian noise to this next action a' and clamp it in a range of values supported by the environment.

STEP 7: The two Critic targets then take each of the couple (s', a') as input and return two Q-values

$Q_{\pi}(s', a')$ and $Q_2(s', a')$ as outputs.

STEP 8: Keep the minimum of these two Q-values as $\min(Q_{\pi}, Q_2)$, which represents the approximated value of the next state.

STEP 9: Get the final target of the two Critic models, which is $Q_t = r + \gamma * \min(Q_{\pi}, Q_2)$, where γ is a discount factor.

STEP 10: The two Critic models take each of the couple (s, a') as input and return two Q-values

$Q_1(s, a')$ and $Q_2(s, a')$ as outputs.

STEP 11: Then compute the Loss coming from the two Critic models.

Critic Loss = $\text{MSE_Loss}(Q_1(s, a'), Q_t) + \text{MSE_Loss}(Q_2(s, a'), Q_t)$

STEP 12: Backproagate the Critic Loss and then update the parameters of the two Critic models with Stochastic Gradient Descent (SGD) optimizer

STEP 13: Once every two iterations, update the Actor model by performing Gradient Ascent on the output of the first Critic model: $\nabla_{\theta} J(\theta) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a) |_{a=\pi_{\theta}(s)} \nabla_{\theta} \pi_{\theta}(s)$, where θ and θ_1 are the weights of the Actor and the Critic respectively.

STEP 14: Once again at every two iterations, update the weights of the Actor target by Polyak Averaging:

$$\theta'_1 \leftarrow \tau \theta_1 + (1 - \tau) \theta'_1$$

STEP 15: Once again at every two iterations, update the weights of the Critic target by Polyak Averaging:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

3. RESULTS AND DISCUSSIONS

In this current research work, we have modeled and trained an Intelligent 3D 4-Legged-Ant robot to interact with a continuous control environment to walk and run across a field using a current state-of-the-art Twin-Delayed DDPG algorithm. The main objectives were to present the evolution of the TD3 algorithm, develop a model using the TD3 algorithm and evaluate its performance with other state-of-the-art models. Within 10,000 iterations, we allowed the agent to explore on its own without training. After the exploration, the agent achieved an Average Reward of approximately 140, which can be seen from (figure 1). This Average Reward value was detected at the end of the 10,000 iterations.

After exposing the Agent to the training, the Agent then learned from the past pattern of the model. The agent improved upon its performance by achieving an approximate Average Reward of 2272 as seen from (figure 2) after the end of the 500,000 iterations. We observed a Maximum Average Reward of approximately 2364 at around 450,323 iterations as seen from (figure 3). In the original paper [10], in order to evaluate their TD3 algorithm, they used the MuJoCo continuous control task (Ant-v1) and interfaced it through the OpenAI Gym. In their experiment, they exposed the agent to training by performing 1,000,000 iterations. Their TD3 model then achieved a Maximum Average Reward of approximately 4372. The result of their model outperformed other start-of-the-art models such as DDPG [5], PPO [9], TRPO [8], ACKTR [14] and SAC [3] as seen from their work in Table 1[10].

Whereas in this current research, we experimented with the pybullet continuous control task (AntBulletEnv-v0) (figure 4) and interfaced it through the OpenAI Gym. Apart from the continuous control task (AntBulletEnv-v0) and the number of iterations (500,000) which were changed, we maintained all the other parameters as in [10]. We exposed the agent to training by performing 500,000 iterations because of limited amount time we had when using a virtual machine server through the google colabatory. Our current model then achieved a Maximum Average Reward of approximately 2364, Which makes sense, since we performed half of the iteration of the original paper. Our current model was as effective as the original paper when compared to other state-of-the-art models such as Deep Deterministic Policy Gradient (DDPG) [5], Proximal policy optimization algorithms (PPO) [9], Trust region policy optimization (TRPO) [8], Actor-Critic using using Kronecker-factored Trust Region (ACKTR) [14], and Soft Actor-Critic (SAC) [3].

The output of the model was a video (upon request can be released), representing the virtual AI application where the 3D 4-Legged-Ant walk and run across a field in a continuous control environment as in (figure 5). The direction of the movement was from left to right.

```
-----
Total Timesteps: 6172 Episode Num: 7 Reward: 481.26159425581857
Total Timesteps: 7172 Episode Num: 8 Reward: 496.2648954826422
Total Timesteps: 8172 Episode Num: 9 Reward: 486.4186150269747
Total Timesteps: 9168 Episode Num: 10 Reward: 52.01461964189676
Total Timesteps: 9374 Episode Num: 11 Reward: 4.867869991715229
Total Timesteps: 9513 Episode Num: 12 Reward: 53.85860286848365
Total Timesteps: 9167 Episode Num: 13 Reward: 385.0356488144698
Total Timesteps: 18167 Episode Num: 14 Reward: 511.822178935436885
-----
Average Reward over the Evaluation Step: 130.570623
-----
```

Figure 1. The screenshot of Average Reward obtained by the Agent after exploring on its own without training within 10,000 iterations.

```
-----
Total Timesteps: 496323 Episode Num: 525 Reward: 3313.174950819184
Total Timesteps: 497323 Episode Num: 527 Reward: 1304.2822467583345
Total Timesteps: 498323 Episode Num: 528 Reward: 1382.1050112316285
Total Timesteps: 499323 Episode Num: 529 Reward: 3181.065575478909
-----
Average Reward over the Evaluation Step: 2271.713488
-----
```

Figure 2. The screenshot of the Average Reward obtained by the Agent after the training over 500,000 iterations.


```

Total timesteps: 449325 episode Num: 478 Reward: 2335.98974896024
Total timesteps: 447325 episode Num: 477 Reward: 2386.987354077358
Total timesteps: 446325 episode Num: 476 Reward: 2379.8597728388404
Total timesteps: 449325 episode Num: 479 Reward: 2348.8461431735525
Total timesteps: 450325 episode Num: 480 Reward: 2387.8307345697518
Average Reward over the Evaluation Step: 2364.483472

```

Figure 3. The screenshot of the Maximum Average Reward obtained by the Agent within 450,323 iterations.

Table 1. Maximum Average Reward over 10 episodes of 500,000 evaluation time steps. Comparing the result of our TD3 model with other state-of-the-art after they experimented their algorithms.

TD3	DDPG	PPO	TRPO	ACKTR	SAC
2364.48	1005.30	1083.20	-75.85	1821.94	655.35

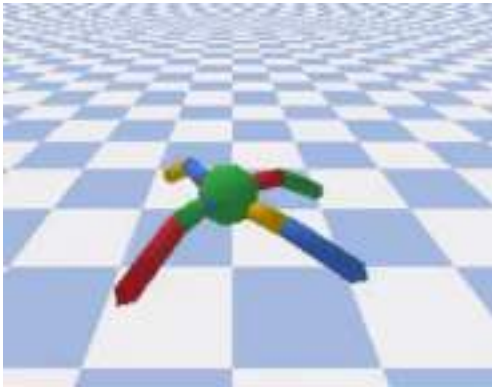


Figure 4. The pybullet environment (AntBulletEnv-v0).

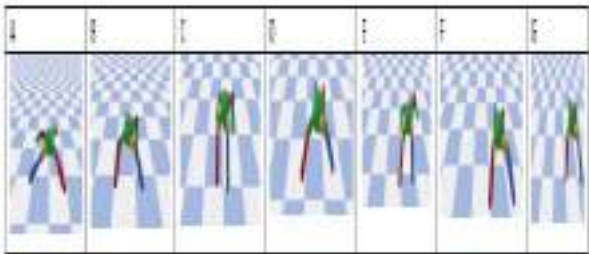


Figure 5. The screenshot of the continuous movement by the 3D Intelligent Ant robot agent from left to right (A-G).

4. CONCLUSION

In this current research work, we modeled and trained an Intelligent 3D 4-Legged-Ant robot to interact with a continuous control environment to walk and run across a field using a current state-of-the-art Twin-Delayed DDPG algorithm. We have elaborated on the evolution process of the TD3 algorithm, as from DQN-DPG-DDPG-TD3. We have evaluated the TD3 algorithm by using the pybullet continuous control task environment and by interfacing it through the OpenAI Gym. Our current TD3 model performed well by achieving a higher Average Reward as compared with other state-of-the-art models such as the DDPG, PPO, TRPO, ACKTR and SAC. We observed that even 500,000 iterations were enough for our model to display such tremendous performance. In future works, this model is applicable to other

continuous control tasks such as the Walker2DBulletEnv-v0, HalfCheetahBulletEnv-v0 etc.

Therefore, we can truly state, Twin-Delayed Deep Deterministic Policy Gradient (TD3) algorithm has prominently improved both the learning speed and performance of the DDPG in a challenging task in a continuous control setting.

ACKNOWLEDGMENTS

First of all, we thank God for his strength and wisdom. Sincere thanks to the Professors and Lab mates in the school of Automation Engineering, University of Science and Technology for their directions. We express our gratitude to the SuperDataScience Team for their technical ideas. we thank the Google Team for providing us with a free virtual machine server for 12 hours. Then lastly, we thank the IASED organizers for given us the opportunity to present our work to the International community.

REFERENCES

- [1] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym, 2016.
- [2] Erwin Coumans, Yunfei Bai and Jasmine Hsu. PyBullet. Available on: <https://pypi.org/project/pybullet/>. Retrieved: 5/30/2019
- [3] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. arXiv preprint arXiv:1801.01290, 2018
- [4] Kingma, Diederik and Ba, Jimmy. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.
- [5] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971, 2015. Available online: <https://arxiv.org/abs/1509.02971>
- [6] Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.
- [7] R.S. Sutton, D.A. McAllester, S.P. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, in: Advances in Neural Information Processing Systems, 2000, pp. 1057–1063.
- [8] Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. Trust region policy optimization. In International Conference on Machine Learning, pp. 1889–1897, 2015.
- [9] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [10] Scott Fujimoto, Herke van Hoof and David Meger, Addressing Function Approximation Error in Actor-Critic Methods. <https://arxiv.org/pdf/1802.09477.pdf>, 2018
- [11] Silver, David, Lever, Guy, Heess, Nicolas, Degris, Thomas, Wierstra, Daan, and Riedmiller, Martin. Deterministic policy gradient algorithms. In ICML, 2014.
- [12] Sutton, S. Richard and Andrew G. Barto. Reinforcement learning: An introduction. 2nd edition, The MIT Press, Cambridge, Massachusetts, ISBN 9780262039246. Available online: <http://www.incompleteideas.net/book/the-book.html>, 2018.



A TD3-based multi-agent deep reinforcement learning method in mixed cooperation-competition environment

Fengjiao Zhang^b, Jie Li^a, Zhi Li^{b,*}

^a The Center of Data Science and Service, Computer science Institute, Beijing university of Posts and Telecommunications, No. 10 Xitucheng Road, Haitian District, Beijing, China

^b Sichuan University, No. 24 South Section 1, Yihuan Road, Chengdu, Sichuan, China

ARTICLE INFO

Article history:

Received 20 November 2019

Revised 25 April 2020

Accepted 31 May 2020

Available online 12 June 2020

Communicated by Peter. W Vamplew

Keywords:

Reinforcement learning

Overestimation error

Dual-critic

MADDPG

MATD3

ABSTRACT

We explored the problem about function approximation error and complex mission adaptability in multi-agent deep reinforcement learning. This paper proposes a new multi-agent deep reinforcement learning algorithm framework named multi-agent time delayed deep deterministic policy gradient. Our work reduces the overestimation error of neural network approximation and variance of estimation result using dual-centered critic, group target network smoothing and delayed policy updating. According to experiment results, it improves the ability to adapt complex missions eventually. Then, we discuss that there is an inevitable overestimation issue about existing multi-agent algorithms about approximating real action-value equations with neural network. We also explain the approximate error of equations in the multi-agent deep deterministic policy gradient algorithm mathematically and experimentally. Finally, the application of our algorithm in the mixed cooperative competition experimental environment further demonstrates the effectiveness and generalization of our algorithm, especially improving the group's ability of adapting complex missions and completing more difficult missions.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Reinforcement learning (RL) is able to solve sequential decision problems in Markov Decision Processes (MDPs), and makes a tremendous progress recently such as AlphaGo [1], AlphaStar, OpenAI Five on Dota 2. With the development of deep reinforcement learning, single agent algorithm is gradually on the right track and has already addressed plenty of difficult problems giving researchers more powerful instruments to study in high dimension and more complex action spaces. As study more on single agent reinforcement learning, researchers realize that the introduction of multi-agent could achieve higher performance on complex mission environments. Jaderberg et al. [2] shows the advantages of multi-agent cooperation in the first-person game Quake III. The successful rate of agents after training surpasses human professional players as their teammates or opponents.

The development of multi-agent deep reinforcement learning is closely related with many basic theoretical problems. For example, there will be an overestimation bias while using a neural network to approximate the true state-action equation. Wang et al. [3] tried to make the value estimate more accurate by improving the neural

network structure. However, some factors cannot be eliminated even if they are reduced due to the application of neural networks. The state-of-the-art model-free reinforcement learning algorithms, whether single-agent or multi-agent reinforcement learning algorithms, are based on Time Division (TD) or Monte Carlo (MC) [4]. These methods have an inevitable overestimation problem and perform badly in large state action space. The utility of policy gradient methods will bring in high variance of the estimation results, making policies more sensitive, even leading to a failed training. We will introduce the same overestimation problems faced by multi-agents in Section 4.1 and 4.2 below.

Inspired by the reduction of overestimation error in real state-action equations in the field of single-agent reinforcement learning [5], we present a new algorithm in multi-agents, which is used to reduce the bias of the function estimation by using dual-centered Q-network. The method using batch update and delayed policy update is adopted to reduce the high variance of estimation results and stabilize the training process. The convergence speed of results is faster and the effect is also improved.

Another important decisive factor in multi-agent reinforcement learning is the stability of state spaces. Q-learning [6] is a widely utilized method in discrete space using MDPs to analyse multi-agent reinforcement learning problems. Another popular framework applying well in the field of single-agent is actor-critic [4],

* Corresponding author.

E-mail address: lizhi@scu.edu.cn (Z. Li).

where the critic is also based on Q-learning and deep Q-network (DQN) [7]. So once other agents change their policy, that is

$$P(s'|s, a, \pi_1, \dots, \pi_N) \neq P(s'|s, a, \pi'_1, \dots, \pi'_N)$$

when any $\pi_i \neq \pi'_i$, where π_i is the policy of agent i , the environment does not meet the assumptions about environmental stability in Q-learning. Our team draws on the centralized critic idea in the multi-agent deep deterministic policy gradient (MADDPG) algorithm [8] allowing individuals in the group to learn the centralized critic architecture proposed separately without interfering with each other during training process. Due to different reward shapes of different individuals, it is possible to allow groups not only to execute monotonous tasks, but also execute cooperative and competitive missions at the same time without designing the reward architecture separately for opposite agents. In summary, we proposed our own multi-agent reinforcement learning algorithm multi-agent time delayed deep deterministic policy gradient (MATD3) for complex mixture environments.

This paper proves that the problem of overestimation and high variance also occurred in MADDPG at continuous motion control scenarios, and under the following assumptions and requirements:

- (1) Individual agents only use local information observed during execution;
- (2) No need to set differential dynamic environment modeling among intelligent community, especially the communication methods;
- (3) Not only suitable for cooperation interactive environment, but also for competitive-cooperative hybrid environments.

We propose an approach solving the critic overestimation and reducing the variance of evaluation results in group reinforcement learning through double centralized critics and delayed policy update. Then, for the target policy smoothing, the correlative actions of individual agents in the group should have similar action values. Double centralized critics absorb some additional information during training in order to stabilize the state space, such as the policies of other agents. This condition can also be mitigated, as done in Lowe et al. [8]. Decentralized implementation of double centralized critics allows us to find a way to reduce the overestimation bias and variance, which can be applied to non-standard stable environments and used to stabilize the training process.

2. Related work

The common failure mode of deep deterministic policy gradient (DDPG) [9] is that the learning of Q-function starts to overestimate the Q-value and eventually breaks down the policy because it takes advantage of errors in the Q-function. TD3 [5] is an algorithm that solves this problem by introducing three key techniques that will be introduced in Section 3.

Estimation error in reinforcement learning algorithm and its effects have been studied in Mannor et al. [10]. We focus on the overestimation of function approximation and high variance of the results of policy network. In the process of double Q-learning to eliminate approximated errors, two separate estimators are used in the single agent domain [11] to make an unbiased estimation. Dueling DQN [3] improves the network structure based on double Q-learning. Another approach is to focus directly on how to reduce the estimated variance in the policy gradient method [12] and minimize overfitting for early high variance [13]. Recently, some researchers also pay attention to risk-aversion [10] and exploration [14].

In order to address the high variance caused by cumulative errors in the TD-like methods, a large part of solutions try to min-

imize the error range of each step or combine off-policy thoughts with MC methods. A new concept of action value is proposed by Nachum et al. [15], which is defined by a Gaussian smoothed version of the expected Q-value, and takes into account the ability to learn the covariance and mean during training. Zheng et al. [16] reduces the error by improving the network structure, but only applies to a single agent environment. The high variance caused by the cumulative error in the monomer is more serious in this situation, so this paper presents a new type of target network for multiple agents to reduce the pre-update error and provide a technique for reducing group variances by regularization and averaging value estimating.

Traditional reinforcement learning methods such as Q-learning and policy gradient are almost powerless for multi-agent environments. One of the reasons is that each agent's policy changes as the training progresses. Thus, the environment becomes very unstable for any independent agent, which poses a challenge for the stability of learning and results in difficulties in using experience replay buffer directly. But a stable past experience replaying is a very significant condition for stabilizing DQN training. There is also an option that a single agent can be optimized using a model-based policy gradient method, which have the ability to learn the optimization strategy through the backpropagation (BP) algorithm. But this requires each agent to have a discernible model that can be distinguished from the dynamic changes and assumptions of the world.

Some recent multi-agent reinforcement learning algorithms such as MADDPG and QMIX [17] do not have made more considerations for joint overestimation in multi-agent environments. Our work builds directly on the idea of MADDPG and the relationship between them is discussed in Section 4. The MATD3 algorithm proposed in this paper can reduce the overestimation error of critic networks and also be extended to any other multi-agent reinforcement learning algorithm. Chen et al. [18] introduces an application of reinforcement learning in adapted optimal control. Its algorithm combines off-policy and experience replay to make the agent study in linear time system. However it does not refer how to deal with group experience replay and off-policy methods that are vital problems in multi-agent RL. Abed-alguni et al. [19] and He et al. [20] aim to reduce the errors of overestimate of Q-value by using more estimators. The latter chooses to use two famous estimators, maximum estimator (ME) and double estimator (DE) alternately. However, both of them do not consider about multi-agent overestimation problem and only useful for single agent in discrete training space.

Both Abed-alguni et al. [21] and Hengst et al. [22] explore the hierarchy structure that decomposes complex tasks in reinforcement learning, which improves the efficiency of agents study. The former considers about multi-agent situation but only suitable for cooperative environments and do not talk about the reward structure in competitive environments. Both of them do not deal with overestimation problems that definitely occurs during the estimation of Q-value as [5] referred. The latter paper and Kulkarni et al. [23] only consider about single-agent problem, but it performs well in environments with sparse rewards.

3. Background

Multi-agent reinforcement learning is to maximize the cumulative reward of community agents. Our environment can be modeled as a community Markov game that includes N agents. MDPs have a typical form of sequential decision-making. In MDPs, actions not only affect the immediate rewards obtained by the current agent, but also affect the subsequent environmental conditions. State $s \in S$ defines the true state of the environment. The

observation set of individual agent $i = \{1, 2, \dots, N\}$ is $O_i \in O = \{O_1, O_2, \dots, O_N\}$, the action space of agent i is $A_i \in A = \{A_1, A_2, \dots, A_N\}$. The initial state defined by distribution $\rho: S \rightarrow [0, 1]$. Given a state $s_t \in S$ at every discrete time step t , agent $i \in \{1, 2, \dots, N\}$ receives a local observation $o_i: s_t \rightarrow O_i$, then agent i will interactive with environment following random policy $\pi_{\phi_i}: A_i \times O_i \rightarrow [0, 1]$ abbreviated as $\pi_i: A_i \times O_i$, take an action $a_t \in A$ receive immediate reward R_{t+1} . Afterwards, the environment will transfer to next state $s_{t+1} \in S$ according to state transition function $\Gamma: S \times A_1 \times \dots \times A_N \rightarrow S$, meanwhile original environment state will transfer from the old state $s_t \in S$ to the new state $s_{t+1} \in S$. Individual rewards are related with the state and action of agent i as $r_i: S \times A_i \rightarrow \mathbf{R}$. The goal of every agent in the community is to maximize the expected return $R_i = \sum_{t=0}^T \gamma^t r_i^t$, where γ is a discount factor that limits the length of time and T is the end time.

Q-learning is a popular reinforcement learning method based on TD approach, and there are also lots of examples to apply it to the multi-agent domain [24]. Q-learning evaluates the value of the state-action pair called Q-value $Q^\pi(s, a) = E_{r, s'}[R|s_t = s, a_t = a]$, rewritten as the form of the Bellman equation $Q_\pi(s, a) = E_\pi[R_{t+1} + \gamma Q(s', a')]$. The formula for updating the Q-value is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (1)$$

Q-value introduces an overestimation error while calculating the max operation. Hasselt [11] and Van Hasselt et al. [25] use two Q-value networks and reduce the estimation error with the smaller one. However, as the state space dimension becomes higher and higher, Q-learning's tabular method is too difficult to search for the calculation of action-value functions. Therefore, Mnih et al. [4] proposed the DQN method based on Q-learning theory, where one trick for stable training is to use a dual network structure that one is the Q-network and the other one is the target Q-network q_{tar} with delayed update outputting $\bar{q}$. The Q-function network learns an action-value function Q^* by minimizing the loss: $L(\theta) = E[(r + \gamma \max_{a'} \bar{q}^*(s', a'|\theta^-) - q^*(s, a|\theta))^2]$, where $y = r + \gamma \max_{a'} \bar{q}^*(s', a'|\theta^-)$ is called TD target. θ, θ^- are the neural network parameter and target network parameter respectively. Another way of DQN stability training is to utilize the experience replay buffer in which experience tuple $\tau = (s, a, s', r)$ gained in the interaction with the environment that is stored breaking data association and improving its utilization. In a multi-agent environment, a simple idea is to let the agents in the community use their own Q-function network and training separately [26–28], but this makes the environment very unstable and affect the convergence conditions of Q-learning seriously. And experience replay method is no longer applicable, since $P(s'|s, a, \pi_1, \dots, \pi_N) \neq P(s'|s, a, \pi'_1, \dots, \pi'_N)$ when any $\pi_i \neq \pi'_i$.

In the continuous action space, there is another more common method better than Q-learning, called policy gradient (PG) [29]. The PG algorithm does not parameterize the Q-value like Q-learning, but directly parameterizes the policy, which is $\pi_\phi(s)$, and updates the parameters by the gradient descent method $\phi \leftarrow \phi + \alpha \nabla_\phi J(\phi)$, where α is the learning rate, $\nabla_\phi J(\phi) = E[\nabla_\phi \log \pi(a|s) q^\pi(s, a)]$ is the policy gradient of the stochastic policy. Finding the optimal parameters through the policy gradient maximizes the expectation of the cumulated return $E[\sum_{t=0}^T R_{t+1} | \pi_\theta]$. The random policy outputs a probability distribution $\pi_\theta(a|s) = P[a|s; \theta]$. When a random strategy is adopted, different actions may be taken even in the same state. The deterministic policy gradient (DPG) algorithm [30] extends on this basis. When the agent is in the same state, the action is uniquely determined that is the deterministic policy $\mu_\theta(s) = a$. DDPG [9] algorithm introduces neural networks based on DPG using neural

networks to approximate policy μ_ϕ and critic $Q_\theta(s, a)$ through the gradient accent and off-policy which can extract experience trajectories from the replay buffer.

Although DDPG can reach good performance sometimes, it is very sensitive for hyper-parameters and other types of adjustments. The common failure mode of DDPG is the overestimation of Q-value when starting to learn the Q-function. Twin Delayed DDPG (TD3) [5] is an algorithm that solves this problem by introducing three key techniques. The first one is clipped dual Q-networks. TD3 learns two Q-functions and uses the smaller of the two Q-values to form the target of the Bellman error $y(r, s', d) = r + \gamma(1 - d) \min Q_{\theta_i, tar}(s', \mu'(s)), i = 1, 2$. The second is delayed policy update. The policy network parameters are updated after dual Q-function networks are updated. Thirdly, smoothed target policy utilization makes all action values within the specified range $\mu'(s') = clip(\mu_{\theta_{tar}}(s') + clip(\xi, -c, c), a_L, a_H)$, $\xi \sim \mathcal{N}(0, 1)$.

The MADDPG algorithm extends the DDPG algorithm to the multi-agent domain, and the agent i takes the centralized Q-value as $Q_i^\mu(s, a)$, where $\mu = \mu_1, \mu_2, \dots, \mu_N$ is $\kappa = \kappa_1, \kappa_2, \dots, \kappa_N$ parameterized group deterministic policy set. s is the state of the current environment. $a = a_1, a_2, \dots, a_N$ is the set of actions of N agents, and $\mathbf{x} = (o_1, o_2, \dots, o_N)$ is the set of observations of all agents. Another reason for stable training in MADDPG is that since we know the actions taken by all agents, $P(s'|s, u, \mu_1, \dots, \mu_N) = P(s'|s, u, \mu'_1, \dots, \mu'_N)$ will be sufficient even $\mu_i \neq \mu'_i$. A stable training environment makes it possible to use the experience replay. Taking centralized critic discretized execution with an off-policy way, the policy gradient of agent is $\nabla_{\phi_i} J(\mu_i) = E_{s, aB}[\nabla_{\phi_i} \mu_i(a_i | o_i) \nabla_{a_i} Q_i^\mu(s, a) | a_i = \mu_i(o_i)]$, where B is the replay buffer storing experience tuple (x, a, r, x') .

4. Method

4.1. Double critic

We set up two independent centralized critic networks with corresponding target networks and one policy network with its target network for individual agents in the group. Its purpose is reducing overestimation by selecting a smaller Q-value when the neural network approximates the real state-action function.

Fig. 1 shows a double centralized critic network. Agent i obtains local observations and actions by interacting with the environment. For double critic networks of agent i , the input layer absorbs not only local observations o_i and action a_i , but also local observations and action information sets of other agents. The network out-

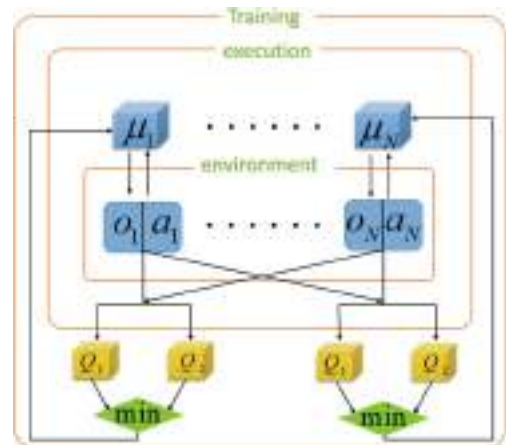


Fig. 1. Double centralized critic network.

puts an approximation of the true state-action equation, and selects a smaller Q-value through the minimization operator to the input layer of the policy network as a guide for its policy gradient.

4.2. Population overestimation

In Q-learning with discrete action space, the update formula for TD target is $y = r + \gamma \max_{a'} Q(s', a')$. Because each step involves a maximizing greedy operation, if the Q-value of a state action pair is overestimated, an error is introduced, which is accompanied by the Bellman update formula used by Q-learning which finally made

$$\mathbb{E}_{\omega} [\max_{a'} (Q(s', a') + \omega)] \geq \max_{a'} Q(s', a') \quad (2)$$

Thrun and Schwartz [31]. In some current reinforcement learning algorithms, like Fujimoto et al. [5] based on actor-critic, further proves that when the learning rate is satisfied, a single agent will have an overestimate error based on Thrun and Schwartz [31], as

$$\mathbb{E} [Q_{\theta}(s, \mu_{\phi_{true}}(o))] \geq \mathbb{E} [Q^{\pi}(s, \mu_{\phi_{true}}(o))], \quad \text{when } \alpha < \min(\omega_1, \omega_2) \quad (3)$$

where the real Q-function is $Q^{\pi}(s, a)$ and the approximate Q-function that actually estimates the real Q-function is $Q_{\theta}(s, a)$. ω_1 and ω_2 limit the learning rate to a very small value range. And it has proved that this deviation appeared in the DPG experiment. The most significant method of stabilizing training environment used in MADDPG is that the Q-network does not only absorb local observation and action of one agent when calculating Q-value but also absorbs observations and actions of other members if necessary. Thus, it doesn't change the intrinsic property of the Q-network. The overestimation in multi-agent scenarios still exist, as the input of formula (5) from local information changes to global information which means every individual's Q-network takes observations and actions from all members in the same 'cooperative' group. The experiments also proved that the overestimation error does exist in multi-agent environments as shown in Fig. 2. In the multiple agent reinforcement learning scenario, due to the environmental stability assumptions by Lowe et al. [8], the following errors exist:

$$\mathbb{E}_{\omega} [\max_{a'} Q_{\theta,i}(\mathbf{x}', \mathbf{a}') + \omega] \geq \max_{a'} Q^{\pi}(\mathbf{x}', \mathbf{a}'), \quad \text{where } \mathbf{x} = (o_1, o_2, \dots, o_N) \quad (4)$$

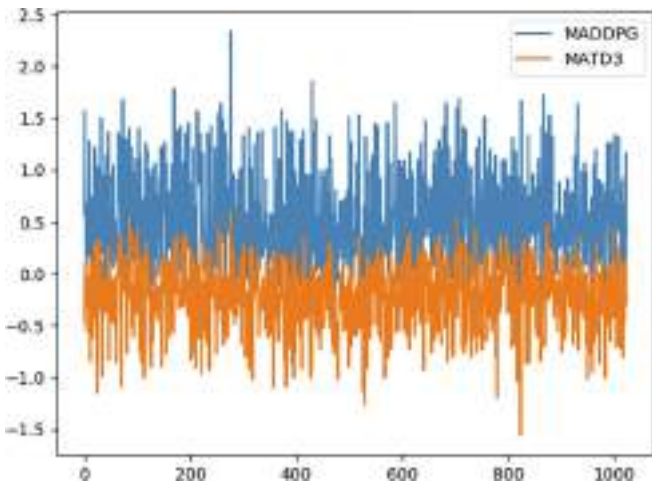


Fig. 2. The original estimated Q-value in MADDPG and the Q-value after taking reduction in MATD3.

This is an observation set for all agents. o_i is the local observation of agent i . $\mathbf{a} = \{a_1, a_2, \dots, a_N\}$ is the action set of all agents. a_i is the action of the agent i .

Our policy updating method in this paper is DDPG-like way. We let the real Q-function network of multiple agent reinforcement learning be annotated as $Q_{\theta,i}(x, a)$, and the actor guided by $Q_{\theta,i}(x, a)$, which $\mu_{\phi_{i,approx}}$ is abbreviated as $\mu_{i,approx}$. The approximate Q-function network is expressed as $Q_i^{\pi}(x, a)$ (unknown at training time), and the actor under its guidance work is $\mu_{\phi_{i,true}}$ abbreviated as $\mu_{i,true}$. The group policy parameters are updated by:

$$\phi_{i,approx} = \phi + \frac{\alpha}{K_1} \mathbb{E}_{\mathbf{x} \sim \mu} [\nabla_{\phi} \mu_{\phi}(\mathbf{x}) \nabla_{a_i} Q_{\theta,i}(\mathbf{x}, a) |_{a_i = \mu_{\phi_i}(o_i)}] \quad (5)$$

$$\phi_{i,true} = \phi + \frac{\alpha}{K_1} \mathbb{E}_{\mathbf{x} \sim \mu} [\nabla_{\phi} \mu_{\phi}(\mathbf{x}) \nabla_{a_i} Q_i^{\pi}(\mathbf{x}, \mathbf{a}) |_{a_i = \mu_{\phi_i}(o_i)}] \quad (6)$$

K_1 and K_2 are used to normalize the gradient. The overestimation error will still appear under some more stringent conditions without normalized gradient [5], because gradient direction is a local optimum rate of increase of the equation. Thus existing ω_1 , when ω_1 enough small, if $\alpha < \omega_1$, the true Q-function value under the guidance of $\mu_{i,approx}(x_i)$ will limit the true value of the Q-function under the policy $\mu_{i,true}(o_i)$ guidance:

$$\begin{aligned} \mathbb{E} [Q_{\theta,i}(o_1, \dots, o_N, a_1, \dots, a_N) |_{a_i = \mu_{i,approx}(o_i)}] \\ \geq \mathbb{E} [Q_{\theta,i}(o_1, \dots, o_N, a_1, \dots, a_N) |_{a_i = \mu_{i,true}(o_i)}] \end{aligned} \quad (7)$$

On the contrast, there exist ω_2 while it is enough small, if $\alpha < \omega_2$, the true value of Q-function under the policy $\mu_{i,approx}(x)$ guidance will limit that under the policy $\mu_{i,true}(x)$ guidance:

$$\begin{aligned} \mathbb{E} [Q_i^{\pi}(o_1, \dots, o_N, a_1, \dots, a_N |_{a_i = \mu_{i,true}(o_i)})] \\ \geq \mathbb{E} [Q_i^{\pi}(o_1, \dots, o_N, a_1, \dots, a_N |_{a_i = \mu_{i,approx}(o_i)})] \end{aligned} \quad (8)$$

When both of their policies are assumed to be $\mu_{\phi_{i,true}}$ because of the formula (1) and $\alpha < \min(\omega_1, \omega_2)$, the value of the group Q-function value will be overestimated:

$$\begin{aligned} \mathbb{E} [Q_{\theta,i}(o_1, \dots, o_N, a_1, \dots, a_N |_{a_i = \mu_{i,approx}(o_i)})] \\ \geq \mathbb{E} [Q_i^{\pi}(o_1, \dots, o_N, a_1, \dots, a_N |_{a_i = \mu_{i,approx}(o_i)})] \end{aligned} \quad (9)$$

Although this overestimated error is relatively small at each update, the presence of this error can cause two concerns. First, if not confirmed, this overestimation bias may accumulate as a large deviation. Second, an unhealthy inaccurate value estimation may also cause the strategy to be updated in a bad direction. In the current multi-agent algorithm MADDPG, we show the difference between the estimate of the true value and the true value of the estimated value after the elimination. As shown in Fig. 2, it can be clearly seen that the deviation does exist.

4.3. Reduce the population overestimate

There have been some ways to reduce the overestimation bias that occurs when using Q-learning. In the field of multi-agents, Zheng et al. [16] explores the extension of double Q-learning to the multi-agent domain, but only suitable for cooperative situation. We improve the critic under the actor-critic setting, and make the algorithm suitable for the mixed cooperative competition environment. Besides, we also reduce the overestimation bias of the real state-action equation.

In double Q-learning, the update to the TD target is through two independent Q-networks, each of which is used to update the esti-

mate of another Q network. If the estimates of values are independent of each other, they can select actions for the policy, making an unbiased estimate of actions chose by the other's policy.

In WMA double weighted multi-agent [16], the update of two Q-networks Q_v and Q_u randomly select a Q-network for updating, and one network is used as the target network of the other network. Both Q_u and Q_v are Q-networks in Fujimoto et al. [5] which balance two Q-networks with β as Eq. (9). If one of them overestimates issues during evaluating the Q-value of same pair of observation and action, it is able to reduce overestimation partly using this method. Because the Q-value is balanced with β , Q-target calculating will be more precise as shown in Eq. (10). The TD target update function is:

$$Q_U^w(s', a*) \leftarrow \beta Q^U(s', a*) + (1 - \beta) Q^V(s', a*), a* \leftarrow \arg \max_a Q^U(s', a) \quad (10)$$

$$Q_x^{\text{target}}(s', a*) \leftarrow R(s, a) + Q_U^w(s', a*) \quad (11)$$

Where U is the Q-function network 1, V is the Q-function network 2, or turn them upside down.

Due to MADDPG is under actor-critic setting, its TD target update function is:

$$y = r_i + \gamma Q_i^{\mu'}(s', a'_1, \dots, a'_N) |_{a'_j = \mu'_{\phi_j}(o_i)} \quad (12)$$

In the experiment, we found that if the policy is slowly changed in the actor-critic structure, the difference between the current and target value network will be very small as well as the update, and the improvement effect will not be obvious. Let critics of the agent i be $(Q_{i,\phi_1}, Q_{i,\phi_2})$, and policies under their guidance are $(\mu_{i,\phi_1}, \mu_{i,\phi_2})$ respectively. We draw on the TD update method in WMA Double DQN and MADDPG, and propose our own TD target update mode of MATD3 agent i :

$$y_i = r_i + \gamma \min_{n=1,2} Q_{i,\phi_n}(s', a'_1, \dots, a'_N) |_{a'_j = \mu'_{\phi_j}(o_i)} \quad (13)$$

Another benefit of setting the independent variable of the estimated error to be variable is that we can see that the minimized operator should provide higher values for states with low variance estimation errors, because the expected smallest value of random variables set will decrease as the variance of the random variable increases. This effect means that the minimum of Eq. (9) will have some preference when estimating state value with low variance. This preference is beneficial and can lead to a more secure policy update that stabilizes learning. This method reduces the drawbacks that the algorithms only apply cooperative environment caused by using DQN and MADDPG cannot alleviate the overestimate error.

4.4. Population policies variance

If the group overestimate error does not be suppressed or reduced in time, it will cause the accumulation of the overestimate error of the group, and finally seriously affect the policy update. This means that when a certain state-value pair is evaluated, there will be a large divergence, causing a large variance in value evaluation so that seriously affects the policy to update in a beneficial direction. Sutton [29] mentions that the high variance of the estimate results will produce a noise gradient when the policy is updated, which can be reduced by decreasing the learning rate, but it is not easy to perform well in actual training. In this section, we emphasize:

- (1) The description of relationship between target network and estimate error.

- (2) The importance of minimizing population error in each time step of updating.
- (3) Proposing how to reduce variance of estimate results based on actor-critic.

4.4.1. Accumulated population error

The policy update of our algorithm uses a time difference method similar to DDPG update. Each evaluation of state action pairs is based on the evaluation of next state, so agent i will introduce an estimation error at each time step t :

$$\delta_{i,t}(s_1, \dots, s_N, a_1, \dots, a_N) = \delta_{i,t}(s, a) \quad (14)$$

$$Q_{\theta_i}(s_1, \dots, s_N, a_1, \dots, a_N) = r + \gamma \mathbb{E}[Q_{\theta_i}(s'_1, \dots, s'_N, a'_1, \dots, a'_N)] - \delta_{i,t}(s_1, \dots, s_N, a_1, \dots, a_N) \quad (15)$$

$$Q_{\theta_i}(s, a) = r + \gamma \mathbb{E}[Q_{\theta_i}(s', a')] - \delta_{i,t}(s, a) \quad (16)$$

Although we hope that this estimation error is as small as possible, it may be accumulated and finally makes the policy update to the local optimal direction. The above formula can be rewritten as:

$$\begin{aligned} Q_{\theta_i}(s_t, a_t) &= r_t + \gamma \mathbb{E}[Q_{\theta_i}(s'_{t+1}, a'_{t+1})] - \delta_{i,t}(s, a) \\ &= r_t + \gamma \mathbb{E}[r_{t+1} + \gamma \mathbb{E}[Q_{\theta_i}(s_{t+2}, a_{t+2})] - \delta_{i,t+1}] - \delta_{i,t} \\ &= \mathbb{E}_{s_1 \sim \mu, a_1 \sim \mu} \left[\sum_{j=t}^T \gamma^{j-t} (r_j - \delta_{i,j}) \right], \text{ while } i = 1, \dots, N \end{aligned} \quad (17)$$

As can be seen from the function 11, when we explicitly write the estimated error into the equation for the Q-value, the variance of the Q-value is not only related to the expected variance of the future reward, but also of the estimation error. If the discount factor γ is large at each step of updating, and does not pay attention to the reduction of the estimation error, the accumulation of errors will be accumulated, so that the variance of the Q-value will increase rapidly every time an update is made. Gradient updates only reduce errors in this mini-batch, but do not help reduce the remaining errors in the entire set of value estimates.

4.4.2. Individual target network and delayed population policy update

The target network is an important tool for stable deep learning. Because the approximator of function needs multi-step gradient update to converge, the target network provides a stable target and makes possible that the training data more effectively convergent during the individual learning process in the group. Without a fixed personal target, each update may have residual errors and begin to accumulate. The accumulation of errors can be harmful to itself. when the residual error appear with policy maximization along the estimation of values, it will result in a uncontrollable divergence.

Fig. 3 shows that the failure of actor critic may be due to the interaction between critic and actor. If our individual policy is updated with the Q-value estimated by its dual centralized critic after each individual critic update, it is easy to update in a terrible direction due to the high variance and inaccurate estimation of the Q-value distribution. And because we adopt a centralized critic, bad policies will spread throughout the group. So it is better to consider updating when critic is stable.

If the policy is updated on a high error state, it will lead to decentralized behavior. And since the target network can reduce the cumulative error caused by multi-step updates, our individual target network should be updated with low frequency. Our algorithm allows the individual's target network to be updated after the critic updating. In order to wait for the critic to be slightly stable, the value error is small enough before updating the policy

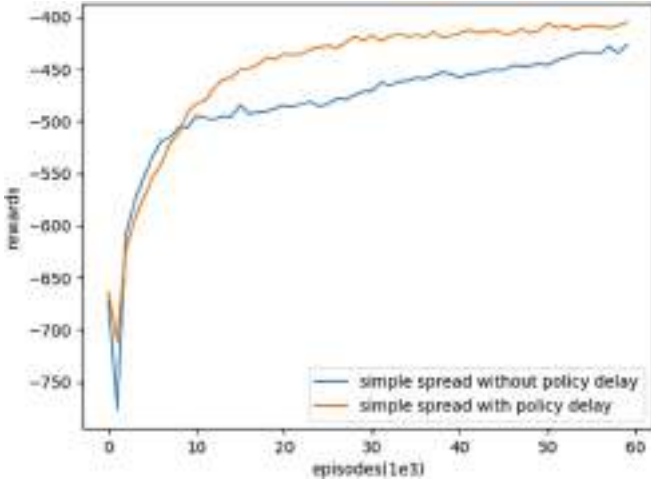


Fig. 3. Comparison of policy delay update.

and target network. We limit the possibility of repeated updates for updating the policy with appropriate delays when the critic does not change. The lower the frequency of policy updates, the higher probability of using low variance for value estimation in principle making the policy update in a better direction. The effectiveness of this method has been demonstrated in the experiment of Section 5, and the performance has been effectively improved.

4.5. Target policy smoothing

The shortcoming of MADDPG is that it may overfit to local peaks. The TD target of individual agent i is:

$$y_i = r_i + \gamma Q_i^{\mu'}(x', a'_1, \dots, a'_N) |_{a'_j = \mu'_k(o_j)} \quad (18)$$

It can be seen that when calculating the TD target of the agent i , two approximate networks are adopted. One is the approximate network of the value network Q_i , and the other is the approximate network of the deterministic policy set μ' , so the error of the TD target due to the approximation of the equation is very sensitive. Therefore, the variance of the TD target is also increased that can be mitigated by regularization. By referring to TD3s thoughts that a single agent should have similar Q-values under similar state-actions, our algorithm mitigates the bias and variance caused by the approximation of the equation with smoothing the target policy μ'_i of each agent i :

$$y_i = r_i + \mathbb{E}_{\omega_i} [Q_{i,\theta'}(x', a'_1, \dots, a'_N) + \omega_i] |_{a_j = \mu_{j,\phi'}(o'_j)} \quad (19)$$

In the actual experiment, the processed random noise ω_i is added to the target policy $\mu_{i,\phi'}$ of each agent i :

$$y_i = r_i + \gamma Q_{i,\theta'}(x', a'_1, \dots, a'_N) + \xi_i |_{a_j = \mu_{j,\phi'}(o'_j)}, \xi_i \sim \text{clip}(N(0, \delta), \text{action_low}, \text{action_high}) \quad (20)$$

where action low and action high are the minimum and maximum of the total action space respectively. $v(0, \delta)$ is a Gaussian distribution with an expectation of 0 and a variance of δ .

5. Experiments

In this section, we show that the MATD3 algorithm has a better performance than the multi-agent environment used by the MADDPG algorithm. With the critic of truncated duel centralized Q-network to reduce the overestimation bias of a single agents

critic networks in the population, the stability of the training environment is increased and the overestimation error is reduced. Besides, the performance of the algorithm is improved. In MATD3, there are two Q-networks for critic, which are used to compare the estimate values, and take a smaller estimate value as the reference Q-value used in actor network update. Critics are updated by minimizing the difference between the target network and the Q-value estimation network. The target network and actor are updated after critics updating certain times.

5.1. Comparison

We performed our algorithm testing in MADDPG's self-built multi-agent particle environment [32]. This environment includes nine scenarios, and the detailed description of the scene is in the MADDPG article, we will not repeat them. The nine scenarios are: simple, physical deception, covert communication, keep-away, simple reference, cooperative communication, cooperative navigation, predator-prey and simple world comm. Our algorithm is compared to DQN, DDPG and MADDPG in nine scenarios. Our results include tables and images shows as following. In our algorithm, each agent i has two centralized critic networks and one actor network, and those networks have their own corresponding target network respectively. The neural network structure is used to set up adversaries and good agents are the same as what uses three full-connected layers as Fig. 4. There is one active unit Relu between each layer, and every layer has 64 units. The parameters of network are updated by Adam. The learning rate is $10e-2$ and set 1024 transitions for mini-batch training which are uniformly sampled from the experience replay buffer. The two independent centralized critic target networks of each agent receive the state set $x = o1, o2, \dots, oN$ and action set $a_1, a_2, \dots, a_N$ in the tuple uniformly is sampled from the replay buffer as the input of the first layer. We compare two outputs and take a smaller Q-value to update the target value of the two critics.

From the Fig. 4, double centralized critic networks have their own streams to estimate the Q-value of current population state-action set and output a smaller Q-value to the policy network by the minimize operator.

To achieve target policy smoothing, the action is eventually limited to the action space of corresponding environment by adding noise $\xi \in N(0, \text{noise})$ to the action. After critic updating certain times, all target networks and actor networks are updated. We use the Q-estimated-value of critic network 1 to update the agent's unique actor network parameters. Update the actor by policy gradient: $\nabla_{\phi} J(\phi_i) = \mathbb{E}_{x,a} [\nabla_{a_i} Q_i^{\mu}(x, a_1, \dots, a_N) \nabla_{\phi_i} \mu_{\phi_i}(o_i) |_{a_i = \mu_{\phi_i}(o_i)}]$. Each task of MATD3 ran between 60,000 and 160,000 times as we marked in tables and figures, and the policy evaluation was conducted every 1000 times.

We don't make any special changes to other three algorithms' original setting when comparing with our work. It should be

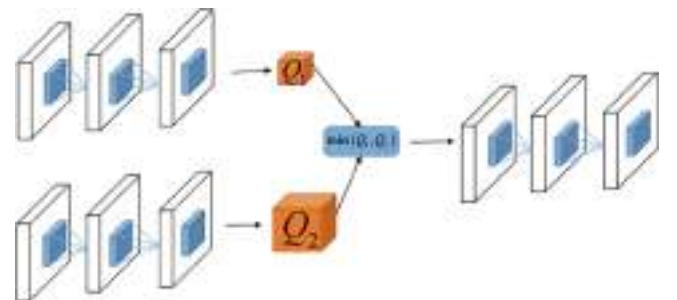


Fig. 4. The double centralized critic networks.

emphasized that the network structure of these algorithms adopts the same network structure and parameter setting. We did not make changes to achieve our goals deliberately. The detail of our algorithms is shown in the appendix.

Our learning curves are shown in Fig. 5. From this figure, we can see that the curves of DQN are strongly undulant and do not reach at a stable state in most of scenarios within limited number of iterations. What's more, MATD3 has smaller oscillation range. For the purpose of more detailed evaluation, the size and figure are adjusted appropriately as Fig. 6 in order to compare MATD3 and other algorithms.

Benefited from reducing overestimation errors, smoothing the target policy and delayed policy update, our algorithm finally performs well in most of scenarios and accelerates the convergence speed of previous period.

In the cooperation scenario, the reward obtained by MATD3 in cooperative navigation is higher than the reward obtained by the original algorithm in the environment. In the convergence of the previous period of MATD3 in cooperative communication, the convergence speed is accelerated due to the reduction of the overestimation error. In the competition scenario, we uniformly set good agents to the agent using MADDPG algorithm, and set adversaries to one of DQN, DDPG, MADDPG and MATD3, then compare good ones and adversaries. In the most complex scene simple world

comm, our algorithm performs much better, especially it has improved performance in the confrontation with MADDPG.

5.2. Competition experiments

In the scenario physical deception, we let adversaries' algorithm be MATD3, and good agents be MADDPG. Compare the confrontation between two sides as we showed in Table 1.

The definition of adv success once is that the center distance between the agent and the target landmark less than a certain threshold which is set to the sum of the radius of the agent and the target landmark. Take the 20,000 times after the reward is stabilized to calculate the number of successes in 60,000 times. Take the 50,000 times after the reward is stabilized to calculate the number of successes in 100,000 times. successful times is the number of MATD3 (adversaries successful times) minus MADDPG (good agents' successful times).

5.3. Complexity experiment

The number of agents and targets involved in the nine experiments are shown in Table 2. The most complicated scenario is simple world comm which involves the largest agents number and includes communication and competition at the same time. In

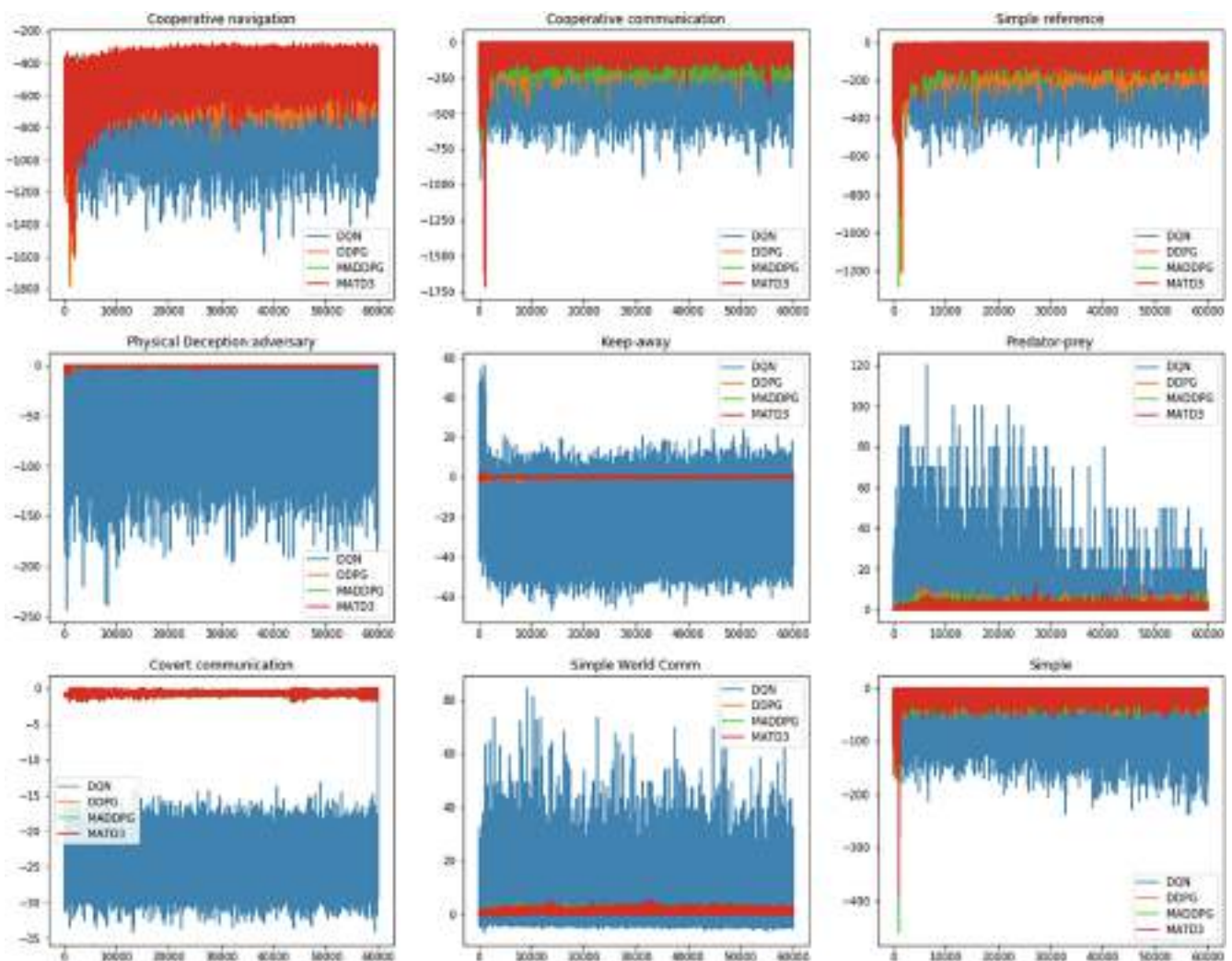


Fig. 5. MATD3 compared with DQN, DDPG and MADDPG in nine scenarios.

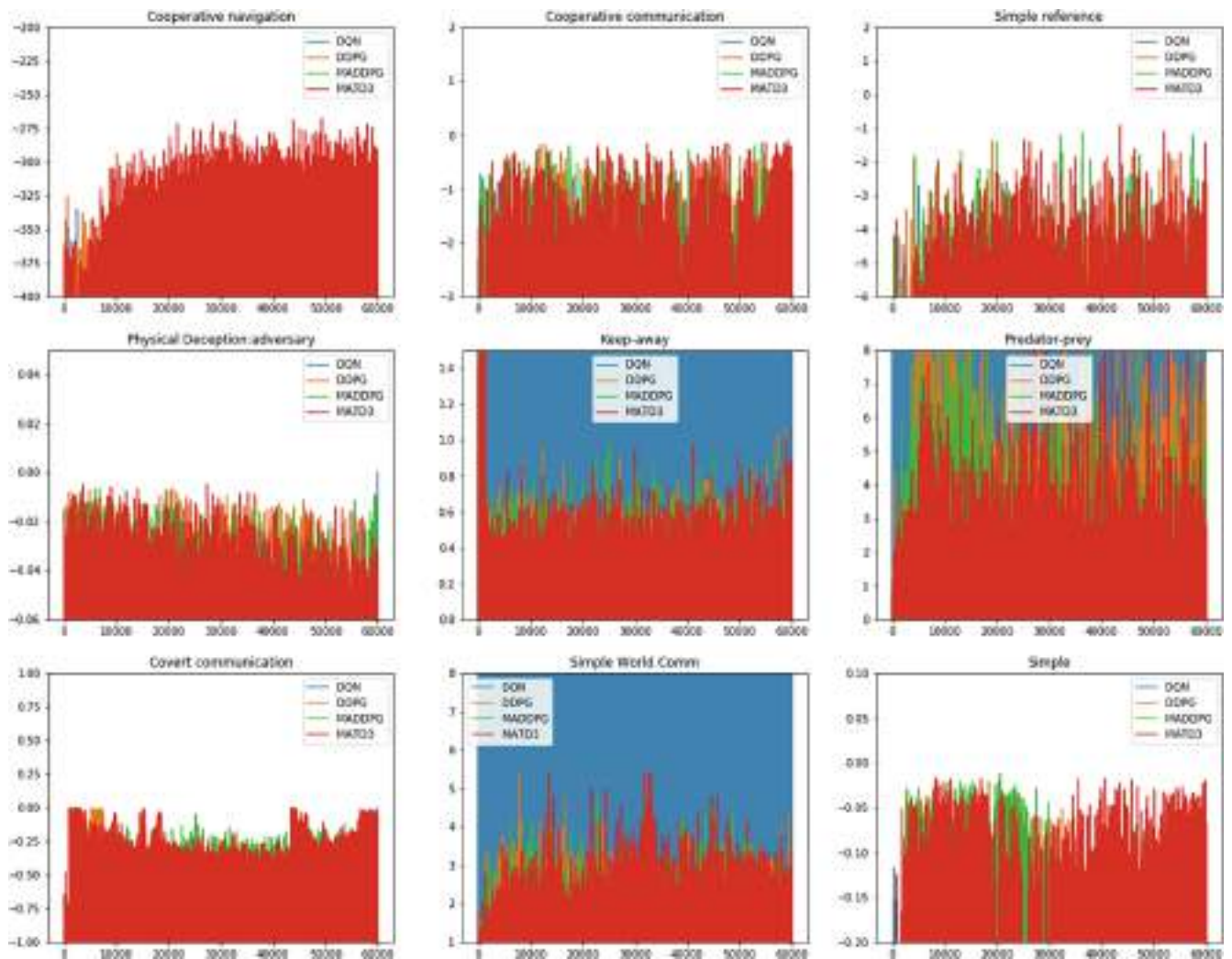


Fig. 6. Adjusted MATD3 compared with DQN, DDPG and MADDPG in nine scenarios.

Table 1

Results in competitive scenario physical deception.

Good agent 1, 2	Adversary agent	Good successful times	Adv successful times	δ successful times
MADDPG	MATD3	49,137	10,863	1369 (60 k)
MATD3	MADDPG	50,405	9494	1369 (60 k)
MATD3	MATD3	50,485	9515	1695 (60 k)
MADDPG	MADDPG	48,790	11,210	1695 (60 k)
MADDPG	MATD3	75,672	24,328	3538 (100 k)
MATD3	MADDPG	79,210	20,790	3538 (100 k)

Table 2

Complexity and improvement of space and tasks in nine scenarios.

Number	Experiment name	Competitive?	Comm?	Number of Agents	Number of	Raise by (%)
1	Simple	N	N	1	1	9.55%
2	Keep-away	Y	N	2	2	12.05%
3	Simple reference	N	Y	2	3	3.48%
4	Cooperative communication	N	Y	2	3	−3.64%
5	Physical deception	Y	N	3	2	−1.04%
6	Cooperative navigation	N	N	3	3	6.93%
7	Predator–prey	Y	N	4	2	−7.75%
8	Covert communication	Y	Y	3	2	12.08%
9	Simple world comm	Y	Y	6	5	25.04%

Fig. 5, it clearly shows that we get the highest score increase in this scenario, which confirms that the more complex the scenario of our MATD3 algorithm, the higher the execution of the group task.

Adv vs good means the competition between adversaries and good agents. Simple world comm involves six agents. Correspondingly, the adversaries group has four agents, the good group has two, and the landmark has five. The landmarks consist of two green areas representing the forest, two blue areas representing the water source, and a black obstruction area. Good agents try to get close to the water source to get rewards, and the adversaries prevents good agents from getting close to the water source. Agents hiding in the forest cannot be discovered by agents other than leaders. Agents in the same group learn to cooperate and can communicate to learn tasks such as chasing and pushing away.

6. Conclusion

This paper explores the overestimation errors and high variance problems in multiple agent deep reinforcement learning. We alleviate the overestimation problem by reducing the population centralization Q-value, and constrain the individuals action value additionally. The delayed policy update further stabilizes the multi-agent training environment, finally achieves good results in the experimental scenarios.

One of the advantage of delayed update policy is that the policy will improve itself towards a better direction when the critic is stable. However, we found that each agent's individual delayed policy update will cause a large group delay when extending the single agent TD3 to the multi-agent field, which limits the convergence speed and may even lead to lower exploration rate. Therefore, future research can consider increasing the exploration rate and increasing the utilization of high-value trajectories in the experience replay buffer.

Appendix A

Algorithm 1: Delayed Double Critics DDPG for N-Agents Population

```

1: Initial two critic networks for each agent  $i$ ,  $Q_{\theta_{i,1}}^\mu, Q_{\theta_{i,2}}^\mu$  and actor network  $\mu_{\phi_i}$  with random parameters  $\theta_{i,1}, \theta_{i,2}, \phi_i$ . Initial target networks for each agent  $i$ ,  $\theta'_{i,1} \leftarrow \theta_{i,1}, \theta'_{i,2} \leftarrow \theta_{i,2}, \phi'_i \leftarrow \phi_i$ , Initial replay buffer  $\mathcal{B}$ 
2: bf forepisode = 1 to Num-episodes do
3:   Initial a random noise  $\xi$  for exploring actions, and receive initial state  $\mathbf{x} = (o_1, o_2, \dots, o_N)$ 
4:   for  $t = 1$  to Max-length-episodes do
5:     for each agent  $i$ , select random action  $a_i \sim \mu_i(o_i) + \xi_i$ , w.r.t exploration and the current deterministic policy,  $\xi_i \sim \text{clip}(N(0, \delta), a_{\text{low}}, a_{\text{high}})$ 
6:     Execute actions  $\mathbf{a} = (a_1, \dots, a_N)$  and observe reward  $r$  and new state  $\mathbf{x}'$ 
7:     Store experience tuple  $(\mathbf{x}', \mathbf{a}, r, \mathbf{x}')$  in replay buffer  $\mathcal{B}$ , and update state  $\mathbf{x} \leftarrow \mathbf{x}'$ 
8:     foreach agent = 1 to  $N$  do
9:       Randomly sample a mini-batch including  $N$  samples  $(\mathbf{x}^j, \mathbf{a}^j, r^j, \mathbf{x}^j)$  from  $\mathcal{B}$ 
10:      Set target  $y^j = r^j + \gamma \min_{n=1,2} Q_{\theta'_{i,n}}^\mu(\mathbf{x}^j, \mathbf{a}^j, \dots, \mathbf{a}_k^j, \dots, \mathbf{a}_N^j) |_{a_k=\mu'_k(o'_i)}$ 
11:      Update critic  $\theta_{i,n} \leftarrow \text{argmin}_{\theta_{i,n}} N^{-1} \sum (y^j - Q_{\theta_{i,n}}^\mu(\mathbf{x}^j, \mathbf{a}^j, \dots, \mathbf{a}_N^j))^2$ 
12:      if  $t \bmod d$  then
13:        Update actor parameter  $\phi$  by policy gradient:  $\nabla_{\phi_i} J(\phi_i) = N^{-1} \left( \sum_j \nabla_{\phi_i} \mu_i(o_i^j) \nabla_{a_i} Q_{\theta_{i,1}}^\mu(\mathbf{x}^j, \mathbf{a}^j, \dots, \mathbf{a}_i^j, \dots, \mathbf{a}_N^j) |_{a_i=\mu_i(o_i^j)} \right)$ 
14:        Update target networks:
15:         $\theta'_{i,n} \leftarrow \tau \theta_{i,n} + (1 - \tau) \theta'_{i,n}, n = 1, 2$ 
16:         $\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$ 
17:      end if
18:    end for
19:  end for
20: end for

```

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Fengjiao Zhang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing. **Jie Li:** Supervision. **Zhi Li:** Resources, Data curation, Project administration, Funding acquisition.

Acknowledgments

This work was supported by the Key Research and Development Project of Science & Technology Department of Sichuan Province under Grant 2019YFG0192.

References

- [1] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484.
- [2] M. Jaderberg, W.M. Czarnecki, I. Dunning, L. Marris, G. Lever, A.G. Castaneda, C. Beattie, N.C. Rabinowitz, A.S. Morcos, A. Ruderman, Human-level performance in first-person multiplayer games with population-based deep reinforcement learning, *arXiv preprint arXiv:1807.01281*.
- [3] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, N. De Freitas, Dueling network architectures for deep reinforcement learning, *arXiv preprint arXiv:1511.06581*.
- [4] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: *International Conference on Machine Learning*, pp. 1928–1937.
- [5] S. Fujimoto, H. van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, *arXiv preprint arXiv:1802.09477*.
- [6] C.J. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (3–4) (1992) 279–292.

Scaling CrossQ with Weight Normalization

Daniel Palenicek

Technical University of Darmstadt
hessian.AI
palenicek@robot-learning.de

Florian Vogt

University of Freiburg
vogtf@informatik.uni-freiburg.de

Jan Peters

Technical University of Darmstadt
hessian.AI
German Research Center for AI (DFKI)
Robotics Institute Germany (RIG)
jan@robot-learning.de

Abstract

Reinforcement learning has achieved significant milestones, but sample efficiency remains a bottleneck for real-world applications. Recently, CrossQ has demonstrated state-of-the-art sample efficiency with a low update-to-data (UTD) ratio of 1. In this work, we explore CrossQ’s scaling behavior with higher UTD ratios. We identify challenges in the training dynamics which are emphasized by higher UTDs, particularly Q-bias explosion and the growing magnitude of critic network weights. To address this, we integrate weight normalization into the CrossQ framework, a solution that stabilizes training, prevents potential loss of plasticity and keeps the effective learning rate constant. Our proposed approach reliably scales with increasing UTD ratios, achieving competitive or superior performance across a range of challenging tasks on the DeepMind control benchmark, notably the complex `dog` and `humanoid` environments. This work eliminates the need for drastic interventions, such as network resets, and offers a robust pathway for improving sample efficiency and scalability in model-free reinforcement learning.

Keywords: Reinforcement Learning, Batch Normalization, Weight Normalization

Acknowledgements

This research funded by the Research Clusters “Third Wave of AI” and “The Adaptive Mind”, funded by the Excellence Program of the Hessian Ministry of Higher Education, Science, Research and the Arts, hessian.AI.

1 Introduction

Reinforcement Learning (RL) has shown great successes in recent years. One fundamental question in RL that remains is how to increase the sample efficiency of the algorithms. This is one crucial part for making RL methods applicable to real-world scenarios like robotics. To that end, model-based RL attempts to increase sample efficiency by learning dynamic models that reduce the need for collecting real data, a process often expensive and time-consuming [Sutton, 1990, Janner et al., 2019, Cowen-Rivers et al., 2022]. Contrary to this approach, there have been attempts to increase the sample efficiency of purely model-free algorithms. Either by increasing the number of gradient updates on the available data, also referred to as update-to-data (UTD) ratio [Chen et al., 2021, Hiraoka et al., 2021, Nikishin et al., 2022, D’Oro et al., 2022], changes to the network architectures [Bhatt et al., 2024], or both [Hussing et al., 2024, Nauman et al., 2024].

In this work, we build upon CrossQ [Bhatt et al., 2024], a model-free RL algorithm that recently showed state-of-the-art sample efficiency on the MuJoCo [Todorov et al., 2012] continuous control benchmarking tasks. Notably, the authors achieved this by carefully utilizing Batch Normalization (BN) within the actor-critic architecture. A technique that has previously been thought to not work in RL as famously reported by Hiraoka et al. [2021] and others. The insight that Bhatt et al. [2024] offered, is that one needs to carefully consider the different state-action distributions within the Bellman equation and handle them correctly to succeed. This novelty allowed maintaining a low UTD ratio of 1 gradient step per environment interaction, while outperforming then state-of-the-art algorithms that scaled the UTD ratio to 20.

This naturally raises the question: *How can we extend the sample efficiency benefits of CrossQ and BN to the high UTD training regime?* Which we address in this manuscript.

Contributions. In this work, we show, that the vanilla CrossQ algorithm can be brittle to tune on DeepMind Control (DMC) environments. We further show that it does not scale reliably with increasing compute. We propose the addition of weight normalization (WN). We demonstrate, that WN is an easy but very effective measure to stabilize CrossQ. Further, we find that these stabilizing effects allow to scale the UTD of CrossQ and consequently its sample efficiency.

2 Preliminaries

Reinforcement learning. Consider a Markov Decision Process (MDP) [Puterman, 2014], which is defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$, with the state space $\mathcal{S} \subseteq \mathbb{R}^n$, action space $\mathcal{A} \subseteq \mathbb{R}^m$, transition probability $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, the reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and discount factor γ . We define the RL problem according to Sutton and Barto [2018]. A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is a behavior plan, which maps a state to a distribution over actions. The goal of RL is to find an optimal policy π^* that maximizes the discounted cumulative reward $\mathcal{R}_t = \sum_{k=t}^{\infty} \gamma^{k-t} r(s_k, a_k)$, also referred to as the *return*. The Q-function $Q^\pi(s, a) = \mathbb{E}_{\pi, \mathcal{P}}[\mathcal{R}_t | s_t = s, a_t = a]$ defines the expected return when taking action a in state s .

For a detailed introduction to CrossQ, we refer the reader to Bhatt et al. [2024], due to space constraints. In a nutshell, CrossQ introduces BN into the critic (and optionally actor) networks, widens the layers and removes target networks.

Normalization techniques in RL. Normalization techniques are widely recognized for improving the training of neural networks, as they generally accelerate training and improve generalization [Huang et al., 2023]. There are many ways of introducing different types of normalizations into the RL framework. Most commonly, authors have used Layer Normalization (LN) within the network architectures to stabilize training [Hiraoka et al., 2021, Lyle et al., 2024]. Recently, CrossQ has been the first algorithm to successfully use BN layers in RL [Bhatt et al., 2024]. The addition of BN lead to substantial gains in sample efficiency. In contrast to LN, however, one needs to carefully consider the different state-action distributions within the critic loss when integrating BN. In a different line of work, Hussing et al. [2024] proposed to project the output features of the penultimate layer onto the unitball to fight Q-function overestimation.

Van Laarhoven [2017] noted that normalization techniques such as BN and LN yield functions f that are scale invariant with respect to the weights w , i.e., $f(\mathbf{X}; cw, \gamma, \beta) = f(\mathbf{X}; w, \gamma, \beta)$. Further, the gradients scale inversely proportional to the scaling factor $\nabla f(\mathbf{X}; cw, \gamma, \beta) = \nabla f(\mathbf{X}; w, \gamma, \beta)/c$. In deep learning, one commonly observes the magnitudes of the network parameters to grow larger over the course of training. Together with inversely proportional scaling of the gradients, this induces an implicit decreasing learning rate schedule. Lyle et al. [2024] recently investigated the effectiveness of combining LN and WN to keep the effective learning rate (ELR) [Van Laarhoven, 2017] constant in order to avoid loss of plasticity in discrete state-action RL.

Increasing update-to-data ratios. Although scaling up the UTD ratio is an intuitive approach to increase the sample efficiency, in practice, it comes with several challenges. Nikishin et al. [2022] demonstrated that overfitting on early training data can inhibit the agent from learn anything later in the training. The authors dub this phenomenon the primacy bias. They suggest to periodically reset the network parameterized while retraining the replay buffer. Many works that followed have adapted this intervention [D’Oro et al., 2022, Nauman et al., 2024]. While often effective, regularly resetting is a very drastic intervention and by design induces regular drops in performance. Since the agent has

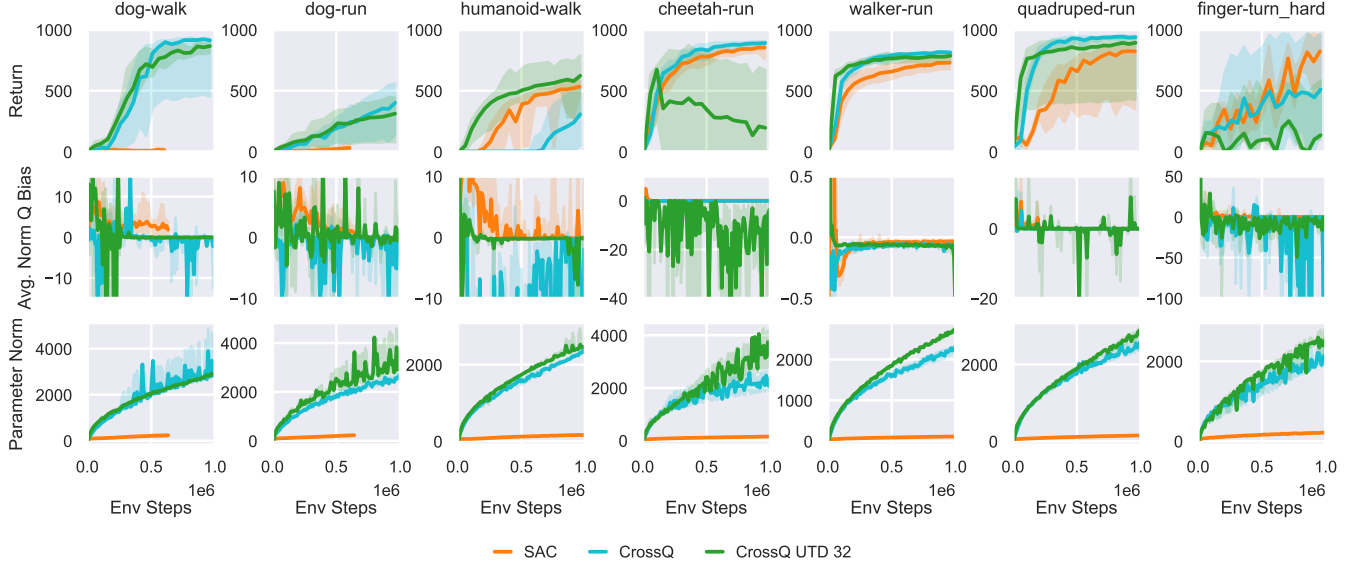


Figure 1: **Q-bias and weight norms.** In general CrossQ shows much larger weight norms than SAC. High UTD can emphasize this problem even more. Large and high variance Q-bias can often be linked to poor performance.

to repeatedly start learning from scratch, it is also not very compute efficient. Finally, the exact reasons why parameter resets work so well in practice are not well understood [Li et al., 2023]. Instead of resetting there have also been other types of regularization that allowed practitioners to train stably with high UTD ratios. Janner et al. [2019] generates additional modeled data, by virtually increasing the UTD. In REDQ, Chen et al. [2021] leverage ensembles of Q-functions, while Hiraoka et al. [2021] use dropout and LN to effectively scale to higher UTD ratios.

3 Failure to Scale Stably

Bhatt et al. [2024] demonstrated CrossQ’s state-of-the-art sample efficiency on the original MuJoCo task suite [Todorov et al., 2012], while at the same time, being very computationally efficient. However, on the more extensive DMC task suite, we find that CrossQ can require tuning. We further find that it works on some, but not all environments stably and reliably.

Figure 1 shows CrossQ training performance on a subset of DMC tasks. We picked the tasks to show a diverse set of behaviors, from successful learning to instabilities during learning. The results on the remaining tasks are similar but were left out due to space constraints. The second and third rows show the Q-function bias and the norm of the network weight matrices of the critic network, respectively. On the one hand, vanilla CrossQ can show instabilities during training or trains slowly, if at all. On the other hand, high UTD ratios can emphasize this effect in some environments. We can link bad performance of CrossQ to fluctuating and exploding Q-bias values. Which in turn have a connection to the common issue of Q-function overestimation.

Further, we notice the growing magnitude of the critic weight matrices. While all three baselines show growing network weights, the effect is vastly emphasized for CrossQ and even more with higher UTD ratios. Growing network weights are a phenomenon which has been linked to loss of plasticity, and can therefore, lead to premature convergence. Additionally, the growing magnitudes poses a challenge for the optimization in itself, similar to the issue of growing activations, the latter of which has been analyzed by Hussing et al. [2024].

4 Successfully Scaling CrossQ to High Update-To-Data Ratios with Weight Normalization

As we demonstrate in the previous section and reported in many other works [Chen et al., 2021, Hiraoka et al., 2021], the Q-bias can be linked to poor training performance. Particularly, in high UTD settings the Q-bias tends to increase and explode if no countermeasures are taken. Inspired by the combined insights of Hussing et al. [2024] and Lyle et al. [2024] we propose to combine CrossQ with WN as a means of counteracting Q-bias explosion and instabilities of high UTD ratios. Our reasoning goes is as follows: Due to the use of BN in CrossQ, the critic network is scale invariant, as noted by Van Laarhoven [2017]. Therefore, we can employ WN to counteract the growth of the networks weight norms. Lyle et al. [2024] showed, that for LN + WN, this helps against loss of plasticity. The scaling of the output is now left to

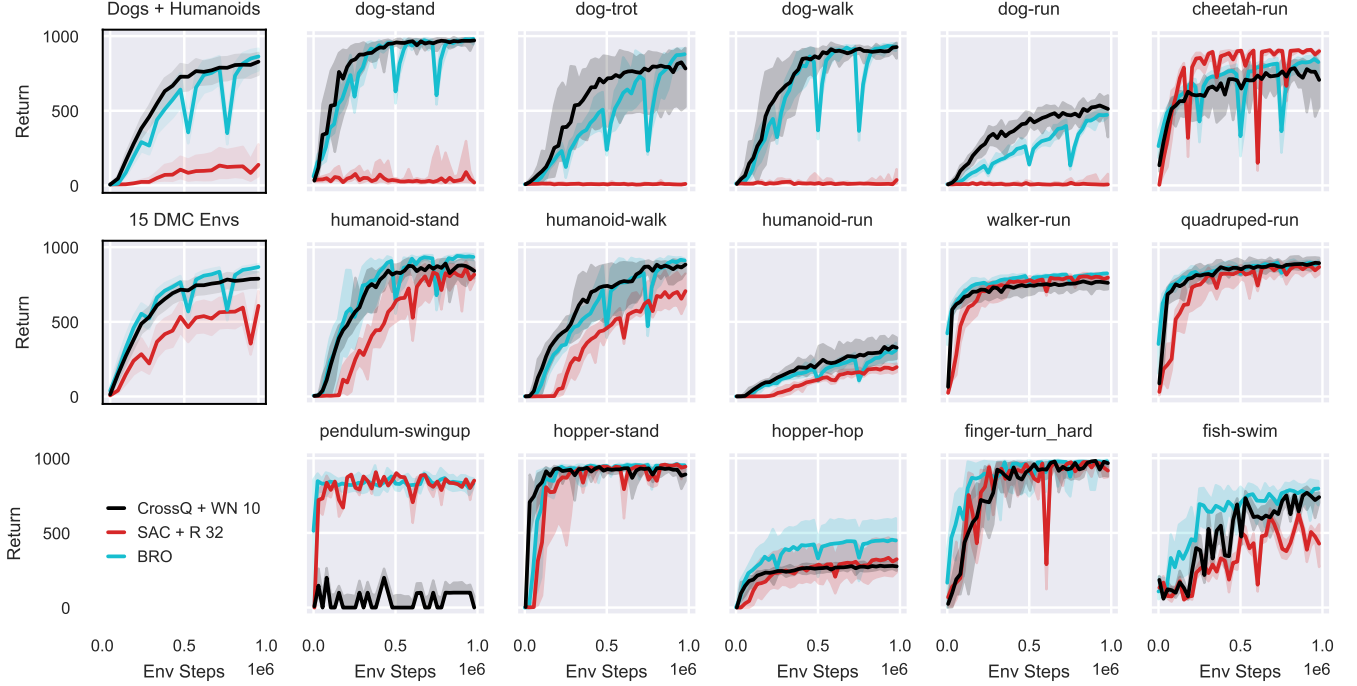


Figure 3: **CrossQ WN UTD=10 against baselines.** We compare our proposed CrossQ WN UTD=10 against two baselines, BRO [Nauman et al., 2024] and SAC with UTD=32 and resets. Results are reported on all 15 DMC tasks and the IQM and 90% inter percentile ranges over 10 random seeds. The leftmost plots (black border) provide aggregated results over the dog and humanoid environments and over 15 environments, respectively. Our proposed approach proves competitive to BRO and outperforms the SAC baseline. We want to note, that our approach achieves this performance without requiring resetting and is also much simpler than BRO.

the very last layer of the network only, similar to the unit ball normalization of the penultimate layers outputs proposed by Hussing et al. [2024]. And the ELR is kept constant.

Weight normalization allows CrossQ to scale effectively. Next, we provide empirical evidence for our hypothesis. We show that the addition of WN helps to stabilize CrossQ and allows to stably increase the UTD ratio. On 15 DMC tasks, we compare our proposed CrossQ + WN to the recently proposed BRO algorithm [Nauman et al., 2024] and SAC with a UTD=32 and periodic resetting. For computational reasons, for the BRO baseline we plot the official training data the authors published.

Figure 3 shows the results of our experiments. The leftmost column shows aggregated performances over the dog and humanoid environments, and over all 15 DMC environments respectively. The aggregation was performed according to Agarwal et al. [2021]. These results show, that CrossQ + WN UTD = 10 is competitive to the BRO baseline, especially on the more complex dog and humanoid tasks. For individual environments, we plot the IQM return and 90% inter percentile ranges across 10 seeds each. Again, we observe that our proposed CrossQ + WN UTD = 10 trains very stably. The combination with WN manages to eliminate the stability problems of the base algorithm. In most environments, it can compete with the much more complex BRO baseline, with the exception of hopper-hop and pendulum-swingup.

For hopper-hop, it seems to be environment specific, as Bhatt et al. [2024] also did not achieve good results on the MuJoCo Hopper-v4. For pendulum-swingup, we can attribute the low performance to the task’s sparse reward. We leave investigation of this to future work. More interestingly, CrossQ + WN UTD=10 matches or even outperforms BRO on the majority of the most difficult humanoid and dog tasks. Further, it also does not suffer from the periodic drops in performance caused

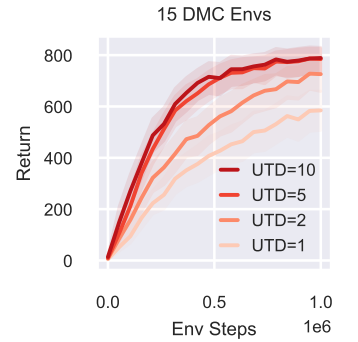


Figure 2: **CrossQ WN UTD scaling behavior.** We plot the IQM return and 95% confidence intervals for different UTD ratios $\in \{1, 2, 5, 10\}$. The results are aggregated over 15 DMC environments and 10 random seeds each according to Agarwal et al. [2021]. The sample efficiency scales reliably with increasing UTD ratios.

by frequent network resetting. This leads us to conclude, that CrossQ benefits from the addition of WN, which results in stable training and scales well with high UTD ratios. The resulting algorithm can match or outperform state-of-the-art baselines on the continuous control DMC suite.

Stable scaling of CrossQ + WN across different UTD ratios. This ablations shows the stable scaling behavior of our proposed CrossQ + WN UTD = 10 with increasing UTD ratios. Figure 2 provides aggregated training curves $UTD \in \{1, 2, 5, 10\}$ aggregated over all 15 DMC tasks. The learning curves are nicely ordered in accordance with their UTD ratio. The same is true for individual learning curves in the environments, however, due to space constraints, we do not include this figure here.

5 Conclusion

In this work, we have addressed the instability and scalability limitations of CrossQ in RL by integrating WN. Our empirical results demonstrate that WN effectively stabilizes training and allows CrossQ to scale reliably with higher UTD ratios. The proposed CrossQ + WN approach achieves competitive or superior performance compared to state-of-the-art baselines across a diverse set of DMC tasks, including the complex humanoid and dog environments. By eliminating the need for drastic interventions such as network resets, this extension preserves simplicity while enhancing robustness and scalability.

References

- R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 2021.
- A. Bhatt, D. Palenicek, B. Belousov, M. Argus, A. Amiranashvili, T. Brox, and J. Peters. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International conference on learning representations*, 2024.
- X. Chen, C. Wang, Z. Zhou, and K. Ross. Randomized ensembled double Q-learning: Learning fast without a model. In *International conference on learning representations*, 2021.
- A. I. Cowen-Rivers, D. Palenicek, V. Moens, M. A. Abdullah, A. Sootla, J. Wang, and H. Bou-Ammar. Samba: Safe model-based & active reinforcement learning. *Machine learning*, 2022.
- P. D’Oro, M. Schwarzer, E. Nikishin, P.-L. Bacon, M. G. Bellemare, and A. Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International conference on learning representations*, 2022.
- T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. In *International conference on learning representations*, 2021.
- L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, and L. Shao. Normalization techniques in training dnns: Methodology, analysis and application. *IEEE transactions on pattern analysis and machine intelligence*, 2023.
- M. Hussing, C. Voelcker, I. Gilitschenski, A.-m. Farahmand, and E. Eaton. Dissecting deep rl with high update ratios: Combatting value overestimation and divergence. *arXiv preprint arXiv:2403.05996*, 2024.
- M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in neural information processing systems*, 2019.
- Q. Li, A. Kumar, I. Kostrikov, and S. Levine. Efficient deep reinforcement learning requires regulating overfitting. In *International conference on learning representations*, 2023.
- C. Lyle, Z. Zheng, K. Khetarpal, J. Martens, H. van Hasselt, R. Pascanu, and W. Dabney. Normalization and effective learning rates in reinforcement learning. In *Neural information processing systems*, 2024.
- M. Nauman, M. Ostaszewski, K. Jankowski, P. Miłoś, and M. Cygan. Bigger, regularized, optimistic: scaling for compute and sample-efficient continuous control. In *Advances in neural information processing systems*, 2024.
- E. Nikishin, M. Schwarzer, P. D’Oro, P.-L. Bacon, and A. Courville. The primacy bias in deep reinforcement learning. In *International conference on machine learning*, 2022.
- M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. *Machine learning*, 1990.
- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2018.
- E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *International conference on intelligent robots and systems*, 2012.
- T. Van Laarhoven. L2 regularization versus batch and weight normalization. *arXiv preprint arXiv:1706.05350*, 2017.